



SAP Cloud Integration

Generated on: 2026-10-06 11:57:10 GMT+0000

SAP Cloud Integration | Cloud

Public

Original content: https://help.sap.com/docs/CLOUD_INTEGRATION/368c481cd6954bdfa5d0435479fd4eaf?locale=en-US&state=PRODUCTION&version=Cloud

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit <https://help.sap.com/docs/disclaimer>.

Basic Concepts of Integration Design

This section provides an overview of the concepts of integration content design.

Related Information

[Elements of an Integration Flow](#)

[The Camel Data Model in a Nutshell](#)

[Updating your Existing Integration Flow](#)

[Adapting Standard Integration Content to Your Requirements](#)

[Runtime Profiles](#)

Elements of an Integration Flow

An integration flow allows you to specify how a message is processed on a tenant.

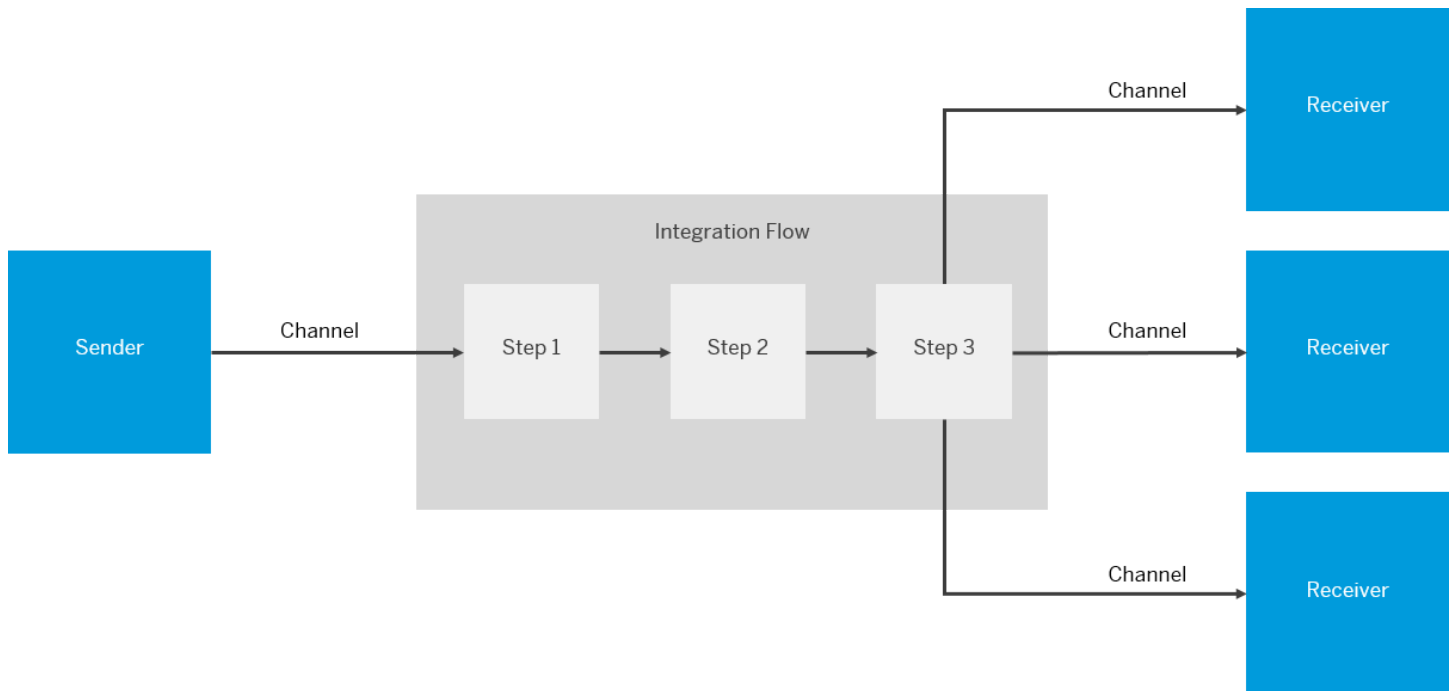
You can use integration flows to specify specific integration patterns like mapping or routing.

A graphical editor allows you, the **integration developer**, to model the message processing steps and specify in detail what happens to the message during processing.

In detail, you define the following aspects in an integration flow:

- The senders and receivers of the message
- How the senders and receivers are connected to the tenant (adapters)
- The steps that define the message processing

The following figure provides a simplified and generalized representation of an integration flow.



Senders and Receivers

You define a participant of an integration scenario as a **sender** or **receiver**. The senders and receivers typically represent the customer systems that are connected to the tenant and exchange messages with each other.

Connectivity (Adapters)

An integration flow **channel** allows you to specify which technical protocols should be used to connect a sender or a receiver to the tenant.

i Note

To specify an adapter, click the connection arrow between the sender/receiver and the Integration Process box.

You use integration flow steps to specify what should happen to a message during processing. Various step types support the wide range of **integration capabilities** of the Cloud-based integration platform.

i Note

To insert a step into an integration flow, drag and drop the desired step type from the palette on the right of the graphical modeling area.

Message Flows

You use message flows to connect various integration flow elements.

Related Information

[Configure Adapter in Communication Channels](#)

Data Storages

Cloud Integration comes with the following data storage features.

Cloud Integration Data Storages

i Note

For guidelines how to use these features during integration flow design, check out [Using Data Storage Features When Designing Integration Flows](#).

For information on the size limits of each data storages both in the Neo and Cloud Foundry environment, see [What Is SAP Cloud Integration?](#)

Physical Database	Description
Main memory (only available during the processing of an integration flow)	During the processing of an integration flow, the system can write data to and read data from the main memory. In particular, we talk about data containers that come with the Camel data model (see The Camel Data Model in a Nutshell). The data is accessible from there as long as the processing of a message lasts. After message processing for an integration flow has finished, this data isn't available anymore. Constraints: • Data only available as long as message processing lasts (there's no persistence) • Overall storage size for headers and properties restricted • No tenant storage capacity required The following data containers are available: • Exchange (where you can store properties) • Message header • Message body Cloud Integration can transfer message headers and the message body to remote components (other integration flows or connected systems). However, exchange properties exist within a dedicated data container that is only available as long as processing of a single integration flow lasts. This container is referred to as exchange. i Note Exchange properties can't be transferred across integration flows. Message headers and the message body can be shared with remote components or with other integration flows (for example, using the ProcessDirect adapter). See About Headers and Exchange Properties
Tenant database	During the processing of an integration flow, Cloud IntegrationCloud Integration tenant. The database keeps the data for a restricted amount of time. The database is associated with a subaccount of SAP BTP. You can store the following data in the tenant database: • Global/local data store content (retention period: up to 180 days; default: 30 days) • Global/local variable (retention period: 400 days)

Physical Database	Description
	• Data required at runtime when aggregating messages (retention period: 60 min (depends on configured timeout); as soon as the data has been consumed successfully, it isn't available anymore in the data store) • Message store content (retention period: 30 days) • Message processing log attachments • Partner Directory content (retention period: No limitation) i Note To be more precise from a technical perspective, when you use Cloud Integration in the Cloud Foundry environment, the following assets are stored in the SAP BTP object store: • Message processing log attachments • Message store content
JMS queues	To asynchronously decouple inbound and outbound processing, Cloud Integration can store messages in message queues and read from there (using the JMS adapter). can write data to and read data from a database that comes with the As soon as the message has been consumed successfully, it isn't available anymore in the queue. Constraints: • Limited storage capacity, but can be enhanced More information: Cloud Integration – JMS Resource and Size Limits 📄 In particular, there's a limit of 4 MB for storing headers and properties. • Store message body and header (and, optionally, properties) • Retention period: 90 days (default) can write data to and read data from a database that comes with theThe system deletes messages from the queue that weren't consumed within this period. • As soon as the message has been consumed successfully, it isn't available anymore in the queue.

Remote Data Storages

Cloud Integration also provides options to integrate various kinds of remote data storage systems:

- Remote archiving system
The monitoring application can store message processing logs (MPLs) in a remote archiving system.
- Remote database system
If you like to integrate a permanent data storage in your scenario, you can set up a remote database system and connect Cloud Integration to it using the JDBC adapter.

The Camel Data Model in a Nutshell

An integration flow is a BPMN (Business Process Model and Notation)-like model that allows you to specify how a message is to be processed on a tenant.

To interpret an integration flow model at runtime, it is transformed into an XML structure that is compatible with Apache Camel (<http://camel.apache.org> 🐘), an Open Source integration framework for Java that supports the mediation and routing of messages of any format.

The only prerequisite for a message that is to be processed by the Camel framework is that it comprises the following elements:

- Headers
Contain information related to the message, for example, information for addressing the message sender.
- Attachments
Contain optional data that is to be attached to the message.
- Body
Contains the payload (usually with the business-related data) to be transferred in the message.

For as long as a message is being processed, a data container (referred to as *Exchange*) is available. This container is used to store additional data besides the message that is to be processed. An Exchange can be seen as an abstraction of a message exchange process as it is executed by the Camel framework. An Exchange is identified uniquely by an Exchange ID. In the [Properties](#) area of the Exchange, additional data can be stored temporarily during message processing. This data is available for the runtime during the whole duration of the message exchange.

Using Camel Simple Expression Language

Cloud Integration supports a subset of the expressions defined in Camel Simple Expression Language (see <https://camel.apache.org/components/next/languages/simple-language.html> ).

You can use Camel Simple Expression Language when configuring the following components:

Components that Support Camel Simple Expression Language

Component	Use Camel Simple Expression to ...	More Information
Content modifier	Define: - Message body - Message header - Exchange property (Type parameter set to Expression)	Define Content Modifier
Write variables	Define a variable. (Type parameter set to Expression)	Define Write Variables
Router	Define a routing condition. (Expression Type parameter set to Non-XML)	Define Router
Content filter	Define conditions for XPath elements. To review some examples, see Examples .	Define Filter

Expressions Supported by Content Modifier and Write Variables Step

The following table lists the expressions from Camel Simple Expression Language that are supported by the content modifier and write variables step.

Expression	Description	Example
exchangeId	Exchange Id.	<code>\${exchangeId}</code>
id	Message Id	<code>\${id}</code>
body	Incoming message body	<code>\${body}</code>
in.body	Incoming message body i Note The expression <code>\${in.body}</code> is deprecated and may stop working in future major releases of Apache Camel. We recommend using <code>\${body}</code> to ensure future compatibility. For more information, see 3383659 	<code>\${in.body}</code>
bodyAs(<class>)	Message body converted to compatible class	<code>\${bodyAs(String)}</code>
mandatoryBodyAs(<class>)	Message body is Mandatory converted to compatible class	<code>\${mandatoryBodyAs(String)}</code>
in.header.foo	Refers to foo (forward declaration) in header	<code>\${in.header.mandatory.length}</code>
in.header.foo.OGNL	Refers to foo (forward declaration) in header with OGNL (Object-Graph Navigation Language) expression	<code>\${in.header.fname.split(',')[0]}</code>
headerAs(<key>, <class>)	Refers to header with name <key> converted to compatible class	<code>\${headerAs(name, String)}</code>

Expression	Description	Example
<code>sys.foo</code>	Refers to foo (forward declaration) in system (runtime container)	<code>\${sys.file.separator}</code>
<code>sysenv.foo</code>	Refers to foo (forward declaration) in system environment	<code>\${sysenv.property1}</code>
<code>date:<command>:<pattern></code>	Creates date specified in command and uses <code>SimpleDateFormat</code> notation	Processed at <code>\${date:now-2h}</code> Other example: To write the current timestamp into a message header, define a header with the following value (and Type set to Expression): <code>\${date:now:yyyy-MM-dd HH:mm:ss}</code>
<code>date-with-timezone:<command>:<timezone>:<pattern></code>	Same as <code>date</code> , but with timezone	Expires at <code>\${date-with-timezone:now+2h30m:IST:yyyy-MM-dd'T'HH:mm:ss.SSSZ}</code>

i Note

The content modifier does not process *Equality* or *Comparison* expressions.

Example: The following expression is not supported:

```
${header.branch} == "HR"
```

Updating your Existing Integration Flow

Some new features might not be available for your current integration flow due to the following reason:

- The feature is not available for the selected runtime profile. See [Runtime Profiles](#)
- A feature for a particular adapter or step was released after you created the corresponding shape in your integration flow. You have the following options:
 - You can use the new integration flow shape (adapter or step).

To do this, select the integration flow component and choose **Update Version** from the property sheet. See [Update Version of an Integration Flow Component](#).
 - You can continue using the older version of the integration flow shape (adapter or step).

⚠ Caution

Updating & Redeploying Your Integration Flow

To minimize disruptions of the message processing during updating or redeploying active integration flows, redeployments are postponed up to 1 min after redeployment has been triggered manually. This set-up allows Cloud Integration enough time to finish the current message processing.

If the message processing cannot be completed within 1 min, an error is thrown. In exceptional cases, the integration flow needs to be redeployed again.

Related Information

[Versioning of Artifacts](#)

Adapting Standard Integration Content to Your Requirements

When you use standard (predefined) integration content published on SAP Business Accelerator Hub, there are two different ways how to adapt such content to meet the requirements of your concrete business scenario.

- Modifying/editing the standard integration flow
- Configuring externalized integration flow parameters

Modifying/Editing the Standard Integration Flow

Using this option, you modify the integration flow in **Edit** mode and change its design, for example, by adding new integration flow steps or channels. Modifying an integration flow this way allows you also to change parameter values predefined in the integration flow model.

However, be aware of the fact that when using this option you can run into conflicts when SAP provides an update of the associated integration package. Note that as soon as you've modified a predefined integration flow, you don't get any further updates to it when SAP provides an update of the related integration package. Furthermore, if you copy an **updated** standard integration package from SAP Business Accelerator Hub to the **Design** workspace of your tenant, you overwrite all changes that you have made to that package before this action.

If you, nevertheless, like to modify standard integration content, it's recommended that you decouple your modifications from the standard content.

More information: [Keep Modifications Separate from Prepackaged Integration Content](#)

i Note

You can only modify content of a standard integration package if its mode is set to **Editable**.

There are dedicated standard integration packages that don't allow a user to modify its content (**Configure-only** mode). In such a case, you can only configure externalized parameters of the contained integration content (as described under *Configuring Externalized Integration Flow Parameters*).

Configuring Externalized Integration Flow Parameters

When you use a predefined integration flow, it's likely that certain parameters are defined as *externalized parameters*. When having copied an integration package from SAP Business Accelerator Hub to the **Design** workspace of your tenant, you have the option to configure the values of externalized integration flow parameters. Choosing this option allows you to adapt the integration flow without the need to modify the integration flow model in **Edit** mode.

Typically, the integration developer who provides the standard integration flow has externalized parameters such like, for example, address parameters of adapters. Such parameters typically can't be foreseen by the provider of the standard integration package. The reason is that the values of such parameters depend on the concrete environment and system landscape where the integration flow is finally to be deployed. To enable the integration flow to run in a dedicated environment, such parameters anyway need to be adapted by the consumer of the standard integration package.

When you have configured externalized parameters of integration flow parameters, you can still receive updates of the associated standard integration package.

More information: [Configure Externalized Parameters of an Integration Flow](#)

Versioning of Artifacts

Cloud Integration offers an easy version management capability for your integration artifacts.

You can version both the integration package and its artifacts and documents.

When editing an artifact, you can either choose **Save as Version** to save your changes. Choose **Save** to save your artifact as **Draft** version. Choose **Save as Version** to create a new version of the artifact. In this case, you can specify the version in the upcoming **Version Information** dialog. In the **Comment** section, you can add additional information specific to the artifact for later reference. This helps you determine the purpose of each version.

- The artifact version management is specific to the tenant that you are using it in. It will not work across tenants.
- When you create a new artifact, it is created with the version 1.0.0 by default.
- If you choose **Save** after editing an artifact, the artifact version will be **Draft**. You can only see the latest **Draft** version.
- If you choose **Save as Version**, by default the micro version is increased, for example, from 1.0.0 to 1.0.1. You can either confirm the proposed version number or change according to your policy.

By default, artifact versions follow the versioning convention of <major>.<minor>.<micro>. Although Cloud Integration does not prevent you from maintaining any version number, it is recommended to follow this convention. In combination with a well-defined versioning policy, this measure helps you to understand to which extent changes of the package or artifact were done.

i Note

Example

- If you add a new integration flow step to an existing integration flow, increase the micro version, for example, from version 1.0.1 to 1.0.2.
- If you enhance an integration flow with a further integration process or an exception subprocess, increase the minor version, for example, from version 1.0.1 to 1.1.0.
- If you have updated an existing integration flow within an integration package, increase the micro version of the package.
- If you have added a new integration flow to an integration package, increase the minor version of the package.

By default, you only see the latest version of the artifact in the integration package. However, the older versions of the artifact are available for you in the backend. If you want to switch to a different version of that artifact, you can do that by selecting the version and choosing the ↺ icon corresponding to the version. This feature allows you to replace integration content of the current version with the selected version.

 Example

You are currently using the 1.2.0 version of the artifact, but you want to switch to version 1.1.8. You select the version number 1.2.0, and in the [Version History](#) dialog, you choose  icon next to 1.1.8 to replace the integration content of the 1.2.0 version with 1.1.8 version. You can choose [More](#) to see comments about each version, if any.

Restrictions

The Integration Designer allows you to model specific patterns which are handled at runtime in an unexpected way.

Restrictions and Alternative Configuration Settings			
Modelled Pattern	Expected Behavior at Runtime	Actual Behavior at Runtime	Alternative Modeling Option
Integration flow step with more than one outgoing sequence flows For example, after a Message Persistence step the message is supposed to be sent to multiple receivers in parallel.	The same message is processed in parallel after the integration flow step.	The messages are delivered to the different receivers in a sequence. Hereby, the order in that the messages are delivered is randomly generated. In addition to that, the following behavior may occur: the message which results from the processing in the previous sequence flow is taken as input for the next sequence flow. i Note As an example, consider two parallel sequence flows where the first one contains an encryption step and the second one not. In that case, the receiver of the second sequence flow also gets an encrypted message (although in the second sequence flow no encryption step has been configured).	Configure only one outgoing sequence flow and parallel processing using a multicast of messages.

Comment on Database Transactions

The following step types include *transactional database processing*.

If one of the following listed steps is contained in an integration flow, the processing of the message is executed in one transaction.

- Data Store Operations step
 - Select (in case delete=true)
 - Write
 - Get (in case delete=true)
 - Delete
- Usage of Write variables
- Aggregator step
- Content Enricher

 Caution

Such steps might lead to resource shortages because long running transactions can cause node instability and impede other processes that are running in transactions.

Some of the above mentioned steps or adapters persist data in the database. In case of an error, the whole process is rolled back and the original state is being reestablished. That means, data from failed processes remain and, in case message processing fails, customers normally cannot access data about the failed processing (due to the roll-back).

In case an error is propagated back to the calling component, all data that have been written in the course of the (failed) transaction are being removed (in other words: not persisted in the database). For the calling component, an error implies, therefore, to restart the integration flow.

Transactional processing is also to be considered in scenarios that contain asynchronous decoupling. Let's assume integration flow A contains a Data Store Operation step. Integration flow B contains a Select operation on the Data Store and runs into an error. In that case, that data is preserved that has been written to

the database by integration flow A. This behavior makes sense in particular when you consider the case that integration flow B changes or deletes the data that has been stored by integration flow A. In case integration flow B fails, the original data from integration flow A can be retrieved.

Working with Integration Packages

Related Information

[Working with Prepackaged Integration Content](#)
[Configure Multiple Integration Flows](#)
[Generating Integration Content using APIs](#)
[Add Integration Packages to the Customer Workspace](#)
[Creating Custom Tags](#)
[Importing Content from SAP Process Orchestration System](#)

Working with Prepackaged Integration Content

SAP delivers rich prepackaged integration content out of the box that enables you get started quickly. These integration packages are created for some of the commonly used integration scenarios. By using this content, you can easily get started with minimal integration developer effort:

1. Choose **Discover** > **Integration** to find and explore the list of packages.
2. Choose a package based on your requirements.
 - o In the **Overview** tab of the package, you can find detailed description about the package. You also find the **Support Information** that shows the correct support component for the package. The support component helps you report incidents to the right SAP support team when you face issues with the content of the package.
 - o In the **Artifacts** tab of the package, you can find artifacts like integration flows, value mappings, message mappings, documentation, and so on.
3. Choose **Copy** to create a copy of the package to your design workspace that you can later edit, configure, and deploy. To know more about how to best work with prepackaged content, see [Use Prepackaged Integration Content Provided by SAP](#).

Note

- You can also find integration content from SAP partners in the **Discover** > **Integration** menu. You can find the vendor packages by filtering for the **Vendor** with any other available value apart from SAP.

You can't download content from integration packages by SAP partner vendors.

- You can also access prepackaged content on the SAP Business Accelerator Hub: [SAP Business Technology Platform - Packages](#).

Community Packages

Community packages are assets created and packaged by the SAP integration community.

What is Community Package

SAP has a thriving community of integration experts involved in the creation of community packages. Community packages consist of open-source and editable integration content. Licensed by Apache v2.0, these packages are created through user feedback. The integration content of these packages can be readily reused in consulting and commercial projects.

How do you consume

You can find these community packages in the **Discover** section, identified by the label **Community Package**.

Like other prepackaged packages, you can copy the community packages to the **Design** workspace. This content is also available on the GitHub. For any support, you can raise an issue with the content under the associated GitHub project.

What does it contain

Each community package contains documents associated with the integration content and the following links:

- The Apache license, Version 2.0
- Link to the package on the GitHub repository.

- [Link to the Contribute.md document](#). This document lists the guidelines to contribute and enhance the community content.

Configure Multiple Integration Flows

You use this feature to configure parameters of multiple integration flows. You can save and deploy all from a single window with minimal effort.

Context

The feature offers a mass update capability where you can configure multiple integration flows. It allows you to enter the configuration details in the mass configuration view. Using the embedded configure view you need to configure the integration flows one by one. You can save and deploy all the configured integration flows in one click and it minimizes your efforts.

With the mass configuration, you can configure all the externalized parameters of the integration flow. If there are any errors during configuration of the fields, it's notified as a check error along with detailed message. You can see an indicator referring to the respective integration flow after you close the error message window.

Benefits

- Reduction in steps and time for configuring multiple integration flows.
- Save and deploy all integration flows simultaneously.
- Validation checks help you to resolve the errors with the clear message.

Procedure

You use the following procedure to configure multiple integration flows.

1. Launch SAP Cloud Integration using the application URL.
2. Choose  **Design** .
3. Select the package that contains the integration flows that you want to configure and choose **Package content**.
4. In the **Artifacts** screen area, select one or more integration flows you want to configure.
5. Choose  **Actions**  **Configure** .
6. Select the integration flows one after the other in the mass configure view and make necessary changes.
7. If you want to save the configuration, choose **Save All**.
8. If you want to deploy the integration flows, choose **Deploy All**.

You can see a message confirming that the integration flows has been deployed.

Generating Integration Content using APIs

Use APIs to generate integration flows.

Context

i Note

This information is relevant only when you use SAP Cloud Integration in the Neo environment.

Based on specific integration requirement, you can filter and choose prepackaged APIs from the **Discover** page. These prepackaged APIs are from the SAP Business Accelerator Hub. Prepackaged API content contains different API types, and each API contains a set of resources and operations. The operations are available in the open API-specification format.

Supported APIs and Operations

APIs	Operations
OData	GET, POST, and DELETE
REST	GET, POST, PUT, and DELETE

For each API contained in an API package, you can generate an integration flow. The generated integration flow exposes an endpoint equivalent to the chosen API. Using this endpoint, you can send a request to the API exposed on a target tenant. The generated integration flow contains a sender adapter according to the chosen API and operation. For example, from an OData API with GET operation, you generate an integration flow with OData sender adapter and GET operation.

i Note

Generating integration flows is only possible for APIs that support basic authentication and DELETE, GET, POST, PUT, and GET_ID operations.

To generate an integration flow and add it to the workspace of an existing integration package, do as follows:

Procedure

1. Choose **Discover > APIs** to access the prepackaged APIs.
2. Choose an API content package.

For example, if you like to generate an integration flow for one of the SAP Cloud Integration OData APIs (on SAP Business Accelerator Hub at <https://api.sap.com/package/CloudIntegrationAPI/odata>), choose the package **SAP Cloud Integration**.

3. Choose the **Artifacts** tab.
4. Select the API for which you like to generate an integration flow.

For example, if you've chosen the SAP Cloud Integration OData API package, the following APIs are available: **Integration Content**, **Log Files**, **Message Processing Logs**, **Message Stores**, **Partner Directory**, and **Security Content**. Let's assume that you like to generate an integration flow to read all SAP Cloud Integration message processing logs from a target tenant. Consequently, choose **Message Processing Logs**.

5. Choose **Generate Integration Flow**.
6. Specify the following parameters:

Parameter	Setting
Package	Select the package where to add the generated integration flow.
Name	Specify the name of the generated integration flow.
Resource	Select the resource of the chosen API. For example, select /MessageProcessingLogs .
Operation	Select the operation. In the chosen example, only GET is available.
URL	Specify the URL of the target system that exposes the API.

7. Choose **Next**.
8. Specify the following parameters:

Parameter	Setting
Entity Set	Specify an entity set for the API, for example, /GetAllMPLs . This parameter is added to the sender adapter of the generated integration flow. In the chosen example, the generated integration flow contains an OData sender adapter, and this parameter specifies the Entity Set parameter of the sender adapter.
Receiver Address	Specify the URL under which the API can be reached on the target tenant.
Credential Name	Specify a name for a User Credentials artifact. This artifact contains the credentials that are used to authenticate the integration flow against the target tenant (only basic authentication is supported). The credential name is added to the receiver adapter of the generated integration flow.

These parameters are added to the generated integration flow as externalized parameters.

Results

You have generated an integration flow and added it to an existing integration package.

Add Integration Packages to the Customer Workspace

Prerequisites

You have logged-on to the Web application with the integration developer role.

Context

You have to add the integration packages to your customer workspace ([Design](#) tab page). This enables you to access the artifacts in that package, configure, and deploy them. There are two options to do this:

1. Copy integration package from catalog ([Discover](#) tab page) to your customer workspace.
2. Upload integration package from your local file system to your customer workspace.

You use this procedure to add integration packages to your customer workspace.

i Note

If you want to copy a package that already exists in your workspace, you can decide to overwrite the existing package or to add a suffix to the package name and create a new one. When you decide to add a suffix, the suffix is added to both the package ID and the IDs of all existing artifacts/integration flows. This way, updates for packages and artifacts with suffixes function the same way like for any other standard content. See: [Updates for SAP's Integration Packages](#).

If you want to import a package, you can decide to overwrite the existing package or to cancel the import. You don't have the option to add a suffix.

Procedure

1. If you want to copy integration package in catalog to your customer workspace, mouse-over the integration package tile and choose .
2. If you want to upload an integration package from your local file system to your customer workspace, perform the following substeps.
 - a. Choose , then [Design](#).
 - b. Choose [Import](#).
 - c. Navigate to the folder in your local file system in which you have saved the integration package.

i Note

The integration package must be a .zip file.

- d. Choose [Open](#).

You see the integration package that you uploaded in your customer workspace.

Creating Custom Tags

You can maintain custom attributes in your editable integration packages.

You can tag your integration package using custom attributes that can be created under the [Settings](#) tab. The tags can help you identify and track your packages based on attributes, for example, such as Author, Line of Business etc.

→ Remember

You must have the tenant administrator role assigned to access the [Custom Tags](#) settings. See [Tasks and Permissions](#).

1. Navigate to the [Settings](#) () in the web UI and choose [Custom Tags](#).
2. Choose [Edit](#) and select [Add](#) to add custom attributes.
3. Enter a name for the tag under [Tag Name](#).
4. By default, the [Mandatory](#) check box is enabled to enforce integration developers to add specific values before saving the package. Uncheck the check box if you don't want to set your tag mandatory.
5. In the [Permitted Values](#) field, type in the possible custom values from which you want the integration developers to choose. For example, with a custom tag name [Department](#), you can provide permitted values like [HR](#), [Finance](#), [Sales](#), and [Marketing](#). Leave the field blank, if you want the integration developers to type their own values.
6. Choose [Save](#). You can't rename the tags after saving them. The tags become read-only.

i Note

- After adding the necessary custom tags, you can inform the integration developers to maintain the custom tags in the existing packages.
- For transport scenarios, the custom tags can be exported from source tenants and imported into the target tenant before the package transport, to reflect the custom tag values maintained at source.

The integration developers can now start maintaining the attribute values under the [Tags](#) tab in the newly created packages. For existing packages without the attributes, the values can be added by editing the package or using OData API.

To know how to work with custom tags using OData API, see [Working with Custom Tags](#) and [Create New Custom Tags Configuration](#).

Saving of an edited package or a newly created package fails if the mandatory attributes aren't maintained.

i Note

Deleting a custom tag in the **Settings** that is already maintained in a package only hides the tag from the **Tags** section of the package. But the value of the tag along with the tag name is persisted in the package for easy retrieval. Read this [blog](#) to know more about custom tags and their behaviours.

Content Transport

Reuse content across multiple tenants. Export integration content from one (source) tenant and import it on another (target) tenant.

i Note

SAP Integration Suite supports the transport of various artifacts, including Integration Flows, Message Mappings, APIs, MCP Servers, and more.

Let us consider an example where you have two tenants – test and production. You design and test an integration artifact on the test landscape and it works as expected. Instead of redesigning the integration artifact on the production landscape, you can reuse the integration content that you've tested in the test landscape in the production landscape as well.

→ Remember

The procedure to set up content transport depends on whether you use Cloud Integration in the Cloud Foundry or in the Neo environment.

Content transport across two different environments isn't supported. You can't transport content between a Neo and a Cloud Foundry tenant.

There are four content transport options:

→ Remember

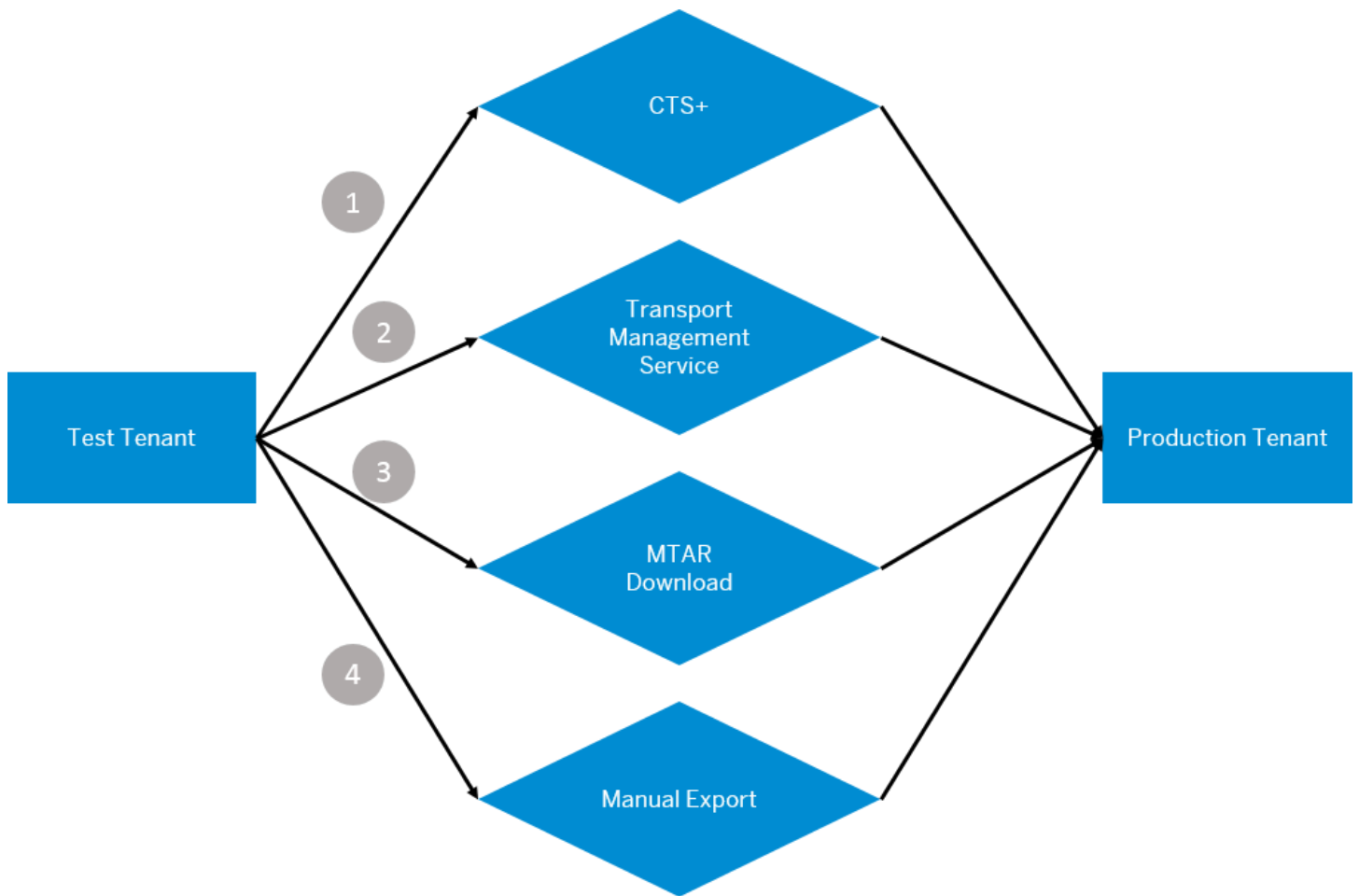
There are different criteria that help you to decide on a transport option. Before you pick your choice of transport option, see [Decision Help for Choosing the Right Content Transport Option](#).

Transport Option	Description
CTS+	With a single step, transport integration content from one tenant to another through the CTS+ system. For more information, see Content Transport Using CTS+ .
Transport Management Service	With a single step, transport integration content from one tenant to another through the Cloud Transport Management service. For more information, see Content Transport Using Cloud Transport Management .
MTAR Download	Download an MTAR file from the source tenant and manually upload the same file to a CTS+ system or Cloud Transport Management service. In the Neo environment, you can deploy an MTAR file to a target Cloud Integration tenant from the subscriber account (through Solutions). For more information, see Content Transport using MTAR Download .
Manual export and import	Manually export integration content from the source tenant and manually import this content to the target tenant. For more information, see Content Transport using Manual Export and Import .

i Note

- If the configuration or the transport results in an error, the error codes are displayed along with the error message.
- The externalized parameters/configured values of the integration artifacts aren't overwritten during a content re-transport.
- If the standard/pre-shipped integration packages are transported to the target tenant, the updates on those packages aren't available in the target tenant.

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.



Please note that image maps are not interactive in PDF outputs.

Selective Transport of Integration Artifacts

You can select one or more integration artifacts that were modified or updated, and transport them to another tenant. See blog on: [Transport at Artifact Level](#).

→ Remember

- A package is the container for an artifact. So, when you transport only a selected artifact, you only see the package in your transport request. The transported package in the target tenant contains only the artifact that you transported.
- Artifacts in the draft version can't be transported.
- A Function Libraries artifact can't be transported independently. You must transport the package that contains the Function Libraries artifact.

You must ensure that the source and target tenants are hosted on the same cloud environment. This means – you can move selected artifacts from a tenant classified for testing purpose to a production tenant only if they both are hosted on Neo or both on Multi-Cloud environment.

You can't transport those artifacts that are protected with access policies in the source tenant. The constraint is also applicable when you transport an artifact that is already present in the target tenant protected with access policies. For more information in how to manage access policies for cloud environments, see [Creating Custom Roles for Access Policies, Cloud Foundry Environment](#).

For Neo environment, see [Creating Custom Roles for Access Policies, Neo Environment](#)

Related Information

[Enabling Content Transport, Cloud Foundry Environment](#)

[Enabling Content Transport, Neo Environment](#)

[Guidelines and Best Practices for Content Transport](#)

Enabling Content Transport, Neo Environment

All the tasks mentioned here are one-time activities. It's recommended that the tenant administrator performs these tasks to make content transport possible.

i Note

This information is relevant only when you use SAP Cloud Integration in the Neo environment.

i Note

To use the automatic transport options (CTS+ or Cloud Transport Management), you need to enable an additional service – Solutions Lifecycle Management. This service is responsible for packaging the integration content objects (integration packages with their artifacts) into MTA files and handing them over to the transport mechanism. This service is always needed independently of the transport mechanism, whether CTS+ or Cloud Transport Management.

Enabling Solutions Lifecycle Management

You enable the [Solutions Lifecycle Management](#) service in the subaccounts where your source and target tenants are hosted. This step is a one-time activity. You can do that by performing the following steps:

1. Log in to your SAP BTP account.
2. Choose [Services](#) > [Solutions Lifecycle Management](#) .
3. Choose [Enable](#).

Selecting the Transport Mode

The content transport mechanism provides you the flexibility of directly exporting integration content to CTS+ system and Cloud Transport Management, or exporting the content as an MTAR file to a local file system. To use one of these transport modes, you must enable the [Transport Mode](#) found in the [Settings](#) ( *(settings)*) of your source tenant.

1. Navigate to [Tenant Settings](#) () in the Web UI.
2. Select the [Transport](#) tab and choose [Edit](#).
3. In the [Transport Mode](#) list, select one among [CTS+](#), [Transport Management Service](#), and [MTAR Download](#) based on your requirement.

→ Remember

- Note that this step needs to be performed on the tenant from which you want to export content. For example, if you're exporting content from your **Test** tenant, you need to perform this step only in the **Test** tenant.
- [Transport Settings](#) are available for you only if you have the tenant administrator role assigned to your account. Assign the role to your user if you don't see the [Settings](#)() icon. [Transport Settings](#).

Next Steps

You must create HTTP destinations to make connections between your tenant, Solutions Lifecycle Management service, and the CTS+/Cloud Transport Management systems. If you want to use:

- [MTAR Download](#) option – create an HTTP destination in Solutions Lifecycle Management.
- [CTS+](#) option – create 3 HTTPS destinations and use Cloud Connector to connect your CTS+ system with your Cloud Integration source tenant. Then, create transport nodes and routes.
- [Cloud Transport Management](#) option – create 2 HTTP destinations in Solutions Lifecycle Management and 1 HTTP destination each in your SAP BTP Neo and Cloud Foundry subaccounts. Then, create transport nodes and routes.

For detailed information about these destinations, see [Creating HTTP Destinations and Transport Route](#).

Creating HTTP Destinations and Transport Route

Create destinations, transport nodes, and transport routes to configure the content transport landscape.

Prerequisites

- Enable the **Solutions Lifecycle Management** (SLM) service. For information, see [Enabling Content Transport, Neo Environment](#).
- To use CTS+ as your transport option, complete the following steps:
 - Set up CTS+ for the transport purpose. See [Activating and configuring CTS Export Web Service](#).
 - Install and configure Cloud Connector to connect your on-premise system with SAP Cloud Integration. See [Cloud Connector](#).
- To use Cloud Transport Management as your transport option, complete the initial setup. See [Cloud Transport Management Setup](#).

Context

i Note

This information is relevant only when you use SAP Cloud Integration in the Neo environment.

HTTP Destinations and Transport Route

The steps involved in creating HTTP destinations and transport route are different for different transport modes. Choose a transport mode (box) to view the detailed procedure for creating destinations and transport routes.

The information within a transport mode is independent of other modes.



Please note that image maps are not interactive in PDF outputs.

Enabling Content Transport, Cloud Foundry Environment

All the tasks mentioned here are one-time activities. The tenant administrator performs these tasks to enable content transport.

i Note

To use the automatic transport options (CTS+ or Cloud Transport Management), you must enable an additional service – [Content Agent Service](#). This service is responsible for packaging the integration content objects (integration packages with their artifacts) into MTAR files and handing them over to the transport mechanism. This service is always needed independently of the transport mechanism, whether CTS+ or.

i Note

This information is relevant only when you use SAP Cloud Integration in the Cloud Foundry environment.

Enabling Content Transport involves the following major steps:

Subscribing to Content Agent Service

You can follow one of the approaches to use the Content Agent Service:

- Create a service instance of Content Agent Service – using which you trigger the transport from Cloud Integration but you must create an additional destination in SAP BTP cockpit. See [Create Instance](#) and [Create Service Key](#).
- Create a subscription of Content Agent Service – using which you trigger the transport from Content Agent Service web UI (instead of Cloud Integration UI) while there's one less destination to be created. See [Subscribe to Content Agent Service](#).

i Note

The Content Agent Service UI method works only if you use Cloud Transport Management as the transport mechanism.

Irrespective of what choice you make, this action must be done in the SAP BTP subaccount where the source Cloud Integration tenant is hosted.

Subscribing to Process Integration Runtime

Follow the instructions mentioned in [Creating Service Instance and Service Key for Inbound Authentication](#) for the service plan `api`.

When prompted to select a role for the service plan, choose the role `WorkspacePackagesTransport`.

Selecting the Transport Mode

The content transport mechanism provides you the flexibility of directly exporting integration content to CTS+ and Cloud Transport Management system, or exporting the content as an MTAR file to a local file system. To use either of the modes, you must enable [Transport Mode](#) found in [Settings](#).

Navigate to [Tenant Settings](#) and move to the [Transport](#) tab. Based on your requirement, select either [CTS+](#), or [Transport Management Service](#), or [MTAR Download](#) from the dropdown list.

→ Remember

- This step must be performed on the tenant from which you want to export content. For example, if you're exporting content from your **Test** tenant, you need to perform this step only in the **Test** tenant.
- You must have the administrator role assigned for Cloud Integration to access the **Transport Settings**. Else reach out to your administrator to enable transport.

Next Steps

You must create HTTP destinations to ease connections between your source and target tenants to transport integration packages and artifacts. See [Creating HTTP Destinations and Transport Route](#).

Creating HTTP Destinations and Transport Route

Create destinations, transport nodes, and transport route to configure the content transport landscape.

Prerequisites

- Complete the tasks mentioned in [Enabling Content Transport, Cloud Foundry Environment](#).
- To use Cloud Transport Management as your transport option, complete the initial setup. See [Cloud Transport Management Setup](#).

Context

i Note

This information is relevant only when you use SAP Cloud Integration in the Cloud Foundry environment.

Destinations are used to enable content transport from a source tenant to the target tenant.

Creating HTTP Destinations for Cloud Transport Management

Create four HTTP destinations in the SAP BTP subaccount where your source Cloud Integration tenant is hosted. Later, create transport nodes and route in Cloud Transport Management .

1. Create a destination for Content Agent Service. See [Create SAP Content Agent Service Destination](#).

i Note

Skip creating this destination if you want to use the Content Agent Service's web UI to trigger the transport.

2. Create a destination for source Cloud Integration tenant. See [Create SAP Cloud Integration Destination](#).
3. Create a destination for Cloud Transport Management . See [Create Cloud Transport Management Destination](#).
4. In Cloud Transport Management service, create the source and target transport nodes. The name for the source node must be the same as the property you defined in **sourceSystemId** of the destination created for Cloud Transport Management service in the previous step. See: [Create Transport Nodes](#).
5. In Cloud Transport Management service, create a transport route using the preceding nodes. See: [Create Transport Routes](#).
6. Create a destination for Cloud Transport Management for the target Cloud Integration tenant. You use this destination in Cloud Transport Management to configure your transport landscape. See [Create Transport Destinations](#).

Content Transport Using CTS+

Prerequisites

i Note

This information is relevant only when you use SAP Cloud Integration in the Cloud Foundry environment.

- You've set the **Transport Mode** as **CTS+** in the tenant **Transport** settings. For more information, see [Enabling Content Transport, Cloud Foundry Environment](#).
- You've created two HTTP destinations on **Solutions Lifecycle Management**. One for the tenant management node. For more information, see [Creating HTTP Destinations and Transport Route](#). The other one is for CTS+ system via cloud connector. For more information, see [Integration with Transport Management Tools](#).

- You've the role [WorkspacePackagesTransport](#) for CF or [IntegrationContent.Transport](#) for Neo assigned.

Context

SAP Cloud Integration provides an option to transport integration content directly to [CTS+](#) system. You can then transport this content from the CTS+ system to your target SAP Cloud Integration tenant. Here's how you can transport content to CTS+.

Procedure

1. Select the integration package that you want to transport.
2. Choose [Transport](#).

If you don't see the [Transport](#) button, contact your tenant administrator to enable transport option in the tenant settings. For more information on the roles, see [Configuring User Access to Cloud Integration](#) and [Tasks and Permissions](#).

3. In the [Transport Comments](#) prompt, you can see the type of transport under the [Mode](#) field configured by the tenant administrator. Provide comments under the [Comments](#) section and choose [Transport](#).

→ Remember

The field has a limit of 512 characters. You can use alphabets, numbers, whitespaces, and the following characters: ' - . _ ~ : \ / ? # [] @ ! \$ & () * + , ; = % ' in your comment.

You can use only English text in a single line. You cannot create a new line in the comment.

You see a prompt with the [Transport ID](#). The integration package is transported to the CTS+ system.

4. Import the content in the target system. For more information, see [Importing transport requests](#).

Content Transport Using Cloud Transport Management

Prerequisites

- You've set the [Transport Mode](#) as [Transport Management Service](#) in the tenant [Transport](#) settings. For more information, see [Enabling Content Transport, Cloud Foundry Environment](#).
- You've created necessary HTTP destinations and transport routes. For more information, see [Creating HTTP Destinations and Transport Route](#).
- You've the role [WorkspacePackagesTransport](#) for CF or [IntegrationContent.Transport](#) for Neo assigned.

Context

Cloud Integration provides an option to transport integration content to Cloud Transport Management system. You can then transport this content from the Cloud Transport Management system to your target Cloud Integration tenant.

Here's how you can transport content to Cloud Transport Management:

Procedure

1. Select the integration package or artifact that you want to transport.

If you want to use the Content Agent Service's web UI to trigger the transport, see: [Export Content Wizard](#).

2. Choose [Transport](#).

If you don't see the [Transport](#) button, contact your tenant administrator to enable transport option in the tenant settings. For more information on the roles, see [Configuring User Access to Cloud Integration](#) and [Tasks and Permissions](#).

3. In the [Transport Comments](#) prompt, you can see the type of transport under the [Mode](#) field configured by the tenant administrator. Provide comments under the [Comments](#) field and choose [Transport](#).

→ Remember

The field has a limit of 512 characters. You can use alphabets, numbers, whitespaces, and the following characters: ' - . _ ~ : \ / ? # [] @ ! \$ & () * + , ; = % ' in your comment.

You can use only English text in a single line. You cannot create a new line in the comment.

If you use Cloud Integration on SAP BTP Neo Environment, you aren't allowed to use the character " _ " in the [Transport Comments](#) field.

You see a prompt with the [Transport ID](#). The content gets transported to the Cloud Transport Management system.

4. Enable the [Propagate logged-in user as transport owner](#) checkbox to send the logged-in user's ID to Cloud Transport Management system so that you can easily identify the artifacts triggered for transport by a particular user. The user ID depends on the attributes configured in the Identity Provider system.
5. To import the content in the target system, follow the steps mentioned in the [official documentation](#) for Cloud Transport Management service.

Content Transport using MTAR Download

Prerequisites

- You have selected **MTAR Download** as your **Transport Mode**. For more information, see [Enabling Content Transport, Cloud Foundry Environment](#).
- You've the role **WorkspacePackagesTransport** for CF or **IntegrationContent.Transport** for Neo assigned.

Context

You can use the **MTAR Download** option to download a MTAR file/s of the integration content to be transported

Procedure

- Select the integration package that you want to transport.
- Choose **Transport**.

If you don't see the **Transport** button, contact your tenant administrator to enable transport option in the tenant settings. For more information on the roles, see [Configuring User Access to Cloud Integration](#) and [Tasks and Permissions](#).

- In the **Transport Comments** prompt, you can see the type of transport under the **Mode** field configured by the tenant administrator. Provide comments under the **Comments** section and choose **Transport**.

→ Remember

The field has a limit of 512 characters. You can use alphabets, numbers, whitespaces, and the following characters: ' - . _ ~ : \ / ? # [] @ ! \$ & () * + , ; = % ' in your comment.

You can use only English text in a single line. You cannot create a new line in the comment.

A file with extension **mtar** is downloaded to your local file system in the download path configured in your browser.

- To import the content in the target system, follow the steps mentioned [here](#).

Content Transport using Manual Export and Import

Prerequisites

You have logged in to SAP Cloud Integration on source and target tenants. Access the  (**Design**) tab (workspace).

Context

One of the options to transport content from one tenant to another is to use the **Export** and **Import** options for your integration package. The application imports the integration package to your local file system in the form of a **.zip** file. You can import the same file in the target tenant using the **Import** option.

Procedure

- Select the integration package that you want to export.
- Choose **Export**.
- A **.zip** file is downloaded to the default browser download location on your local file system.
- Log in to the SAP Cloud Integration web application to which you want to import the content. Choose .
- Choose **Import**.

A new window opens in your file system explorer, allowing you to access your local file system.

- Navigate to the folder path where the **.zip** file was downloaded in step 3.
- Select the file and choose **Open**.

You see a prompt indicating the successful import of the **.zip** file. You can see the imported integration package in the **Design** tab of your target tenant.

Handling Integration Artifacts across Multiple Tenants

SAP Cloud Integration allows you to transport integration packages from a source tenant to a target tenant in order to re-use the content of the integration package (integration flows, APIs, MCP servers, value mappings, OData APIs) on the target tenant.

However, having transported an integration package to a target tenant does not mean that the contained integration artifacts run without any further actions.

For example, you need to make sure that security material and keystore content that is used/referenced by the integration flow or within the API is as well available on the target tenant. If that is not the case, the integration artifact will fail.

This topic provides an overview of how to handle the different kinds of artifacts to make sure that integration content can be executed without errors also on the target tenant.

Web UI Section	Artifact Type	Required Actions to After Content Transport
Design view	Integration flow, APIs, MCP server, value mapping, OData API	Can be transported as part of an integration package (with one of the chosen transport options such like manual export/import, usage of SAP CTS+ or usage of the SAP BTP Transport Service). More information: Content Transport
Design view	Integration flow resources (schemas, mappings)	Are transported along with the integration flow.
Monitor view/Keystore	Public key or certificate	Download public part of key pair (certificate, certificate chain, root certificate, signing request) from source tenant and import to target tenant. More information: Managing Keystore Entries
Monitor view/Keystore	Private/public key pair	You need to newly create the key pair and upload it to the keystore of the target tenant.. More information: Creating a Key Pair/SSH Key Pair
Monitor view/Security Material	User Credentials artifact	Cannot be reused; newly create artifact on target tenant (and use same artifact name in order not to break any references to the artifact). More information: Deploying a User Credentials Artifact
Monitor view/Security Material	Secure Parameter artifact	Cannot be reused; newly create artifact on target tenant (and use same artifact name in order not to break any references to the artifact). More information: Deploying a Secure Parameter Artifact
Monitor view/Security Material	OAuth2 Credentials	Cannot be reused; newly create artifact on target tenant (and use same artifact name in order not to break any references to the artifact). More information: Deploying an OAuth2 Client Credentials Artifact
Monitor view/Security Material	Known Hosts	Newly upload the known hosts file to the target tenant. More information: Deploying an SSH Known Hosts Artifact
Monitor view/PGP Keys	PGP Public Key	Newly upload the key file to the target tenant. More information: Deploying a PGP Public Keyring
Monitor view/PGP Keys	PGP Secret Key	Newly upload the key file to the target tenant. More information: Deploying a PGP Secret Keyring
Monitor view/Access Policies	Access Policy	Newly create access policy on target tenant. More information: Managing Access Policies
Monitor view/Certificate-to-User Mappings (Only relevant in the Neo environment)	Certificate-to-User Mapping	Newly create certificate-to-user mapping on target tenant. You can reuse an existing certificate (by manual upload from your computer). Make sure that the role specified in the related integration flow (sender adapter) is defined on the target tenant and assigned to the user. More information: Managing Certificate-to-User Mappings, Neo Environment
Monitor view/JDBC Data Sources	Operations view/JDBC Data Source	Cannot be reused; newly create artifact on target tenant (and use same artifact name in order not to break any references to the artifact). More information: Managing JDBC Data Sources

Additional Content

Artifact T	More information:
Partner Directory content	You need to migrate the Partner Directory content using the OData API. More information: Parameterizing Integration Flows Using the Partner Directory

When you plan to automate certain manual steps described in this topic, you can use the SAP Cloud Integration OData API.

For example, you can facilitate steps like:

- Reading a security artifact from the source tenant and adding it to the target tenant
- Exporting a certificate from the source tenant and importing it to the target tenant
- Generating a key pair
- Creating a certificate-to-user mapping

Guidelines and Best Practices for Content Transport

Cloud Integration offers different options to transport integration content. There are certain criteria that help you to decide which option to choose for your integration project.

To find out more about the different content transport options, see [Content Transport](#).

The mentioned options allow you to transport always complete integration packages and their content. Integration artifacts transported with an integration package are:

- Integration flows
- APIs
- MCP Servers
- Value mappings
- OData APIs
- REST APIs
- SOAP APIs

i Note

However, additional artifacts are required to use integration content on a tenant that cannot be transported along with integration content.

For example, to run an integration flow or API that uses authentication to a receiver system based on OAuth2 client credentials grant, you also need to deploy an **OAuth2 Client Credentials** artifact on the tenant and refer to the artifact alias in the integration flow or API.

After you have transported an integration flow or API using this authentication type, you need to separately deploy a new **OAuth2 Client Credentials** artifact on the target tenant (ideally with the same alias as used on the source tenant).

Other examples for non-transportable content are keystore entries (key pairs and certificates).

For more information on objects that cannot be transported and how to deal with them in a transport scenario, see [Handling Integration Artifacts across Multiple Tenants](#).

Related Information

[Decision Help for Choosing the Right Content Transport Option](#)

[Decoupling Development from Transport Using Externalized Parameters](#)

[Guidelines on Role Assignments](#)

[Content Transport - Sequence of Steps \(Example\)](#)

Decision Help for Choosing the Right Content Transport Option

There are different criteria that help you to decide on a transport option.

The following table lists the characteristics and advantages and disadvantages of the different transport options.

Transport Option	Recommendations
Manual export/import	Use this option only in exceptional cases, for example for urgent fixes in case the transport system is down. This option is easy-to-use out-of-the-box (no configuration required). However, there is no control and logging of the transport events and the involved users. You have the risk of losing control of who is transporting what and when. Using this option, you don't have any advantage of the tool support provided by a transport system, like, for example, logging. For this reason, manual exports and imports should be avoided in an integration project or while operating SAP Cloud Integration. It could be used.

Transport Option	Recommendations
CTS+	This is a recommended option. This option provides a fully-automated transport mechanism (allows the assignment of roles, definition of transport routes, approvals, and comes with additional useful features like logging). If an on-premise CTS+ is already installed and used for other systems or objects, it is an option of choice to use CTS+ also for integration content transport. i Note CTS+ is available for many years to transport on-premise non-ABAP content. This concept has been extended some time ago to support SAP BTP. However, only cloud content that can be packaged in MTAR files can be transported. For integration content transport, this constraint is fulfilled. Therefore, if you use MTAR-based content only in SAP BTP, CTS+ is a valid choice.
Transport Management Service (TMS)	This is a recommended option. This option provides a fully automated transport mechanism (allows the assignment of roles, definition of transport routes, approvals, and comes with additional useful features like logging). If there is no on-premise CTS+ available in your landscape, a good option is to go for TMS. TMS is the solution for transporting content in SAP BTP. Compared to using CTS+, this option initially implies a smaller investment in implementation and operation. Note that TMS is a cloud service (subscription-based and maintenance and updates by SAP). TMS is therefore the way to go for customers planning to implement projects using SAP BTP services.
MTAR Download	This is a semi-automated option (in between manual export/import on the one and using CTS+ or TMS on the other hand). This option allows you to download an integration package manually from a source tenant as an .mtar file. In a subsequent step, you can upload this file to the transport system and, finally, transport it to the target tenant. Using this option, you benefit of having certain advantages of a transport system. However, there is still a manual step. As manual steps should be avoided as far as possible, it is recommended to use this option only in exceptional cases, for example for urgent fixes in case the connection between source tenant and transport system is down.

Both options, CTS+ and TMS are recommended as summarized in the table. In the long run, a good practice is to perform all cloud-related transports via TMS and the on-premise transport with CTS+. For more information and a comparison between TMS and CTS+, see [Interplay of Transport Management, CTS+ and ChaRM in hybrid landscapes](#).

The following table provides a further decision help along certain typical requirements that you encounter in an integration project.

Criteria	Requirement	Recommended Transport Option
Lifecycle phase	Exploring Cloud Integration	Manual Export/Import
Lifecycle phase	Operating Cloud Integration	Automated transport (TMS/CTS+)
Exceptional Case: Urgent Intervention	Transport system down	Manual Export/Import
Exceptional Case: Urgent Intervention	Connection to transport system down	MTAR Download
Facts given by your current landscape	CTS+ already implemented and used (benefit from available infrastructure)	CTS+
Facts given by your current landscape	CTS+ already implemented and used	TMS
Facts given by your current landscape	CTS+ not implemented	TMS
Strategy	On-premise	CTS+
Strategy	Cloud	TMS

Decoupling Development from Transport Using Externalized Parameters

Consider decoupling development from transport when designing integration content.

When designing integration content, there are certain configurations that depend on the landscape where the content is deployed.

Let us assume that you design an integration flow that calls an external system (with OAuth authentication). Let us furthermore assume that you design the integration flow first on a development tenant and, in a subsequent step, transport it to a production tenant.

In such a case, you can assume that the URL used to address the external system and the OAuth credentials differ in the development and the production environment.

If you specify the URL of the external system for the development environment in the related receiver adapter, the integration scenario doesn't work anymore after you've transported it to the production tenant.

To overcome this issue, you or another integration developer need to edit the integration flow on the production tenant after content transport and deploy it again. Such an activity leads to inconsistencies in the transport mechanism. Furthermore, this activity also decreases productivity, as additional steps are needed before deployment.

To avoid this behavior, the best practice is to use externalized parameters. If you've designed your integration flows using externalized parameters, you benefit from the following advantages:

- After you've transported the integration flow and before deploying it on the target tenant, you only need to configure the externalized parameters. You don't have to do any changes in the integration flow model at all.
- Configuring the externalized parameters after transport is usually a one-time activity. After subsequent transports, you don't need to configure these parameters again if the values remain the same (as the initially configured values aren't overwritten).

Related Information

[Externalize Parameters of an Integration Flow](#)

[Externalize Volatile Configurations](#)

Guidelines on Role Assignments

The combined usage of the following role collections, respectively, authorization groups (as described under [Personas](#)) gives you full control of the tenant in terms of design, monitoring, and administration.

Assigning these role collections/authorization groups to a user is a good choice for the source tenant (development system). However, assigning these role collections/authorization groups to a user on target tenants can lead to undesired behavior and inconsistencies with the transport system. To avoid that a user gets too many permissions in the target system, it's necessary to define permissions on a more detailed level on the target tenant (test and production).

Required on Target Tenant	Avoid on Target Tenant	Description
	AuthGroup.IntegrationDeveloper (Neo) PI_Integration_Developer (Cloud Foundry)	Allows user to connect to Cloud Integration and to display, download, and deploy artifacts.
	AuthGroup.BusinessExpert (Neo) PI_Business_Expert (Cloud Foundry)	Allows user to perform business tasks like, for example, examining the payload, monitor integration flows, status of integration artifacts.
	AuthGroup.Administrator (Neo) PI_Administrator (Cloud Foundry)	Allows user to connect to Cloud Integration and to perform administrative tasks.
	WebToolingWorkspace.Write (Neo) WorkspacePackagesEdit (Cloud Foundry)	Allows user to update a package, create/edit/import/delete a package and its artifacts. → Tip Updating, creating, editing, importing, or deleting a package or its artifacts on a target tenant isn't recommended. Changes made on the target tenant are overridden when the package is retransported. In addition, objects not properly saved after editing lead to errors while transporting. For that reason, those roles must not be assigned to a user on the target tenant.
	TransportModule.write (Neo) WorkspacePackagesTransport (Cloud Foundry)	Allows user to export, import, and update packages. → Tip Export for transport and import or update a package from transport must be also restricted on the target tenant (as the transport works with CTS+ or TMS as recommended options). However, still the export, import, and update of packages can be available just in source tenant with these roles.
WebToolingWorkspace.Read (Neo) WorkspacePackagesRead (Cloud Foundry)		Allows user to view packages and artifacts. → Tip On the target tenant, it's still needed to access and view the transported packages and artifacts.

Required on Target Tenant	Avoid on Target Tenant	Description
WebTooling.IntegrationFlowConfigure (Neo) WorkspacePackagesConfigure (Cloud Foundry)		Allows user to configure artifacts (integration flows and value mappings). → Tip After transporting a package from source tenant to target tenant, artifacts (integration flows and value mappings) must be configured on the target tenant.
NodeManager.read, GenerationAndBuild.generationandbuildcontent, NodeManager.deploycontent (Neo) WorkspaceArtifactsDeploy (Cloud Foundry)		Allows user to deploy artifacts. → Tip After transport to target tenant and configuring the artifacts, artifacts must be deployed.
	WebToolingCatalog.OverviewRead, WebToolingCatalog.DetailsRead, WebToolingWorkspace.Write (Neo) CatalogPackagesRead, CatalogPackageArtifactsRead, CatalogPackagesCopy (Cloud Foundry)	Allows user to access SAP Business Accelerator Hub (Discover section). → Tip Standard packages are published on SAP Business Accelerator Hub. A good practice is to subscribe only a single tenant to the SAP Business Accelerator Hub (see Subscribe Only a Single Tenant to SAP Business Accelerator Hub). If you apply this rule, the standard packages are copied only to the workspace of the source tenant and transported to the target tenant. If there's an update for those packages, then this update can only be applied to the source tenant. Standard packages that are transported to a target tenant do not get updates on that target tenant. To have the updates applied, the package with the update applied must be transported from source to target tenant. To implement this behavior, you can restrict the access to the SAP Business Accelerator Hub on the target tenant assigning these roles just to the source tenant.
All roles related to the Monitor area (see Tasks and Permissions)		Allows user to perform tasks associated with monitoring and operations. → Tip There are objects that can't be transported using the transport system. In addition, monitoring- and operations-related tasks must be allowed on the target tenant.
	WebToolingSettingsProductProfiles.savetenantconfiguration	Allows user configure the transport option. → Tip The selection of what transport mechanism to use is done just on the source tenant.

For more information on the individual roles and tasks, see [Tasks and Permissions](#).

Certain roles must be assigned also in the transport system to determine the responsibilities of who can do what (for example, import a transport order into a queue, release an order).

For more information on roles in CTS+, see [CTS Roles and Authorizations](#).

For more information on roles in TMS, see [Security](#).

If you have implemented complex transport routes, it's a good practice to assign attributes to certain roles in TMS to restrict the corresponding authorizations to specific transport nodes only. For example, one person is responsible for releasing one order to test environment while a second one is responsible for production. You can find more information in section *Transport Node-Specific Attributes* under [Security](#).

Content Transport - Sequence of Steps (Example)

There are the following prerequisites:

- The tenant administrator of the source system has configured the transport mode (SAP Cloud Integration under [Settings](#)).
- The integration developer has granted the required transport permissions (see [Guidelines on Role Assignments](#)).

The following diagram shows the typical sequence of steps for transporting integration content, based on the assumption that the content is first transported from a development tenant to a test tenant, and then to a production tenant:



1. The integration developer initiates transport of the desired integration package by creating a transport order in the transport system.
2. The transport responsible logs in to the transport system and releases the request in the test node queue.
3. Once the package is imported on the test tenant, the integration developer configures the integration artifacts (integration flows, APIs or MCP servers) and deploys them on the test tenant. If necessary, the integration developer also configures and deploys the associated security artifacts.
4. If the request is to go to production, the transport responsible forwards transport to production node queue.
5. Either manually or by means of a job, the request is imported in the production node.
6. The integration developer configures the integration artifacts (integration flows, APIs or MCP servers) and deploys them (on the prod tenant). If necessary, the integration developer also configures and deploys the associated security artifacts.

Creating an Integration Flow

Add an integration flow to an integration package.

Prerequisites

You've created an integration package. For more information, see [Creating an Integration Package](#).

Context

Create integration flows in your package either by creating or uploading.

Add Integrations Manually

Create Integration Flow

Procedure

1. Choose [Design](#) to view the list of integration packages.
2. Select the integration package in which you want to add an integration flow and choose [Edit](#).
3. Navigate to the [Artifacts](#) tab and choose [Add](#) [Integration Flow](#).
4. In the [Add Integration Flow](#) dialog, choose the method [Add Integrations manually](#).
5. If you want to create a new integration flow, execute the following substeps:
 - a. Choose the [Create](#) radio button.
 - b. Enter a valid name for the integration flow in the [Name](#) field. The [ID](#) field gets auto-filled.

For a new integration flow you can define two attributes, [Name](#) and [ID](#).

i Note

- The integration flow [ID](#) needs to be unique across the tenant, but you can specify the same [Name](#) multiple times. When you create a new integration flow, by default the entry for [ID](#) is generated automatically based on the string entered for [Name](#) (with underscore replacing space characters).
- However, you can also manually edit the [ID](#) field and, once it's modified manually, changing the name of the integration flow doesn't make any auto-changes to the [ID](#) field again.

c. You can select the runtime profile for the integration flow in the [Runtime Profile](#) field. The integration flow templates used during creation adhere to the latest version of a component available in the runtime profile. For more information, see [Runtime Profiles](#).

d. Enter [Description](#) for the integration flow.

e. Choose  *search values* to add **Sender** and **Receiver** systems. The search values provide a list of sender and receiver systems maintained by SAP.

→ Tip

If you don't find a sender or receiver system of your choice, or want to add your own values, type in the **Sender** and **Receiver** fields and press  button in your keyboard. You can also add multiple values if it's necessary.

f. Choose **Add** to just create an integration flow or **Add and Open in Editor** to open the integration flow in its editor. Alternatively, you can press the   key to create an integration flow.

6. If you want to upload an integration flow, execute the following substeps:

a. Choose the **Upload** radio button.

b. Choose **Browse** to select the integration project archive from your local file system. When you select the archive file, the **Name** and **ID** fields in the dialog box are filled automatically.

i Note

- The integration flow **ID** needs to be unique across the tenant, but you can specify the same **Name** multiple times. When you create a new integration flow, by default the entry for **ID** is generated automatically based on the string entered for **Name** (with underscore replacing space characters).
- However, you can also manually edit the **ID** field and, once it's modified manually, changing the name of the integration flow doesn't make any auto-changes to the **ID** field again.

c. Choose **Add** to just create an integration flow or **Add and Open in Editor** to open the integration flow in its editor. Alternatively, you can press the   key to create an integration flow.

Results

An integration flow with the following shapes is created:

- **Sender**: Represents your sender system.
- **Receiver**: Represents the receiver system.
- **Integration Process**: The shape where you add all the steps that define how a message is processed in the tenant.

Upload Integration Flow

Procedure

1. Choose **Design** to view the list of integration packages.
2. Select the integration package in which you want to add an integration flow and choose **Edit**.
3. Navigate to the **Artifacts** tab and choose  **Add** .
4. In the **Add Integration Flow** dialog, choose the method **Add Integrations manually**.
5. To upload an integration flow, execute the following substeps:

a. Choose the **Upload** radio button.

b. Choose **Browse** to select the integration project archive from your local file system. When you select the archive file, the **Name** and **ID** fields in the dialog box are filled automatically.

i Note

- The integration flow **ID** needs to be unique across the tenant, but you can specify the same **Name** multiple times. When you create a new integration flow, by default the entry for **ID** is generated automatically based on the string entered for **Name** (with underscore replacing space characters).
- However, you can also manually edit the **ID** field and, once it's modified manually, changing the name of the integration flow doesn't make any auto-changes to the **ID** field again.

c. Choose **Add** to just create an integration flow or **Add and Open in Editor** to open the integration flow in its editor. Alternatively, you can press the   key to create an integration flow.

Results

An integration flow with the following shapes is created:

- **Sender**: Represents your sender system.
- **Receiver**: Represents the receiver system.

- **Integration Process:** The shape where you add all the steps that define how a message is processed in the tenant.

Related Information

[Configure Integration Flow Components](#)

Using Flow Step Recommendation

The flow step recommendation in an integration flow development helps you to easily add flow steps. You get the recommendations based on prepackaged SAP integration content.

Prerequisites

- You've created an integration package. For more information, see [Create an Integration Package](#).
- You've created an integration flow. For more information, see [Creating an Integration Flow](#).

Context

With flow step recommendation, there's no need to find the next flow steps from the palette. You get a dialog with a list of flow steps in the editor itself where you need them. You get a suggestion for next possible flow step at the time of integration scenario development. For example, when you're adding a content modifier in your integration flow, it comes with a flow step recommendation that helps you select the next flow step without navigating to the palette. The selected flow step automatically gets added to the sequence connector.

You see the following sections in the flow step recommendation dialog:

- **Recommended Steps:** From an existing flow step or sequence connector, you get recommendations for the next step. The recommendations are made by a machine learning study on the prepackaged SAP integration content.
- **All Steps:** The dialog also provides a list of all flow steps where you can either scroll or search for the required flow step.

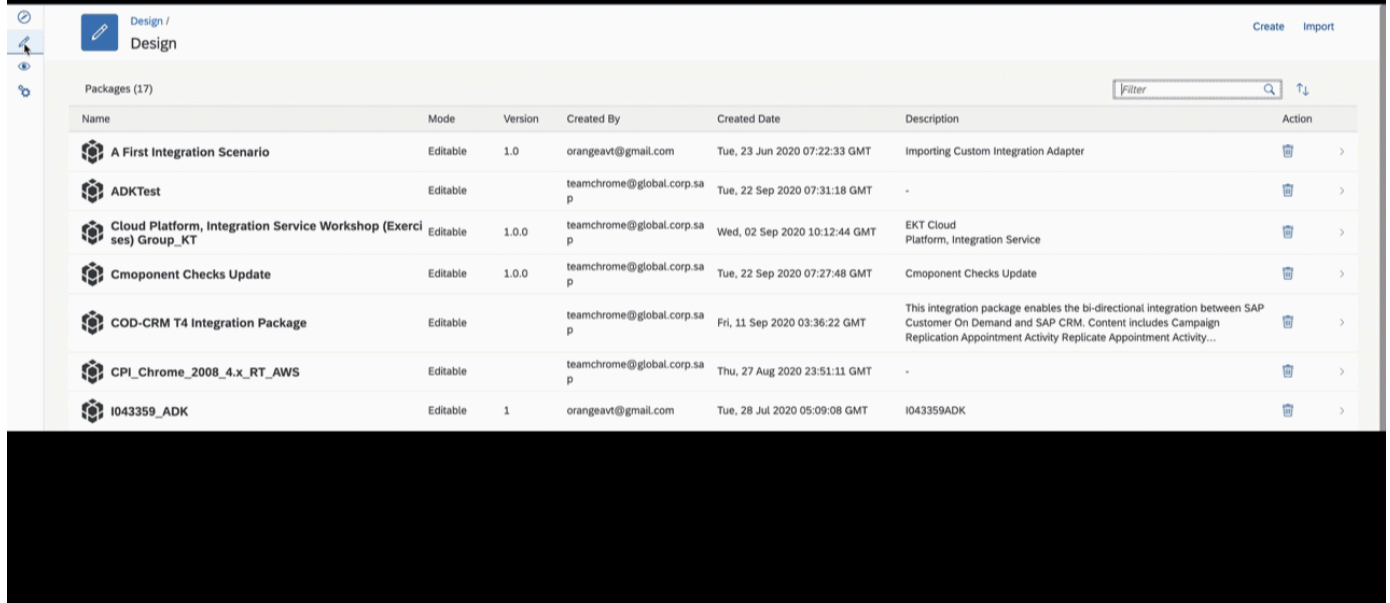
Procedure

Here's how you can use flow step recommendation for adding flow step in an integration flow.

1. Choose  (**Design**) tab to access your workspace.
2. Choose  **Integration Package**  **Artifacts**  **Integration Flow**  **Edit** .
3. Select an existing flow step or a sequence connector.
4. Choose  (**Add Flow Step**) button.

A dialog opens up showing the recommended steps first and then all the steps.

5. Select a step that is required for your integration flow.



The screenshot shows the SAP Design workspace interface. At the top, there's a header with 'Design / Design' and buttons for 'Create' and 'Import'. Below the header, a sidebar on the left shows navigation icons. The main area displays a table of 'Packages (17)' with a search filter. The table has columns for Name, Mode, Version, Created By, Created Date, Description, and Action. The packages listed are:

Name	Mode	Version	Created By	Created Date	Description	Action
A First Integration Scenario	Editable	1.0	orangeavt@gmail.com	Tue, 23 Jun 2020 07:22:33 GMT	Importing Custom Integration Adapter	[Trash] >
ADKTest	Editable		teamchrome@global.corp.sap	Tue, 22 Sep 2020 07:31:18 GMT	-	[Trash] >
Cloud Platform, Integration Service Workshop (Exercises) Group_KT	Editable	1.0.0	teamchrome@global.corp.sap	Wed, 02 Sep 2020 10:12:44 GMT	EKT Cloud Platform, Integration Service	[Trash] >
Cmpoent Checks Update	Editable	1.0.0	teamchrome@global.corp.sap	Tue, 22 Sep 2020 07:27:48 GMT	Cmpoent Checks Update	[Trash] >
COD-CRM T4 Integration Package	Editable		teamchrome@global.corp.sap	Fri, 11 Sep 2020 03:36:22 GMT	This integration package enables the bi-directional integration between SAP Customer On Demand and SAP CRM. Content includes Campaign Replication Appointment Activity Replicate Appointment Activity...	[Trash] >
CPI_Chrome_2008_4.x_RT_AWS	Editable		teamchrome@global.corp.sap	Thu, 27 Aug 2020 23:51:11 GMT	-	[Trash] >
I043359_ADK	Editable	1	orangeavt@gmail.com	Tue, 28 Jul 2020 05:09:08 GMT	I043359ADK	[Trash] >

Integration Flow Extension

SAP Cloud Integration allows you to extend the capabilities of standard integration content provided by SAP. This approach allows you to implement specific integration scenarios relevant to your business use case without changing the content provided by SAP.

SAP (or SAP partners) provide prepackaged integration content (also referred to as *standard* content) that covers a large variety of integration requirements. This content usually comprises a set of integration flows, value mappings, and other integration artifacts that cover a standard integration use case. If you want to adapt these capabilities to specific business requirements, you might consider directly editing and modifying the related integration flows contained in the standard integration package.

However, this does have some disadvantages. For example, you will not get any further updates to modified standard integration content. Furthermore, if you copy the updated standard package from the SAP Integration Content Catalog to the **Design** workspace of your tenant, you will overwrite all changes that you have made to that package.

To avoid such implications, we recommend decoupling the modifications (required for your specific business scenario) from the **standard content** provided by SAP. To support this pattern, SAP Cloud Integration comes with dedicated concepts and features. Using these features, a customer can specify requirements in one or more dedicated **custom integration flows**, without touching any integration flows from the standard content. Both the standard and the custom integration flows are deployed on the same tenant and are processed in conjunction, communicating with each other using a specific adapter, the *ProcessDirect* adapter.

i Note

Using the described concepts, the customer can keep the standard integration content unchanged and specify any required enhancements (for example, specific mappings) in one or more custom integration flows separate from the standard integration flow.

We can summarize the advantages as follows:

- Ensures better lifecycle management of prebuilt integration flows. If you make custom changes to a standard integration flow (prebuilt by SAP), this has an impact on lifecycle management, as you will not receive future updates related to modified artifacts on your tenant. Additionally, if you copy the updated standard package from the content catalog to the **Design** workspace of your tenant that contains the modified packages, you will overwrite all changes that have been made to that package. Therefore, defining all required changes in a dedicated integration flow helps you to better manage the lifecycle of your integration packages.
- Provides flexibility to integration developers to customize their integration flows without modifying them entirely.
- Enables the reuse of integration flows for mapping across different integration projects.

Involved Target Groups

As an integration flow extension is based on the interplay of standard integration content with one or more custom integration flows, two target groups are involved in the overall process:

- *Content publisher* (at SAP or an SAP partner)
- Customer doing the actual extension (referred to as *customer* in the following)

This documentation is structured as follows to serve both groups:

Section	Target Group
Integration Flow Extension - Concepts Provides an overview of the concept and the involved components; shows how standard content and extended content work in conjunction.	Content publisher Customer
Mapping Extension Step by Step (Demo Example) Provides a step-by-step description of a simple demo example and covers <i>both sides</i> of the extension story: • Creating the Standard Integration Flow Shows how to design the standard integration content (with a post-exit step) • Creating the Custom Integration Flow with the Post-Exit Mapping Shows how to design a post-exit integration flow (with the customer mapping extension)	Content publisher Customer This scenario is also feasible as a learning scenario: You should be able to set up and run this example in about an hour.
Mapping Extension Step by Step (Example from SAP Hybris C4C) Shows how a mapping extension is done for a real-life use case (covering pre-exit and post-exit mappings).	Customer

Integration Flow Extension - Concepts

This section provides an overview of the concepts and components required to implement an extension.

i Note

The use cases that we use to explain the concept in detail are all mapping extensions, and we use variations of mapping extensions for our examples and tutorials as well.

Note that you can, however, extrapolate the concept to any other extension.

Customer Exits

This is the basic concept: A **standard integration flow**, predefined by SAP or an SAP partner as part of a standard integration package, contains one or more customer exits. Through these exits, one or more **customer integration flows** (designed by the customer who is extending the standard content) are called. The standard integration flow and the custom integration flows (called through the customer exit) have to be deployed on the same tenant.

To support mapping extensions, standard integration flows can provide 2 types of customer exits:

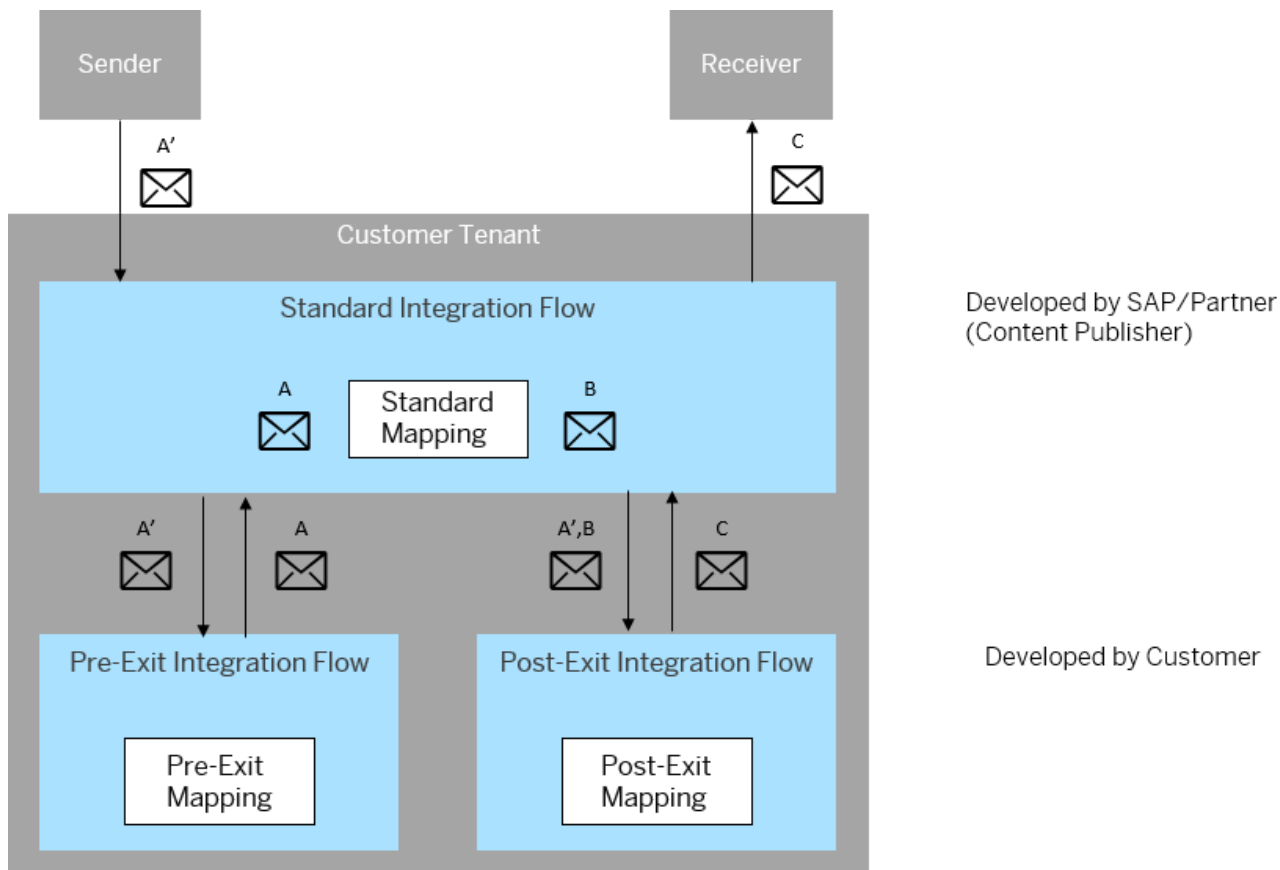
- Pre-exit

This exit is only required if the customer needs to extend the source message of the mapping. Using this exit, the standard integration flow calls a **pre-exit integration flow** (which is designed by the customer). Within the pre-exit integration flow, the extended source message is mapped to the message that is consumed by the mapping predefined in the standard integration flow.

- Post-exit

This exit is always required. It calls a post-exit custom integration flow that maps the output of the standard mapping and the extended source structure to the extended target structure.

The following figure shows the general overview of the message flow when a pre-exit and a post-exit are involved in the extension scenario.



The message flow in such an extension scenario works in the following way:

Let us assume that the mapping predefined in the standard integration flow defines the transformation of an original **message A** to **message B**.

We also assume that the customer has enhanced the source structure of the mapping (resulting in **message A'**). The pre-exit integration flow contains a mapping that transforms message **A'** to message **A**. Message **A** is then handed over to the standard integration flow and can be consumed by the predefined mapping.

The predefined mapping transforms message **A** to message **B**. Message **B** is then passed to the post-exit integration flow, where it's mapped against the actual enhancement of the target structure (defined as **message C**).

As indicated in the figure, the standard integration flow merges the original message (with the extended structure, message **A'**) with message **B** before handing it over to the post-exit integration flow. The post-exit mapping then consumes the merged message. Merging the message is a prerequisite for using fields from the original message **A'** that might have got lost during the standard mapping step in the post-exit mapping.

i Note

Technically, the communication between the standard integration flow and the pre-exit and post-exit integration flow is accomplished through the ProcessDirect adapter.

This adapter allows you to directly connect different integration flows deployed on the same tenant. In contrast to any of the available HTTP-based adapters (for example, the HTTP or SOAP adapter), communication through the ProcessDirect adapter is not routed through the load balancer and, therefore, does not imply any network latency.

The ProcessDirect adapter (both sender and receiver) has only 1 parameter, which is **Address**. When 2 integration flows need to communicate with each other through this adapter, the value specified for **Address** (in the related sender and receiver ProcessDirect adapters) has to match in both integration flows.

More information: [ProcessDirect Adapter](#)

i Note

The standard integration flow is developed as part of an integration package that is made available in the Integration Content Catalog (either by SAP or an SAP partner). For the sake of simplicity, we refer to this target group as the *integration content publisher*.

The custom integration flows (pre-exit and post-exit integration flows) are developed by customers who want to consume the integration package and build the extension on top. For the sake of simplicity, we refer to this target group as the *customer*.

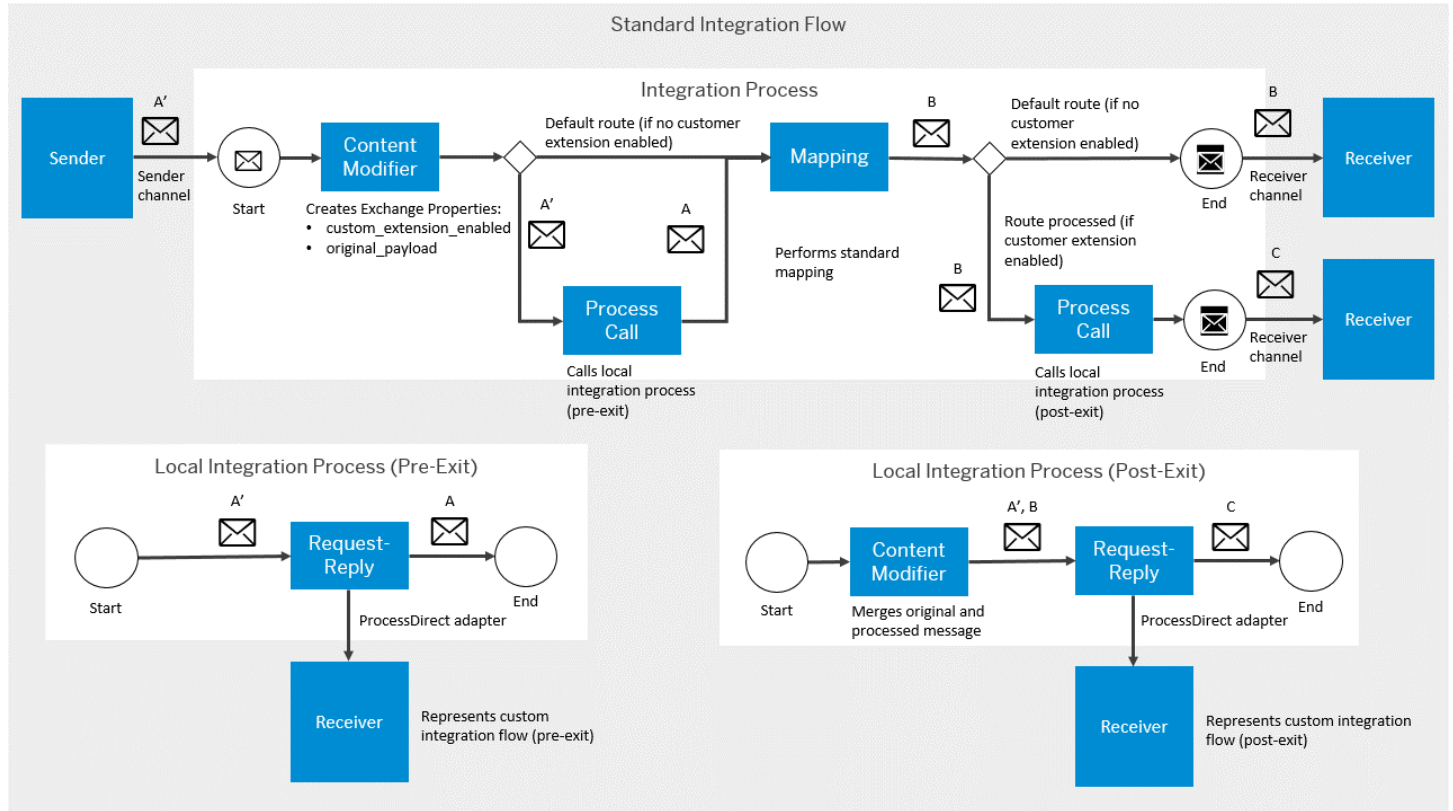
Depending on whether the standard integration flow contains both a pre-exit and a post-exit, or only a post-exit, the customer needs to create either 2 integration flows (a pre-exit and a post-exit integration flow) or only 1 post-exit integration flow to specify its own extensions.

Structure of the Standard Integration Flow

We first describe the structure of the standard integration flow, and then the structure of the custom (pre-exit/post-exit) integration flows.

The standard integration flow that supports mapping extensions is structured in the following way. It contains one or more *customer exits* (pre-exit or post-exit). These exits are encapsulated within local integration processes. From the local integration process, the related pre-exit or post-exit integration flow is called by the ProcessDirect receiver adapter.

The following figure shows the general structure of the **standard integration flow** containing both a pre-exit and a post-exit. Furthermore, the message flow is illustrated using the same notation for the different messages as already used above.



The standard integration flow contains a main integration process shape and 2 local integration process shapes (as we assume that both a pre-exit and a post-exit are involved in this scenario):

- The main integration process contains the main business logic (in this section, we'll just look at a mapping) and 2 routing steps leading to 2 different message processing paths (both containing a Process Call step).
- The local integration processes contain further steps to modify the message and – most important – the outbound call to a receiver. The receiver represents the pre-exit or post-exit integration flow. The outbound communication with the receiver is defined in a ProcessDirect adapter.

In the model, the local integration process on the left represents the pre-exit, and the local integration process on the right represents the post-exit.

Let's go into more detail. The integration flow is designed so that the message is processed in the following way at runtime:

Main Integration Process

- The sender system sends a message (A', which is the extended source message structure) to SAP Cloud Integration through a sender channel.

i Note

If as **Authorization** the option **User Role** is specified in the sender channel, the **User Role** parameter is externalized.

This enables consumers of the standard integration flow to apply their security role concept by providing a custom role for the integration flow.

- The Content Modifier creates 2 exchange properties:

- custom_extension_enabled

This exchange property is used as a parameter to control whether the extension concept should be applied to the standard integration flow or not. If its value is set to **true**, the pre-exit and post-exit integration flows are called. If its value is **false**, the standard mapping is applied without any further extension.

This parameter is externalized so that the value can be specified during integration flow configuration without the need to edit the integration flow model.

- `original_payload`

In this exchange property, the content of the original message received by the sender is stored (to make it available in a later step where the original message is merged with the transformed message as preparation for the post-exit mapping step).

- The first router step opens 2 alternative processing paths. Which path is processed depends on the routing condition, which uses the exchange property `custom_extension_enabled` defined in the preceding Content Modifier.
 - If the value of the property `custom_extension_enabled` is `false`, the default route (upper path) is taken.
The default route directly leads to the standard mapping.
 - If the value of the property `custom_extension_enabled` is `true`, the 2nd route (lower path) is taken. This route contains a Process Call step that passes the message to the local integration process on the left (pre-exit).

This route is processed under the condition that the value of the exchange property `custom_extension_enabled` is `true`.
- The mapping step transforms the inbound message to the target message **B**. The mapping step in this integration flow contains the standard mapping predefined by SAP or an SAP partner. This step always expects a predefined original message structure (message A). If the customer has extended the source structure (to message A'), the message needs to be processed by the pre-exit before the standard mapping step, to be transformed to the original structure A (as already explained above).
- The subsequent router step also opens 2 alternative processing paths, using the same routing condition as the first router step (using the exchange property `custom_extension_enabled` defined in the Content Modifier).
 - If the value of the property `custom_extension_enabled` is `false`, the default route (upper path) is taken.
The default route now ends message processing. In this case, the message isn't subjected to any further processing other than the standard mapping.
 - If the value of the property `custom_extension_enabled` is `true`, the 2nd route (lower path) is taken. This route contains a Process Call step that passes the message to the local integration process on the right (post-exit).

Local Integration Process (Pre-Exit)

If the message is first handed over to the local integration process with the pre-exit, the following steps are executed on the message:

A Request-Reply step calls a receiver (which represents the pre-exit integration flow) through a ProcessDirect receiver adapter.

The address of the ProcessDirect adapter is externalized and can be configured during integration flow configuration. The customer needs to make sure that the address of the ProcessDirect receiver adapter (of the standard integration flow) is the same as the one used in the ProcessDirect sender adapter of the pre-exit integration flow (to be connected).

Local Integration Process (Post-Exit)

If the message is first handed over to the local integration process with the post-exit, the following steps are executed on the message:

- A Content Modifier constructs a message body that is composed of the following 2 parts:
 - The original payload received from the sender (contained in the `original_payload` exchange property created in the Content Modifier of the main integration process)

(message **A'** in the figure)
 - The message that results as output from the Mapping step in the main process

(message **B** in the figure)

With this step, the original payload from the sender system (**A'**) is merged with the output message (**B**) resulting from the standard mapping step. The *concatenated* message is denoted as message **A',B**.

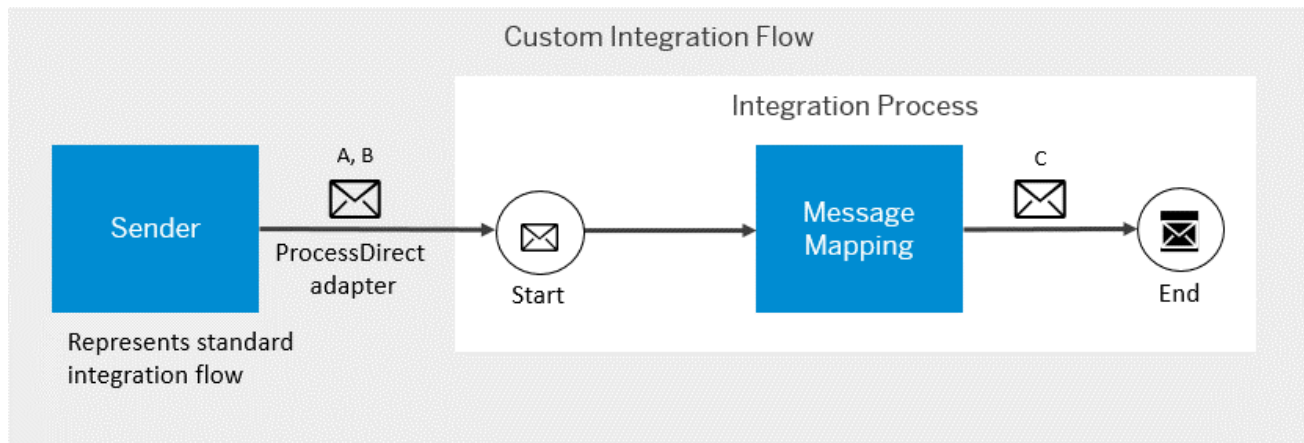
This measure is required for the following reason: The standard mapping step transforms the original message into a new one (**B**), which means that certain fields from the original message (**A**) may no longer be available in the resulting message **B**. However, the customer requires such fields in the custom mapping step (which is executed in the custom integration flow, as explained below). The message merge is done in order to make sure that all fields of the original message are still available in the message that is handed over to the post-exit integration flow.

- A Request-Reply step calls a receiver (which represents the post-exit integration flow) through a ProcessDirect receiver adapter.

The address of the ProcessDirect adapter is externalized and can be configured during integration flow configuration. The customer needs to make sure that the address of the ProcessDirect receiver adapter (of the standard integration flow) is the same as the one used in the ProcessDirect sender adapter of the post-exit integration flow (to be connected).

Structure of the Pre-Exit and Post-Exit Integration Flows (Designed by Customer)

The following figure shows the general structure of a pre-exit or post-exit integration flow.



- The sender shape represents the standard integration flow, which passes on the original message. The sender component is connected to the integration process component through a ProcessDirect sender adapter.
- Detailed instructions about the mapping are provided in a separate topic. In short, the mapping steps in the pre-exit and the post-exit integration flow process the inbound message in the following way:
 - Pre-exit mapping: Transforms the extended source message **A'** into the original message **A**, which can be consumed by the standard mapping.
 - Post-exit mapping: Transforms the message **A', B** from the standard integration flow into the final message **C**.

Procedure

The following list contains the main tasks in an integration flow extension process:

Tasks accomplished by the content publisher (SAP or SAP partner):

Create and publish the standard integration flow. Make sure that it contains the required customer exits (post-exit and, if applicable, pre-exit point). Externalize those parameters that the customer needs to configure the required parameters (for example, the addresses of the involved ProcessDirect adapters) without changing the standard integration flow.

Tasks accomplished by the customer (extending the content):

i Note

It is a prerequisite that you, the customer, have imported the integration package that contains the standard integration flow (that is to be extended) into your own workspace.

1. Copy the desired standard integration flow (as contained in the Integration Content Catalog) into your own workspace.
2. Create the required custom integration flows (with the pre-exit and post-exit steps).

Make sure you deploy the custom integration flows on the same tenant as the standard integration flow.

3. Configure the standard integration flow so that it can be used together with the custom integration flows.

To access the configuration user interface with the externalized parameters, open the standard integration flow and select **Configure**.

Examples:

- For the **custom_extension_enabled** parameter, enter the value **true**.
- For the **Address** parameter of the ProcessDirect receiver adapter, enter the same value as used in the **Address** field of the custom integration flow's ProcessDirect sender adapter.

Related Information

[ProcessDirect Adapter](#)

[Configure Externalized Parameters of an Integration Flow](#)

[Define Process Call](#)

[Define Local Integration Process](#)

[Define Content Modifier](#)

Mapping Extension Step by Step (Demo Example)

A demo example, explained step by step, covers the key aspects of integration flow extension.

This section provides a more detailed view of the concept overview given in [Integration Flow Extension - Concepts](#). We walk you through the most important concepts and involved components using a simple demo scenario.

You should be able to set up and run the scenario within approximately one hour.

i Note

The demo scenario covers both the design of the standard integration flow and of the custom integration flow. Consider the following:

- This demo scenario covers *both sides of the extension concept*: the design of the standard integration content and the customer extension (in a post-exit integration flow). Note that these tasks are typically accomplished by different target groups. Therefore, the demo scenario is aimed at experts from both target groups who want a more detailed understanding of the concepts and also hands-on experience of the tasks.

The tasks described in the two parts of the demo scenario are associated with the different target groups as follows:

- [Creating the Standard Integration Flow](#)

Describes the tasks accomplished by the **content publisher** (at SAP or an SAP partner organization) who makes this integration flow available as part of a predefined integration package that customers can copy into their own workspace.

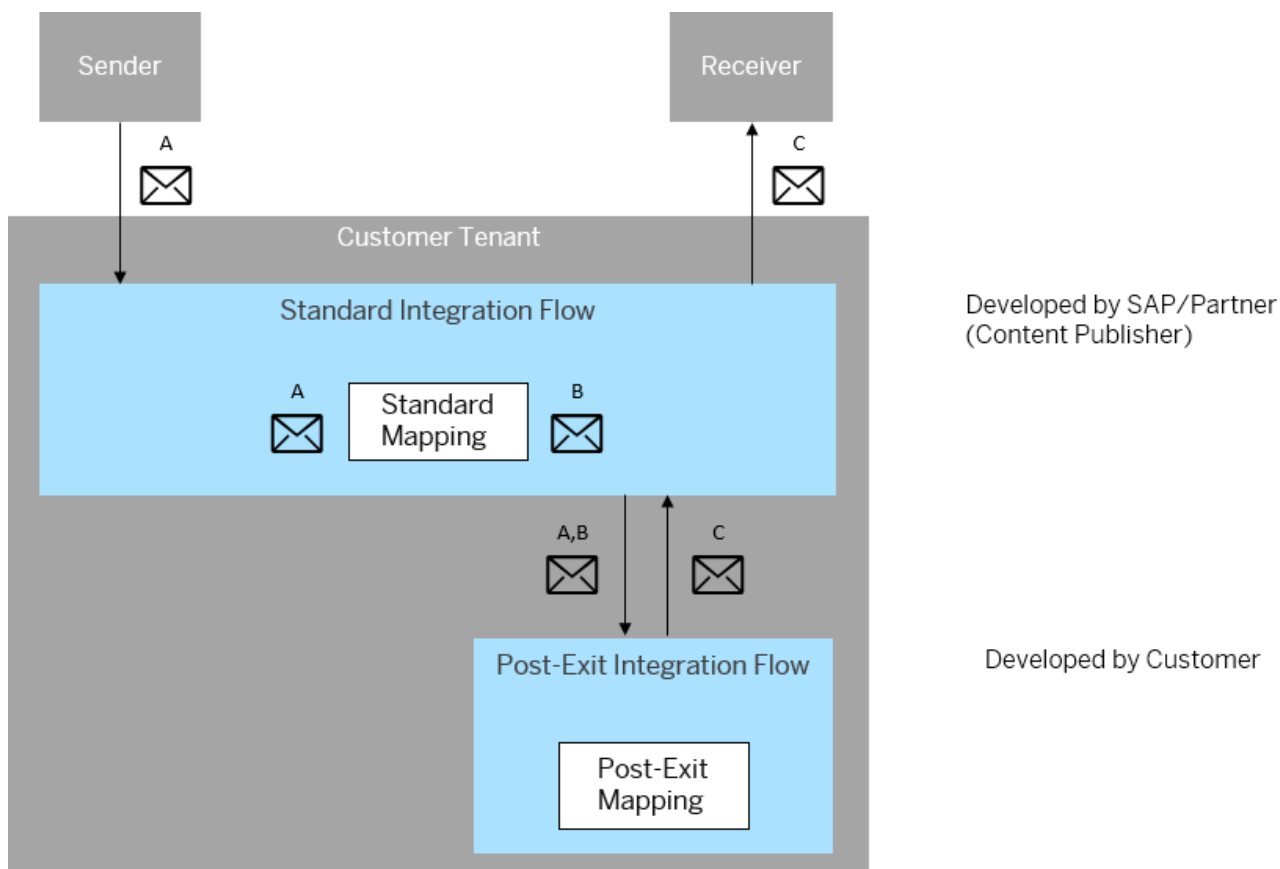
- [Creating the Custom Integration Flow with the Post-Exit Mapping](#)

Describes the tasks accomplished by the **customer** who is extending the content.

- The example scenario does **not** reflect a real-life integration use case. We keep the mapping as simple as possible and focus on the main principles and concepts so that you can see the differences between the standard mapping provided by SAP and the customer extension. To make it easy for you to reproduce the example, the integration flows are based on the scenario described in [SOAP Sender Adapter: Example Integration Flow](#).
- To find instructions for a mapping extension based on a real-life example, check out [Mapping Extension Step by Step \(Example from SAP Hybris C4C\)](#).
- The demo scenario does **not** use a pre-exit step.

Overview of the Demo Scenario

This section covers the following extension scenario with a post-exit.



In this scenario, the standard integration flow passes a message to a post-exit integration flow (which contains the custom mapping).

In this demo scenario, the sender system is simulated by a SOAP client, which sends a SOAP message to SAP Cloud Integration. The original message has a simple structure (only four elements). The standard mapping is also kept simple and concatenates (me.res) two of the original fields

More information: [Standard Mapping](#)

The mapping extension in the post-exit integration flow (the customer extension) adds an additional field to the target message and fills it with the value of a field from the original message (changing uppercase letters to lowercase letters).

More information: [Post-Exit Mapping](#)

To learn how to design the standard integration flow, see [Creating the Standard Integration Flow](#).

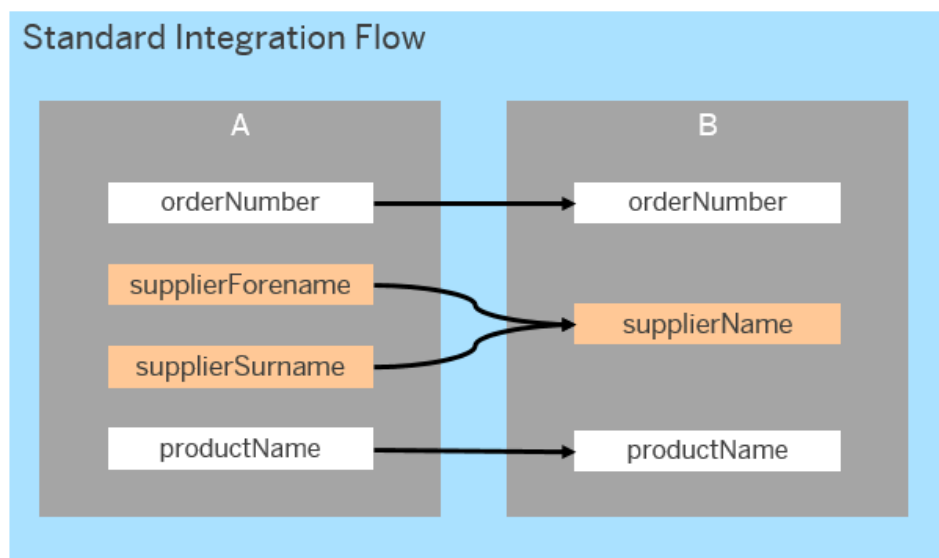
To learn how to design the post-exit integration flow, see [Creating the Standard Integration Flow](#).

To learn how to execute the scenario, see [Executing the Scenario](#).

Standard Mapping

The following figure shows the mapping used in this demo scenario.

The figure also indicates which fields correspond to the messages A and B as used in the concept overview in [Integration Flow Extension - Concepts](#).

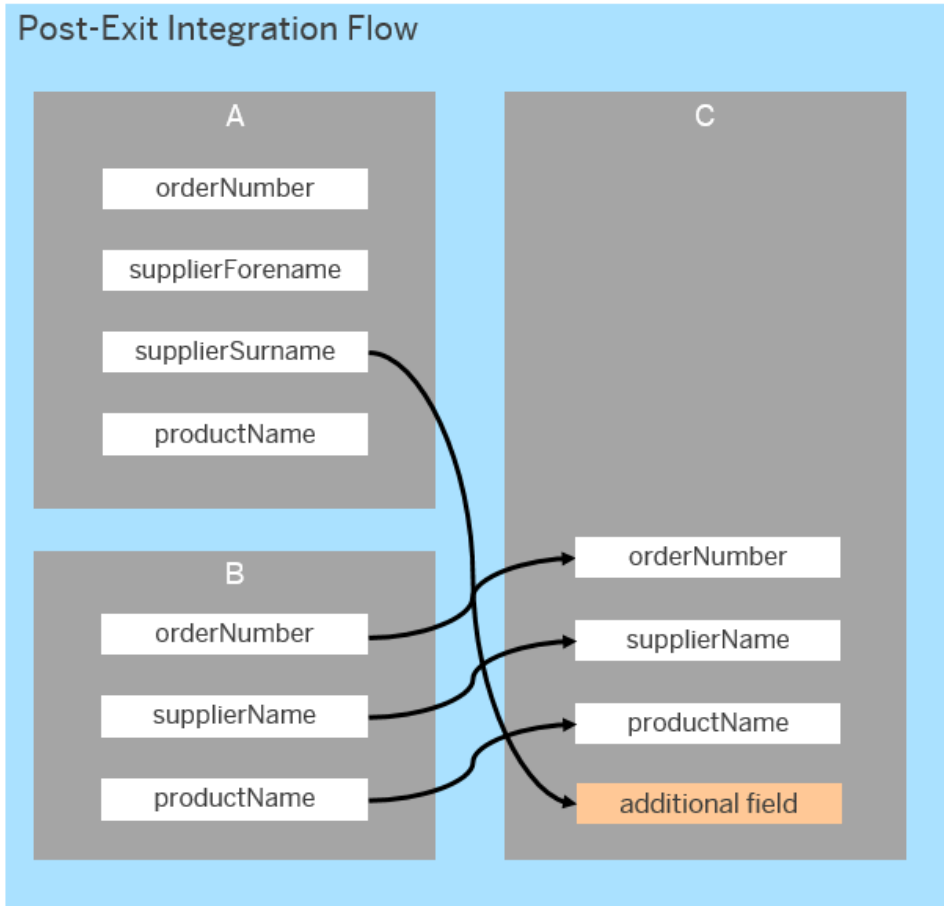


In the standard mapping, the original message A is transformed in the following way:

The fields `supplierForename` and `supplierSurname` are concatenated (resulting in the field `supplierName` of target message B).

Post-Exit Mapping

The following figure shows the custom mapping extension in the post-exit integration flow.



Here, the only mapping extension is the addition of the field `AdditionalField` to the target structure.

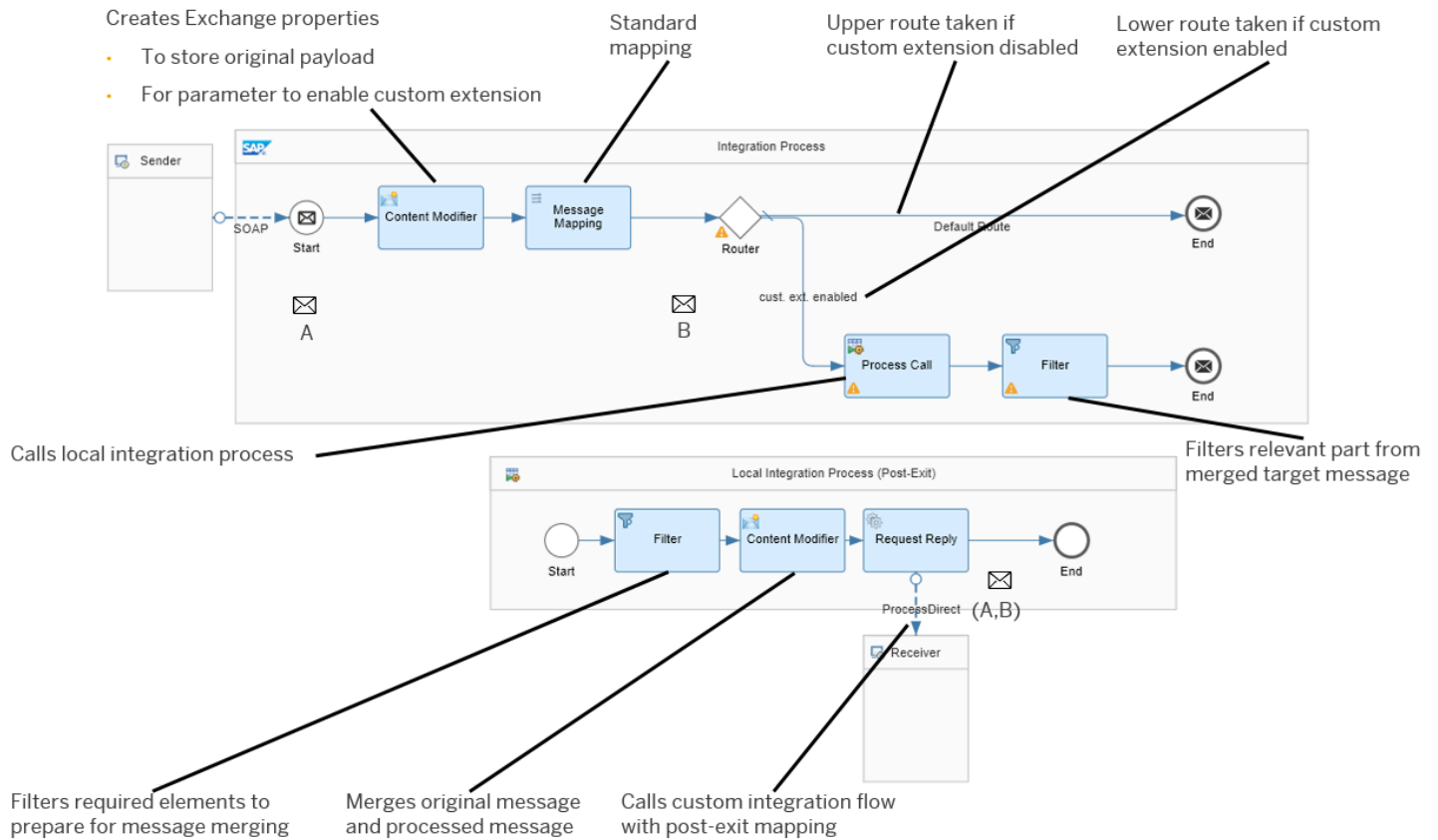
The custom mapping maps the `supplierSurname` field of the source message to the new field `AdditionalField` and applies a simple function (changing uppercase characters to lowercase characters).

Creating the Standard Integration Flow

i Note

This section is targeted at the content publisher at SAP or an SAP partner organization.

The standard integration flow is depicted in the following figure, where the key elements are highlighted.



This is a summary of the elements of the integration flow. Check out the detailed topics below to find out how to design these shapes.

- Sender and SOAP sender adapter

The sender is simulated by using a SOAP client that calls the integration flow endpoint using the SOAP 1.x sender adapter. The SOAP client sends message A to SAP Cloud Integration.

- Content Modifier

This step performs the following tasks:

- Stores the body of the inbound message in an exchange property (`originalpayload`).
- Creates another exchange property `custom_extension_enabled`, which is used in the subsequent routing step to separate message processing depending on whether the custom extension is activated or not.

We show how to externalize this parameter so that you can easily specify its value during the integration flow configuration without having to redesign the integration flow.

- Message Mapping

This step defines the standard mapping according to [Standard Mapping](#).

- Router

The Router step leads to two different message processing paths. The routing conditions are expressed using the exchange property `custom_extension_enabled` in the following way:

- Default route: Ends message processing without any further message processing.
- Route taken if the following condition is met: `custom_extension_enabled=true`

This message processing path contains a Process Call step, which calls the local integration process (post-exit).

After the Process Call step, a filter is added. This step filters the relevant parts from the message that is received from the post-exit integration flow.

The local integration process has the following elements:

- Filter

This step filters the elements from message B (which is the result of the standard mapping step) that are needed in the merged message (A,B), which is to be sent to the post-exit integration flow. This step is used to prepare the content of message B before it is merged with the original message A in the subsequent Content Modifier step.

- Content Modifier

Merges message A and message B by constructing a new message body using dynamic expressions for the original message A and the processed message B.

- Request-Reply step connected through ProcessDirect adapter with receiver

Uses the ProcessDirect receiver adapter to call the custom integration flow.

At runtime, the target integration flow is called based on the endpoint address configured in the adapter. You will learn how to externalize this parameter.

The integration flow does not have a receiver component. The result of the mapping can be analyzed in the response that is sent back to the SOAP client.

Related Information

[Configure the SOAP \(SOAP 1.x\) Sender Adapter](#)

[Defining the Content Modifier](#)

[Defining the Message Mapping](#)

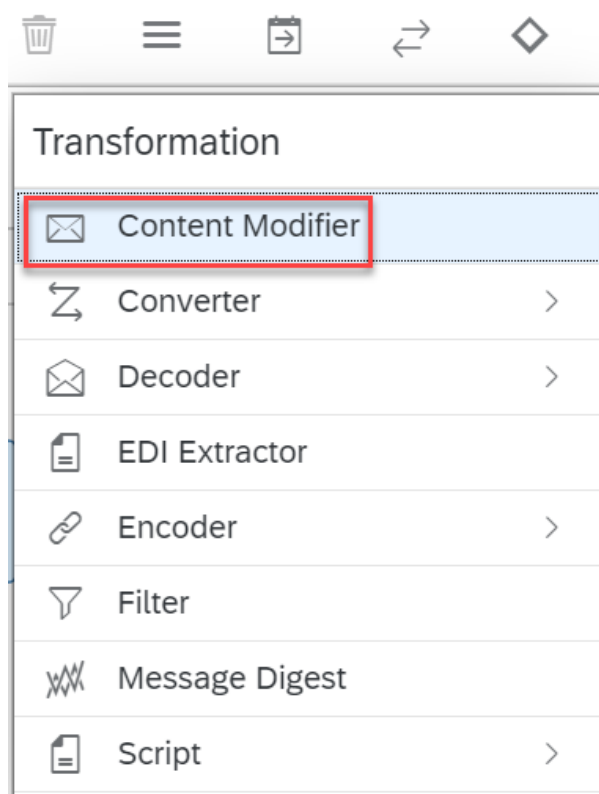
[Defining the Local Integration Process](#)

[Defining the Router, Process Call, and Filter Steps](#)

Defining the Content Modifier

1. Add a Content Modifier after the start message event.

You find the Content Modifier shape in the palette under **Message Transformation > Content Modifier**.



2. On the **Exchange Property** tab, enter the following property (by selecting **Add** and specifying the property):

Parameter	Select Option/Enter
Name	Enter original_payload
Type	Select Expression
Value	Enter \${in.body}

This property stores the content of the inbound message (message A, which is retrieved from the WebShop) by dynamically evaluating the Camel simple expression `${in.body}` at runtime.

We define the second property as an externalized parameter.

1. Select **Externalize**.
2. Go to the **Exchange Property** tab.

3. Enter `custom_extension_enabled` under **Name**, select **Expression** as the **Type**, and enter `{{ custom_extension_enabled }}` in the **Value** field.

Action	Name	Type	Data Type	Value	Default
Create	custom_extension_enabled	Expression		{{custom_extension_enabled}}	

4. Click **Define Value** (in the **Value** field), and on the next screen enter `false`. Then, choose **OK**

The final Content Modifier has the following settings:

Action	Name	Type	Data Type	Value	Default
Create	custom_extension_enabled	Expression		false	
Create	original_payload	Expression		\${in.body}	

When the integration flow is being processed, the Content Modifier does the following:

- It writes the complete message body (retrieved from the external OData API) into the property `original_payload`.
- Secondly, it writes the value specified during integration flow configuration for the externalized parameter `custom_extension_enabled` into the Exchange property. This property is used in the subsequent routing step to determine whether the default route is taken (where only the SAP mapping is processed and nothing else is done to the message) or whether the customer exit is taken (if the value of `custom_extension_enabled` is set to `true`).

Related Information

[Define Content Modifier](#)

Defining the Message Mapping

Before defining the standard mapping, you need to upload to the tenant certain resource files (Web Services Definition Language, or WSDL files for short) that define the structure of source message (A) and target message (B) of the mapping. To facilitate the design of the demo scenario, we provide you with the content of the WSDL files in this section.

To upload the WSDL files, perform the following steps:

- Copy the content of the following coding example into a text editor and save it as a file with extension `.wsdl` (for example, `A.wsdl`).

This WSDL file defines the structure of the source message of the standard mapping (corresponding to the original **message A**, which is sent to the integration flow through the SOAP client).

Sample Code

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions name="SendOrder_Async"
  targetNamespace="http://cpi.sap.com/demo" xmlns:p1="http://cpi.sap.com/demo"
  xmlns:wsp="http://schemas.xmlsoap.org/ws/2004/09/policy"
  xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/">
  <wsdl:documentation />
  <wsp:UsingPolicy wsdl:required="true" />
  <wsp:Policy wsu:Id="OP_SendOrder_Async" />
  <wsdl:types>
    <xsd:schema targetNamespace="http://cpi.sap.com/demo"
      xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://cpi.sap.com/demo">
      <xsd:element name="Order_MT" type="Order_DT" />
      <xsd:complexType name="Order_DT">
        <xsd:sequence>
          <xsd:element name="orderNumber" type="xsd:string" />
          <xsd:element name="supplierForename" type="xsd:string" />
        </xsd:sequence>
      </xsd:complexType>
    </xsd:schema>
  </wsdl:types>
</wsdl:definitions>
```

```

        <xsd:element name="supplierSurname" type="xsd:string" />
        <xsd:element name="productName" type="xsd:string" />
    </xsd:sequence>
</xsd:complexType>
</xsd:schema>
</wsdl:types>
<wsdl:message name="Order_MT">
    <wsdl:documentation />
    <wsdl:part name="Order_MT" element="p1:Order_MT" />
</wsdl:message>
<wsdl:portType name="SendOrder_Async">
    <wsdl:documentation />
    <wsdl:operation name="SendOrder_Async">
        <wsdl:documentation />
        <wsp:Policy>
            <wsp:PolicyReference URI="#OP_SendOrder_Async" />
        </wsp:Policy>
        <wsdl:input message="p1:Order_MT" />
    </wsdl:operation>
</wsdl:portType>
<wsdl:binding name="SendOrder_AsyncBinding" type="p1:SendOrder_Async">
    <soap:binding style="document"
        transport="http://schemas.xmlsoap.org/soap/http" xmlns:soap="http://schemas.xmlsoap.org/wsdl/s
    <wsdl:operation name="SendOrder_Async">
        <soap:operation soapAction="http://sap.com/xi/WebService/soap1.1"
            xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" />
        <wsdl:input>
            <soap:body use="literal" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" />
        </wsdl:input>
    </wsdl:operation>
</wsdl:binding>
</wsdl:definitions>

```

You will notice that the WSDL contains exactly the same fields that are shown in the topic [Standard Mapping](#) for message A.

2. Do the same with the following code sample, but save it under a different name, for example, B.wSDL.

This WSDL file defines the structure of the target **message B** of the standard mapping.

Sample Code

```

<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions name="SendOrder_Async"
    targetNamespace="http://cpi.sap.com/demo" xmlns:p2="http://cpi.sap.com/demo"
    xmlns:wsp="http://schemas.xmlsoap.org/ws/2004/09/policy"
    xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd"
    xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/">
    <wsdl:documentation />
    <wsp:UsingPolicy wsdl:required="true" />
    <wsp:Policy wsu:Id="OP_SendOrder_Async" />
    <wsdl:types>
        <xsd:schema targetNamespace="http://cpi.sap.com/demo"
            xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://cpi.sap.com/demo">
            <xsd:element name="Order_MT" type="Order_DT" />
            <xsd:complexType name="Order_DT">
                <xsd:sequence>
                    <xsd:element name="orderNumber" type="xsd:string" />
                    <xsd:element name="supplierName" type="xsd:string" />
                    <xsd:element name="productName" type="xsd:string" />
                </xsd:sequence>
            </xsd:complexType>
        </xsd:schema>
    </wsdl:types>
    <wsdl:message name="Order_MT">
        <wsdl:documentation />
        <wsdl:part name="Order_MT" element="p2:Order_MT" />
    </wsdl:message>
    <wsdl:portType name="SendOrder_Async">
        <wsdl:documentation />
    </wsdl:portType>

```

```

<wsdl:operation name="SendOrder_Async">
  <wsdl:documentation />
  <wsp:Policy>
    <wsp:PolicyReference URI="#OP_SendOrder_Async" />
  </wsp:Policy>
  <wsdl:input message="p2:Order_MT" />
</wsdl:operation>
</wsdl:portType>
<wsdl:binding name="SendOrder_AsyncBinding" type="p2:SendOrder_Async">
  <soap:binding style="document"
    transport="http://schemas.xmlsoap.org/soap/http" xmlns:soap="http://schemas.xmlsoap.org/wsdl/s
  <wsdl:operation name="SendOrder_Async">
    <soap:operation soapAction="http://sap.com/xi/WebService/soap1.1"
      xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" />
    <wsdl:input>
      <soap:body use="literal" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" />
    </wsdl:input>
  </wsdl:operation>
</wsdl:binding>
</wsdl:definitions>

```

→ Tip

You might have noticed that in this WSDL document the namespace prefix p2 is used instead of the namespace prefix p1 (used in the WSDL for message A). For example, note the following entry: p2="http://cpi.sap.com/demo".

This is an important detail. Using the correct namespace prefixes (and making some further related settings in the integration flow as shown below) makes sure that the merged message that is passed on to the post-exit mapping can be interpreted in the right way. Using namespaces correctly is critical because the merged message contains the structures of messages A and B, which have many elements with the same name.

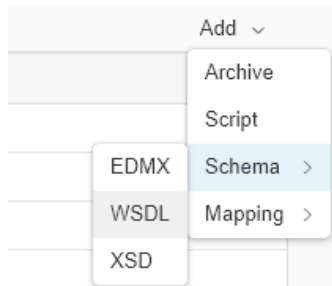
We map the namespaces in the next step to complete this configuration.

3. In the integration flow editor, click somewhere outside the **Integration Process** shape and select **Runtime Configuration**. In field **Namespace Mapping** enter the following:

```
xmlns:p1=http://cpi.sap.com/demo;xmlns:p2=http://sap.com/xi/XI/SplitAndMerge
```

The namespace mapped to the prefix p2 relates to the final merged message.

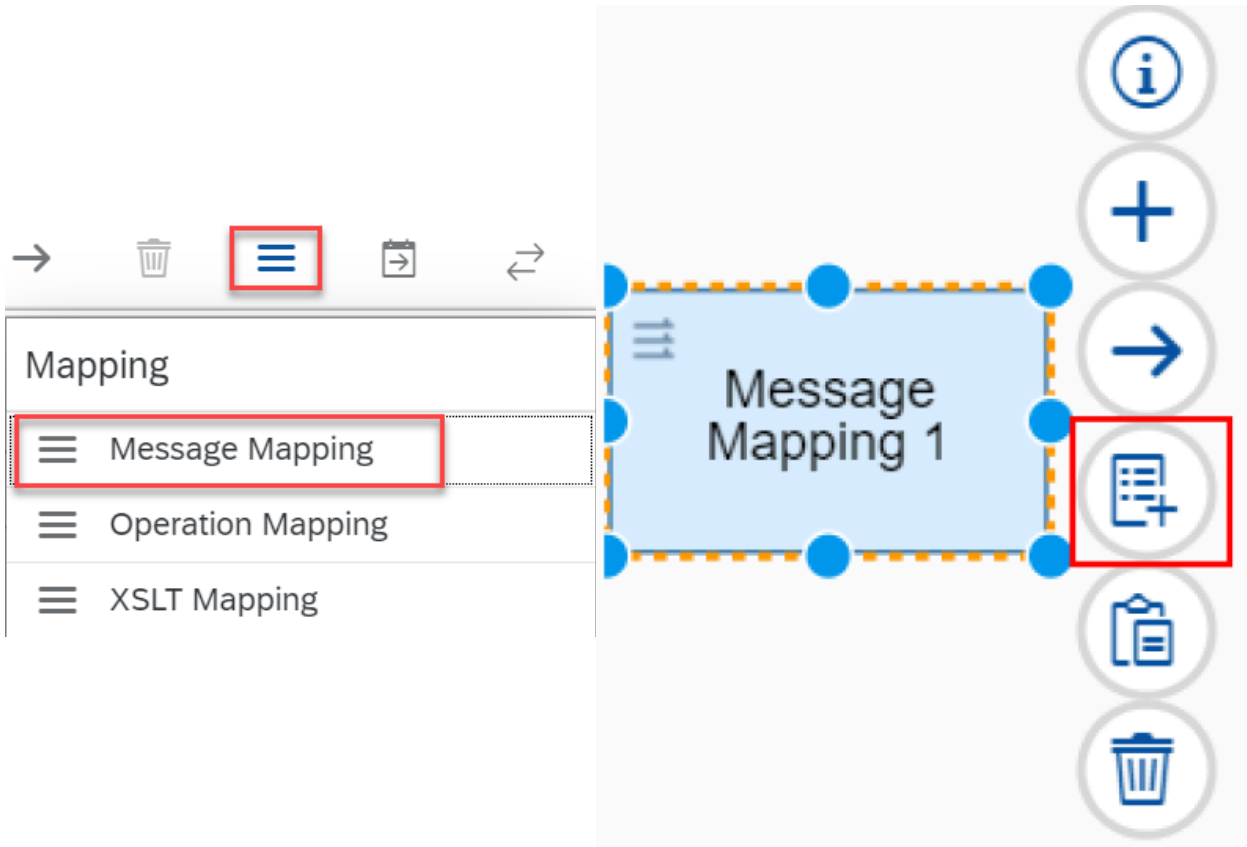
4. In the integration flow editor, click somewhere outside the **Integration Process** shape and select **References**.
5. Choose **Add** **Schema** **WSDL** and browse to the A.wsdL file.



6. Repeat these steps for the B.wsdL file.

Now you can define the mapping step.

1. In the integration flow model, select the **Message Mapping** shape from the palette (under **Mapping**) and place it in the model after the Content Modifier.



i Note

To add message mapping shape in an alternate method, choose **Connector line > Add flow step > Mapping > Message mapping**.

- 2. Choose **Create** icon next to the message mapping shape.
- 3. Enter a name (for example, `standardmapping`) and choose **Create** to submit the message mapping name.

Create Message Mapping

Name:

Create

Cancel

- 4. The graphical mapping editor is opened.
- 5. Choose **Add source message** and browse to the WSDL file of the source message (in our example, the file `A.wsdl`).

i Note

You can also choose **Upload from file system** and browse a different WSDL file.

The WSDL file uploaded from here will not be saved in the **References**.

Add source message

Structure	Occu...
No data	

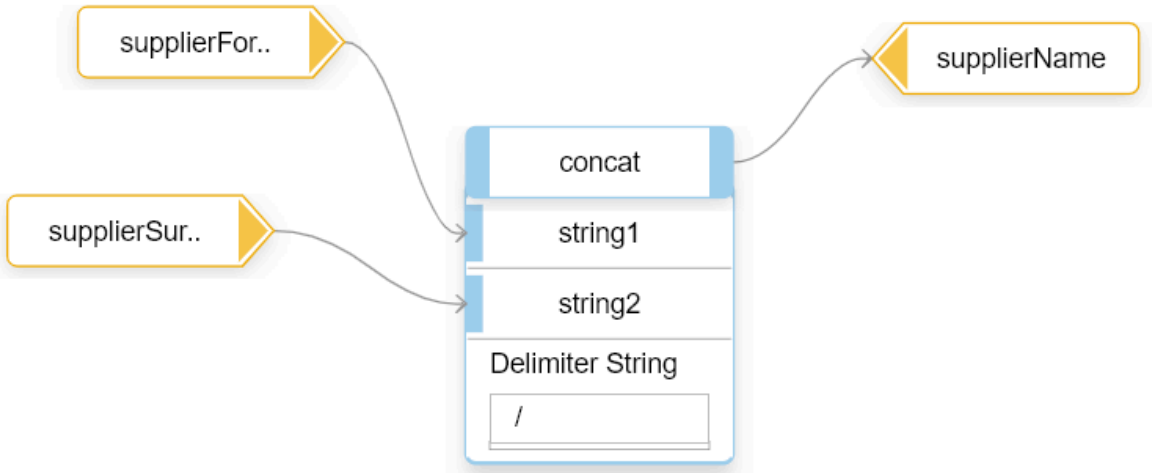
Add target message

Structure	Occu...
No data	

- Choose **Add target message** and repeat the steps with the WSDL file for the target message (B.wsd1).
- Connect the fields of the source and target message in the following way using the cursor:
 - Connect `orderNumber` in the source message with `orderNumber` in the target message.
 - Connect `supplierForename` in the source message with `supplierName` in the target message.
 - Connect `supplierSurname` in the source message with `supplierName` in the target message.
 - Connect `productName` in the source message with `productName` in the target message.
- Click `supplierForename` in the source message. The connectors between `supplierForename` and `supplierSurname` (in the source message) and `supplierName` (in the target message) are highlighted.
- In the section below the graphical mapping editor, select the function **Text > Concat**. Connect the source and target fields in the following way:

Connect `supplierForename` with `string1` and `supplierSurname` with `string2`. Connect `concat` with `supplierName`.

Furthermore, you can add a delimiter string (for example, / or an empty space).



Your mapping should now look like this:

Order_MT

Search		
Structure		Occurrence
ns0:Order_MT		1..1
orderNumber		1..1
supplierForename	A	1..1
supplierSurname		1..1
productName		1..1

Order_MT

Search		
Structure		
ns0:Order_MT		
orderNumber		
supplierName		
productName		

Mapping expression

Functions

Search...

Standard

Arithmetic

supplierFor..

supplierSur..

concat

supplierName

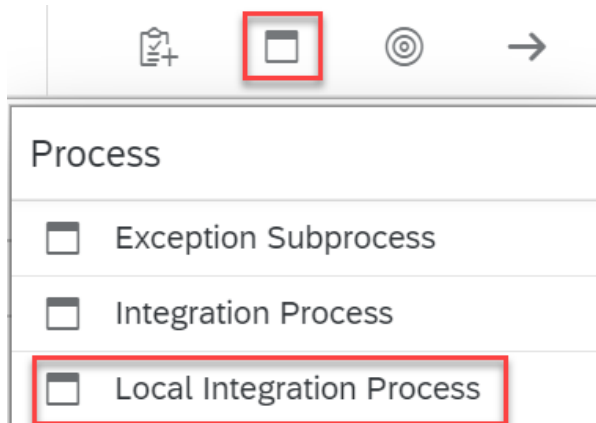
This mapping function merges the fields `supplierForename` and `supplierSurname` from the source message to one single field `supplierName` in the target message (where the values are separated in the target field by the delimiter string).

10. Choose **OK**. The integration flow model is displayed again.

Defining the Local Integration Process

Before continuing to the routing step, we first add a local integration process shape to the integration flow model.

1. In the palette, select **Process > Local Integration Process** and position the shape below the (main) **Integration Process** shape.



2. In the palette, select **Transformation > Filter** and place the filter shape to the right of the start event of the local integration process.

3. Click the **Filter** shape and, on the **Processing** tab, enter the following string in the **XPath Expression** field: `/p1:Order_MT`

Keep the default setting of **Value Type** as **Nodelist**.

This step will filter the required elements from the message structure `Order_MT` (associated with the namespace mapped to prefix `p1`) from the processed message B (to prepare the message for the subsequent merge step).

4. Add a Content Modifier to the right of the Filter step (you can find this under **Message Transformers** in the palette).

On the **Message Body** tab, for type dropdown choose **Expression** and enter the following:

Sample Code

```
<ns1:Messages xmlns:ns1="http://sap.com/xi/XI/SplitAndMerge">
  <ns1:Message1>
    ${property.original_payload}
  </ns1:Message1>
  <ns1:Message2>
    ${in.body}
  </ns1:Message2>
</ns1:Messages>
```

This step creates the message (A,B), which will be passed on to the custom integration flow.

Again, you will notice the following dynamic expressions:

- Expression `${property.original_payload}` is replaced at runtime by the content of the original message A (which was stored prior to the standard mapping step in the Exchange property).
- Expression `${in.body}` is replaced at runtime by the content of the message that enters the Content Modifier. This is message B, which results from the standard mapping.

5. Add a Request-Reply step (you can find this in the palette under **Call > External Call**) to the right of the Content Modifier.

6. Add a receiver shape below the local integration process and connect it to the Request-Reply step.

7. When connecting the shapes, select **ProcessDirect** as the **Adapter Type**.

8. The **ProcessDirect** adapter has only one parameter that you need to configure. Specify the **Address** as an externalized parameter.

Select **Externalize** and, on the next screen, enter `{{endpoint}}` as **Address**.

Externalization

ProcessDirect

CONNECTION DETAILS

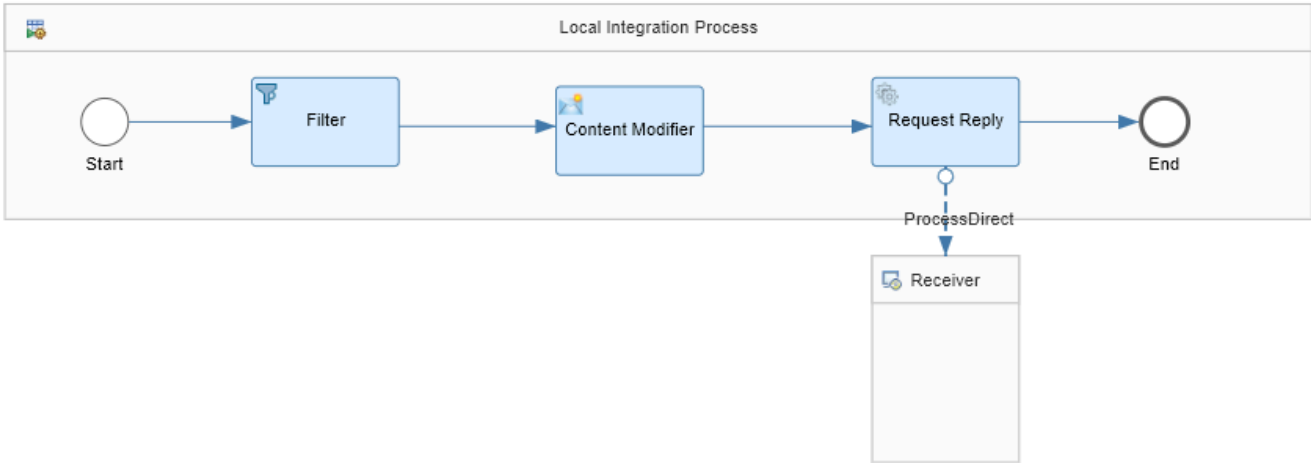
Address:

{{endpoint}}

<Define Value>

- 9. Choose **Define Value** and enter any value starting with /.
- 10. Choose **OK**.

The local integration process should now look something like this:



- Click in the area outside the integration process and the local integration process shapes and select **Externalized Parameters**.
- You will find the two parameters that you have already defined as externalized parameters (one from the Content Modifier on the **More** tab and the endpoint address of the ProcessDirect adapter on the **Receiver** tab).

Configure "Customer Extension Standard Integration Flow"

Receiver

More

Receiver:

Receiver

Adapter Type:

ProcessDirect

Connection

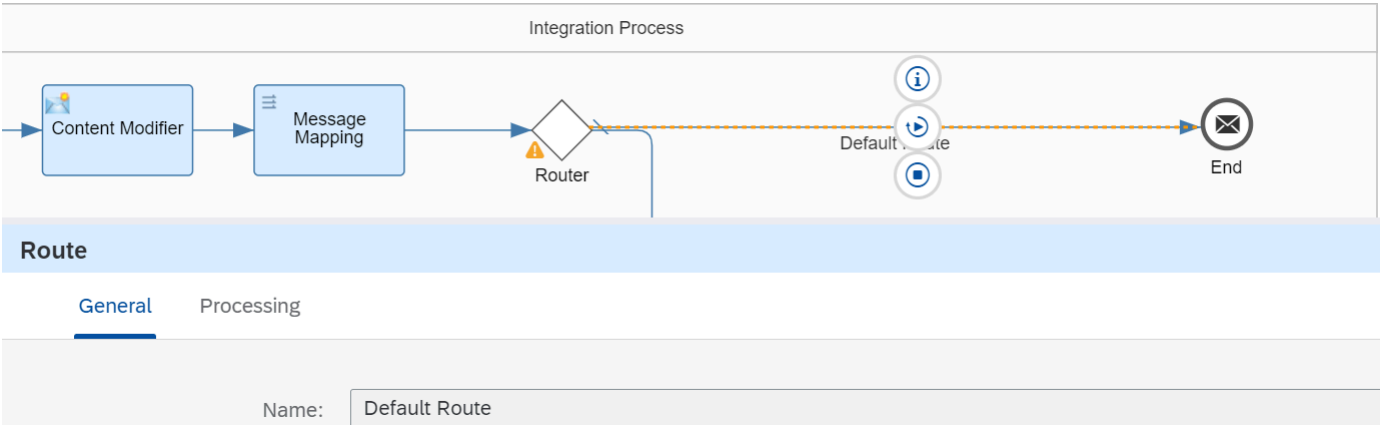
Address:

/customflow

Defining the Router, Process Call, and Filter Steps

Now that you have defined the local integration process, you can continue with the design of the main integration process.

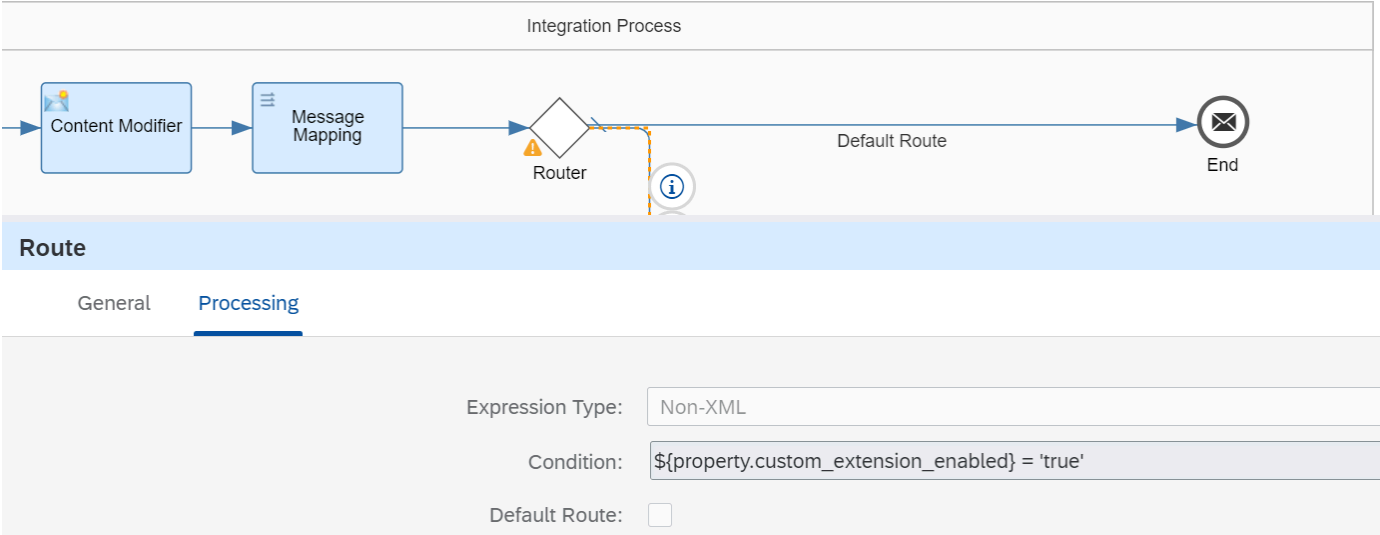
- 1. In the main integration process, add a Router step to the right of the message mapping. You can find the step type in the palette under **Message Routing** **Router**.
- 2. Add another end message event (from the palette under **Events**) and place it below the existing one.
- 3. Connect the Router step with the second end message event.
- 4. Click the upper route (connection of router step and upper end message event) and, on the **Processing** tab, select **Default Route**.



5. Click the lower route and, on the **Processing** tab, specify the following:

- Select **Non-XML** as **Expression Type**.
- Under **Condition**, enter the following routing condition:
`${property.custom_extension_enabled} = 'true'`

This condition will be evaluated at runtime in the following way: The value of the (externalized) parameter `custom_extension_enabled` is checked. If the value is `true`, this route will be processed. If not, the default route is processed (which means that message is not subjected to any further processing after the standard mapping step).

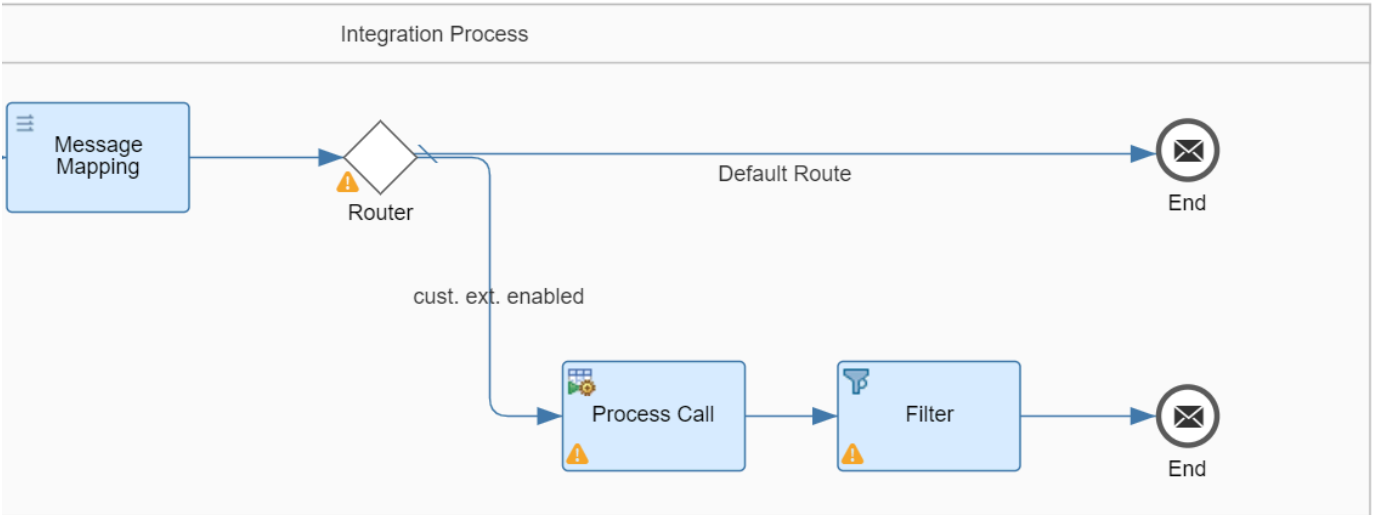


6. In the lower route, add a Process Call step (which you can find in the palette under **Call** > **Local Call**).

Select the Process Call shape and, on the **Processing** tab, choose **Select**. You can now select the local integration process that you have already defined in a preceding step.

7. In the lower route, after the Process Call shape, add a Filter step (you can find this in the palette under **Transformation** > **Filter**).

This part of the integration flow model should now look like this:



8. On the properties sheet of the Filter step, under **Processing**, add the following expressing in the **XPath Expression** field:

```
/p2:Messages/p2:Message1/p1:Order_MT
```

Note that this setting contains two different namespace prefixes, p1 and p2. The mapping of these prefixes with the relevant namespaces is defined on the **Runtime Configuration** tab of the integration flow (as shown in [Defining the Message Mapping](#)).

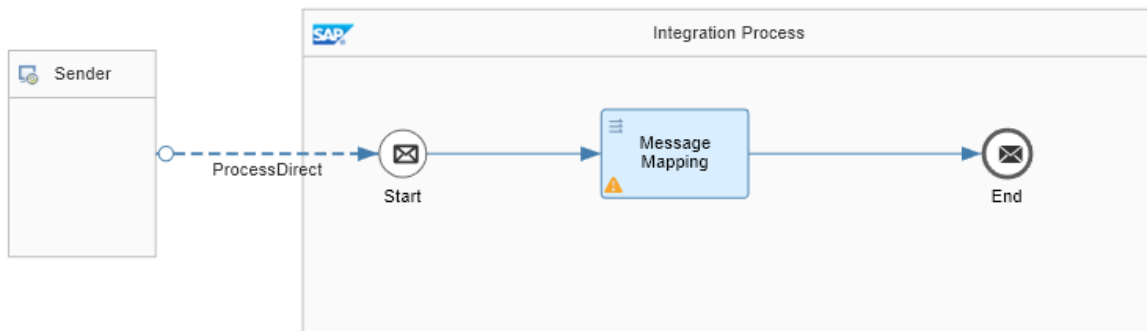
Using different namespace prefixes and addressing them correctly as described is necessary to make sure that the correct parts of the message are filtered. Note that this Filter step comes after the Process Call step, and so it gets the message processed in the post-exit integration flow as input.

Creating the Custom Integration Flow with the Post-Exit Mapping

i Note

This section is targeted at the customer who is doing a mapping extension.

This integration flow is kept very simple.



- The sender shape in this model represents the standard integration flow (which is the source integration or producer integration flow). The custom integration flow receives the message from the producer integration flow through the ProcessDirect adapter (as both integration flows are deployed on the same tenant).
- The post-exit message mapping step contains the mapping as explained under [Post-Exit Mapping](#).

Related Information

[Configuring the ProcessDirect Sender Adapter](#)

[Creating the Custom Mapping](#)

Configuring the ProcessDirect Sender Adapter

1. Create a new integration flow (this will be the custom integration flow).
2. Remove the receiver component.
3. Connect the sender shape with the start message event and select **ProcessDirect** as **Adapter Type**.
4. Configure the address of the ProcessDirect adapter as an externalized parameter in the same way as described in [Defining the Local Integration Process](#) for the ProcessDirect receiver adapter in the standard integration flow.

Creating the Custom Mapping

It is a prerequisite for the custom mapping that you upload the WSDL files defining the source and target message structures to the **Resources** tab of the integration flow (as shown for the standard mapping in [Defining the Message Mapping](#)). As you can see, three message structures, A, B, and C, are involved in the mapping, and the corresponding resource files (WSDLs) have to be made available to the integration flow.

To make it easy for you to define the mapping step, we provide the content of the WSDL files below.

You can use this content to create the WSDL file for the source and target message of the custom mapping:

For the source message, import two WSDL files:

- The WSDL file describing the original **message A** (as contained in the standard integration content)
Use file **A.wsdl** from [Defining the Message Mapping](#).
- The WSDL file describing the message structure after the standard mapping (message B)

Use file B.wSDL from [Defining the Message Mapping](#).

However, note one important detail: In this WSDL file, you need the original namespace prefix p1. Otherwise, message processing results in a namespace conflict and the mapping step cannot be executed without an error.

Correspondingly, to define the WSDL file for **message B** (B.wSDL), copy the following content:

Sample Code

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions name="SendOrder_Async"
  targetNamespace="http://cpi.sap.com/demo" xmlns:p1="http://cpi.sap.com/demo"
  xmlns:wsp="http://schemas.xmlsoap.org/ws/2004/09/policy"
  xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd"
  xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/">
  <wsdl:documentation />
  <wsp:UsingPolicy wsdl:required="true" />
  <wsp:Policy wsu:Id="OP_SendOrder_Async" />
  <wsdl:types>
    <xsd:schema targetNamespace="http://cpi.sap.com/demo"
      xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://cpi.sap.com/demo">
      <xsd:element name="Order_MT" type="Order_DT" />
      <xsd:complexType name="Order_DT">
        <xsd:sequence>
          <xsd:element name="orderNumber" type="xsd:string" />
          <xsd:element name="supplierName" type="xsd:string" />
          <xsd:element name="productName" type="xsd:string" />
        </xsd:sequence>
      </xsd:complexType>
    </xsd:schema>
  </wsdl:types>
  <wsdl:message name="Order_MT">
    <wsdl:documentation />
    <wsdl:part name="Order_MT" element="p1:Order_MT" />
  </wsdl:message>
  <wsdl:portType name="SendOrder_Async">
    <wsdl:documentation />
    <wsdl:operation name="SendOrder_Async">
      <wsdl:documentation />
      <wsp:Policy>
        <wsp:PolicyReference URI="#OP_SendOrder_Async" />
      </wsp:Policy>
      <wsdl:input message="p1:Order_MT" />
    </wsdl:operation>
  </wsdl:portType>
  <wsdl:binding name="SendOrder_AsyncBinding" type="p1:SendOrder_Async">
    <soap:binding style="document"
      transport="http://schemas.xmlsoap.org/soap/http" xmlns:soap="http://schemas.xmlsoap.org/wsdl/s">
    <wsdl:operation name="SendOrder_Async">
      <soap:operation soapAction="http://sap.com/xi/WebService/soap1.1"
        xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" />
      <wsdl:input>
        <soap:body use="literal" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" />
      </wsdl:input>
    </wsdl:operation>
  </wsdl:binding>
</wsdl:definitions>
```

i Note

Both WSDL files together define the structure of the merged message (A, B), which is passed on from the standard integration flow (as you will see when defining the message mapping).

For target **message C**, create a WSDL file with the following content and import it to the integration flow [Resources](#) tab:

Sample Code

```
<?xml version="1.0" encoding="UTF-8"?>
<wsdl:definitions name="SendOrder_Async"
```

```

targetNamespace="http://cpi.sap.com/demo" xmlns:pl="http://cpi.sap.com/demo"
xmlns:wsp="http://schemas.xmlsoap.org/ws/2004/09/policy"
xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/"
<wsdl:documentation />
<wsp:UsingPolicy wsdl:required="true" />
<wsp:Policy wsu:Id="#OP_SendOrder_Async" />
<wsdl:types>
  <xsd:schema targetNamespace="http://cpi.sap.com/demo"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema" xmlns="http://cpi.sap.com/demo">
    <xsd:element name="Order_MT" type="Order_DT" />
    <xsd:complexType name="Order_DT">
      <xsd:sequence>
        <xsd:element name="orderNumber" type="xsd:string" />
        <xsd:element name="supplierName" type="xsd:string" />
        <xsd:element name="productName" type="xsd:string" />
        <xsd:element name="additionalField" type="xsd:string" />
      </xsd:sequence>
    </xsd:complexType>
  </xsd:schema>
</wsdl:types>
<wsdl:message name="Order_MT">
  <wsdl:documentation />
  <wsdl:part name="Order_MT" element="pl:Order_MT" />
</wsdl:message>
<wsdl:portType name="SendOrder_Async">
  <wsdl:documentation />
  <wsdl:operation name="SendOrder_Async">
    <wsdl:documentation />
    <wsp:Policy>
      <wsp:PolicyReference URI="#OP_SendOrder_Async" />
    </wsp:Policy>
    <wsdl:input message="pl:Order_MT" />
  </wsdl:operation>
</wsdl:portType>
<wsdl:binding name="SendOrder_AsyncBinding" type="pl:SendOrder_Async">
  <soap:binding style="document"
    transport="http://schemas.xmlsoap.org/soap/http" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" />
  <wsdl:operation name="SendOrder_Async">
    <soap:operation soapAction="http://sap.com/xi/WebService/soap1.1"
      xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" />
    <wsdl:input>
      <soap:body use="literal" xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/" />
    </wsdl:input>
  </wsdl:operation>
</wsdl:binding>
</wsdl:definitions>

```

Note that message structure C is similar to message structure B; it has one additional field `additionalField`. Save the file with the name `C.wsdl`.

Perform the following steps to define the mapping:

1. Add a message mapping shape between the start message and the end message event.

Choose the **+** icon and define a name for the message mapping.



- 2. Choose **Create**.
- The mapping editor opens.
- 3. Select **Add source message** and browse to the file **A.wsdL**.
- 4. Choose the **Edit message** icon.



The **Source and Target Messages** editor opens.

The structure of message A is already displayed under **Source Messages**.

- 5. Choose the **+** icon beneath **Source Messages** and add file **B.wsdL**.
- 6. In the same way, choose the **+** icon beneath **Target Messages** and add file **C.wsdL**.

After this step, messages A and B are shown under **Source Messages** and message C is shown under **Target Messages**.

Source and Target Messages

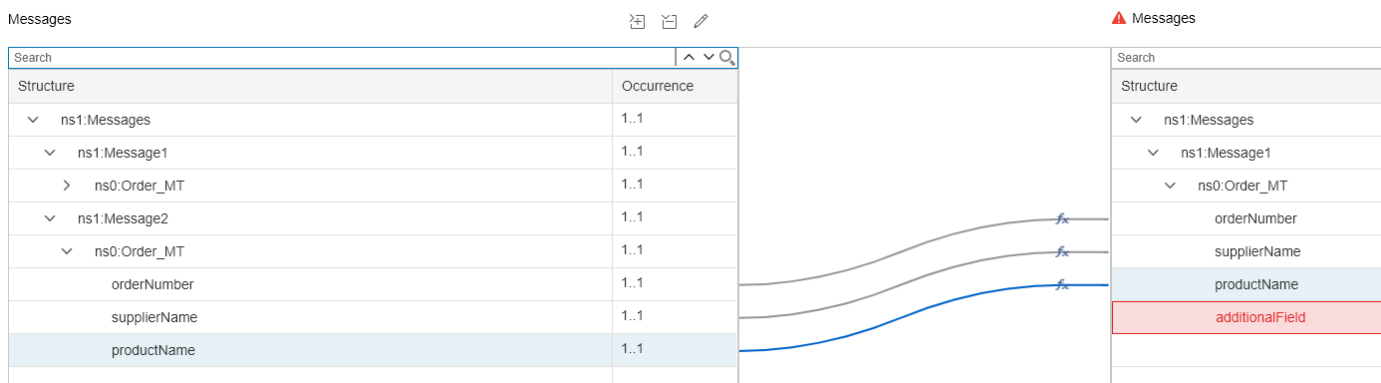
Source Messages

Target Messages

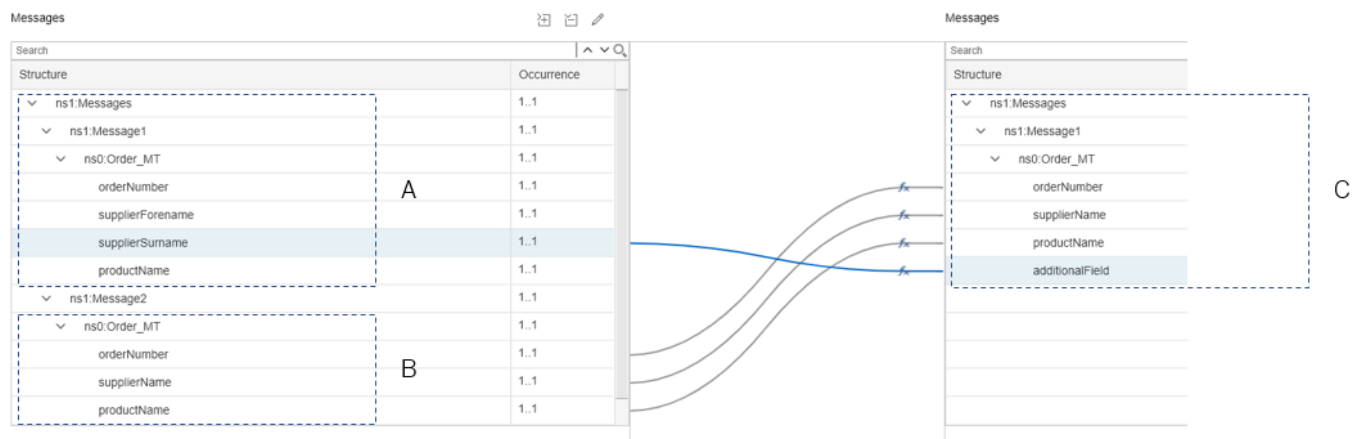
Messages	Occurrence	Actions
Order_MT http://cpi.sap.com/demo	1..1	
Order_MT http://cpi.sap.com/demo	1..1	

Messages	Occurrence	Actions
Order_MT http://cpi.sap.com/demo	1..1	

- 7. Choose **OK** to go back to the mapping editor.
- 8. In the source message structure on the left side, open the nodes under **ns1:Message2** (the lower part of the source structure), and on the right side open the nodes of the target structure. Then, connect each field of the source structure to the field with the same name in the target structure. Leave out the field **additionalField** in the target structure.



- 9. Now, open the upper part of the source structure and connect the field **supplierSurname** in the source structure with the field **additionalField** in the target structure.
- The figure below shows the final mapping and also indicates which parts are associated with the messages A, B, and C.



10. Finally, to apply the mapping function for the connection of the fields `supplierSurname` (source structure) and `AdditionalField` (target structure), open the mapping expression **Text** `toLowerCase`. Connect the source and target field in the following way: `supplierSurname` with `toLowerCase` and connect the latter with `AdditionalField`.
11. Choose **OK** to return to the integration flow model.
12. Save and deploy the integration flow.

Executing the Scenario

To execute the scenario, you need to install a SOAP client (for example, SoapUi) and configure it accordingly. You need to configure the SOAP client so that it calls the endpoint address of the *standard* integration flow.

More information: [Set Up the SOAP Client and Start Message Processing](#)

To define the inbound message (sent by the SOAP client), upload the file `A.wsd1` in the SOAP client. When the SOAP client has been configured accordingly, and the message structure is shown, you can enter any values for the elements, for example:

Sample Code

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/" xmlns:demo="http://cpi.sap.com/demo">
  <soapenv:Header/>
  <soapenv:Body>
    <demo:Order_MT>
      <orderNumber>1</orderNumber>
      <supplierForename>WALTER</supplierForename>
      <supplierSurname>SMITH</supplierSurname>
      <productName>Notebook</productName>
    </demo:Order_MT>
  </soapenv:Body>
</soapenv:Envelope>
```

1. Configure the standard integration flow (open the standard integration flow and choose **Configure**).
2. Specify a value for the **Address** parameter of the **ProcessDirect** adapter (or leave the default value as it is).

Configure "Customer Extension Standard Integration Flow"

Receiver
More

Receiver:

Adapter Type:

Connection

Receiver

ProcessDirect

customflow

3. Go to the **More** tab and leave the default value for the `custom_extension_enabled` parameter as `false`.

Receiver **More**

Type: All Parameters

custom_extension_enabled: false

4. Choose **Save** and then **Deploy**.

5. Choose the **Operations** view of the Web UI and under **Manage Integration Content** check the deploy status of your standard integration flow.

6. As soon as the integration flow deployment has finished (status **Started**), copy the endpoint address to your clipboard and paste it into the address field of your SOAP client.

7. Now configure and deploy the custom integration flow. Make sure that you configure the same ProcessDirect adapter **Address** parameter as in the standard integration flow.

When the custom integration flow is also deployed, trigger message processing from your SOAP client as explained in [Set Up the SOAP Client and Start Message Processing](#).

The SOAP client should receive a response where the standard mapping has been applied to the original message.

The fields `supplierForename` and `supplierSurname` of the original message have been concatenated in the field `supplierName`.

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" ...>
  <soap:Header/>
  <soap:Body>
    <ns0:Order_MT xmlns:ns0="http://cpi.sap.com/demo">
      <orderNumber>1</orderNumber>
      <supplierName>WALTER / SMITH</supplierName>
      <productName>Notebook</productName>
    </ns0:Order_MT>
  </soap:Body>
</soap:Envelope>
```

8. Repeat all these steps, but with the externalized parameter `custom_extension_enabled` of the standard integration flow specified as `true`.

9. The SOAP client should receive a response in which the custom mapping has been applied.

The field `additionalField` is available and contains the content from the original field `supplierSurname` (but uppercase characters have been transformed to lowercase characters).

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" ...>
  <soap:Header/>
  <soap:Body>
    <ns0:Order_MT xmlns:ns0="http://cpi.sap.com/demo">
      <orderNumber>1</orderNumber>
      <supplierName>WALTER / SMITH</supplierName>
      <productName>Notebook</productName>
      <additionalField>smith</additionalField>
    </ns0:Order_MT>
  </soap:Body>
</soap:Envelope>
```

Mapping Extension Step by Step (Example from SAP Hybris C4C)

This section explains how to do mapping extensions based on a real example from the product area SAP Hybris Cloud for Customer (C4C).

i Note

This section is targeted at the customer who is doing a mapping extension.

i Note

For more background information regarding this example, check out the following blog in SAP Community: [Extending standard integration flow to support Customer extensions](#) 🙌.

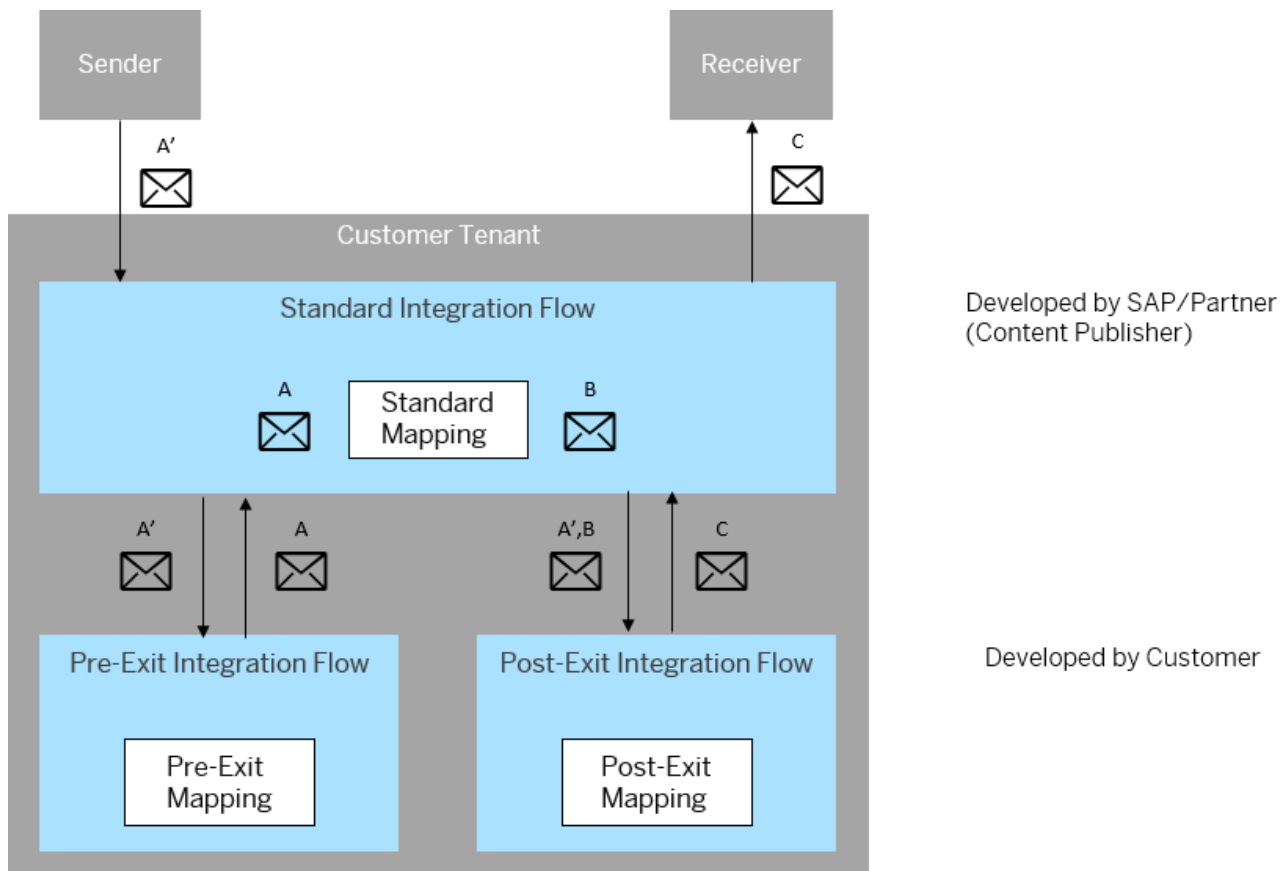
Let's assume that you want to do a mapping extension that also includes the extension of the source message structure. This means that both a pre-exit and a post-exit are involved. The integration flows discussed in this section have the same structure as those explained in [Integration Flow Extension - Concepts](#).

We also assume the following:

- The sender is an SAP ERP system sending an IDoc message to Cloud Integration. An IDoc adapter is used as the sender channel for the standard integration flow.
- The receiver is an SAP Cloud application that expects a SOAP message (through the SOAP adapter).
- You also need to extend the *source* IDoc message. To support this use case, the standard integration flow contains a pre-exit. The pre-exit step will then enable you to design a corresponding pre-exit integration flow in order to map the extended *source* message to the original message structure. The message processed by the pre-exit integration flow can then be consumed by the standard mapping.

You need to create two integration flows, one for the pre-exit mapping and another one for the post-exit mapping. We explain the required steps to extend the mapping and to connect the custom integration flows with the standard integration flow.

The following figure shows the overall message flow between the standard integration flow and the pre-exit and post-exit integration flows, including sender and receiver.



The following message structures are involved in the extension scenario:

- Message A
Contains the original source message structure that is defined by the requirements of the sender system and can be consumed by the standard mapping.
- Message A'
Contains an extended source structure (defined by the customer-specific requirements with regard to the sender system).
Typically, the customer derives message structure A' from message structure A by adding additional fields. In our example, the sender system is an SAP ERP system that sends an IDoc to SAP Cloud for Customer.
Messages A and A' are IDoc messages.
- Message B
This is the target structure of the standard mapping. In other words, the standard integration flow maps message A to message B.
In order to enable the standard mapping to consume the inbound message in the customer-specific scenario, message A' first has to be mapped to the original message structure A, which is defined in the standard integration flow. For this purpose, the message is passed to the pre-exit integration flow and mapped to message A, which can then be mapped within the standard integration flow.
- Message C

This is the message that results from the post-exit mapping.

Note that the letters A, A', and so on correspond to the notation as used in the general concept description in [Integration Flow Extension - Concepts](#).

Related Information

[Prerequisites](#)

[Creating the Pre-Exit Integration Flow](#)

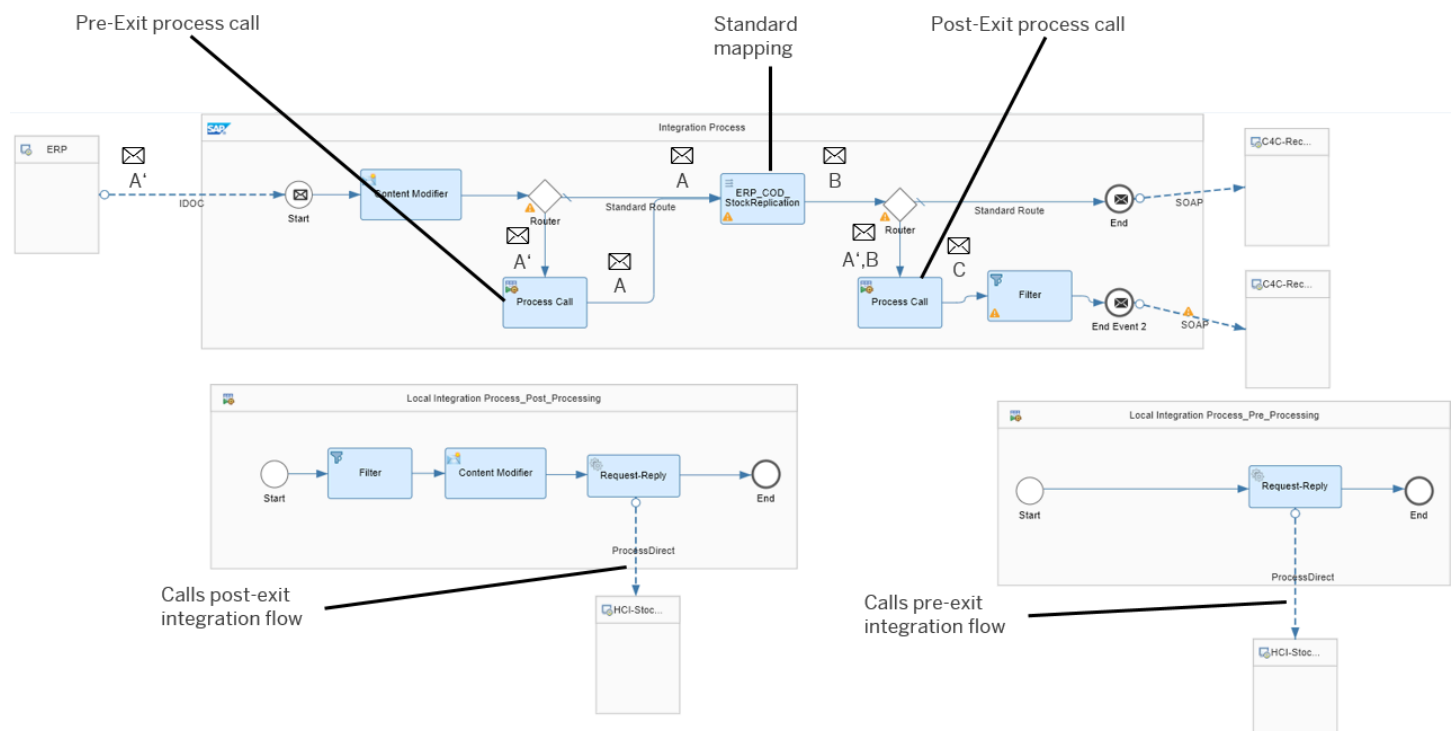
[Creating the Post-Exit Integration Flow](#)

[Connecting Pre-Exit and Post-Exit Integration Flows with the Standard Integration Flow](#)

Prerequisites

Standard Integration Flow with Customer Exits

The standard integration flow (provided as part of a predelivered integration content package) with both a pre-exit and a post-exit has the following design. The message flow as explained above is also depicted.



Required Integration Flow Resources (WSDLs)

Prerequisite: You have copied the integration package with the standard integration flow into your own workspace.

We also assume that you, the customer, have already defined the source message structure (the extended IDoc structure A') and the target message structure that depends on the intended final mapping (this is referred to as message C).

i Note

The WSDL files defining the message structures A and B (source and target message of the standard mapping) are contained as part of the standard integration flow (from the predelivered integration package).

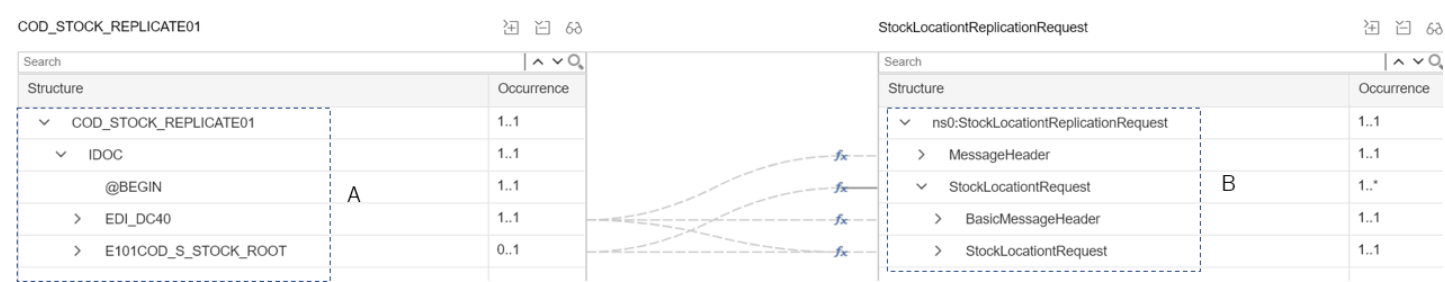
You can get these by deploying the standard integration flow (from the predelivered integration package) on your customer tenant, opening the standard integration flow, clicking in the area outside the (local) integration process shapes, and selecting the [Resources](#) tab. Here, you will find the related WSDL files and you can download them.

The following figure shows the resources as uploaded to the standard integration flow:

Resources (4)		
Name	Type	Actions
▼ Mappings (1)		
ERP_COD_StockReplication	Message Mapping	
▼ Schemas (3)		
COD_STOCK_REPLICATE.COD_STOCK_REPLICATE01	WSDL	↓
DOC_INTF_II_STOCK_LOCATION_REPLIC_IN	WSDL	↓

- COD_STOCK_REPLICATE.COD_STOCK_REPLICATE01.wsdl describes the original source structure for the standard mapping, which is an IDoc message (message A).
- DOC_INTF_II_STOCK_LOCATION_REPLIC_IN.wsdl describes the target structure of the standard mapping (message B)

The standard integration flow contains a mapping that transforms message A to message B. The following figure shows this mapping as displayed in the mapping editor:



Note
You must **not** change the standard integration flow.

Creating the Pre-Exit Integration Flow

Creating the Pre-Exit Mapping

Let's assume that you want to extend the source structure by additional fields.

1. Extend the source structure of the message (to get message A').

You want to add additional elements to the source message, for example, the following element: Z101YANEXT2.

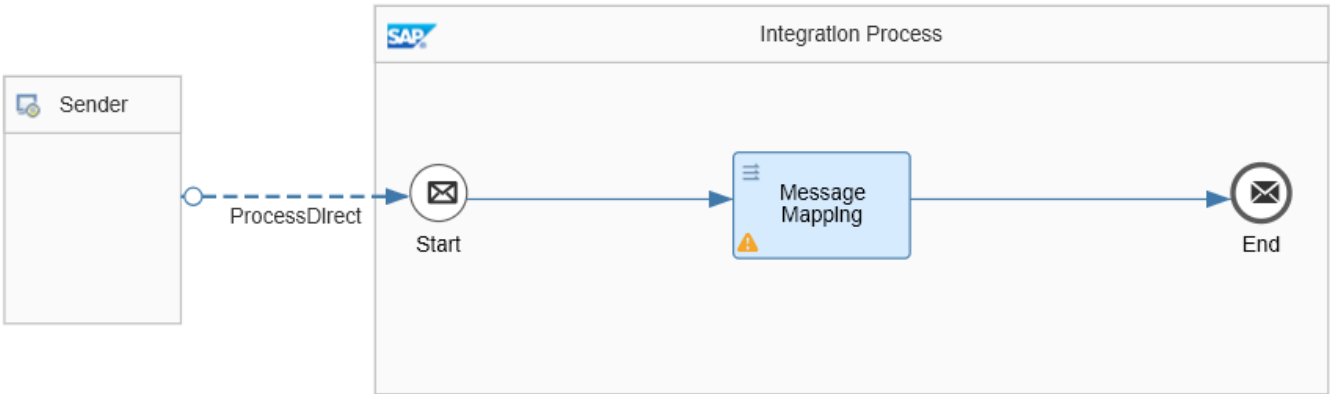
Edit the WSDL file for message A (COD_STOCK_REPLICATE.COD_STOCK_REPLICATE01.wsdl) accordingly (using the WSDL for message A from the [Resources](#) tab of the standard integration flow).

Save the new WSDL file under any name (for example, YANCOD_STOCK_REPLICATE01.wsdl).

2. Create a new integration flow.

We recommend that you add this new integration flow to the package that contains the standard integration flow (both have to be deployed on the same tenant).

3. Model the integration flow according to the following figure:



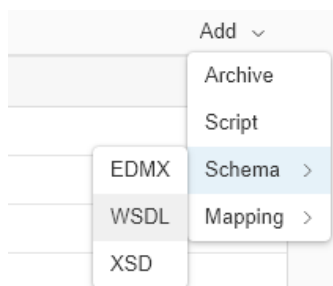
4. When connecting the sender shape with the message start event, select the [ProcessDirect](#) adapter type.

Go to the **Connection** tab of the ProcessDirect adapter and as **Address** enter any value starting with a slash (/), for example `/preexit_flow`.

5. In the integration flow editor, click somewhere outside the **Integration Process** shape and select **Resources**.

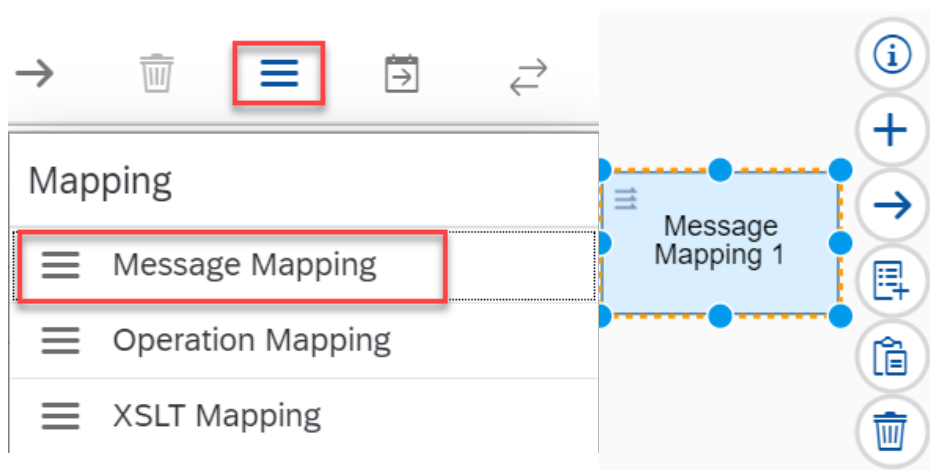
6. Choose **Add** **Schema** **WSDL** and browse to the WSDL file defining the source message (message A', this is file `YANCOD_STOCK_REPLICATE01.wsdl`).

In the same way, add the WSDL file for message A to the integration flow (file `COD_STOCK_REPLICATE.COD_STOCK_REPLICATE01.wsdl`).



7. Repeat these steps for the target message (here, add the WSDL file for message A). Note that the pre-exit steps map the extended source message A' to the original message A. As mentioned above, you get the resource for message A from the standard integration flow, whereas the resource for message A' depends on the specific enhancements of the source structure. Typically, it is similar to message A, but contains additional custom fields.

8. In the integration flow model, select the **Message Mapping** shape from the palette (under **Mapping**) and place it in the model between the message start and message end event.



9. Choose the **+** icon next to the message mapping shape and enter a name for the message mapping (for example, `pre_exit_mapping`).

Create Message Mapping

Name:

Create Cancel

10. Choose **Create**.

11. The graphical mapping editor is opened.

12. Choose **Add source message** and browse to the WSDL file of the source message (message A'). Choose **Upload from file system**.

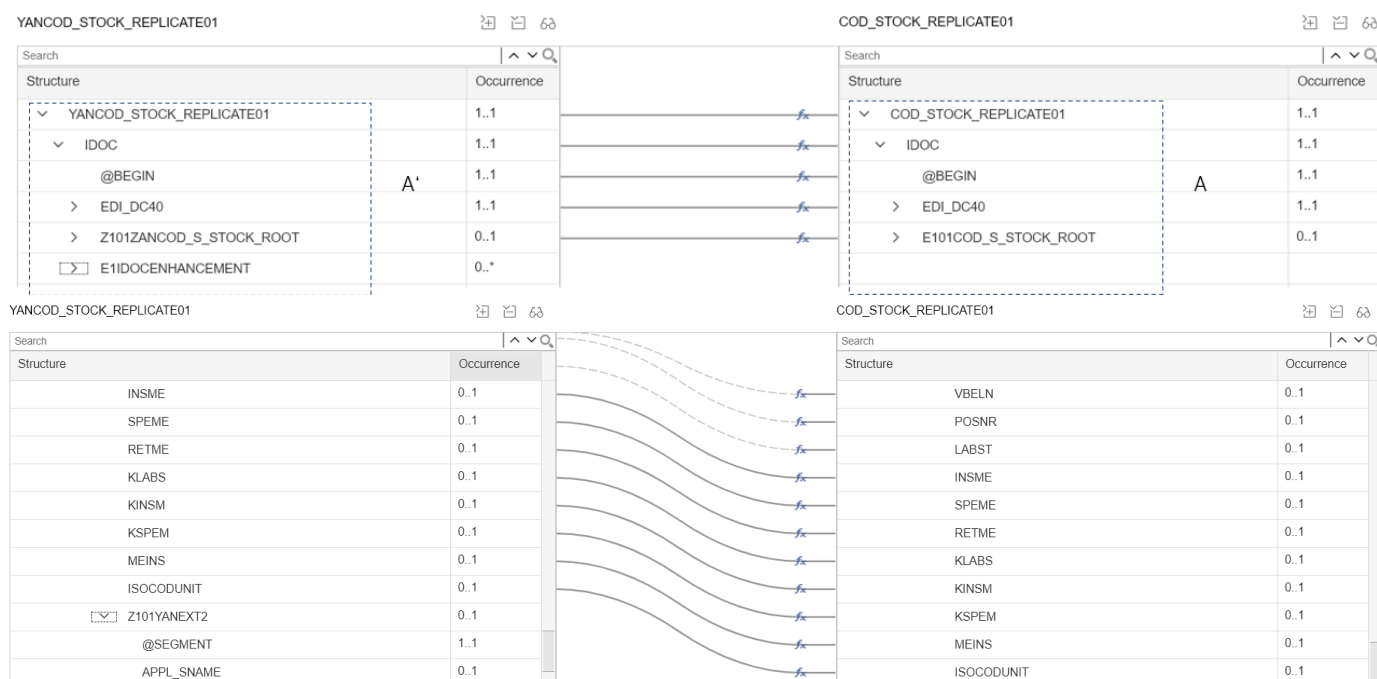
Add source message

Structure	Occu...
No data	

Add target message

Structure	Occu...
No data	

13. Choose [Add target message](#) and repeat the steps with the WSDL file for the target message (message A).
14. Connect all fields that are identical in the source and target mapping without assigning any mapping function. Do **not** connect the additional fields (which have no equivalence in the target message A) to any target field.



15. Save and deploy the integration flow.

Creating the Post-Exit Integration Flow

Creating the Post-Exit Mapping

Create another integration flow with the same shapes as in the pre-exit integration flow (also within the same integration package). Make the following specific settings:

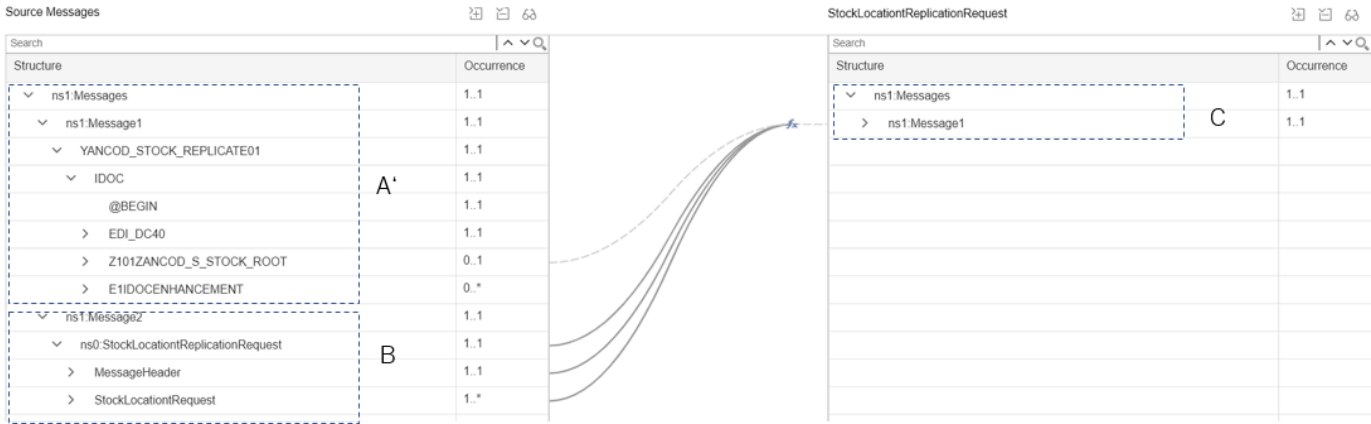
- In the **Address** field of the ProcessDirect adapter, enter a string that is different to the one in the pre-exit integration flow, for example `/postexit_flow`.
- Click in the area outside the integration process shape, go to the **Resources** tab, and upload the WSDL files for the messages A', B, and C.

Note that you get the resource for message B from the standard integration flow, whereas the resource for message C depends on the specific custom enhancements related to the target structure. Typically, message C is similar to message B, but contains additional fields.

- Configure the message mapping in the following way:

For the source message, upload both resource files for messages A' and B. For the target message, upload the resource file for message C.

You can proceed in the same way as described for the demo example in [Creating the Custom Mapping](#).



You will notice that the source message consists of two messages, A' and B.

Connect **all fields** in the structure below Message2 with the corresponding fields in the target structure.

- Define connections **between selected (additional) fields of the extended original message A'** (in the structure below Message1) and selected fields in the target message C (typically, using fields that have been added in the target structure).

Connecting Pre-Exit and Post-Exit Integration Flows with the Standard Integration Flow

Connecting Custom Integration Flows with the Standard Integration Flow

To make sure that the standard integration flow and post- and pre-exit integration flows are processed in conjunction as expected, make sure that the ProcessDirect adapter addresses match in the right way.

The ProcessDirect adapter address of the pre-exit integration flow must match the ProcessDirect adapter address in the pre-exit of the standard integration flow.

The same applies for the ProcessDirect adapter address of the post-exit integration flow and the corresponding ProcessDirect adapter address in the post-exit of the standard integration flow.

Working with Custom Adapters

Importing Custom Integration Adapter, Cloud Foundry Environment

Use the integration-adapter artifact type to import custom integration adapter in your integration package.

Prerequisites

i Note

This information is relevant only when you use SAP Cloud Integration in the Cloud Foundry environment.

- You've developed a custom integration adapter using Adapter SDK. For more information, see [Developing Custom Adapters](#).
- You've exported integration adapter content, which is essentially an *.esa file in your local file system.
- You've assigned the role template [WorkspacePackagesEdit](#) and [WorkspaceArtifactsDeploy](#) that are required to accomplish the various tasks when working with custom integration adapter. For more information, see [Tasks and Permissions](#).
- You've created an integration package. For more information, see [Create an Integration Package](#).

Context

Cloud Integration can connect to a multitude of systems in the cloud and on-premise. It already comes with an impressive number of adapters that allow communication with SAP and Non-SAP systems, security standards, and business-specific requirements.

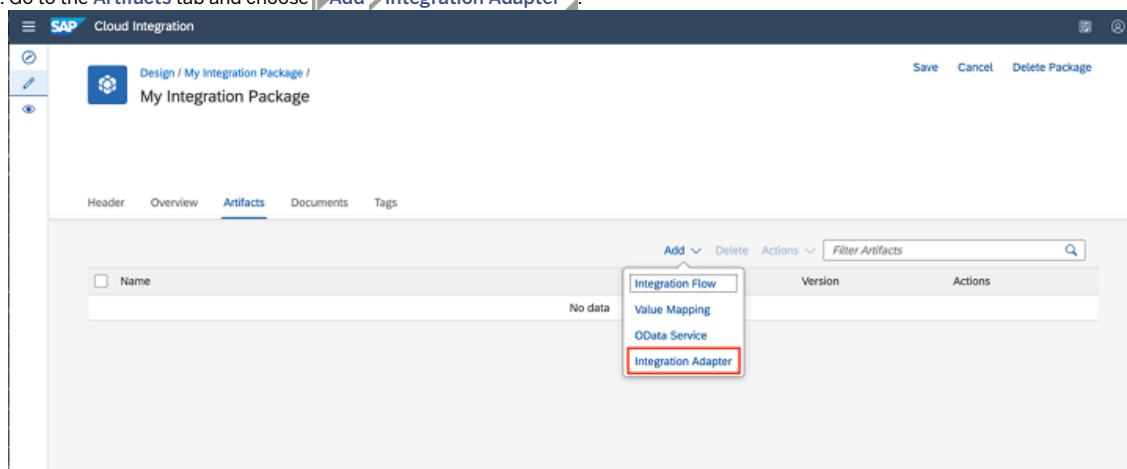
Custom integration adapter eases the integration of a business based on your requirements and it has the following lifecycle operations:

Design Time Operations	Monitoring view Operations
The adapter is available for modeling in the integration flow once the custom adapter is imported as an artifact in the integration package.	You can monitor the status of deployed custom integration adapter. The status details show the status of the integration adapter with regard to its usage at runtime.
The version history shows the versions of the adapters that are maintained in the design workspace.	You can Restart , Undeploy , and Download from the Manage Integration Content view.
You can Delete , Download , and Deploy the imported custom integration adapter through Actions . You can also edit via View metadata .	

Procedure

Here's how you can import an integration adapter in Cloud Integration in the cloud foundry environment.

1. Choose [Design](#).
2. Select the integration package in which you want to add a custom integration adapter.
3. Choose [Edit](#) to enable editing of the package
4. Go to the [Artifacts](#) tab and choose [Add](#) [Integration Adapter](#).

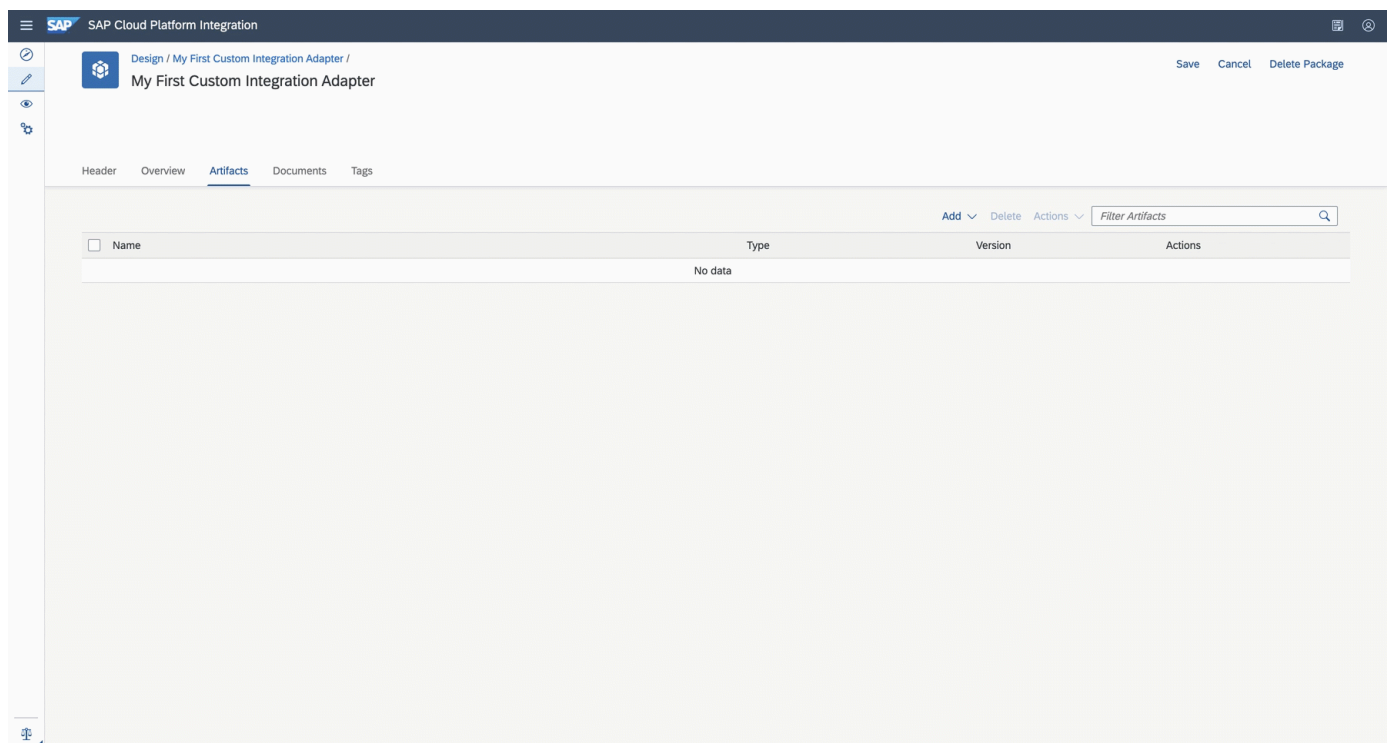


5. Choose [Browse](#) to import integration adapter file (*.esa). [Name](#), [ID](#), and [Version](#) fields get auto filled.
6. In the [Type](#) list, select a type for your adapter.
[Type](#) is used to categorize the adapters based on line of business the adapter is targeted at. For example, Analytics, CRM, ERP, Finance, HCM, Marketing.
7. In the [Application](#) list, select the application for which you're developing the adapter.
[Application](#) refers to the software/application for which the adapter provides either inbound or outbound connectivity. For example, select [Slack](#) when you develop an adapter that provides connectivity to Slack application.
8. Choose [Deploy](#).

i Note

You need to be familiar with the following aspects:

- o The integration adapter [ID](#) needs to be unique across the tenant. When you create a new integration adapter, by default the entry for [Name](#) and [ID](#) is automatically imported from the file.
- o If there's already an integration adapter with the same [ID](#), the system throws an error.
- o To find more information about the integration adapter with the same ID (for example, the integration package name and the integration adapter name), click [Show More](#) below the error message.
- o The integration adapter must be deployed first before deploying the integration flow that consumes the integration adapter.
- o The adapter isn't available for modeling once you delete it from the integration package.
- o Once you undeploy the integration adapter artifact, it's removed from the [Manage Integration Content](#) view.
- o If you're unable to find the imported integration adapter when modeling the integration flow, the browser cache needs to be cleared. To clear browser cache, choose [Empty cache and hard reload](#) from the application browser window.



Deploying an Integration Adapter

You've now completed all steps to develop your adapter project, imported as an artifact in the package, and consumed for modeling an integration flow. To use the adapter in the integration flow for processing, you need to deploy the adapter.

Procedure

1. Open the integration package.
2. Go to the **Artifacts** tab.
3. Select the **Actions** button next to the name of a custom integration adapter.
4. Select **Deploy**.

Simulation of an Integration Flow

The simulation feature allows you to test an integration flow or its subset and see if you can get the desired outcome even before you deploy the integration flow. Based on the simulation result, you can decide to deploy the integration flow or resolve any errors.

Scope of an Integration Flow Simulation

The simulation feature is evolving in numerous directions and incorporating new capabilities with multiple increments of development.

- We allow simulation in the read and edit mode of an integration flow. You can test the integration flow during the development phase itself.
- In edit mode, you can simulate an integration flow without saving the data and settings.

i Note

When you switch from edit to read mode, the simulation output isn't retained.

Benefits of Simulation

This section lists the advantages of using simulation in an integration flow.

- Realistic input for integration developers when designing an integration flow.
- You can simulate dependencies and identify issues at an early stage.
- Any errors identified during the simulation are recorded as a message along with details about how to fix it.

- Improves integration developer productivity and saves time in developing an integration flow.
- The smart upload functionality allows you to upload a zip file as a simulation input payload. The content of the trace message can also be downloaded after simulation.

Elements Supported in Simulation

Category	Elements	Supported Elements
Message Transformers	• Content Modifier • Converter • Decoder • Encoder • Filter • Message Digest • Script	All i Note HTTP calls can't be simulated for the Script flow step.
Call	External Call • Request Reply • Content Enricher • Send	All You can't make the actual call to a receiver system to get the response. However, you can mock the response from the receiver system via receiver adapter. Select the receiver adapter that is connected to Request Reply step and select Add Simulation Response. Provide the payload or headers in the dialog and the same is considered as a response when you run the simulation. If you don't mock the response, the message payload from the previous steps is be considered.
	Local Call • Process Call • Looping Process Call	All
Message Routing	• Aggregator • Join • Router • Splitter • Multicast • Gather	Aggregator isn't supported.
Security Elements	• Decryptor • Encryptor • Signer • Verifier	All
Persistence	• Data Store operations • Persist • Write Variables	You can only simulate the read operation for Write Variables and Data Store.
Validator	XML Validator	XML Validator

Functionalities of Simulation

This section explains the different functionalities and how they work in the simulation tool.

Details of Simulation Functionalities

Functionalities	Icon	Details
Start Point		Place a Start Point at the beginning of your flow path. It feeds a simulation input.
End Point		Place an End Point to end the simulation on your connection or the defined path of simulation.
Add Simulation input		When you choose a Start Point, the Add Simulation Input dialog opens where an integration developer can add the following types of input: • Header • Properties • Body: You can copy and paste your payload content into the body or upload from the file system. Upon uploading the content, if there are any changes required you can edit in the content section and provide the necessary input. • Upload from File system: Upload input payload from the file system. i Note ◦ You can upload not only the standalone zip file, but also individually as headers, properties, and body. ◦ The maximum size of the payload is 1 MB. For more information, see Using Various Types of Body Files in the Simulation.
Add Simulation Response		Allows the integration developer to simulate the response from the receiver adapter connected to: • The Request-Reply step • Read of variable in the content modifier • Read of the data store in the Get element.
Run Simulation		Used to run the simulation once the start and end point have been defined.
Clear Simulation		Removes all of the simulation elements such as the start point, end point, and message processing output.
Message Envelope		When the simulation run is successful, the message envelope appears. You can find these between the start and end points. Choose the message envelope to display the message content.
Download Trace Message Content	Download	After a successful simulation run, do the following: 1. Choose the Message Envelope to open the message content attributes. 2. Choose Download at the right to download the trace message content.

Color Conventions

During the simulation process, certain colors are familiarly associated with particular meanings.

Colors and Their Meanings in the Simulation Tool.

Color Code	Used For	Details
------------	----------	---------

Color Code	Used For	Details
Blue	Start point and Message Envelope	• Start Point: Always remains the same. • Message Envelope: Appears after running the simulation successfully.
Red	End point and Message Envelope	Appears when there are any errors. Choose the respective elements to check error messages.
Black	End point	Normal behavior and no action required.
Green	End point	Appears on successful execution of the simulation.

For detailed steps about using simulation, see [Configure Simulation](#)

Configure Simulation

Use **Simulation** feature to test an integration flow and check the desired outcome even before the deployment.

Prerequisites

i Note

Availability of this feature depends upon the SAP Integration Suite service plan that you use. For more information about different service plans and their supported feature set, see SAP Note [2903776](#) .

- A tenant admin must enable the memory profiling feature on from [Artificial Intelligence](#) settings page.
- If the integration flow contains groovy script step:
 - The Integration flow must be in edit mode as the [optimization](#) inputs for groovy scripts are needed to be opened in the groovy editor.
 - A tenant admin must enable the script optimization feature from Artificial Intelligence settings page. See [Artificial Intelligence](#)

Context

The steps described here explains the process of running a **Simulation** in both read and edit mode of an integration flow.

→ Remember

If your tenant is provisioned in Cloud Foundry Environment, then the input payload size for the simulation must be less than 100KB and the overall output payload must be less than 1MB.

Procedure

1. Choose your package and then select the integration flow.
2. You can see simulation tool on the palette.

The simulation tool includes the following functionalities:-

- a. **Run Simulation:** Use it to execute the simulation process.
 - b. **Clear Simulation:** Use it to clear the previously used simulations. Once you clear the simulation, you cannot undo it.
3. Identify subset of an integration flow that you would like to simulate and choose the connection to locate the **Start Point** and the **End Point** for the simulation.
 4. Choose  as a start point of the simulation on any connection to begin.
 - a. Choose the start point at a connection gives an option to provide the input to simulation and trigger it accordingly.
 - b. A dialog opens to add a simulation input. Provide the necessary details. You can add input that could be **Payload** or **Headers** or **Properties**. You can also upload input payload from your local file system.
 - c. Under AI configuration, select **Enable Memory Profiling** to obtain memory profiling insights after a successful simulation run. Read this [blog](#)  or watch a short [video](#)  for this AI feature.
 - d. Choose **OK** to submit input. For more information, see [Using Various Types of Body Files in the Simulation](#).
 5. Choose  to end the defined simulation flow step.

i Note

- You can't run simulation if you don't define the end point. In this case **Run Simulation** button grayed out.

- Simulation fails if there are any check errors in resources like mapping.
- You can allow maximum of 10 integration flow step per simulation to run. The best practice is to allow lesser than or equal to 10 flow steps for complex integration flows.
- Simulation of looping process call with maximum number of iterations more than 10 is not supported.

6. Choose  to run the simulation. Once the simulation run is successful, a message  envelope appears alongside each flow component with a tracing information. Choosing the envelope you can see the details.

i Note

If you have enabled Memory profiling and the flow step is one of the top memory-contributing flow step, you'll be able to view the memory profiling insights.

If the integration flow contains Groovy script step, choose [Optimize in Editor](#) and then [Optimize](#) to view the recommendations in the script editor. See

7. **(Only if memory profiling is enabled):** Review the reported top memory contributors and the recommendations. Apply the suggested fix to that flow step and re-run simulation to identify other significant memory issues.

8. Choose  to clear the simulation and you can start with fresh flow steps to begin new set of simulation.

i Note

- Changing integration flow mode (Read or Edit) loses the configuration of an integration flow simulation.
- When the simulation is cleared, simulation data is lost.

Using Various Types of Body Files in the Simulation

Use simulation input to upload the payload content. You can simulate Zip and Tar splitters, which require an incoming payload in Zip and Tar format.

Context

You can experience the following behaviour while providing input payload.

- Upload the trace message content in zip comprising of file types such as *.header, *.properties, and *.body.
- Any modification in the contents of the *.body file that has readable contents, for example, xml, json, text, etc., invalidates the file and simulate the modified content.
- If you would like to reset the modified content of the file, you can perform using [Reset](#) option, that brings back the file you uploaded.
- You can upload any other body input files such as *.tar, *.txt, *.xlsx, *.pdf, etc., via [Upload from File System](#).

Procedure

Here's how you can add simulation input while you're working with integration flow simulation.

- Identify a subset of an integration flow that you would like to simulate.
- Choose  *start point* of the simulation on identified connection to begin.
- A dialog opens to add a simulation input. Add the necessary details.
 - You can add input that could be of [Headers](#) or [Properties](#) or [Body](#).
 - You can also upload input payload from your local file system.
- To add [Header](#), choose [Add](#) button and enter the [Name](#) and [Value](#).
- To add [Properties](#), choose [Add](#) button and enter the [Name](#) and [Value](#).
- To upload your local file, choose [Upload from File System](#) that automatically fetches all the values.
- If you have chosen to upload the file, the file appears in the [Body](#) section and the content is displayed.

i Note

- A file with binary content can't be displayed in the body section. The file supports encoding in UTF8 format only.
- You can modify the content by choosing [Edit](#) option.
- If you don't want the modified content to be simulated, select [Reset](#) option and that brings back the uploaded file.

8. Choose [OK](#).

Related Information

[Configure Simulation](#)

Smoke Test Scenario

This is a simple test to verify that your SAP Cloud Integration simulation is working as expected.

Context

In this scenario, you run a simulation without deploying the integration flow and you can check the trace results in the message envelope.

In the course of this exercise, you execute a simulation tool on an integration flow.

i Note

Create an integration flow, add the following details, and save.

- Insert `${in.body}` in the message body of **Content Modifier**.
- Insert XPath Expression as `//root/order` in the processing section of the **Filter**.

Procedure

1. Choose an Integration flow and keep it in a read-only mode.
2. Select the connector line between **Timer** and **Content Modifier** and choose **Simulation Start Point** .
3. Select the connection between **XML to JSON Converter** and **End Message** and choose Simulation End Point .

You can see that the first two icons of the simulation tool have been enabled.

4. We need to provide an input for the simulation to run. You can add a header value or a property or, you can enter a payload in the body. For this test case, let us provide an input payload in the body.
5. Choose the **Simulation Start Point**, a dialog opens to **Add Simulation Input**. Provide an input payload in the **Body** section and choose **OK**. The subset of an integration flow now filters the incoming payload based on the XPath provided in the Filter step.

Use the following payload:

```
<root><order><orderno>0D100</orderno></order></root>
```

6. Select simulation tool and choose **Run Simulation** .
7. You receive a toast message that the simulation completed successfully, the simulation **End Point** turns from black color to green and a message envelope  appears between start and end point.
8. Choose **Message Envelope** located next to **XML to JSON Converter**. A message content opens, check the body section and verify that the input payload has been simulated. Based on the input and the XPath provided, the output message consist of the order details.

You can check that based on the filter criteria, the resulting message has been generated correctly.

9. Choose **Clear Simulation**  located in the simulation tool to clear all defined simulation path.

Deploying Data Flows

Procedure

1. In the **Integration Package Editor**, in the **Artifacts** section, select the required data flow from the list.
2. Choose  to **Deploy**.
3. In the **Settings** dialog box that appears, enter the following details:

- Data Center URI
- Application Name
- Organization Name

Once you enter the above mentioned details, the application forms the Data Service URL.

4. Choose **OK**.
5. In the **LOG ON** screen that appears, enter the organization name.
6. Choose **Log On**.

The Data Integration application opens its **Projects** tab.

This is custom documentation. For more information, please visit [SAP Help Portal](#).

7. In the dialog box that appears, enter the following details as required:

- Name
- Source
- Target

8. Choose **OK**.

The newly deployed data flow appears as an entry in the **Projects** tab page. If required, you can switch back to the SAP Cloud Integration Web application.

i Note

To switch to SAP Cloud Integration Web Catalog view from Data Integration using the **Back Button** in the browser, perform the required substeps:

- If you use Internet Explorer, from the context menu of **Back Button**, select Cloud Integration.
- If you use Google Chrome, click twice on **Back Button**.
- If you use Mozilla Firefox, from the context menu of **Back Button**, select Cloud Integration.

If you want to switch to SAP Cloud Integration Web **Catalog** view from Data Integration using the **Forward Button** in the browser then by default, it displays the **Projects** tab page without the Settings option. This behavior remains the same for Internet Explorer, Google Chrome, and Mozilla Firefox.

If you do not have authorization to access the Data Integration application then you get the following error messages:

- Error message for Google Chrome and Mozilla Firefox:
User is not included in the organization. Contact your security administrator for assistance.
- Error message for Internet Explorer:
HTTP 404: Not found error

Unlocking Integration Packages and Artifacts

When you edit an integration package or an artifact, it is locked and cannot be edited by other users simultaneously. In case you close your browser without saving the changes or there is a session timeout, the package or artifact remains locked, until you unlock it.

Unlocking Packages or Artifacts Locked by You

1. Choose **Design**, and select the locked integration package or artifact.
2. Choose **Edit**.
3. Select **Yes** when you are prompted to close the previous session.

The package or artifact is now unlocked and you can continue editing it.

i Note

If the browser is closed accidentally or the session times out before you could save your changes, the unsaved changes will be lost.

Unlocking Packages or Artifacts Locked by Another User

If the integration package or artifact you want to edit is locked by another user, either request the other user or reach out to your tenant administrator to get them unlocked. For more information, see [Design-time Artifact Locks](#).

Develop API-Based Integration Artifacts

Develop OData API-based integration scenarios, deploy, and manage your artifacts in one place.

You can create integration artifacts of types OData APIs. The API-based artifacts generate a default integration design that you can further extend to meet your requirements. After you complete your integration design, you can save and deploy the artifacts. Finally, you can create your own tiles in the monitoring section to view your API-based artifacts that are deployed. For more information, see [SAP Cloud Integration – Step towards Building API Based Integrations](#).

[Create and Deploy an OData API Artifact](#)

Use this procedure to create and deploy an OData API artifact.

Create and Deploy an OData API Artifact

Use this procedure to create and deploy an OData API artifact.

Context

An OData API artifact creates an OData API-based integration scenario with the OData V2 sender adapter.

Create an OData API Artifact

Procedure

1. Choose **Design**.
2. Select the integration package where you want to create an SOAP API artifact and choose **Edit**.
3. In the **Artifacts** tab, choose **Add > API**.
The **Create API** wizard opens.
4. Select **Cloud Integration** from the list of available runtime profiles. For more information, see [Runtime Profiles](#). Choose **Next**.
5. Choose **OData API > Next**.
6. In the **Select A Method** tab, choose **Create Using Template**.
By choosing **Create Using Wizard** or **Upload**, you start creating an OData API project. See [Developing an OData API Project](#).
7. Enter the name and description for the OData API.
ID and Runtime Profile are uneditable fields.
8. Choose **Create**.
The artifact is added to the integration package and the API artifact opens up in the editor.
9. Choose **Edit**.
A palette appears on top of the integration flow model that provides access to all integration flow steps that you can add to the model. See [Overview of Integration Flow Editor](#).
10. Make your changes and choose **Save**.
11. Choose **Deploy** when your artifact is ready for execution.

i Note

The URL of the OData API endpoint is derived from the name of the OData API artifact.

Next Steps

You can check the status of all your integration artifacts in the **Monitor** view. By default, API-based artifacts don't appear in the **Monitor**. Perform the steps that follow to view the deployed artifact.

View the deployed OData API Artifact

Procedure

1. Choose **Monitor**.
2. Under **Manage Integration Content**, add a new tile.
3. In the **Tile Settings** dialog, do the following:
 - a. Select **All** from the **Status** list.
 - b. Select OData API from the **Type** list.
 - c. Choose **OK** to create a tile.
4. Open the tile to view your artifact and its status.

Next Steps

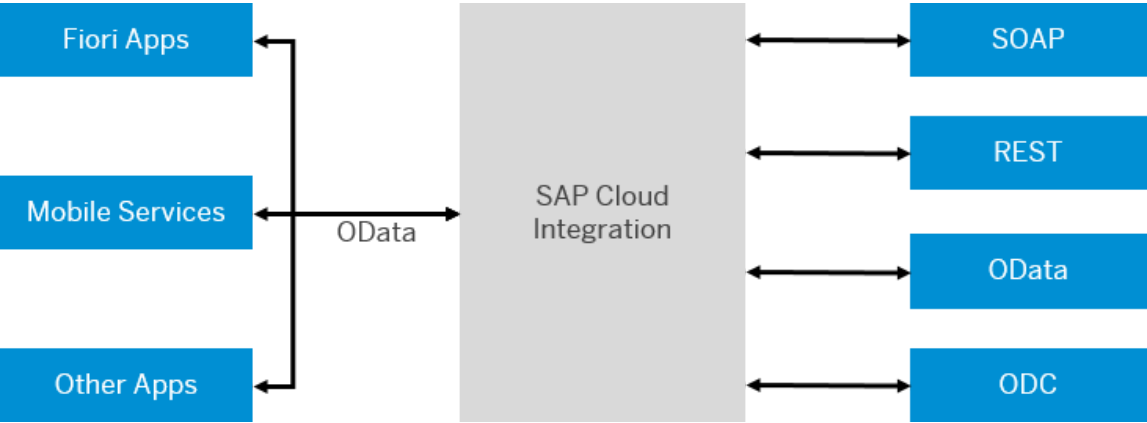
Repeat the same process to create a tile under **Monitor Message Processing** to view the status of your processed messages.

Developing an OData API Project

→ **Remember**

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#).

You can develop OData APIs that expose existing data sources, such as SOAP, as OData endpoints. These OData APIs can be consumed by SAP Fiori apps, SAP BTP Mobile Services, or any other custom app, to implement user-centric scenarios.



Differences Between OData API and Integration Projects

When deciding whether to use OData APIs or integration projects to implement your business scenario, keep the following differences in mind:

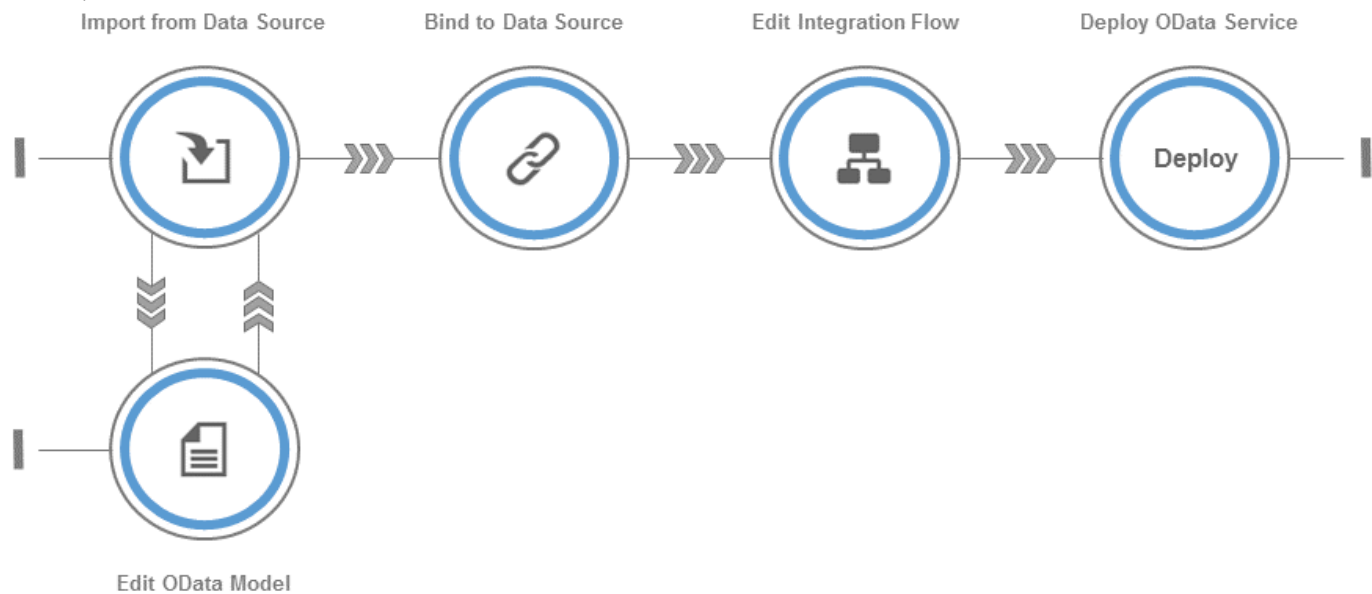
OData APIs	Integration Projects
An OData API can expose multiple entities and multiple associated operations at the same time.	An integration project can expose only one entity and one associated operation at a time.
You can configure any application to consume an OData API. As a result, any number of users with access to these applications can consume the OData API.	In an integration project, the credentials of a limited set of users are used to authenticate the integration scenario between two systems.

Service Development Process

As a service developer, you can create and deploy an OData API without going through the time-consuming process of writing code. You start off the process by creating an OData API artifact in an integration package. For more information, see [Creating an OData API](#). Cloud Integration enables you to further develop the OData API with the following steps:

1. Import an OData model using simple modeling steps. If you prefer writing code, Cloud Integration also lets you handcraft an OData model or edit an existing one using an editor.
2. Connect the OData objects to a data source through integration flows.
3. Tweak the integration flows to enhance business logic.
4. Deploy and monitor the OData API.

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.



Please note that image maps are not interactive in PDF outputs.

i Note

- You can deploy OData API artifact for [SAP Cloud Integration](#) and [SAP Cloud Integration OEM](#) runtime profiles only.
- The OData API artifacts cannot be Externalized.

Authorization

[User Role](#) is the default authorization used in OData APIs. Assign the `ESBMessaging.send` role to users in order for them to access the OData APIs.

Related Information

[Supported OData Features](#)

Supported OData Features

→ **Remember**

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#) .

The following operations are supported when developing an OData API in the Cloud Integration web application:

OData Operation	Description
Create	Maps to HTTP POST method
Read	Maps to HTTP GET method
Update	Maps to HTTP PUT method
Delete	Maps to HTTP DELETE method
Query	Maps to HTTP GET method
Function Imports	Custom operations that can be invoked through HTTP GET or POST method. i Note Currently, Cloud Integration supports only function imports that can be invoked through HTTP GET method.

The OData API supports the data types for the uploaded metadata file (EDMX) defined [here](#)  except:

- Null
- Edm.Single

The following system query options are supported when calling an OData API developed in the Cloud Integration web application:

OData System Query Option	Description
\$select	Specifies a subset of properties to return.
\$top	Determines the maximum number of records to return.
\$skip	Sets the number of records to skip before it retrieves records in a collection.
\$filter	Specifies an expression or function that must evaluate to true for a record to be returned in the collection.
\$count	Returns the number of records in a collection.
\$orderby	Determines which values are used to order a collection of records.
\$expand	Specifies that related records must be retrieved in line with the record or collection being retrieved. For example, you can use \$expand to retrieve a customer and all orders placed by that customer in a single query.
\$inlinecount	Specifies that the response must contain a number of records in the collection of entries identified by the resource path section of the URI. Message property must be an Integer value. This value is returned in the response.
\$format	Specifies that a response to the request must be in the format specified by this query option.

i Note
The system query options \$select and \$count are supported out of the box. You do not have to write a script to enable functionality for these system query options.

i Note
Server side pagination support:

To enable serverside pagination with next link in response, set the message header skipToken with a value for the skiptoken.

🔗 Example
skipToken : 20 adds the next link in response as <link href="Products?\$skiptoken=20" rel="next"/>

Related Information

[Developing an OData API Project](#)

Creating an OData API

Use this procedure to create an integration package that contains an OData API artifact.

Context

→ **Remember**
This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#) .

Procedure

1. Choose **Design**.
2. Choose **Create**.
3. Enter the required data for the integration package. See [Creating an Integration Package](#).
4. In the **Artifacts** tab, choose  **Add**  **OData API** .
5. In the **Add OData API** dialog box, choose one of the following options:

- [Create Using Wizard](#) – to create a new OData API artifact that you can further develop based on your requirements.
- [Upload](#) – to create a new OData API artifact by uploading the necessary resources from your local file directory.

i Note

The [Create Using Template](#) option is targeted for low code API development.

6. Enter the OData API details as listed in the following table.

Field	Description
Name	Name of the OData API.
Namespace (for Create Using Wizard)	Logical grouping of the OData API. Default is <i>SAP</i> .
ID (for Create Using Wizard)	Uniquely identifies the OData API and is a combination of the name and namespace of the OData API in the format <name>_<namespace>_1. This field isn't editable.
Description	Description of the OData API and its purpose.
OData API (only for uploading the artifact)	Upload a ZIP file containing the OData API artifact.
OData Version	Currently, OData V2 is the only supported OData version.

i Note

You can edit some of the OData API details such as Name, Description, and Namespace after the service is created. For more information, see .

7. Choose [Create](#).

You can further develop the OData API by:

- [Creating an OData mode](#)
- [Binding to a data source](#)
- [Editing an Integration Flow](#)

8. Deploy the OData API. For more information, see [Deploying an OData API](#).

9. Save the integration package.

After the iFlow template generation, ensure that the Sender channel is always of type *OData*; for the other types, the tracing functionality won't capture the details of the sender.

Related Information

[Packaging Integration Content in Cloud Integration](#)

Create an OData Model

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#)



You can create an OData model in SAP Cloud Integration by:

- Importing from a data source
- Editing a model from the ground up

Cloud Integration gives you the flexibility to create an OData model in an iterative fashion. For example, you can create an OData model by importing the model definition from a SOAP WSDL file. You can add to this OData model by editing it. Or, you can add to the model definition by importing from a different WSDL file or even a different data source, for example, an EDMX file.

Related Information

[Import from Data Source](#)

[Editing an OData Model](#)

Import from Data Source

Cloud Integration allows you to create an OData model by importing the model definition from an existing data source, such as SOAP and OData.

i Note

When an OData API is generated not all attributes are copied from source metadata. However, once the OData API is generated, you can configure these attributes as needed in the OData Model Editor.

An import wizard guides you in this process with the following steps :

1. Select a data source type.
2. Select a structure.
3. Verify and finalize the EDMX structure.

Related Information

[Importing from SOAP](#)

[Importing from OData](#)

[Importing from ODC](#)

Importing from SOAP

Context

You have created an OData API artifact in an integration package. You can use this procedure to create an OData model or build up an existing one by importing the model definition from SOAP web services.

Procedure

1. If you are creating a new OData API artifact, proceed to the next step. If you are editing an existing OData API artifact, choose [Edit](#) on the Service Designer page.
2. Choose [Import Model Wizard](#) (🔗).
3. In the [Select Data Source Type](#) page, select [SOAP](#) as data source type.
4. Choose [Browse](#) and select the WSDL file from your file system. You can also select the WSDL from the list of existing WSDL files (if any) used in the OData API artifact.

i Note

Composite WSDLs that are flattened are supported. However, multipart WSDLs and other file types are not supported.

5. Choose [Step 2](#). In the [Select Structure](#) page, you can see the elements from the WSDL listed in a table.
6. Select elements that you want to add to the OData model from the [WSDL Hierarchy](#) column. You can expand or collapse WSDL elements to view the hierarchy better.
7. Deselect any child element that you do not want to include in the OData model. You can see that the corresponding parent elements are in a mixed state (neither checked nor unchecked), if they have other child elements that are selected.

i Note

While importing from any data source using the [Import Wizard](#), if the element is of complex type containing a single child element of simple type, the option to deselect only the child element is not supported. In such a scenario, you can select [Flatten](#) from the [Import As](#) dropdown for the parent WSDL element. After completing the [Import Wizard](#), navigate to the [OData Model Editor](#) and modify the structure as required.

8. Select the OData type that each WSDL element has to be converted to in the OData model. To see options available for various types of WSDL elements, refer the following table:

WSDL Type	OData Type
Operation	Complex Type
	Entity Type
	Function Import
Complex Type	Complex Type
Input Message	Complex Type

WSDL Type	OData Type
Output Message	Complex Type
Part	Complex Type

i Note

You can select **Flatten** from the **Import As** dropdown for all WSDL types other than **Operation**. **Flatten** is not an OData type that a WSDL element can be converted to. It is an instruction to the system to skip importing the specified WSDL element into the OData model without affecting the import of its children in the hierarchy.

9. Choose **Step 3**. You can review the mapping of WSDL elements to OData objects in this step.
10. In the **Review and Finish EDMX Structure** page, select a primary key from the properties of entity types.
11. Verify and perform the following additional changes, if required, to the OData model definition:
 - Rename the OData objects listed under **EDM Name**.
 - Select a different EDM type.
 - Update the **Documentation** column.
 - Restore default definition of the OData model by choosing **Restore Default Values**.
12. Once you have completed all the changes, choose **Finish**. The Service Designer page displays the entity sets and function imports from the OData model in a table.
13. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Importing from OData

Context

You have created an OData API artifact in an integration package. You can use this procedure to create an OData model or build up an existing one by importing the model definition from an existing OData API.

Procedure

1. If you are creating a new OData API artifact, proceed to the next step. If you are editing an existing OData API artifact, choose **Edit** on the Service Designer page.
2. Choose  (*Import Model Wizard*).
3. In the **Select Data Source Type** page, select **ODATA** as data source type.
4. Choose **Browse** and select the EDMX file from your file system. You can also select the EDMX from the list of existing EDMX files (if any) used in the OData API artifact.

metadata.edmx is the file used internally in Cloud Integration Web application to store the OData model definition of the OData API artifact.

i Note

- The OData Model Editor supports only OData SAP V2 annotations.
- Only OData models up to 300 KB in size are supported.

! Restriction

Importing of EDMX files with navigation entities are not supported and also there is size restrictions on EDMX files with large number of complex entities. For more information, see the KBA [2844323](#)  and read the blog on [When to use an OData API](#) .

5. Choose **Step 2**. In the **Select Structure** page, you can see entity types and function imports from the EDMX listed in a table. The table also displays the EDM type, primary key and documentation information associated with the OData objects.

i Note

While importing from any data source using the **Import Wizard**, if the element is of complex type containing a single child element of simple type, the option to deselect only the child element is not supported. In such a scenario, you can select **Flatten** from the **Import As** dropdown for the parent WSDL element. After completing the **Import Wizard**, navigate to the **OData Model Editor** and modify the structure as required.

- 6. Select OData objects that you need to add to the model from the [OData Hierarchy](#) column. You can expand or collapse OData objects to view its properties better.
- 7. Choose [Step 3](#).
- 8. In the [Review and Finish EDMX Structure](#) page, verify and perform the following additional changes, if required, to the OData model definition:
 - Rename the OData objects listed under [EDM Name](#).
 - Select another primary key.
 - Select a different EDM type.
 - Update the [Documentation](#) column.
 - Restore default definition of the OData model by choosing [Restore Default Values](#).
- 9. Once you have completed all the changes, choose [Finish](#). The Service Designer page displays the entity sets and function imports from the OData model in a table.
- 10. Choose [Save](#) to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose [Save as version](#).

Importing from ODC

Prerequisites

You have created an OData API artifact in an integration package.

Context

SAP Cloud Integration helps you import a model definition from an OData API, which is created on an on-premise SAP Gateway system. SAP Gateway has a component called Backend Provisioning Component or IW_BEP. This component resides on SAP Business Suite and is used to build OData APIs from data sources like RFCs, BAPIs, and so on. SAP Cloud Integration communicates with the IW_BEP component in SAP Gateway through OData Channel or ODC protocol, which is an SAP proprietary protocol.

i Note

SAP Cloud Integration does not support connecting to an SAP Gateway system based on SAP NetWeaver 7.02 where the IW_BEP component version is SP 06 or lower.

SAP Cloud Integration also does not support connecting to an SAP Gateway system based on SAP NetWeaver 7.40 where the SAP_GWFND component is SP 02 or lower.

You can use this procedure to create an OData model or build up an existing one by importing the model definition from an OData API created in IW_BEP (ODC).

Procedure

- 1. If you are creating a new OData API artifact, proceed to the next step. If you are editing an existing OData API artifact, choose [Edit](#) on the Service Designer page.
- 2. Choose [Import Model Wizard](#) (.
- 3. In the [Select Data Source Type](#) page, select [ODC](#) as data source type.
- 4. Enter the connection details as listed in the following table.

Connection Details	Description
Address	Endpoint URI of the target system
Client	Client of the target system
Namespace	Logical grouping of the service implementation
Service Name	Name of the service implementation
Version	Version of the service implementation
Proxy Type	The type of proxy that you are using to connect to the target system:

Connection Details	Description
	Select On-Premise if you are connecting to an on-premise system using the SAP Cloud Connector. This option is only available if you have chosen Cloud Integration as the runtime profile. If you select this option, the Address field of the adapter refers to a virtual address that has to be configured in the SAP Cloud Connector settings.
Authentication	Select Basic if you want to use user credentials to connect to the target system.
Credential Name	Name of the user credentials artifact deployed on the tenant

5. Choose **Step 2**.

i Note

While importing from any data source using the **Import Wizard**, if the element is of complex type containing a single child element of simple type, the option to deselect only the child element is not supported. In such a scenario, you can select **Flatten** from the **Import As** dropdown for the parent WSDL element. After completing the **Import Wizard**, navigate to the **OData Model Editor** and modify the structure as required.

In the **Select Structure** page, you can see entity types and function imports from the EDMX listed in a table. The table also displays the associated EDM type, primary key, and documentation information.

6. Select entity types and function imports that you need to add to the model from the **OData Hierarchy** column. You can expand or collapse entity types and function imports to view its properties better.

7. Choose **Step 3**.

8. In the **Review and Finish EDMX Structure** page, verify and perform the following additional changes, if required, to the OData model definition:

- Rename the OData objects listed under **EDM Name**.
- Select a different EDM type.
- Select another primary key.
- Update the **Documentation** column.
- Restore default definition of the OData model by choosing **Restore Default Values**.

9. Once you have completed all the changes, choose **Finish**. The Service Designer page displays the entity sets and function imports from the OData model in a table.

10. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Editing an OData Model

Context

You have created an integration package and added an OData API artifact to the package.

You can use this procedure to create an OData model from the ground up. You can also use it to edit an existing OData model.

Procedure

1. If you are creating a new OData API artifact, proceed to the next step. If you are editing an existing OData API artifact, choose **Edit** on the Service Designer page.
2. Choose **OData Model Editor** (🔗) on the Service Designer page.

If you are creating an OData model from the ground up, the OData Model Editor opens a predefined structure based on the selected OData version. The root is a **Schema** element. Below the root, subelements such as **EntityType**, **ComplexType**, **Association**, **EntityContainer**, and so on, can be added if required.

i Note

- In the case of an existing OData API artifact, you can only view the OData model in the OData Model Editor unless you explicitly choose **Edit** on the Service Designer page.

For more information on the features of the OData Model Editor, see [OData Model Editor Features](#).

- o Only OData models up to 300 KB in size are supported.

3. Once you have finished editing the OData model, choose **OK**.
4. Choose **Back** (↩) to get back to the Service Designer page.
5. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

OData Model Editor Features

The OData Model Editor in the Cloud Integration Web application enables you to write an Entity Data Model XML (EDMX). This model serves as metadata for your OData API. OData Model Editor provides you with the following basic features:

- **Code Assist:** Provides you with code suggestions that help speed up your development time. There are two types of code assist:
 - o Schema-based
 - o Model-based
- **Autocomplete:** Predicts the code element you are entering based on the pattern of letters entered.

i Note

You can use the OData Model Editor to edit and fine-tune your OData model, after you have imported it from a data source. However, we recommend that you keep the overall end-to-end scenario in mind when you edit the model using this tool. The OData Model Editor can only provide **semantic** validations. It cannot validate the correctness of the OData model in the context of the OData provisioning scenario that you are working in.

Related Information

[Using Schema-Based Code Assist](#)

[Using Model-Based Code Assist](#)

[Using Autocomplete](#)

[Validation in the OData Model Editor](#)

Using Schema-Based Code Assist

Context

The schema-based code assist feature provides you with code suggestions that adhere to the Conceptual Schema Definition Language (CSDL) specification. These code suggestions are provided based on the current cursor position.

Follow this procedure to use schema-based code assist in the OData Model Editor.

Procedure

1. Choose **OData Model Editor** (🔗) on the Service Designer page. The OData Model Editor page opens.
2. Place your cursor in the context where you require assistance, and press **Ctrl** + **Spacebar**.

Based on the cursor location, a list of the elements that can be used appears.

Each element in the dropdown list is represented by an icon. The following image shows the list of elements and their corresponding icons:

Icons	Elements
	Property Ref, Return Type, On Delete, Association Set
	Association
	Complex Type
	Entity Container
	Entity Type
	Entity Set
	Function
	Using
	Documentation
	End
	Referential Constraint
	Parameter
	Navigation Property
	Key
	Function Import
	Property
	Dependent
	Default image

3. Scroll through the list, select the required element, and press **Enter**. The OData Model Editor inserts a full snippet of code in the desired location.

Using Model-Based Code Assist

Context

The model-based code assist feature provides code suggestions based on the OData model you are working on.

Follow this procedure to use model-based code assist in the OData Model Editor.

Procedure

1. Choose **OData Model Editor**  on the Service Designer page. The OData Model Editor page opens.
2. Place the cursor between the quotes of the attribute of an element such as PropertyRef, EntitySet, or AssociationSet and press **Ctrl** + **Spacebar** to see the relevant suggestions.
3. Scroll through the list and select the required suggestion.

Using Autocomplete

Context

The autocomplete feature predicts the word you are entering based on the pattern of letters entered and provides you with suggestions. The suggestions are context-sensitive. You can get suggestions for elements, attributes, attribute types, properties, and property values.

Follow this procedure to use the autocomplete feature in the OData Model Editor.

Procedure

1. Choose **OData Model Editor**  on the Service Designer page. The OData Model Editor page opens.
2. Place the cursor in the context where you need assistance and start entering letters. Autocomplete suggestions appear next to the letters in context.
3. Use the **Up arrow** and **Down arrow** keys on the keyboard to select the required suggestions and press **Enter**.

i Note

Use the arrow keys if there is more than one suggestion. If there is only one suggestion, simply press **Enter**.

Validation in the OData Model Editor

Validation in the OData Model Editor ensures that your OData model adheres to the CSDL schema and does not deviate from logical or semantic specifications.

After validation, lines of code with errors are highlighted with a red icon in the code editor.

Let us look at an example of how validation works.

Example

In any OData model, remove the value corresponding to the **Name** attribute in an **EntityContainer** element.

Since **Name** is a mandatory attribute, the corresponding line of code is highlighted with a red icon in the code editor. You can see the error message in a tooltip when you hover the mouse over the red icon.

Viewing the OData Model Layout

Context

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#)



You have created an OData model either by importing from a data source or by editing the model manually using the Cloud Integration Web application.

You can use this procedure to view the overall layout of an OData model in the Graphical Model Viewer. This helps you verify the OData model and determine if any additional changes are necessary.

Procedure

1. Choose **Graphical Model Viewer** (🔍) on the Service Designer page. The OData model is rendered in a graphical format.
You can use the navigation control and viewing options to customize your viewing preferences. For more information, see [Graphical Model Viewer Features](#).
2. Choose **Back** (←) to get back to the Service Designer page.

Graphical Model Viewer Features

The Graphical Model Viewer in the Cloud Integration Web application provides you with the following navigation controls and viewing options to customize your viewing preferences:

Fit to Screen

There might be instances where the OData model objects of a big model extend beyond the scope of the Graphical Model Viewer space. In such cases, you can choose **Fit to Screen** to view the entire model within the boundaries of the Graphical Model Viewer. This feature automatically aligns the model to fit the Graphical Model Viewer space.

Expanding and Collapsing OData Objects

The Graphical Model Viewer lets you expand or collapse all the section headers of the objects in the OData model. You can also expand and collapse individual OData model objects.

Collapsed State

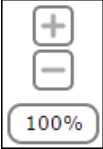
- To collapse all OData objects in the Graphical Model Viewer, choose the **Collapse All** button on the toolbar. The **Collapse All** button toggles to **Expand All** once the model is collapsed.
- If you want to collapse an individual OData object, choose the arrowhead button in the expanded object.

Expanded State

- To expand all OData objects in the Graphical Model Viewer, choose the **Expand All** button on the toolbar. The **Expand All** button toggles to **Collapse All** once the model is expanded.
- If you want to expand an individual OData object, choose the arrowhead button in the collapsed object.

Navigation Controls

Depending on the number of OData objects in the model, objects might appear too small or be too big for the Graphical Model Viewer to accommodate. In such cases, the navigation controls help you view the model better.

Button	Usage
	Use the compass to control the perspective of the model: - Choose  to move right. - Choose  to move left. - Choose  to move down. - Choose  to move up. - Choose  to zoom the model to 100%.
	- Choose  to zoom in to the model. - Choose  to zoom out of the model. - The zoom level indicator  shows the zoom percentage.

Bind to Data Source

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#) .

You have created an OData model in the Cloud Integration Web application. As a result, you can see OData objects associated with the model listed in a table on the Service Designer page.

Next, you need to bind or connect function imports and operations of entity sets to a data source. When you bind an OData object to a data source, a predefined integration flow is autogenerated and set up for you. You can modify this integration flow to suit your scenario.

i Note

The externalise option is not visible for these autogenerated integration flows.

You can bind one or all five CRUDQ operations of an entity set to a data source. This means that each entity set can have multiple integration flows connecting to the same data source. You can also bind entity sets and function imports in an OData API artifact to different data sources.

The following data sources are currently supported:

- SOAP
- OData
- REST
- ODC

Binding Status

To identify the graphical representations of the OData objects and their binding status on the Service Designer page, refer to the following table:

OData Object	Description
	Represents an entity set in an OData model.
	Represents an operation associated with an entity set in an OData model.
	Represents an operation bound to a data source with errors in the associated integration flow. You cannot deploy the OData API unless the errors are corrected.

OData Object	Description
	Represents an operation bound to a data source with warnings in the associated integration flow. We recommend that you verify the warnings and take corrective measures where necessary before deploying the OData API.
	Represents an operation bound to a data source with no errors or warnings.
	Represents a function import in an OData model.
	Represents a function import bound to a data source with errors in the associated integration flow. You cannot deploy the OData API unless the errors are corrected.
	Represents a function import bound to a data source with warnings in the associated integration flow. We recommend that you verify the warnings and take corrective measures where necessary before deploying the OData API.
	Represents a function import bound to a data source with no errors or warnings.

Unused Bindings

After binding an entity set or function import to a data source, if you rename or delete it in the OData Model Editor, the binding is no longer associated with it and becomes unused. You cannot deploy the OData API unless you update the binding. For more information, see [Managing Unused Bindings](#).

Related Information

[Binding to SOAP](#)

[Binding to OData](#)

[Binding to REST](#)

[Binding to ODC](#)

Binding to SOAP

Prerequisites

You are editing an OData API artifact and you have created an OData model. OData objects in the model are listed in a table on the Service Designer page.

Context

You can use this procedure to bind function imports and operations of entity sets to the corresponding SOAP Web service operations.

Procedure

1. Select **SOAP** as the data source for an entity set or function import.
2. Choose **Bind** () for the operation associated with the entity set or function import that you need to bind. The **Configure SOAP Data Source** dialog box appears.
3. Choose **Browse** and select the WSDL file from your file location. If the OData model was imported from a WSDL file, the corresponding file name appears by default. You can change this to any other file name as required.

Note

Composite WSDLs that are flattened are supported. However, multipart WSDLs and other file types are not supported.

4. Select an operation. If the OData model was imported from a WSDL file, operations defined in the WSDL appear in the dropdown list.
5. Enter the Endpoint URI where the operation is located. If the OData model was imported from a WSDL file and an operation was selected from this WSDL, the corresponding Endpoint URI is auto-populated.
6. Choose **OK**. The Service Designer page appears and the **Action** column for this operation now displays **Navigate to Integration Flow Editor** () and **Delete Binding** () buttons.

A predefined integration flow with mapping steps is generated for the operation. For more information, see [Predefined Integration Flows for SOAP](#).

You can update the integration flow to better suit your business scenario by choosing **Navigate to Integration Flow Editor** (). For more information, see [Editing an Integration Flow](#).

In case you need to update the binding information, you can do it in one of the following ways:

- o Choose **Navigate to Integration Flow Editor** () and update the Endpoint URI information in the receiver channel.
- o Choose **Delete Binding** () and repeat the binding procedure.

7. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Binding to OData

Prerequisites

You are editing an OData API artifact and you have created an OData model. OData objects in the model are listed in a table on the Service Designer page.

Context

You can use this procedure to bind function imports and operations of entity sets to the corresponding OData APIs.

Procedure

1. Select **ODATA** as the data source for an entity set or function import.
2. Choose **Bind** (🔗) for the function import or operation associated with the entity set that you need to bind. The **Configure OData Data Source** dialog box appears.
3. Choose **Browse** and select the EDMX file from your file location. You can also select the file from the **Existing Files** list. The **Existing Files** list is populated with EDMX file names used in other bindings in the same OData API artifact.
4. Select an entity set from the **Entity Sets** dropdown list. This list displays all entity sets from the selected EDMX file.
5. Enter the Endpoint URI corresponding to the OData API that you are binding to.
6. Choose **OK**. The Service Designer page appears and the **Action** column for this operation now displays **Navigate to Integration Flow Editor** (🔗) and **Delete Binding** (🗑️) buttons.

A predefined integration flow with mapping steps is generated for the operation or function import. For more information, see [Predefined Integration Flows for OData](#).

You can update the integration flow to better suit your business scenario by choosing **Navigate to Integration Flow Editor** (🔗). For more information, see [Editing an Integration Flow](#).

In case you need to update the binding information, you can do it in one of the following ways:

- Choose **Navigate to Integration Flow Editor** (🔗) and update the Endpoint URI information in the receiver channel.
 - Choose **Delete Binding** (🗑️) and repeat the binding procedure.
7. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Binding to REST

Prerequisites

You are editing an OData API artifact and you have created an OData model. OData objects in the model are listed in a table on the Service Designer page.

Context

You can use this procedure to bind function imports and operations of entity sets to the corresponding REST services.

Procedure

1. Select **REST** as the data source for an entity set or function import.
2. Choose **Bind** (🔗) for the function import or operation associated with the entity set that you need to bind. The **Configure REST Data Source** dialog box appears.
3. Enter the Endpoint URI corresponding to the REST service that you are binding to.
4. Choose **OK**. The Service Designer page appears and the **Action** column for this operation or function import now displays **Navigate to Integration Flow Editor** (🔗) and **Delete Binding** (🗑️) buttons.

A predefined integration flow with mapping steps is generated for the operation or function import. For more information, see [Predefined Integration Flows for REST](#).

You can update the integration flow to better suit your business scenario by choosing [Navigate to Integration Flow Editor](#) (🔗). For more information, see [Editing an Integration Flow](#).

In case you need to update the binding information, you can do it in one of the following ways:

- Choose [Navigate to Integration Flow Editor](#) (🔗) and update the Endpoint URI information in the receiver channel.
- Choose [Delete Binding](#) (🗑️) and repeat the binding procedure.

5. Choose [Save](#) to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose [Save as version](#).

Binding to ODC

Prerequisites

You are editing an OData API artifact and you have created an OData model. OData objects in the model are listed in a table on the Service Designer page.

Context

Cloud Integration helps you bind or connect to an OData API OData API, which is created on an on-premise SAP Gateway system. SAP Gateway has a component called Backend Provisioning Component or IW_BEP. This component resides on SAP Business Suite and is used to build OData APIs from data sources like RFCs, BAPIs, and so on. Cloud Integration communicates with the IW_BEP component in SAP Gateway through OData Channel or ODC protocol, which is an SAP proprietary protocol.

i Note

Cloud Integration does not support connecting to an SAP Gateway system based on SAP NetWeaver 7.02 where the IW_BEP component version is SP 06 or lower.

Cloud Integration also does not support connecting to an SAP Gateway system based on SAP NetWeaver 7.40 where the SAP_GWFND component is SP 02 or lower.

You can use this procedure to bind function imports and operations of entity sets to the corresponding OData APIs created in IW_BEP (ODC).

Procedure

1. Select **ODC** as the data source for an entity set or function import.
2. Choose [Bind](#) (🔗) for the function import or operation associated with the entity set that you want to bind.
The [Configure ODC Data Source](#) dialog box appears. The entity set and CRUDQ operation information is prepopulated.
3. Enter the following information related to the ODC destination.

Connection Details	Description
Entity Sets	A list of entity sets that is populated from the ODC source when importing the OData model. If you have coded the OData model, then the list is populated directly from your model. Select an entity set that you want to bind to.
Address	Endpoint URI of the target system
Client	Client of the target system
Namespace	Logical grouping of the service implementation
Service Name	Name of the service implementation
Version	Version of the service implementation
Proxy Type	The type of proxy that you are using to connect to the target system: ◦ Select On-Premise if you are connecting to an on-premise system using the SAP Cloud Connector.

Connection Details	Description
	This option is only available if you have chosen <i>HANA Cloud Integration</i> as the product profile. If you select this option, the Address field of the adapter refers to a virtual address, which has to be configured in the SAP Cloud Connector settings.
Authentication	Select Basic if you want to use user credentials to connect to the target system.
Credential Name	Name of the user credentials artifact deployed on the tenant

4. Choose **OK**. The Service Designer page appears and the **Action** column for this operation now displays **Navigate to Integration Flow Editor** (🔗) and **Delete Binding** (🗑️) buttons.

A predefined integration flow with mapping steps is generated for the operation or function import. For more information, see [Predefined Integration Flows for ODC](#).

You can update the integration flow to better suit your business scenario by choosing **Navigate to Integration Flow Editor** (🔗). For more information, see [Editing an Integration Flow](#).

If you want to update the binding information, you can do it in one of the following ways:

- Choose **Navigate to Integration Flow Editor** (🔗) and update the Endpoint URI information in the receiver channel.
- Choose **Delete Binding** (🗑️) and repeat the binding procedure.

5. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Managing Unused Bindings

Context

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#).



You are editing an OData API artifact and you have bound a function import or operations of an entity set to a data source. At this point, let us assume you changed the name of the function import or entity set, or even deleted it in the OData Model Editor. The binding associated with the renamed or deleted entity set or function import is now unused.

The unused bindings appear in a separate table on the Service Designer page. You cannot deploy the OData API unless you update the unused bindings.

Cloud Integration Web application helps you manage unused bindings by providing you with the following options:

Reconfigure Binding

Context

You can use this procedure to reconfigure the binding from the earlier entity set or function import to one in the updated OData model.

Procedure

- Choose **Bind** (🔗) for the entity set or function import associated with the unused binding. The **Reconfigure Binding** dialog box appears.
- Select the entity set or function import in the updated OData model to which the unused binding needs to be associated.
- Choose **OK**. The selected entity or function import in the updated OData model is now associated with the binding. The **Unused Bindings** table disappears if there are no other unused bindings.

SAP Cloud Integration updates the sender channel with the new binding information. However, you have to update other integration flow components that references the earlier entity set or function import, such as mapping and scripts.

- Choose **Save** to save your changes to the OData API artifact.

i Note

Delete Binding

Context

You can use this procedure to delete the unused binding so that it is no longer associated with any entity set or function import in the OData model.

Procedure

1. Choose **Delete Binding** () for the associated entity set or function import.
2. Confirm the deletion. The **Unused Bindings** table disappears if there are no other unused bindings.
3. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose [Save as version](#).

Editing an Integration Flow

Context

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#)



You are editing an OData service artifact and you have created an OData model. You have provided binding information for at least one OData object in the model.

Once a function import or operation of an entity set is bound to a data source, a predefined integration flow is set up for you. Within this integration flow, the OData adapter is selected as the default for the sender channel. The data source to which the operation is bound is selected as the default adapter for the receiver channel.

You can use this procedure to update the predefined integration flow to suit your business scenario.

Procedure

1. Choose **Navigate to Integration Flow Editor** () for an operation or function import that is bound to a data source. The associated integration flow opens in the Integration Flow Editor.
2. Edit the integration flow according to your requirements.

i Note

The following points need to be kept in mind when editing integration flows in an OData service artifact:

- The OData sender channel is not editable.
- The OData sender channel, participant and start event cannot be deleted.
- A new sender participant cannot be dragged and dropped into the integration flow.
- If you modify the default name of the receiver adapter, you need to make sure it is unique across all integration flows in an OData service artifact.
- If your service requires [Session Handling](#), ensure that session handling is enabled for each integration flow in your OData service project.
- You need to ensure that the Namespace Mapping entered in the runtime configuration, is consistent across all integration flows in an OData service artifact.

For example, you can enter the namespace-prefix pair `xmlns:test=http://sapcd.com/testABC` in multiple integration flows in an OData service artifact. You cannot enter `xmlns:test=http://sapcd.com/testABC` in one integration flow and enter `xmlns:test=http://sapab.com` in another integration flow in the same OData service artifact. However, it is perfectly valid to enter `xmlns:test=http://sapcd.com/testABC` in one integration flow and `xmlns:test2=http://sapab.com` in another.

- You can add other receiver/outbound adapters in the integration flow, but ensure that one of the following outbound adapters is present in the integration flow:
 - OData V2
 - ODC

- HTTPS
- SOAP

For more information about configuring integration flow components, see [Configure Integration Flow Components](#).

3. Once you have finished editing the integration flow, choose **OK**.
4. Choose **Back** (↩) to get back to the Service Designer page.
5. Choose **Save** to save your changes to the OData service artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Related Information

[Predefined Integration Flows](#)

Predefined Integration Flows

The Cloud Integration Web application sets up predefined integration flows when OData objects are bound to a data source. These integration flows have the sender channel preconfigured to the OData adapter type.

A predefined integration flow has the minimum setup required for an integration scenario. You have to configure this integration flow to suit the specific requirements of your scenario.

Related Information

[Predefined Integration Flows for SOAP](#)

[Predefined Integration Flows for OData](#)

[Predefined Integration Flows for REST](#)

[Predefined Integration Flows for ODC](#)

Predefined Integration Flows for SOAP

The Cloud Integration Web application sets up predefined integration flows when the CRUDQ (Create, Read, Update, Delete, and Query) operations of entity sets and function imports are bound to a SOAP Web service.

Related Information

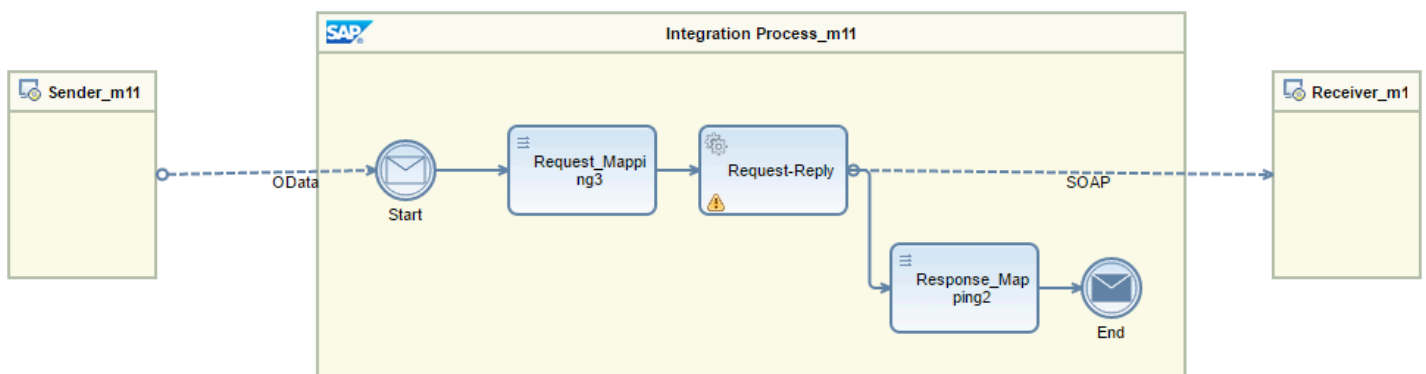
[Predefined Integration Flows for Create and Read Operations](#)

[Predefined Integration Flows for Update and Delete Operations](#)

[Predefined Integration Flow for Query Operations](#)

[Predefined Integration Flow for Function Imports](#)

Predefined Integration Flows for Create and Read Operations



This integration flow is autogenerated for Create and Read operations. In order to complete the binding to the data source, you have to edit at least the following elements:

- Request Mapping
- Response Mapping

To understand this integration flow, let's look at an example for the Create operation.

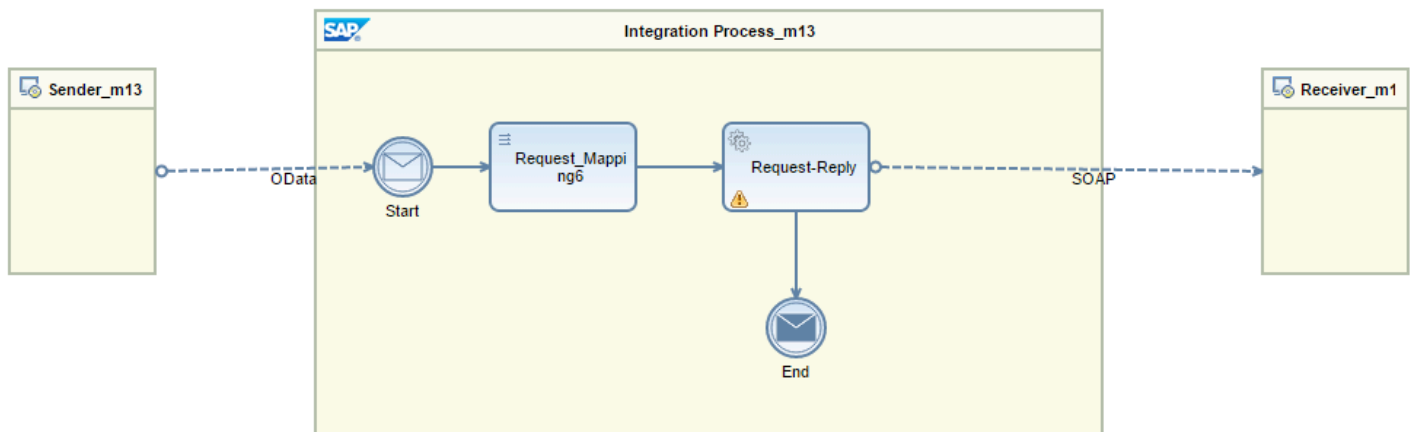
Example

You open an app on your smartphone to book a hotel room in the city of New York. This application sends the booking information to a hotel booking system. The hotel booking system receives this data with the help of a SOAP Web service, stores it, and sends the booking status back to you.

The mobile application communicates with the hotel booking system by calling a URI to create a booking record and passes the booking information as the payload. When the OData sender receives the request, it passes it to the Request Mapping element. This mapping element contains information that maps the fields from the OData structure to the SOAP structure and passes the create request to the SOAP receiver.

The SOAP receiver processes the request and creates a booking record in the back end. If the operation is successful, it returns the status to the Response Mapping element. The Response Mapping element contains information that maps the fields from the SOAP structure to the OData structure. It passes the booking status and other mapped details such as the confirmation number to the OData sender. OData sender in turn passes the response to the mobile application.

Predefined Integration Flows for Update and Delete Operations



This integration flow is autogenerated for Update and Delete operations. In order to complete the binding to the data source, you have to edit at least the Request Mapping element.

To understand this integration flow, let's look at an example of an Update operation.

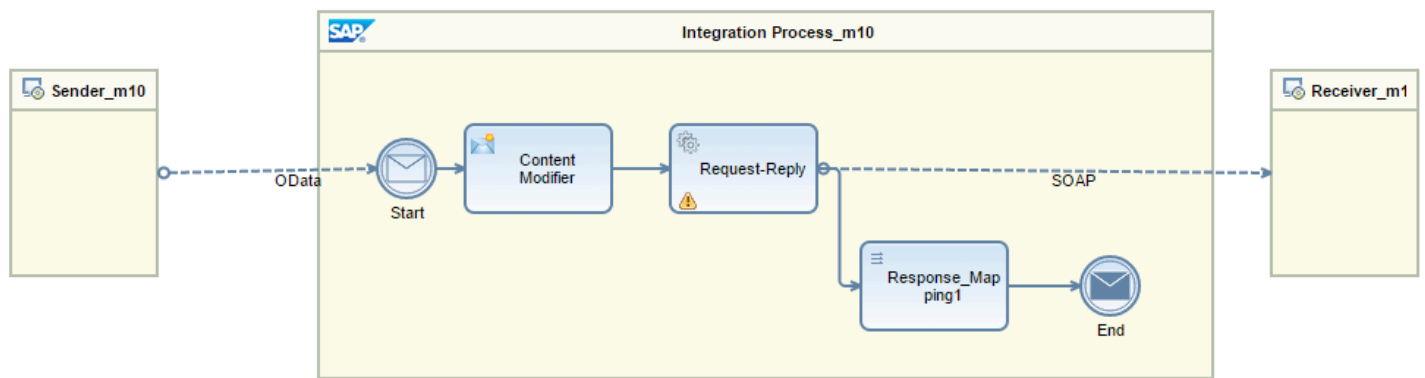
Example

You open an app on your smartphone to extend your stay in a hotel room in the city of New York. This application sends the booking information to a hotel booking system. The hotel booking system receives this data with the help of a SOAP Web service and updates it.

The mobile application communicates with the hotel booking system by calling a URI to update a booking record and passes the booking information as the payload. When the OData sender receives the request, it passes it to the Request Mapping element. This mapping element contains information that maps the fields from the OData structure to the SOAP structure and passes the update request to the SOAP receiver.

The SOAP receiver processes the request and updates the booking record in the back-end. The status of the update operation is then sent to the OData sender. The OData sender in turn sends it to the mobile application.

Predefined Integration Flow for Query Operations



This integration flow is autogenerated for Query operations. In order to complete the binding to the data source, you have to edit at least the following elements:

- Content Modifier
- Response Mapping

Note

If you want to enable system query options such as \$top, \$skip, and \$filter, you can replace the Content Modifier element with a Script element.

To understand this integration flow, let's look at an example.

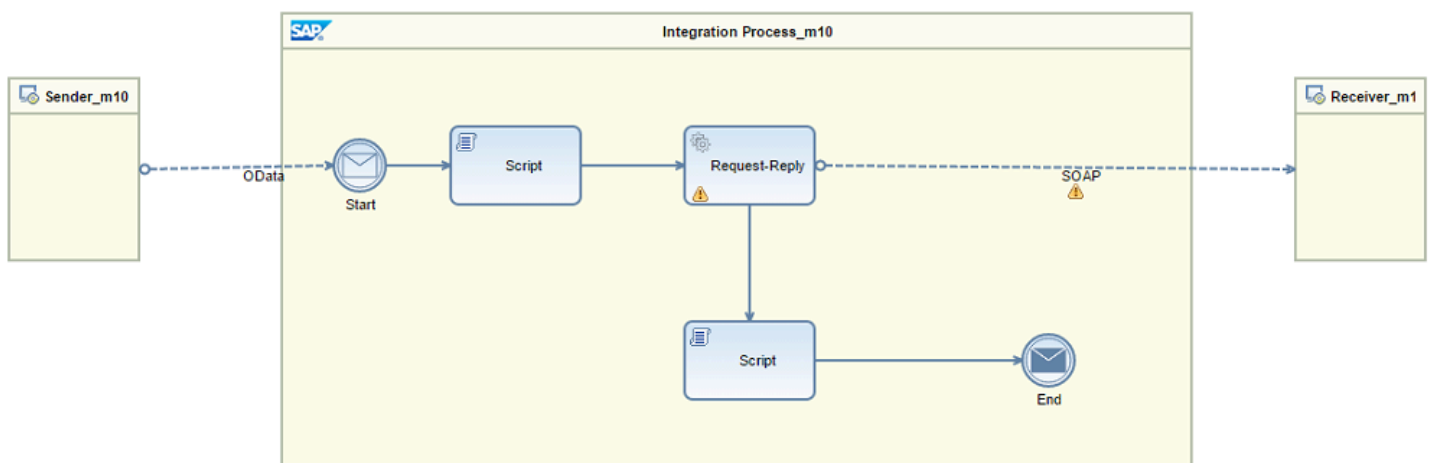
Example

A mobile application needs to get a list of hotels available in the city of New York from a hotel booking system. The hotel booking system publishes this data in the form of a SOAP Web service. The mobile application places this request in the form of a URI, where the city of New York is passed as a parameter.

When the OData sender channel receives the URI request, it passes it to the Content Modifier element. The Content Modifier element adds additional information to the head and body of the incoming message so that the SOAP receiver can understand it.

The SOAP receiver processes the request and, if there are no issues with the request, returns a list of hotels as the response. The Response Mapping element contains information that maps the fields from the SOAP structure to the OData structure. The mapped data is then sent to the mobile application through the OData sender channel.

Predefined Integration Flow for Function Imports



This integration flow is autogenerated for function imports. In order to complete the binding to the SOAP Web service, you have to edit at least the Script elements for request and response mapping.

To understand this integration flow, let's look at an example.

Example

You open an app on your smartphone to check the most frequently visited hotel in the city of New York and the number of guests that visited this hotel in the previous year. The hotel booking system publishes this data in the form of a SOAP Web service. The mobile application places this request in the form of a URI, where the city of New York is passed as a parameter.

When the OData sender channel receives the URI request, it passes it to the Script element. The Script element maps the request so that the SOAP receiver understands it.

The SOAP receiver processes the request and, if there are no issues with the request, returns the most frequently visited hotel and the guest count as the response. The Script element contains information that maps the fields from the SOAP structure to the return type defined in the OData model. The mapped data is then sent to the mobile application through the OData sender channel.

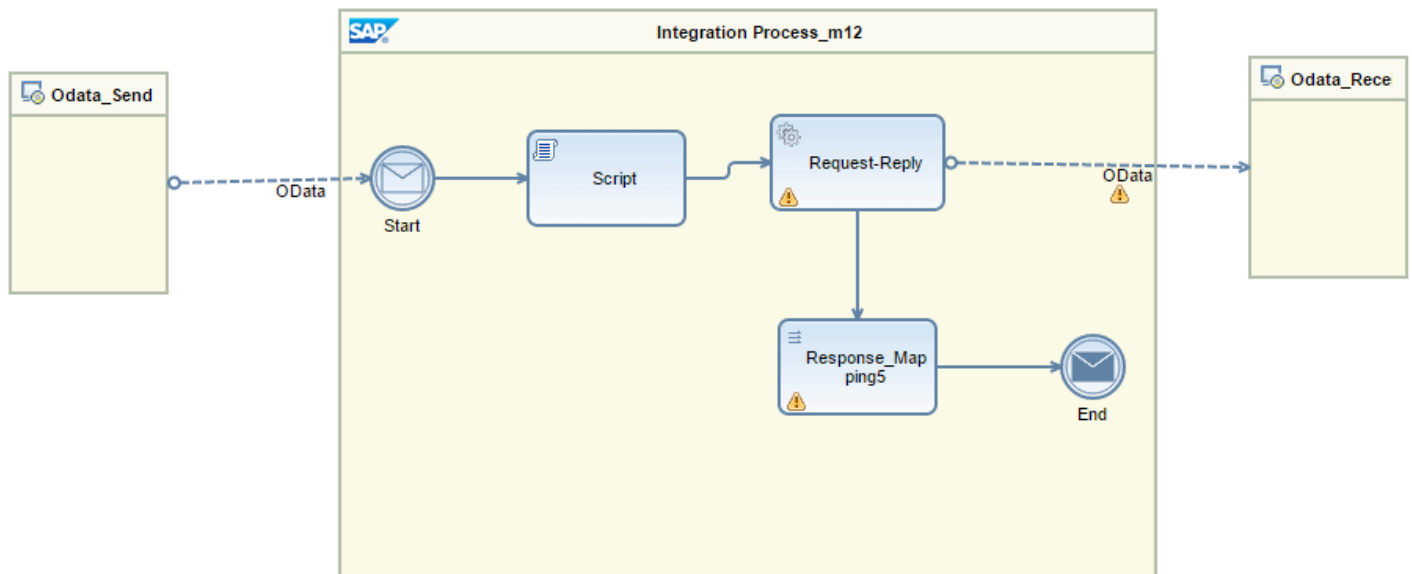
Predefined Integration Flows for OData

The Cloud Integration Web application sets up predefined integration flows when the Create, Read, Update, and Query operations of entity sets are bound to an OData API.

i Note

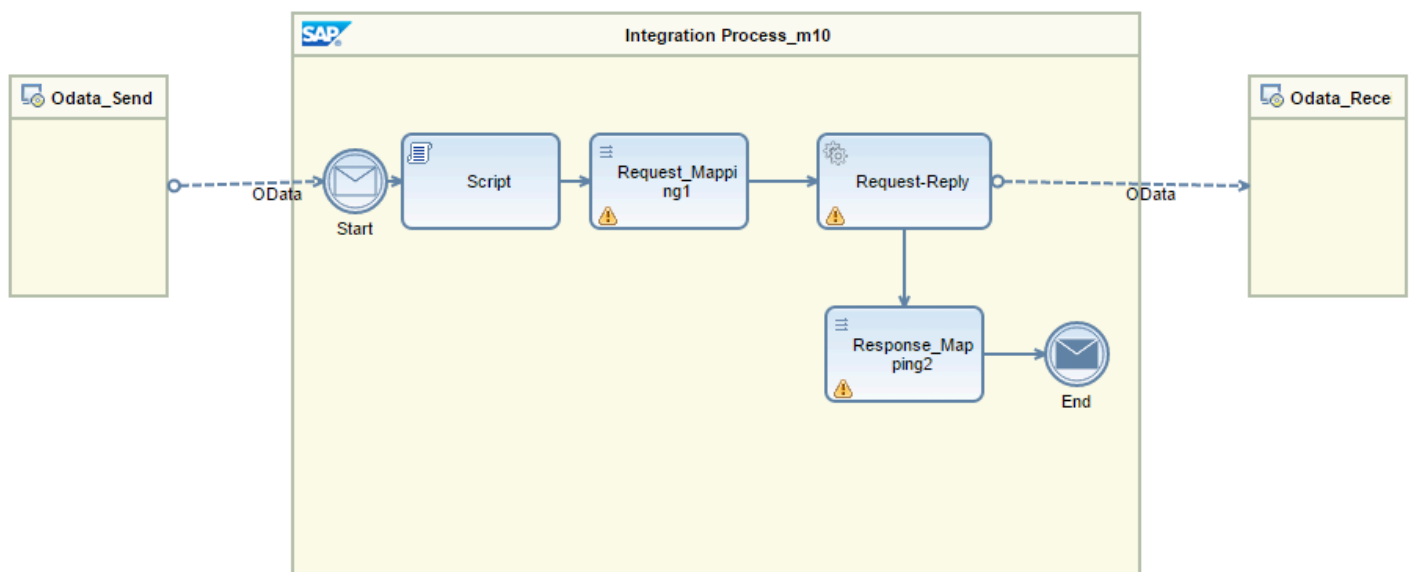
Function imports are not available in the OData receiver channel. We recommend that you use another adapter like HTTP for it.

Predefined Integration Flows for Query and Read Operations



This integration flow is autogenerated for Query and Read operations. Script step in this integration flow takes the query options from the incoming URI and sets it into an exchange header `<odataURI>`. In the case of Query operations, you can use this exchange header to dynamically pass the query options to the OData receiver channel. And in the case of Read operations, you can use this exchange header to dynamically pass the key fields of the entity set to the OData receiver channel.

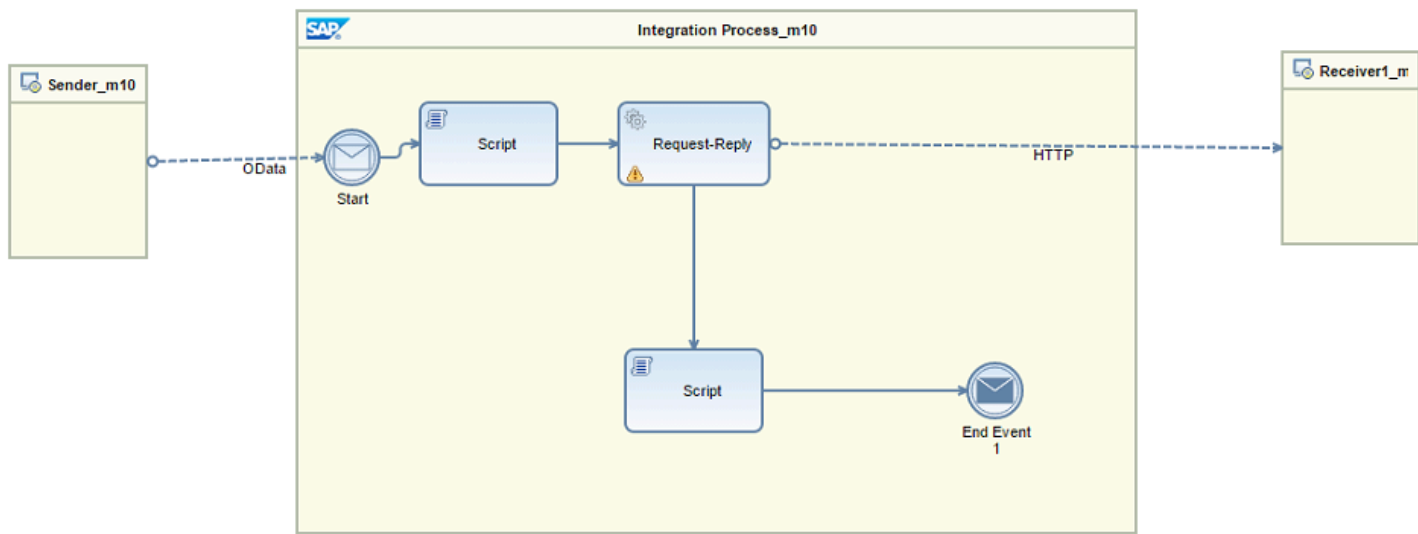
Predefined Integration Flows for Create and Update Operations



This integration flow is autogenerated for Create and Update operations. Script step in this integration flow takes the query options from the incoming URI and sets it into an exchange header `<odataURI>`. In the case of Update operations, you can use this exchange header to dynamically pass the key fields of the entity set to the

Predefined Integration Flows for REST

The Cloud Integration Web application sets up predefined integration flows when the CRUDQ (Create, Read, Update, Delete, and Query) operations of entity sets and function imports are bound to a REST web service.

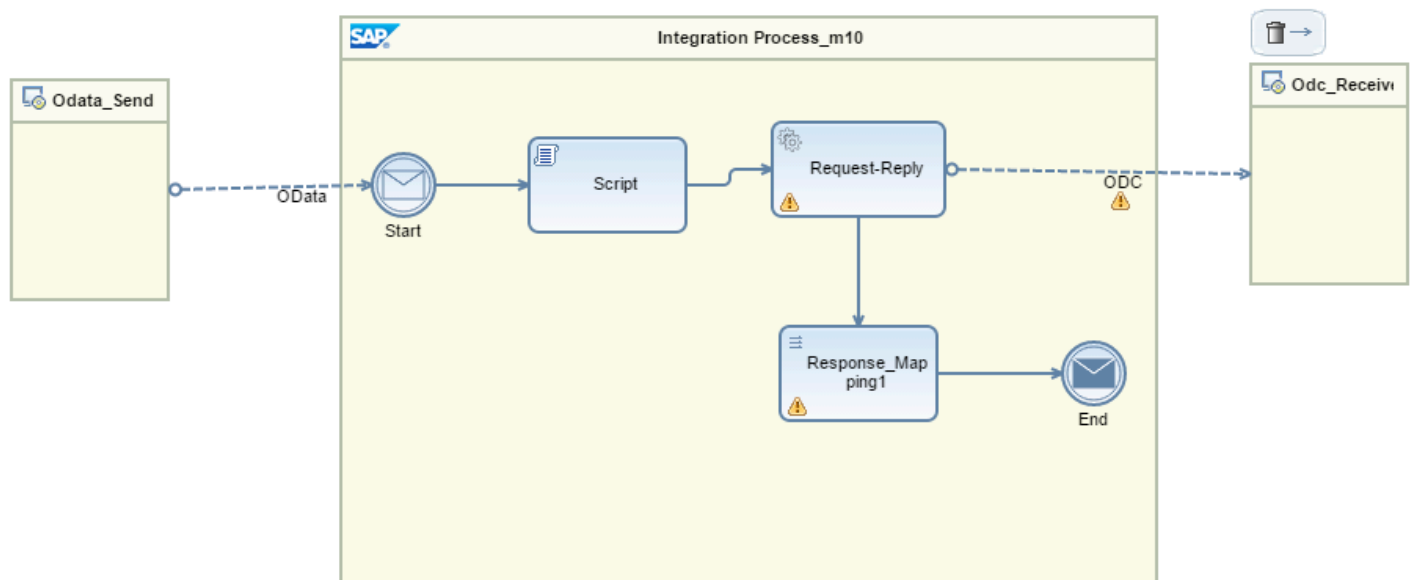


This integration flow is autogenerated for all CRUDQ operations when the receiver is a REST service. You can edit the Script elements in order to define the request and response mappings.

Predefined Integration Flows for ODC

The Cloud Integration Web application sets up predefined integration flows when the CRUDQ (Create, Read, Update, Delete, and Query) operations of entity sets and function imports are bound to an ODC service.

Predefined Integration Flow for Query and Read Operations



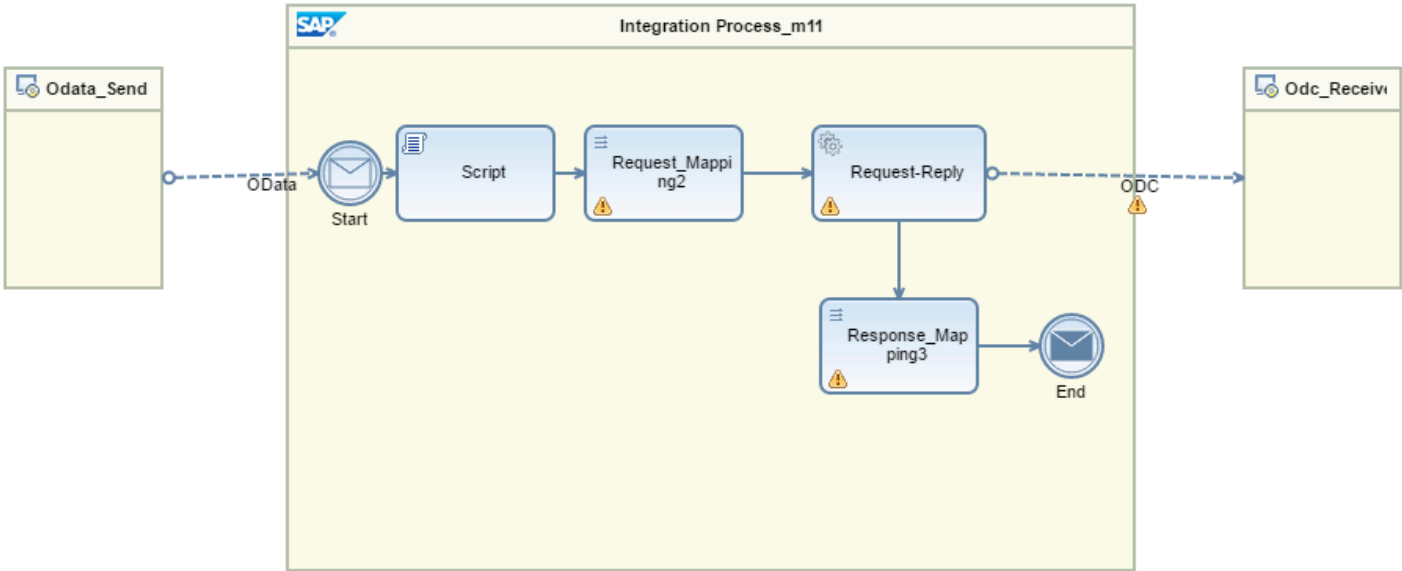
This integration flow is autogenerated for Query and Read operations. Script step in this integration flow takes the query options and key fields of entity sets from the incoming URI and sets it into an exchange header `<odataURI>`.

i Note

Currently, the ODC receiver channel does not support any query options or key fields of entity sets that are dynamically passed using the exchange header.

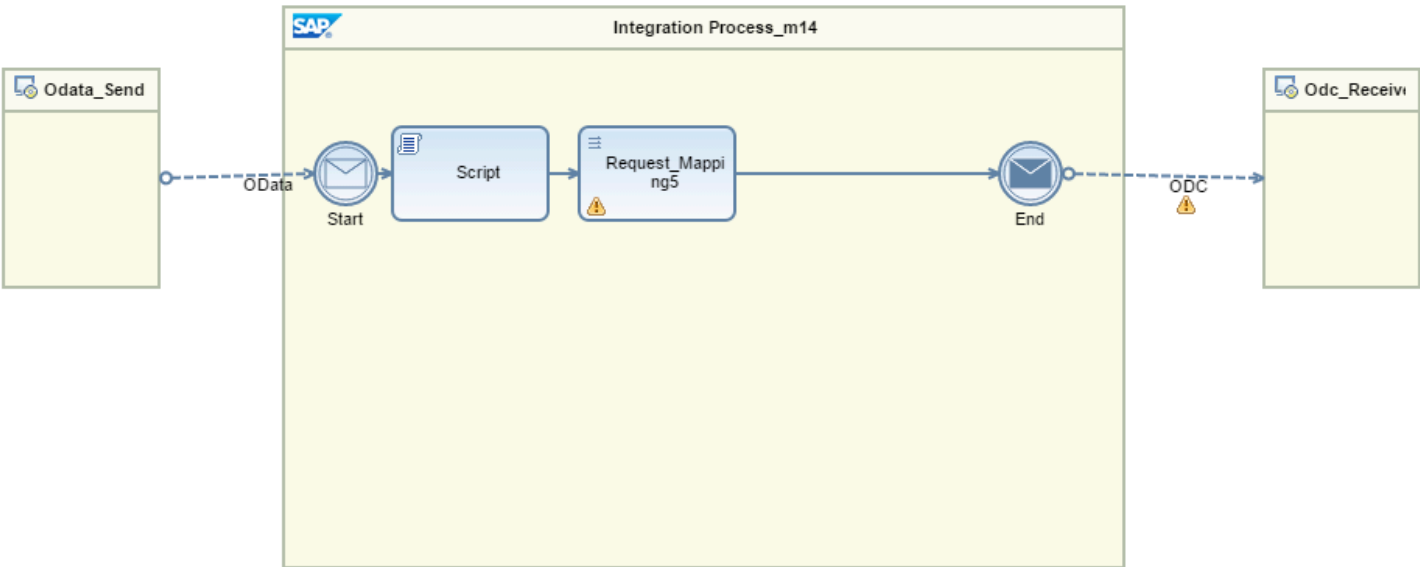
You can map the fields from the ODC structure to the OData structure in the Response Mapping element.

Predefined Integration Flow for Create Operation



This integration flow is autogenerated for Create operation. You can map the fields from the OData structure to the ODC structure in the Request Mapping element. Similarly, map the fields from the ODC structure to the OData structure in the Response Mapping element.

Predefined Integration Flow for Update Operation



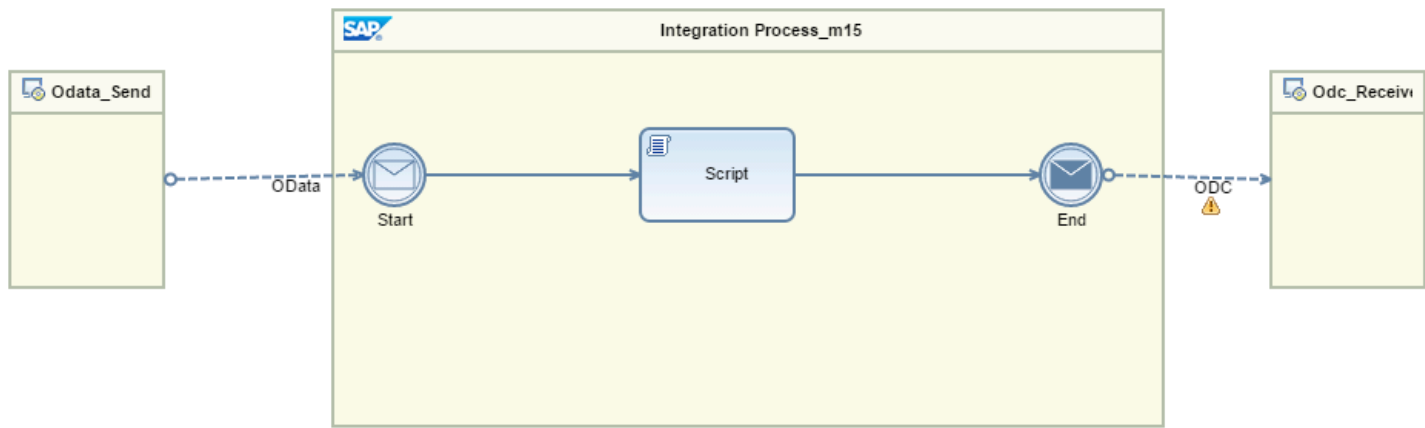
This integration flow is autogenerated for Update operation. Script step in this integration flow takes the key fields of entity sets from the incoming URI and sets it into an exchange header `<odataURI>`.

i Note

Currently, the ODC receiver channel does not support any key fields of entity sets that are dynamically passed using the exchange header.

You can map the fields from the OData structure to the ODC structure in the Request Mapping element.

Predefined Integration Flow for Delete Operation



This integration flow is autogenerated for Delete operation. Script step in this integration flow takes the key fields of entity sets from the incoming URI and sets it into an exchange header `<odataURI>`.

i Note

Currently, the ODC receiver channel does not support any key fields of entity sets that are dynamically passed using the exchange header.

Configuring an OData API

Context

→ **Remember**

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#).

You have created an OData API artifact.

You can use this procedure to modify the parameters that you entered when creating the artifact. These parameters identify the OData API.

Procedure

- 1. Choose **Design**.
- 2. Choose the integration package that contains the OData artifact for which the details need to be changed.
- 3. Choose and then choose **View metadata** to view or modify the metadata of the OData API.
- 4. Edit or modify the parameters listed in the following table, as required.

Field	Description
Name	Name of the OData API.
Description	Description of the OData API and its purpose.
Namespace	Logical grouping of the OData API.

- 5. Choose **Save** to save your changes to the OData API artifact.

i Note

To retain a copy of the current artifact, choose **Save as version**.

Deploying an OData API

Context

→ **Remember**

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#).

You've created an OData API artifact with a valid OData model. You've provided binding information for at least one OData object in the model. You've verified that all integration flows are valid.

You can use this procedure to deploy the OData API.

Procedure

1. Choose **Deploy** on the Service Designer page.
2. Confirm that you do want to deploy. If there are no validation errors, a message is displayed confirming that the service is triggered for deployment.
3. Choose **OK**. The OData API is now deployed.

The OData API starts on the tenant, after deployment. You can confirm that the status of the OData API is **Started** in the Integration Content Monitor. For more information, see [Manage Integration Content](#).

It can take a few moments for the OData API to get active. Once it's active, the OData API is ready for consumption.

i Note

The URL of the OData API endpoint is derived from the name of the OData API artifact.

Invoking an OData API

Context

→ Remember

This component or some of its features might not be available in the Cloud Foundry environment. For more information on the limitations, see SAP Note [2752867](#).



You have deployed an OData API successfully; the service is in active status ready to be consumed.

Procedure

1. Choose **Monitor**.
2. On the Overview page, in the section **Manage Integration Content**, choose the tile that shows the services with the status *Started*.
3. Search for and open the service that you deployed.
4. On the **Endpoints** tab, you can view the endpoint URI. This endpoint URI is the OData API Root URI.
5. You can copy this URI by choosing the copy icon and invoke it from the calling application.

You can invoke an OData API in *\$batch* mode by appending the term */ \$batch* to the service root URI and invoking the same using the relevant payload.

- Currently only one operation is supported for each *changeset*.
- The Message Processing Log (MPL) is generated for each *changeset*

i Note

If your MPL shows an error message **FAILED** without any error in the trace, then it means that your Response does not match the Metadata defined in the adapter.

- To obtain a JSON response, add **Accept HTTP header** with application/json value.
- All the headers available in the integration flow pipeline at the time of message processing, will be returned as response headers when the OData API is invoked.
- POST calls support deep structure responses.

Developing Custom Adapters

Develop adapters tailored for your specific requirements.

Though Cloud Integration provides a set of adapters to meet standard connectivity needs, you can develop and import your own custom adapters for your specific needs.

Adapter Development Prerequisites

You can contribute to an Apache Camel component as Cloud Integration adapter. To integrate camel component into Cloud Integration framework, specify the detailed set of adapter attributes as component metadata. Component metadata defines the adapter attributes, dependencies between them and the structure of configuration interface for a specific adapter, which is used for user interface generation.

Following are the guidelines that you need to understand and follow while developing camel component:

- **Protocol and Exchange Headers** - While you are specifying the protocol headers as camel headers or vice-versa from your custom integration adapter, we strongly recommend to consider these rules for security reasons:
 - Propagate only compliant headers or the headers required by adapter protocol.
 - Camel-specific arbitrary headers should not be used. You may still propagate the headers compliant with the adapter protocol following a naming convention conforming to adapter protocol guidelines.
 - Provide the configuration in the custom integration adapter to allow the non-compliant exchange headers to be propagated.
 - Include the propagated headers in the adapter documentation.
- **Camel Component Scheme** – Make sure that you use a unique camel component scheme to avoid runtime collisions with other registered schemes. For example, the camel component scheme can be sap-dropbox.
- This is the new content.
- **Secure Programming** – Store secure parameters in runtime secure store and accessed through alias configured in adapter.
- **Security Aspects** –
 - Always scan for virus in the Adapter workspace / project before deployment.
 - The Adapter design time content bundle shouldn't contain any confidential information (For example: hard-coded secrets, passwords, users, or documents).
 - Perform ftify and java static code checks.
- **Logging** – Use Slf4j logging
- **API Consumption**
 - To access platform-specific features, you can use public APIs that are available with the Adapter Development Kit (ADK). Public APIs for the ADK are available in Cloud Integration tools page.
 - The two maven coordinates available in the central maven for Public API consumption are:

`com.sap.cloud.adk:generic.api`

`com.sap.cloud.adk:adapter.api`

Refer the central maven repository to obtain the latest version number.
 - To consume the APIs, you need to use the code snippet `ITApiFactory.getApi(apiType, context)`; wherein, `apiType` is the class of the requested API interface. The camel context can be null if it isn't required for a particular API. For Example: For accessing `KeystoreService`, use `ITApiFactory.getApi(KeystoreService.class, context)`

i Note

The Java standard libraries of Java 8 can be used.

Cloud Integration supports the XML Document Object Model (DOM) to process XML documents.

- **Manifest.MF + Allowlisting** – Only ADK APIs, Camel core and Logging are accessible at runtime. However, if necessary you can import packages for dependent libraries bundled along with project.
- If you want to read a resource in integration flow project at runtime, use the following code snippet:

≡, Sample Code

```
InputStream is = ResourceHelper.resolveMandatoryResourceAsStream(getCamelContext().getClassResolver(), "/
```

Adapter Development Process

Understand the steps involved in developing custom adapters.

[Develop Adapters Using Archetype](#)

You can develop custom adapters using Adapter SDK in Cloud Integration.

[Consuming Non-OSGi Jar](#)

OSGi bundles are normal jar components with extra manifest headers, which you use to develop adapters. In OSGi environment, every service we use must be an OSGi bundle.

[Component Metadata](#)

Component metadata specifies the attributes of sender and receiver adapters used to define the UI. Each tab can contain multiple attributes. You can group the attributes into multiple attribute groups within the tabs. You can use the same attribute across the sender and receiver adapters by using attribute reference.

[Blueprint Metadata](#)[Enabling Endpoint Visualization for Custom Adapter](#)

Configure a custom adapter with endpoint visualization to view the adapter endpoint URL.

[Managing Cluster Lock in Custom Adapters](#)

Custom Integration Adapters provide a mechanism to acquire cluster-level locks, ensuring that specific actions (e.g., message processing) are executed by only one worker or node instance at a time. This prevents duplicate processing and maintains data consistency across distributed environments.

[Enabling ADK Persistency](#)[Enabling Authentication for Servlet Based Adapters](#)[Enabling Tracing for Custom Adapter](#)

Configure a custom adapter with a tracing feature that records the processing of the incoming and outgoing messages in an adapter.

[Enabling Scheduler Support for ADK Sender Adapter](#)[Enabling Connection Status for Integration Flow](#)[Accessing User Credentials](#)[Accessing Trust and Key Managers](#)[Accessing On-Premise Application using Cloud Connector](#)

Follow the steps here to extend the connectivity of Cloud Integration with remote on-premise application.

[Versioning Rules for Custom Adapters](#)

Follow the guidelines mentioned below to version a custom adapter.

Develop Adapters Using Archetype

You can develop custom adapters using Adapter SDK in Cloud Integration.

Prerequisites

- You are aware of the technical details of component/domain that you have to connect with Cloud Integration through the newly developed adapter type.
- You have Java 8 version.
- You are familiar with using Apache Camel and Java.
- You have the Camel component and its dependencies as OSGi bundles.
- You have the camel core runtime version 3.14.
- You have the simple logging facade for java (SLF4J).

Context

You use an adapter in an integration flow for outbound and inbound communication with different applications during runtime. In such scenario if an adapter does not support your communication requirements you use SAP ADK framework to create an adapter and deploy it in an integration flow. .

There are two types of adapter project template to create a new adapter project:

- **Adapter Project**

Use this template when you have a requirement to create a new SAP adapter and a new Camel component or customize an existing runtime component.

- **Component reuse Adapter Project**

Use this template if you want to re-use an existing runtime component without any customization.

Here we describe how to build and deploy an adapter during runtime using these two templates.

Procedure

1. Adapter Project Template

- a. Use any IDE for creating a project from the archetype or use the generate command and enter the following:

Use the below maven archetype artifacts:

```
com.sap.cloud.adk:com.sap.cloud.adk.archetype-adapter
```

Refer to the central maven to obtain the latest version number.

≡ Sample Code

```
mvn archetype:generate -B -DarchetypeGroupId=com.sap.cloud.adk \
-DarchetypeArtifactId=com.sap.cloud.adk.archetype-adapter \
-DarchetypeVersion=1.0.1 \
-DcomponentName=SampleComponentName \
-DadapterName=AdapterName \
-DadapterId=AdapterNameId \
-DadapterVendor=VendorName \
-DadapterVersion=1.0.0 \
-Dscheme=scheme \
-DgroupId=project.group \
-DartifactId=project.artifact \
-Dversion=1.0-SNAPSHOT
```

Component Description	
Component	Description
archetypeGroupId archetypeArtifactId archetypeVersion	Artifacts of maven archetype
componentName scheme	Artifacts of your runtime component componentName is the name of your camel component. scheme is the scheme name of the component.
adapterName adapterId adapterVendor adapterVersion	Artifacts of your integration adapter
groupId artifactId version	Artifacts of your maven project

b. The following table describes different components created for the custom adapter project using maven:

Component	Description
*Component.java	Sample runtime component.
*Consumer.java	Sample sender component.
*Endpoint.java	Logger information is found.
*Producer.java	Sample receiver component.
metadata.xml	UI values are coded here.
*ComponentTest.java	Sample JUnit test
pom.xml	Contains configuration details and dependencies.
*config.adk	Contains the adapter configuration parameters.
sap-sample	Represents the endpoint scheme. i Note If you deploy multiple adapters make sure the filenames are unique. When you change the filename do the following: ■ In the endpoint class, change scheme value. For example, @UriEndpoint(scheme = "sap-sample", syntax = "", title = "").

Component	Description
	■ In metadata.xml change the attribute value. For example, <code>gen:RuntimeComponentBaseUri="sap-sample"</code>.

i Note

You can update to the latest ADK version by manually changing the version in `com.sap.cloud.adk.build.archive` artifact found in the `pom.xml` file. Refer the central maven repository to get the latest ADK version.

2. **Component reuse Adapter Project Template**

a. Use any IDE for creating a project from the archetype or use the generate command and enter the following:

Use the below maven archetype artifacts:

`com.sap.cloud.adk:com.sap.cloud.adk.archetype-adapter-component-reuse`

Refer to the central maven to obtain the latest version number.

i Note

You can reuse an existing camel component in your adapter project using this template.

 Sample Code

```
mvn archetype:generate -B -DarchetypeGroupId=com.sap.cloud.adk \
-DarchetypeArtifactId=com.sap.cloud.adk.archetype-adapter-component-reuse \
-DarchetypeVersion=1.0.1 \
-DadapterName=AdapterName \
-DadapterId=AdapterNameId \
-DadapterVendor=VendorName \
-DadapterVersion=1.0.0 \
-DgroupId=project.group \
-DartifactId=project.reuse.artifact \
-Dversion=1.0-SNAPSHOT
```

Component Description

Component	Description
archetypeGroupId archetypeArtifactId archetypeVersion	Artifacts of maven archetype
adapterName adapterId adapterVendor adapterVersion	Artifacts of your integration adapter
groupId artifactId version	Artifacts of your maven project

b. The following table describes different components created for the custom adapter project using maven:

Component	Description
component	A folder where runtime component JAR must be placed.
libs	A folder where runtime component dependencies JARs must be placed.
metadata	A folder where component and blueprint (optional) metadata will be generated.
pom.xml	Contains configuration details and dependencies.

i **Note**

You can update to the latest ADK version by manually changing the version in com.sap.cloud.adk.build.archive artifact found in the pom.xml file. Refer the central maven repository to get the latest ADK version.

c. Copy runtime component **JAR** into **component** folder and their dependencies into the **libs** folder.

3. Run the command `mvn clean install` to build the adapter. This command will generate runtime component metadata and ESA with a component.

Blueprint metadata is not generated by default with this command. To generate this metadata, refer the next step.

4. Other maven goals that you can use are

o **Generate runtime component metadata**

This will generate component metadata using the camel component.

```
mvn com.sap.cloud.adk:com.sap.cloud.adk.build.archive:generate-cmd
```

o **Generate blueprint metadata**

This will generate the blueprint metadata for custom generation.

```
mvn com.sap.cloud.adk:com.sap.cloud.adk.build.archive:generate-bmd
```

o **Check adapter project**

This is used explicitly for validation. This step is included by default in the project build step.

```
mvn com.sap.cloud.adk:com.sap.cloud.adk.build.archive:check
```

Consuming Non-OSGi Jar

OSGi bundles are normal jar components with extra manifest headers, which you use to develop adapters. In OSGi environment, every service we use must be an OSGi bundle.

Prerequisites

- You have created a simple adapter.
- The adapter is an OSGi esa file with type as Composite. The dependencies are bundled within this esa.
- You are aware of Java development.

Context

To integrate camel component into Cloud Integration framework, camel component and its dependencies should be in a OSGi bundle. To convert the jar file to OSGi bundle, refer to the blog, [here](#)👉.

Results

The consumed non-osgi jar is available inside the component jar as bundle classpath.

Component Metadata

Component metadata specifies the attributes of sender and receiver adapters used to define the UI. Each tab can contain multiple attributes. You can group the attributes into multiple attribute groups within the tabs. You can use the same attribute across the sender and receiver adapters by using attribute reference.

Component Metadata

This set of metadata allows you to define components details.

Component Metadata

Property	Datatype	Cardinality	Description
ComponentId	xsd:string	Attribute	Specifies a unique Id for the component. The following are technical identifiers and should not contain any space or : Example: ctype::Adapter/cname::sap:dropbox/version::1.0.0 ctype is type of component, which is adapter in this particular example

Property	Datatype	Cardinality	Description
			cname is component name, which is sap:dropbox in this example. Here, sap is vendor and dropbox is name
ComponentDisplayName	xsd:string	Attribute	Specifies display name of the component.
Variant		Child Element	Different variants for a specific component type - for example: Sender and Receiver variants of an adapter
gen:RuntimeComponentBaseUri	xsd:string	Attribute	This attribute specifies the name with which the component is identified. If not added, then the baseUri value is take TP (Transport Protocol). Example for specifying attribute in the CMD: ⌘ Sample Code <pre><ComponentMetadata ComponentId="ctype::Adapter/cname::sap:sampleadapter/version::1.0.0 ComponentName="sap:sampleadapter" UIElementType="Adapter" IsDefaultGenerator="true" hci:RuntimeComponentBaseUri="dropbox"></pre> Example for generated bean.xml containing camel:from ⌘ Sample Code <pre><camel:from id="MessageFlow_1_1435812482630" uri="dropbox:foo?path=/input/test.txt& method=get&sendEmptyMessageWhenIdle=false& initialDelay=1000&greedy=false&startScheduler=false&timeUnit=MILLISECONDS& useFixedDelay=false&delay=500&runLoggingLevel=TRACE"/></pre>
IsRequestResponse	xsd:string	Attribute	This attribute is used for enabling the component to allow the service call for Request Reply. Default value is false. Henceforth, you should define it at variants level only.
hci:FirstUriPart	xsd:string	Attribute	Specifies the initial part of the URI. Customer can provide custom values for the first part of the URI string. Default v. 'foo'.
adapterInstanceID	xsd:string	Attribute	Use cluster lock to prevent same message being processed by different runtime nodes simultaneously. i Note You must define the following property at the endpoint. ⌘ Sample Code <pre><Metadata> <AdditionalMetadata> <Name>requiredIntegrationFlowProperties</Name> <Value>adapterInstanceID</Value> <GuiLabels> <Label language="en"/> </GuiLabels> </AdditionalMetadata> </Metadata></pre> Use the above parameter as the lock name in the cluster lock API.

i Note

IsDefaultGenerator and Metadata Version must always be true and should never be set to false. Metadata Version attribute is generated with default value as 2.0.

Variants

This set of metadata allows you to define variants for a component.

Variants

Property	Datatype	Cardinality	Description
Id	xsd:string		Provides a unique Id for the component variant. Example: ctype::AdapterVariant/cname::sap:dropbox/tp::dropbox/mp::dropbox/direction::Send tp stands for transport protocol

Property	Datatype	Cardinality	Description						
			mp stands for message protocol direction refers to Sender or Receiver system						
Groups		Child Element	Specifies collection of attributes defining metadata.						
VariantDisplayName	xsd:string	Attribute	Specifies display name of the variant.						
Input Content Tag used <InputContent>		Child Element	Input content specifies the type of data the adapter processes. <table><tr><th>Properties</th><th>Attributes/Child Elements</th><th>Description</th></tr><tr><td>ContentType</td><td>Child Element</td><td>Actual content type Supported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries></td></tr></table>	Properties	Attributes/Child Elements	Description	ContentType	Child Element	Actual content type Supported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries>
Properties	Attributes/Child Elements	Description							
ContentType	Child Element	Actual content type Supported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries>							
Output Content		Child Element	Output content specifies the type of data the adapter produces. Example: XML. <table><tr><th>Properties</th><th>Attributes/Child Elements</th><th>Description</th></tr><tr><td>ContentType</td><td>Child Element</td><td>Actual content type <ManifestEntries> <ManifestEntry> <Name>Import-PackageSupported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries></td></tr></table>	Properties	Attributes/Child Elements	Description	ContentType	Child Element	Actual content type <ManifestEntries> <ManifestEntry> <Name>Import-PackageSupported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries>
Properties	Attributes/Child Elements	Description							
ContentType	Child Element	Actual content type <ManifestEntries> <ManifestEntry> <Name>Import-PackageSupported content types: Xml, S Any, Json, Binary, Zip, GZip, Integer, NonXml, None and Text. <ManifestEntries> <ManifestEntr Package</Name> <Value>com.sap.gateway.core.service.ljs.registry:resolution:=optional,com.sap.gateway.core.l </ManifestEntry> </ManifestEntries>							
Metadata		Child Element	Helps to add additional metadata in the component. Specifies the display name of transport protocol and message protocol. Example for specifying attribute in the CMD:  Sample Code <pre><Metadata> <AdditionalMetadata> <Name>mp</Name> <GuiLabels guid="e7fa4f1d-ec6a-4248-9752-920ea9616bc9" > <Label language="EN" >AS2</Label> <Label language="DE" >AS2</Label> </GuiLabels> </AdditionalMetadata> <AdditionalMetadata> <Name>tp</Name> <GuiLabels guid="e7fa4f1d-ec6a-4248-9752-920ea9616bc9" > <Label language="EN" >HTTPS</Label> <Label language="DE" >HTTPS</Label> </GuiLabels> </AdditionalMetadata> </Metadata></pre> i Note You should refer message protocol as mp and transport protocol as tp .						
ManifestEntries		Child Element	Specifies which manifest entries you can add in manifest file. You can add or update Import-Package entries in manifest file. Example for specifying attribute in the CMD:  Sample Code						
Allowed Headers		Child Element	Specifies which headers can be passed in the integration flow pipeline from sender component. Example for specifying attribute in the CMD:  Sample Code						

Property	Datatype	Cardinality	Description
			<code><AllowedHeaders></code> <code>Header1,Header2</code> <code></AllowedHeaders></code>
hci:FirstUriPart	xsd:string	Attribute	Specifies the initial part of the URI. Customer can provide custom values for the first part of the URI string.
IsRequestResponse	xsd:string	Attribute	This attribute is used for enabling the variant to allow the service call for Request Reply
supportsPolling	xsd:boolean	Attribute	This attribute depicts if the component has the polling behavior. (supportsPolling). It also helps in populating the timer icon decorated. Default value is false. It is applicable for sender channel only.
AttachmentBehavior	xsd:string	Attribute	This attribute helps to identify the component behavior towards attachment. • Modify - Create or update an attachment • Preserve - Pass through for the attachments • Drop - Attachment will be lost If attachment behavior is not specified; system considers that the component lost the attachment.
ReferencedComponents		Child Element	Specifies the reused component to be referred by the adapter.

i Note

If you want to rename a variant id or delete a variant from adapter, first undeploy the adapter with old variants and then deploy updated adapter.

You can still view the older variants if you do not undeploy the adapter with old variants.

Tabs

This set of metadata allows you to specify tab pages for the adapter configuration user interface.

Tabs

Property	Datatype	Cardinality	Description
Id	xsd:string		Specifies the tab identifier.
GuiLabels	xsd:string		Specifies a language-specific GUILabel. Sample GUILabel is shown below: <pre><GuiLabels> <Label language="EN">Address</Label> </GuiLabels></pre> Here, Label language = "EN" represents that the text of guilabel is in english. Address is the text for the attribute name. Currently, guilabel supports only English language.

Attribute Group

This set of metadata allows you to group adapter attributes.

Attribute Group

Property	Datatype	Cardinality	Description
Name	xsd:string		
Id	xsd:string		Specifies the attribute group identifier. Examples: Dropbox Endpoint Dropbox Security Settings
AttributeReference		Child Element	Specifies ReferenceName, ErrorMessage, Restriction, and Description
GuiLabels			Specifies a language-specific GUI label.

Property	Datatype	Cardinality	Description
Value - Datatype		Child Element	Datatype for value xsd:string xsd:integer xsd:boolean xsd:long xsd:password xsd:id xsd:idref i Note If an attribute is marked with xsd:id in metadata, then its value should be unique across elements of integration flow, during design time. If an attribute is marked with xsd:idref then it specifies a reference to another resource. If resource path starts with /<folder_name>/<resource_name>, then checks will validate if specific resource path exists in src\main\resource folder of project or not.

The sample attribute group is shown below:

```
<AttributeGroup id = "Connection Details">
  <GuiLabels>
    <Label language="EN">Connection Details</Label>
  </GuiLabels>

  <AttributeReference>
    <ReferenceName>address</ReferenceName>
    <!-- Restriction tag defines java regular expression for field value -->
    <Restriction>Constraint.isValidRegex([A-Za-z]+[0-9_~\-\.\.]*)</Restriction>
    <!-- This tag defines message to be displayed if field value is invalid -->
    <ErrorMessage>Enter a valid address. E.g. /file</ErrorMessage>
    <!-- Description tag defines tool tip for field value -->
    <description>Relative endpoint address on which.</description>
  </AttributeReference>

  <AttributeReference>
    <ReferenceName>wsdlURL</ReferenceName>
    <Restriction>Constraint.isValidRegex:^(https://){1}(.+)</Restriction>
    <ErrorMessage>Enter a valid URL. E.g. /wsdl/abc.wsdl;</ErrorMessage>
    <description>URL to the WSDL.</description>
  </AttributeReference>

  <AttributeReference>
    <ReferenceName>cxfoNewayRobust</ReferenceName>
    <description>Used for reliable one-way message exchanges.</description>
  </AttributeReference>
</AttributeGroup>
```

Attribute Reference

Attribute Reference Table

Property	Data Type	Cardinality	Description
ReferenceName	xsd:string		Specifies that in the CMD there is an attribute definition associated with this reference name.
description	xsd:string		Brief description of the attribute, which is shown as the tool tip.
Restriction	xsd:string		Details given below in the table.
ErrorMessage	xsd:string		If the constraint specified in the restriction tag fails, the system shows the error message set in this tag. If this field is not set, then the system shows a default error message at the time of check failure.

List of Constraints that are supported for Restriction tag

Constraint	Description
------------	-------------

Constraint	Description
isValidRegex	Method will check a value against a passed regex. Returns true in case value matches regex ⌚ Sample Code <AttributeReference> <ReferenceName>stepid</ReferenceName> <Restriction>Constraint.isValidRegex([a-zA-Z0-9]+)</Restriction> <ErrorMessage>StepID not defined for Persist Message step</ErrorMessage> </AttributeReference>
isAlphaNumeric	Checks whether the pass value is alphanumeric
isStartsWithLetter	Checks the passed value starts with Letter
isValidURIString	Returns true if it is valid URI String. It can start with http(s), ftp, file or ldap
isValidXMLString	Returns true if passed string is a valid XML String
isValidNCName	Return true if passed string value is valid NCName
isValidXpath	Returns true if passed string value is valid xpath expression

Attribute Metadata

This set of metadata allows you to define the details of all adapter attributes. This includes the definition of fixed values for input help, and the kind of user interface element used for an adapter attribute (for example if it is a checkbox or a dropdown list).

The sample attribute metadata should be as shown below:

```
<AttributeMetadata>
  <Name>address</Name>
  <Usage>true</Usage>
  <Default>/BSN/paymentOrder</Default>
  <DataType>xsd:string</DataType>
  <isparameterized>true</isparameterized>
  <Length>200</Length>
  <FixedValues>
    <FixedValue>
      <Value>Atleast Once</Value>
      <GuiLabels>
        <Label language="EN">Atleast Once</Label>
      </GuiLabels>
    </FixedValue>
  </FixedValues>
  <GuiLabels>
    <Label language="EN">Address</Label>
  </GuiLabels>
</AttributeMetadata>
```

Attribute Metadata

Property	Datatype	Cardinality	Description
Name	xsd:string		Specifies the parameter id. Example: address
Usage	xsd:boolean		Specifies if the field is mandatory or not. If the value is true then usage of field is mandatory. If the value is false then usage of field is not mandatory.
Default	xsd:string		Specifies a default value. Example: /BSN/paymentOrder
Datatype			Specifies the datatype for the value. Examples: xsd:string xsd:integer

Property	Datatype	Cardinality	Description												
Length	xsd:integer		Specifies the maximum character length for field values.												
Minlength	xsd:integer	Child Element	Specifies the minimum character length for field values.												
HelpService		Child Element	Helps to configure specified resources in adapters. Example for specifying attribute in the CMD: ≡ Sample Code <pre><HelpService> <HelpServiceName> resourceLookupService </HelpServiceName> <HelpServiceQueryAttributes> <HelpServiceQueryAttribute> <Name> Type </Name> <Value> xml </Value> </HelpServiceQueryAttribute> <HelpServiceQueryAttribute> <Name> ResourceLocation </Name> <Value> edmx </Value> </HelpServiceQueryAttribute> </HelpServiceQueryAttributes> </HelpService></pre> Once you add the above code, then a Browse button gets enabled. You can click on the button and select resource file from the project of file system. ResourceType specifies resource extension. Currently, only one extension can be specified. ResourceLocation specifies relative path to project from where files are read or written.												
GUILabel		Child Element	Specifies a language-specific GUI label.												
FixedValue(s)		Child Element	Specifies a fixed list of allowed values. <table><tr><th>Properties</th><th>Attributes/Child Elements</th><th>Description</th></tr><tr><td>FixedValue</td><td>Child Element</td><td>Specifies fixed list of allowed values. Multiple FixedValue tags together are called a combo.</td></tr><tr><td>isEditable</td><td>Child Element</td><td>If the element is set to 'true', then the dropdown and combo fields are editable.</td></tr><tr><td>Sort</td><td>Child Element</td><td> If the element is set to 'true', then the values are arranged either in ascending or descending order. Use the following formats to arrange the values: - To arrange the values in ascending order, use <Sort order="ascending">true</Sort> - To arrange the values in descending order, use <Sort order="descending">true</Sort> i Note If the element is set to 'false', then the values are arranged as per the order defined in the cmd. </td></tr></table>	Properties	Attributes/Child Elements	Description	FixedValue	Child Element	Specifies fixed list of allowed values. Multiple FixedValue tags together are called a combo.	isEditable	Child Element	If the element is set to 'true', then the dropdown and combo fields are editable.	Sort	Child Element	If the element is set to 'true', then the values are arranged either in ascending or descending order. Use the following formats to arrange the values: - To arrange the values in ascending order, use <Sort order="ascending">true</Sort> - To arrange the values in descending order, use <Sort order="descending">true</Sort> i Note If the element is set to 'false', then the values are arranged as per the order defined in the cmd.
Properties	Attributes/Child Elements	Description													
FixedValue	Child Element	Specifies fixed list of allowed values. Multiple FixedValue tags together are called a combo.													
isEditable	Child Element	If the element is set to 'true', then the dropdown and combo fields are editable.													
Sort	Child Element	If the element is set to 'true', then the values are arranged either in ascending or descending order. Use the following formats to arrange the values: - To arrange the values in ascending order, use <Sort order="ascending">true</Sort> - To arrange the values in descending order, use <Sort order="descending">true</Sort> i Note If the element is set to 'false', then the values are arranged as per the order defined in the cmd.													

Attribute Reference Table

Property	Data Type	Cardinality	Description
----------	-----------	-------------	-------------

Property	Data Type	Cardinality	Description
AttributeName		Child Element	Name of the attribute
AttributeValue		Child Element	Value of the attribute
AndCondition		Child Element	Attribute Reference Property This is complex constraint that is used when there are two or more complex or basic constraints that need to be satisfied to achieve certain criteria. Example of Complex condition ☰ Sample Code <pre> <AndCondition> <EditCondition> <AttributeName>reference1</AttributeName> <AttributeValue>reference1value</AttributeValue> </EditCondition> <EditCondition> <AttributeName>reference2</AttributeName> <AttributeValue>reference2value</AttributeValue> </EditCondition> </AndCondition> </pre> As you can see in the above example AndCondition is complex condition that contains two or more EditConditions(basic conditions) and for the condition to be satisfied both of these basic conditions is satisfied.
OrCondition		Child Element	Attribute Reference Property This is complex constraint that is used when satisfaction of any one of the nested complex or basic constraint is required to achieve certain criteria The structure of OrCondition is similar to And condition, with only change from <AndCondition> to <OrCondition>.
NotCondition		Child Element	Attribute Reference Property This is complex constraint that is mostly used with other complex constraints, this constraint reverses the result returned by the nested complex or basic condition Not condition can contain only one edit condition but multiple complex conditions. These complex conditions can be used with each other. ☰ Sample Code <pre> <AndCondition> <OrCondition> <EditCondition></EditCondition> <EditCondition></EditCondition> </OrCondition> <NotCondition> <EditCondition></EditCondition> </NotCondition> </AndCondition> </pre>
EditCondition		Child Element	Attribute Reference Property Constraint to be executed, this is on top of base constraint defined at Attribute Level Environment Variables: If the Attribute Name starts with \$ then next part is considered as an Environment variable. There can be two types of variables: a) \$PROFILE.NAME - environment variables starting with profile are properties fetched on profile configured for project b) \$ENVVARIABLE - environment variables starting with env are system properties for vm
AttributeBehavior	xsd:string	Child Element	If you assign AttributeBehavior as SecureAlias for a particular field, then the field refers to alias of security parameter artifact.

Attribute Table Metadata

Property	Data Type	Cardinality	Description
Name	xsd:string		

Property	Data Type	Cardinality	Description
Usage	xsd:boolean		
HelpService		Child Element	Helps to configure certificate in adapters. Example for specifying attribute in the CMD: ≡ Sample Code <pre><HelpService> <HelpServiceName> certificateLookupService </HelpServiceName> </HelpService></pre> Once you add the above code, then a Browse button gets enabled. You can click on the button and select certificate file. This attribute supports *.crt and *.cer types of certificates only.
AttributeReference			Specifies reference name that is header of the column. i Note Each reference name constitutes to one column in the table.
GuiLabels	xsd:string		Specifies the label of the table.

The sample attribute table metadata should be as shown below:

≡

Sample Code

```
<AttributeTableMetadata>
  <Name>clientCertificates</Name>
  <Usage>true</Usage>
  <GuiLabels guid="01c2bd59-8dd1-4efb-b795-869851eb979b">
    <Label language="EN">Client Certificate Authorization</Label>
    <Label language="DE">Client Certificate Authorization</Label>
  </GuiLabels>
  <AttributeReference>
    <ReferenceName>clientCertificate.subjectDN</ReferenceName>
  </AttributeReference>
  <AttributeReference>
    <ReferenceName>clientCertificate.issuerDN</ReferenceName>
  </AttributeReference>
</AttributeTableMetadata>
```

i

Note

- You must not use any special character in the table fields.
- You can reload component metadata in the following three ways:
 - You can reopen eclipse tool.
 - You can reconnect to server.
 - You can use  icon (on eclipse toolbar) to manually download it.

Blueprint Metadata

Blueprint metadata defines the attributes of the sender and receiver adapters that contain multiple tags.This document mainly describes all the tags and attributes apart from CMD and camel blueprint tags for providing additional generation injection for ADK development.

This set of tags and attributes allows you to define the metadata.

Attributes	Datatype	Cardinality	Description
createOnce	xsd:string		If you set the string value as "true", then the bean instantiation in the blueprint generation is done only once. By default the value is 'false'.
createCondition	xsd:string		This attribute can be used to conditionally instantiate the bean. The syntax followed is similar to XSLT test attribute. For example: <pre><=, Sample Code</pre> <pre><bean id="123" class="com.example.Someclass.java" createCondition="\$.cmd.attribute(proxyType)='onPremise'" /></pre> Here Someclass.java is generated in the blueprint only if the UI variable proxyType contains the value onPremise.

Tag	Description
<code><TransformationTags xmlns:bp="http://www.adk.gen/xmlns/blueprint/"></code>	This is the parent tag for the <code><blueprint></code> tag. This tag does not contain any generation specific information as part of attribute. The <code><blueprint></code> tag must be included within this tag.
<code><bp:blueprint></code>	This tag can be used to specify additional namespaces if needed.
<code><bp:bean></code>	Beans are declared using <code><bean></code> element.
<code><bp:reference></code>	To generate References this tag can be used. This tag does not contain any new attributes. It is similar to the camel blueprint's reference tag. Refer the XSD above for all supported attributes.
<code><cxfc:bus xmlns:cxfc="http://cxf.apache.org/blueprint/core"></code>	Refer http://cxf.apache.org/schemas/blueprint/core.xsd for all attributes supported.
<code><cxf:cxfEndpoint xmlns:cxf="http://camel.apache.org/schema/blueprint/cxf"></code>	Refer https://camel.apache.org/schema/cxf/camel-cxf.xsd for all the attributes supported.
<code><camel:camelContext xmlns:camel="http://camel.apache.org/schema/blueprint"></code>	This is the root camel context tag just for the reference. This contains all the properties a final blueprint's camelContext contains as part of g&b generation. The user is not allowed to make any change to this tag. Any changes/additions will simply be ignored.
<code><http-conf:conduit xmlns:http-conf="http://cxf.apache.org/transport/http/configuration" ></code>	This is same as that of Camel Blueprint specification for conduit tag. Refer http://cxf.apache.org/schemas/configuration/http-conf.xsd .
<code><camel:route xmlns:camel="http://camel.apache.org/schema/blueprint"></code>	This is same as that of Camel blueprint specification <code><route></code> . Must appear within <code><camelContext></code> .. Refer the XSD for the attributes. Multiple routes are supported.
<code><properties></code>	Can be contained in the DSL just for reference. This tag must be same as g&b's generated tag. It has no significance. i.e. any modification to this tag can simply be ignored by the DSL processor.
<code><streamCaching></code>	Provided just for the reference.

Enabling Endpoint Visualization for Custom Adapter

Configure a custom adapter with endpoint visualization to view the adapter endpoint URL.

The following sections describe the guidelines that you need to understand and follow:

Adding Libraries

You can find the latest ADK API in the [Adapter Development Kit](#) section. After downloading the *.zip files, add the following libraries to the `src > main > resources` folder that is in your ADK project:

- com.sap.it.public.adapter.api.jar
- com.sap.it.public.generic.api.jar

For successful code compilation, right-click the *.jar file and choose `Build Path > Add to Build Path` to add the jar files to the Referenced Libraries of the adapter project.

Adding Dependencies

Open the `pom.xml` file and add the dependencies shown here:

Source Code

```
<dependency>
<groupId>com.sap.it.public</groupId>
<artifactId>adapter.api</artifactId>
<version>...</version>
<scope>system</scope>
<systemPath>${project.basedir}/src/main/resources/adapter.api.<version of the downloaded file>.jar</systemPath>
</dependency>
<dependency>
<groupId>com.sap.it.public</groupId>
<artifactId>generic.api</artifactId>
<version>...</version>
<scope>system</scope>
<systemPath>${project.basedir}/src/main/resources/generic.api.<version of the downloaded file>.jar</systemPath>
</dependency>
```

Note

Make sure that you use the latest version of the *.jar files.

After adding the dependency, add the `com.sap.it.public` group ID to the `<excludeGroupIds>` tag.

Adding Endpoint Information Service

You must create a **Class** and use the following method to retrieve the list of all endpoints for all integration flows:

Source Code

```
import java.util.List;

import com.sap.it.api.adapter.monitoring.AdapterEndpointInformation;
import com.sap.it.api.adapter.monitoring.AdapterEndpointInformationService;

public class AdapterEndpointInfo implements AdapterEndpointInformationService{
    /**
     * Gets the list of all endpoints for all IntegrationFlows used by the
     * adapter
     *
     * @return the list of endpoints
     */
    @Override
    public List<AdapterEndpointInformation> getAdapterEndpointInformation() {
        // TODO Auto-generated method stub
        return null;
    }

    @Override
    public List<AdapterEndpointInformation> getAdapterEndpointInformationByIFlow(String arg0) {
        // TODO Auto-generated method stub
        return null;
    }
}
```

The service implementation must contain the following method, which supports the retrieval of all endpoints exposed across all integration flows:

Source Code

```
package com.sap.it.api.adapter.monitoring;

import java.util.List;
```

```

/**
 * Interface, which adapter developers have to implement to return the endpoints
 * (URLs) used by their adapter
 */
public interface AdapterEndpointInformationService {

    /**
     * Gets the list of all endpoints for all IntegrationFlows used by the
     * adapter
     *
     * @return the list of endpoints
     */
    public List<AdapterEndpointInformation> getAdapterEndpointInformationList();

    /**
     * Gets the list of endpoints for a given IntegrationFlow. If the given
     * Integration Flows does not use the adapter, it has to return an empty
     * instance list.
     *
     * @param integrationFlowId
     *         the id (bundle symbolic name) of the Integration Flow
     * @return the endpoint information for the given Integration Flow
     */
    public AdapterEndpointInformation getAdapterEndpointInformation(String integrationFlowId);
}

```

The class describing the **adapter endpoint information** is mentioned here:

Source Code

```

package com.sap.it.api.adapter.monitoring;

import java.util.List;

/**
 * Class representing endpoint information for an Integration Flow
 */
public class AdapterEndpointInformation {
    [...]

    /**
     * Get the endpoint instance list for this Integration Flow.
     *
     * @return the endpoint instance list
     */
    public List<AdapterEndpointInstance> getAdapterEndpointInstanceList()
    [...]

    /**
     * Set the endpoint instance list for this object.
     *
     * @param adapterEndpointInstanceList
     *         the endpoint instance list
     */
    public void setAdapterEndpointInstanceList(List<AdapterEndpointInstance> adapterEndpointInstanceList)
    [...]

    /**
     *
     * @return the integration flow id (bundle symbolic name)
     */
    public String getIntegrationFlowId()
    [...]

    /**
     * sets the integration flow id
     */
}

```

```

    * @param integrationFlowId
    *         the integration flow id (bundle symbolic name)
    */
    public void setIntegrationFlowId(String integrationFlowId)
    {
    }

```

The class describing the **adapter endpoint instance** is mentioned here:

Source Code

```

package com.sap.it.api.adapter.monitoring;

/**
 * The endpoint instance consists of an <code>endpointCategory</code> which
 * describes the kind of endpoint and an URL relative to the worker node. It is
 * mainly about the distinction between endpoints to which messages can be sent
 * and API definition endpoints from which metadata about the API can be
 * retrieved.<br/>
 * The consumer of the API is responsible for retrieving the correct dispatcher
 * URL for the worker node (group) on which the Integration Flow is
 * deployed.<br/>
 * There is also an optional attribute available to provide additional info
 * about the character of the endpoint in case the category is not sufficient.
 * For CXF based endpoints, the WSDL download is offered in several flavors, for
 * instance, so the adapter can provide this information in that attribute.
 */
public class AdapterEndpointInstance {
    [...]

    /**
     * instantiates the object
     *
     * @param endpointCategory
     *         the endpoints category, see also {@link EndpointCategory}.
     * @param relativeEndpointUrl
     *         the URL relative to the worker nodes dispatcher URI starting
     *         with a forward slash ('/')
     */
    public AdapterEndpointInstance(EndpointCategory endpointCategory, String relativeEndpointUrl)
    {
    [...]

    /**
     * instantiates the object
     *
     * @param endpointCategory
     *         the endpoints category, see also {@link EndpointCategory}.
     * @param relativeEndpointUrl
     *         the URL relative to the worker nodes dispatcher URI starting
     *         with a forward slash ('/')
     * @param additionalInfo
     *         additional information (optional) in order to provide more
     *         information if the category is not self explanatory. For example, in
     *         case the WSDLs are offered in different flavors for different
     *         consumers
     */
    public AdapterEndpointInstance(EndpointCategory endpointCategory, String relativeEndpointUrl, String additionalInfo)
    {
    [...]

    /**
     *
     * @return the endpoints category
     */
    public EndpointCategory getEndpointCategory()
    {
    [...]

    /**
     *
     * @return the URL relative to the worker nodes dispatcher URI starting with

```

```

*          a forward slash ('/')
*/
public String getRelativeEndpointUrl()
    [...]

/**
 *
 * @return additional information (optional) in order to provide more
 *         information if the category is not self explanatory. For example, in case
 *         the WSDLs are offered in different flavors for different
 *         consumers. Can be null
 */
public String getAdditionalInfo()
    [...]
}

```

The *EndpointCategory* can have the following values:

Source Code

```

package com.sap.it.api.adapter.monitoring;

public enum EndpointCategory {
    ENTRY_POINT, DEFINITION_OAS_JSON, DEFINITION_OAS_YAML, DEFINITION_RAML_JSON, DEFINITION_RAML_YAML, DEFINITION_WSDL, DEF
}

```

Binding the Service with the Implementation

Next, you link the implementation to the service by creating a `beans.xml` file in the `src/main/resources/OSGI-INF/blueprint/` path and adding the following snippet:

Source Code

```

<blueprint xmlns="http://www.osgi.org/xmlns/blueprint/v1.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.osgi.org/xmlns/blueprint/v1.0.0 http://www.osgi.org/xmlns/blueprint/v1.0.0/blueprint.
    <bean id="iAdapterEndpointInformationService" class="<add the implementation class path>"
        activation="eager"></bean>
    <service id="IAdapterEndpointInformationService" ref="iAdapterEndpointInformationService"
        interface="com.sap.it.api.adapter.monitoring.AdapterEndpointInformationService">
    </service>
</blueprint>

```

To support the binding snippet, add the following dependency in the `pom.xml` file:

Source Code

```

<dependency>
<artifactId>camel-blueprint</artifactId>
<groupId>org.apache.camel</groupId>
<version>${camel.version}</version>
</dependency>

```

Updating the Dependency to Export the Service

Ensure that the following dependency version is always greater than or equal to **1.25.0**. to export this service from your subsystem.

Source Code

```

<dependency>
<groupId>com.sap.cloud.adk</groupId>
<artifactId>com.sap.cloud.adk.build.archive</artifactId>
<version>1.25.0</version>
</dependency>

```

Deploying the Adapter

After completing the process, deploy the adapter and use the Cloud Integration Web application to consume the adapter in an integration flow. Now, go to the [Manage Integration Content](#) section to verify the endpoint information.

Related Information

[Endpoint Visualization](#)

Managing Cluster Lock in Custom Adapters

Custom Integration Adapters provide a mechanism to acquire cluster-level locks, ensuring that specific actions (e.g., message processing) are executed by only one worker or node instance at a time. This prevents duplicate processing and maintains data consistency across distributed environments.

Prerequisites

- You have the necessary permissions to utilize the public APIs. See [SDK API](#).
- Your adapter should be configured to support cluster lock functionalities. See [Adapter Development Prerequisites](#).

Context

The LockManager interface offers following methods to acquire, release, and manage cluster-level locks.

- **newLock**: Creates a new lock with a specified expiration duration.

Source Code

```
Lock lock = lockManager.newLock("vendor", "component", "lockName", Duration.ofMinutes(30));
```

- **getAllLocksByVendor**: Retrieves all locks associated with a specific vendor.

Source Code

```
Set<LockInfo> locks = lockManager.getAllLocksByVendor("vendor");
```

- **getAllLocks**: Retrieves all locks for the current tenant.

Source Code

```
Set<LockInfo> locks = lockManager.getAllLocks();
```

- **releaseLocks**: Releases a set of locks identified by their lock IDs.

Source Code

```
int releasedCount = lockManager.releaseLocks(lockIds);
```

- **isLockAcquired**: Checks if a lock with a specific ID is currently held.

Source Code

```
boolean isAcquired = lockManager.isLockAcquired("lockId");
```

Procedure

1. Obtain LockManager Service.

Source Code

```
LockManager lockManager = ITApiFactory.getService(LockManager.class, null);
```

2. In your adapter's metadata, define the adapterInstanceID property:

Source Code

```
<Metadata>
  <AdditionalMetadata>
    <Name>requiredIntegrationFlowProperties</Name>
    <Value>adapterInstanceId</Value>
    <GuiLabels>
      <Label language="en"/>
    </GuiLabels>
  </AdditionalMetadata>
</Metadata>
```

This adapterInstanceId now can be accessed in your adapter runtime using an endpoint field. You can now use this value as lockname.

3. Use the newLock or newRuntimeLock method to create a lock.

Source Code

```
Lock lock = lockManager.newLock("vendor", "component", "lockName", Duration.ofMinutes(30));
```

4. Use the tryLock method to attempt acquiring the lock.

Source Code

```
boolean lockStatus = lock.tryLock();
```

5. Once the task is completed, release the lock.

Source Code

```
lock.unlock();
```

API Interface:

Sample Code

```
package com.sap.it.api.clusterlock;

import com.sap.it.api.ITApi;
import com.sap.it.api.clusterlock.types.LockInfo;

import java.time.Duration;
import java.util.Set;
import java.util.concurrent.locks.Lock;

/**
 * LockManager helps adapter developers to acquire, release and get information related to cluster level locks
 *
 */
public interface LockManager extends ITApi {

    /**
     * Gets a new {@link Lock} object which will be used for acquiring the lock
     * @param vendor - this should be the vendor name of the adapter. 'SAP' vendor name is reserved for internal usage
     * @param component - this should be the adapter name which is used while creating the adapter. Length restricted
     * @param lockName - a unique lockName (suggestion: Use either bundle symbolic name or the channelId)
     * @param expireAfter - time period after which lock expires
     * @return {@link Lock}
     */
    Lock newLock(String vendor, String component, String lockName, Duration expireAfter);

    /**
     * Gets a new {@link Lock} object which will be used for acquiring the lock with Runtime Scope.
     * This lock will be automatically released by the cleanup job if the node/worker/instance restarts due to a crash
     * @param vendor - this should be the vendor name of the adapter. 'SAP' vendor name is reserved for internal usage
     * @param component - this should be the adapter name which is used while creating the adapter. Length restricted
     * @param lockName - a unique lockName (suggestion: Use either bundle symbolic name or the channelId)
     * @param expireAfter - time period after which lock expires
     * @return {@link Lock}
     */
    Lock newRuntimeLock(String vendor, String component, String lockName, Duration expireAfter);

}
```

```
* Gets all {@link LockInfo}s by vendor name for the current tenant
* @param vendor - this should be the vendor name of the adapter. 'SAP' vendor name is reserved for internal usage
* @return {@link Set} of {@link LockInfo}s
*/
Set<LockInfo> getAllLocksByVendor(String vendor);

/**
 * Gets all {@link LockInfo}s for the current tenant
 * @return {@link Set} of {@link LockInfo}s
 */
Set<LockInfo> getAllLocks();

/**
 * Releases all the locks held with lockIds
 * @param lockIds - lock identifiers
 * @throws com.sap.it.api.clusterlock.types.LockException (is a RuntimeException)in case of failures
 * @return number of released locks
 */
int releaseLocks(Set<String> lockIds);

/**
 * Returns true if lock with lockId is held else false
 * @param lockId - lock identifier
 * @return true or false
 */
boolean isLockAcquired(String lockId);
```

Troubleshooting

Issue	Impact	Mitigation
Database connection unavailability	Inability to create or release locks.	Implement retry mechanisms in the adapter.
Lock deletion job failure	Locks may persist beyond their expiration, preventing new lock creation.	Implement retry mechanisms in the adapter.
Deployment inconsistencies	Potential for overload on a single node during deployment.	Implement random wait times for lock acquisition to distribute load evenly.
Unreleased locks due to system failures	Message processing downtime.	Release the lock from monitoring manually. See .
Inconsistent shutdown	Potential downtime if old worker nodes fail to shut down properly.	Release the lock from monitoring manually. See .

Results

Your custom adapters can now execute critical tasks by a single node with cluster-level locks, preventing duplicate processing and preserving system integrity.

Enabling ADK Persistency

As an integration developer you can now utilize an adapter API to store and retrieve critical information ensuring data persistence even after node restarts or crashes. This also ensures flexibility and reliability in data management.

You can also integrate this feature into your workflows to enhance system resilience and maintain seamless operations. It has following benefits:

- Reduces event duplication, ensuring efficient and streamlined processing.
- Avoids unnecessary costs by maintaining continuity.

To utilize this feature,

- Add an additional metadata entry in the metadata.xml file for your adapter:

```
<Metadata>
<AdditionalMetadata>
  <Name>ADKStoreRequired</Name>
  <Value>true</Value>
  <GuiLabels>
```

```

        <Label language="en"/>
    </GuiLabels>
</AdditionalMetadata>
</Metadata>

```

- Ensure to refer the adapter.api dependency in the pom.xml file. See [SDK API](#)
- In the endpoint class, the ADK store can be accessed by using the following getter and setter methods:

```

private ADKStore adkStore;

public ADKStore getAdkStore() {
    return adkStore;
}

public void setADKStore(ADKStore adkStore) {
    this.adkStore = adkStore;
}

```

i Note

This API should not be used to store any personal or sensitive information such as passwords and keys.

Frequent calls may impact the performance of the worker.

Enabling Authentication for Servlet Based Adapters

Context

Both Basic and Certificate based authentication can be enabled by performing the steps described here:

Procedure

1. Open the web.xml file and add the filter definition given below:

Source Code

```

<filter>
    <filter-name>ADKAuthenticationFilter</filter-name>
    <filter-class>com.sap.it.api.asdk.cloud.authentication.ADKAuthenticationFilter</filter-class>
</filter>
<filter-mapping>
    <filter-name>ADKAuthenticationFilter</filter-name>
    <url-pattern>/*</url-pattern>
</filter-mapping>

```

2. Open the pom.xml file and include the below import package instruction in the maven-bundle-plugin:

Source Code

```

<Import-Package>com.sap.it.api.asdk.cloud.authentication,*</Import-Package>

```

3. Save the changes.

Results

Now you can connect to the adapter's endpoint either using Basic or Certificate based authentication.

Enabling Tracing for Custom Adapter

Configure a custom adapter with a tracing feature that records the processing of the incoming and outgoing messages in an adapter.

In the integration flow model, the logs generated by the adapter tracing help you to do the following:

- Monitor the overall status of the incoming or outgoing messages
- Troubleshoot specific errors encountered by the adapter during runtime

i Note

The incoming or outgoing messages in an adapter are only recorded if you have set the log level to **Trace** in the Cloud Integration Web application.

Adding Libraries

You can find the latest ADK API in the [Adapter Development Kit](#) section. After downloading the *.zip files, add the following libraries to the **src > main > resources** folder that is in your ADK project:

- com.sap.it.public.adapter.api.jar
- com.sap.it.public.generic.api.jar

For successful code compilation, right-click the *.jar file and choose **Build Path > Add to Build Path** to add the jar file to the Referenced Libraries of the adapter project.

Adding Dependencies

Open the pom.xml file and add the following dependencies to enable the tracing feature:

Source Code

```
<dependency>
  <groupId>com.sap.it.public</groupId>
  <artifactId>generic.api</artifactId>
  <version>2.12.0</version>
  <scope>system</scope>
  <systemPath>${project.basedir}/src/main/resources/com.sap.it.public.generic.api-2.12.0.jar</systemPath>
</dependency>
<dependency>
  <groupId>com.sap.it.public</groupId>
  <artifactId>adapter.api</artifactId>
  <version>2.12.0</version>
  <scope>system</scope>
  <systemPath>${project.basedir}/src/main/resources/com.sap.it.public.adapter.api-2.12.0.jar</systemPath>
</dependency>
```

i Note

Make sure that the version of the *.jar files is greater than **2.12.0**.

After adding the dependency, add the com.sap.it.public group ID to the `<excludeGroupIds>` tag.

Enabling Tracing

The tracing feature is only relevant for adapters that transform the message either before sending or upon reception.

→ Recommendation

- Inbound messages are traced before the transformation takes place, which means that the user can see the incoming message with its original payload and headers. Outbound messages (headers and payload) are traced after the transformation has taken place.
- The trace data is written even if errors occur. This can help the user to analyze transformation errors for inbound messages or connection errors for outbound messages.
- The inbound message is traced after authentication and authorization have taken place. This is to prevent denial-of-service attacks that flood the trace database table.
- Security-relevant header values must be obfuscated.
- Trace data is only written if the adapter trace is enabled for the integration flow.
- If possible, also set the character encoding for the trace payload.

The following procedure helps you to implement adapter tracing in either Consumer or Producer component:

1. Use the following code snippet to retrieve the *Adapter Message Log Factory* from your endpoint:

Source Code

```
/** get Adapter Message Log Factory */
AdapterManagerLogFactory msgLogFactory = (AdapterManagerLogFactory) this.endpoint.getCamelContext().getRegistry()
.lookupByName(com.sap.it.api.msglog.adapter.AdapterMessageLogFactory.class.getName());
```

2. Use the following code snippet to create the *Adapter Message Log* instance from the *Adapter Message Log Factory*.

Source Code

```
AdapterManagerLog mplLog = adapterMessageLogFactory.getMessageLog(exchange, description, "<CMD component ID of the ada
UUID.randomUUID().toString());
```

3. Use the snippet to create the trace message:

Source Code

```
AdapterTraceMessage traceMessage = mplLog.createTraceMessage(type, body, truncated );
```

4. Use the snippet to write the trace message to the MPL log:

Source Code

```
mplLog.writeTrace(traceMessage);
```

The interfaces available for *Adapter Message Log* are:

```
/**
 * Returns <code>true</code> if the adapter tracing for the integration flow to
 * which this message log belongs is active, otherwise <code>false</code>.
 */
boolean isTraceActive();

/**
 * Creates a trace message which can be used in the method
 * {@link #writeTrace(AdapterTraceMessage)}.
 *
 * The supplied message body shall not exceed 25 MB = 25 * 1024 * 1024 bytes =
 * 26214400 bytes. We strongly recommend to supply a truncated copy of the
 * original message body if the original message body exceeds 25 MB. If, despite
 * of this rule, the supplied message body still exceeds 25 MB, then the body
 * will be truncated internally and the property <tt>isBodyTruncated</tt> will
 * be set to <code>true</code>.
 *
 * @param traceMessageType
 *         type of the trace message
 * @param body
 *         message body, shall not exceed 25 MB, truncate the body if
 *         necessary
 * @param isBodyTruncated
 *         indicator whether the supplied message body is truncated
 * @throws IllegalArgumentException
 *         if <tt>traceMessageType</tt> or <tt>body</tt> is
 *         <code>null</code>
 */
AdapterManagerLog createTraceMessage(AdapterTraceMessageType traceMessageType, byte[] body, boolean isBodyTruncated)

/**
 * Writes a trace message into the adapter trace if the trace for the
 * integration flow of the adapter is active.
 *
 * <p>
 * <b>Attention:</b>Camel endpoints, which are used in several from elements of
 * different Camel routes cannot write traces. This method will not have any
 * effect for such kind of endpoints.
 * <p>
 * Messages incoming to the adapter (inbound messages) shall be traced, if the
 * adapter transforms the message before it is handled over to the Camel route;
 * the trace shall be written <b>before</b> the transformation takes place.
 * Transformation here means changes to the message body or headers.
 * <p>
 * Messages outgoing from the adapter (outbound messages) shall be traced if the
 * adapter transforms the message before it is sent out; the trace shall be
 * written <b>after</b> the transformation.
 * <p>
 * Typical usage:
 *
 * <pre>
 * {
 *     if (this.isTraceActive()){
 *         byte[] body = getTraceBody(originalMessage);
 *         boolean isBodyTruncated = isBodyTruncated(body,originalMessage);
```

```

*          // specify the correct adapter message type, in this example we use AdapterTraceMessageType.SENDER_INBOUND
*          AdapterTraceMessage traceMessage = this.createTraceMessage(AdapterTraceMessageType.SENDER_INBOUND, body, :
*          // fill the optional properties of the trace message: encoding and headers
*          traceMessage.setEncoding(getEncoding());
*          traceMessage.setHeaders(getHeaders());
*          this.writeTrace(traceMessage);
*      }
*  </pre>
*
*  @param traceMessage trace message
*/
void writeTrace(AdapterTraceMessage traceMessage);

```

Write the trace data. You can use the following example for reference:

❖ Example

≡ Sample Code

```

import org.apache.camel.Exchange;
import com.sap.it.api.msglog.adapter.AdapterMessageLog;
import com.sap.it.api.msglog.adapter.AdapterMessageLogFactory;
import com.sap.it.api.msglog.adapter.AdapterTraceMessage;
import com.sap.it.api.msglog.adapter.AdapterTraceMessageType;

// OSGi service
private final AdapterMessageLogFactory adapterMessageLogFactory;

void writeTrace(Exchange exchange, byte[] traceData, boolean isOutbound) {
    // replace "<adapter type>" by your adapter type!
    String text = isOutbound ? "Sending <adapter type> message" : "Receiving <adapter type> message";
    // replace "<CMD component ID of the adapter>" by your CMD component ID
    AdapterMessageLog mplLog = adapterMessageLogFactory.getMessageLog(exchange, text, "<CMD component ID of the adapte
        UUID.randomUUID().toString());
    if (!mplLog.isTraceActive()) {
        return;
    }
    // if you have a fault inbound message then specify AdapterTraceMessageType.RECEIVER_INBOUND_FAULT,
    // if you have a fault outbound message then specify AdapterTraceMessageType.SENDER_OUTBOUND_FAULT
    // for synchronous adapters you may also need AdapterTraceMessageType.SENDER_OUTBOUND and AdapterTraceMessageType.
    AdapterTraceMessageType type = isOutbound ? AdapterTraceMessageType.RECEIVER_OUTBOUND : AdapterTraceMessageType.SE

    AdapterTraceMessage traceMessage = mplLog.createTraceMessage(type, traceData, false );//Setting isTruncated as fal
    // Encoding is optional, but should be set if available.
    traceMessage.setEncoding("UTF-8");
    // Headers are optional and do not forget to obfuscate security relevant header values.
    mplLog.writeTrace(traceMessage);
}

```

5. Build and deploy the adapter.

Viewing the Trace Data

To show the trace information details, select an integration flow from the overview list, go to the [Log](#) section, and open the link by clicking the [Trace](#). For more information, see [Message Processing Log - Adapter Tracing](#).

Logs

[Open Text View](#)

Trace data is removed after the configured retention time, typically 1 hour.

Log Level: **Trace**

Process ID:

Related Information

[Setting Log Levels](#)

Enabling Scheduler Support for ADK Sender Adapter

Context

Referenced scheduler component is equipped with an in-built locking feature for sender integration adapter. Also, a scheduler component comes with a standard User Interface. Follow these steps to use scheduler component in your custom adapter:

Procedure

1. Implement the polling sender by extending `<ScheduledPollingConsumer>`. Other consumers aren't supported.
2. Add the referenced component for the Sender Variant.

Source Code

```
<ReferencedComponents>
  <ReferencedComponent>
    <ReferencedComponentId>ctype::ExtensionVariant/cname::sap:Scheduler/version::1.0</ReferencedComponentId>
    <AttributeMetadataConfiguration>
      <Name>scheduleKey</Name>
      <AttributeBehavior>Scheduler_ScheduleOnDay,Scheduler_ScheduleToRecur</AttributeBehavior>
    </AttributeMetadataConfiguration>
  </ReferencedComponent>
</ReferencedComponents>
```

i Note

Currently, only the above-listed behavior options `<Scheduler_ScheduleOnDay>` and `<Scheduler_ScheduleToRecur>` are supported.

3. Set the variant level attribute `<useDefaultScheduler>` as FALSE or skip it.

Source Code

```
<Variant VariantName="VariantName" VariantId="ctype::AdapterVariant/cname::GreetingsDs/vendor::sap/tp::sap-sample/mp::
  MetadataVersion="2.0" gen:RuntimeComponentBaseUri="sap-sample" AttachmentBehavior="Preserve"
  useDefaultScheduler="false">
  ...
</Variant>
```

4. Ensure that all the scheduler-related references (UI) aren't present in CMD.

Enabling Connection Status for Integration Flow

Prerequisites

This is custom documentation. For more information, please visit [SAP Help Portal](#).

Context

You can use the public API to publish the status of the connection to the integration flow monitoring for ADK based integration adapters. For example, this can provide you more insight on consumption of integration flow connection status.

Use the below interface, which is available for the ADK based integration adapters:

Source Code

```
package com.sap.it.api.adapter.iflowmonitoring;

import com.sap.it.api.ITApi;
import org.apache.camel.Endpoint;

/**
 * Service to submit an event on the status of the Connection as and when necessary.
 * <p>
 * The class can be initialized via the following statement:
 * </p>
 * <code>
 * IFlowMonitorService monitorService = ITApiFactory.getService(IFlowMonitorService.class, null);
 * </code>
 * <p>
 * EndPoint and EventDetails need to be passed.<br><br>
 * </p>
 * This interface must
 * neither be extended nor implemented by clients.
 * * @since 2.28.0
 */
public interface IFlowMonitorService extends ITApi {
    /**
     * Publishes event for the integration flow
     * @param endpoint The camel endpoint
     * @param eventDetails {@link EventDetails } Contains the details of the event
     * @throws IFlowMonitorServiceException The exception is thrown whenever endpoint/camelContext/EventStatus is invalid
     */
    void publishEvent(Endpoint endpoint, EventDetails eventDetails) throws IFlowMonitorServiceException;
}
```

Procedure

1. Invoke the `getService` method to get the integration flow monitor service API.

Source Code

```
IFlowMonitorService monitorService = ITApiFactory.getService(IFlowMonitorService.class, null);
```

2. Based on your requirement, invoke the `publishEvent` based on the event status type. You should use this API to submit an event on the integration flow.

Sample Code

Below example shows event submission in success state.

```
EventDetails eventDetails =new EventDetails();
eventDetails.setEventStatus(EventStatus.OK);
monitorService.publishEvent(endpoint, eventDetails);
```

Sample Code

Below example shows event submission in error state.

```
EventDetails eventDetails =new EventDetails();
eventDetails.setEventStatus(EventStatus.ERROR);
eventDetails.setException(exception);
```

```
monitorService.publishEvent(endpoint, eventDetails);
```

Accessing User Credentials

Context

Follow the procedure below to get user credentials in an ADK project.

Procedure

1. Download the latest ADK API from the [Adapter Development Kit](#) section.
2. Add the `com.sap.it.public.generic.api.jar` file to the `resources` folder that is in your ADK project.
3. Open the `pom.xml` file and add the dependency below:

Source Code

```
<dependency>
  <groupId>com.sap.it.public</groupId>
  <artifactId>api</artifactId>
  <version>2.8.0</version>
  <scope>system</scope>
  <systemPath>${project.basedir}/src/main/resources/com.sap.it.public.generic.api-2.8.0.jar</systemPath>
</dependency>
```

4. Add the `com.sap.it.public` group ID to the `<excludeGroupIds>`.
5. To import user credentials, use the code below:

Source Code

```
SecureStoreService secureStoreService = IApiFactory.getService(SecureStoreService.class, null);
UserCredential userCredential = secureStoreService.getUserCredential();

/**
 *Provide an alias name in getUserCredential()
 */
```

Add the following import declarations:

Source Code

```
import com.sap.it.api.securestore.SecureStoreService;
import com.sap.it.api.securestore.UserCredential;
import com.sap.it.api.securestore.exception.SecureStoreException;
import com.sap.it.api.IApiFactory;
```

Accessing Trust and Key Managers

Context

Follow the procedure below to get trust and key managers in an ADK project.

Procedure

1. Download the latest ADK API from the [Adapter Development Kit](#) section.
2. Add the `com.sap.it.public.generic.api.jar` file to the `resources` folder that is in your ADK project.
3. Open the `pom.xml` file and add the dependency below:

Source Code

```
<dependency>
  <groupId>com.sap.it.public</groupId>
  <artifactId>api</artifactId>
  <version>2.8.0</version>
```

```
<scope>system</scope>
<systemPath>${project.basedir}/src/main/resources/com.sap.it.public.generic.api-2.8.0.jar</systemPath>
</dependency>
```

4. Add the `com.sap.it.public` group ID to the `<excludeGroupIds>`.

5. Depending on your requirements, add the code below to either the `*Producer.java` (receiver adapter) or `*Consumer.java` (sender adapter) file. This allows you to import all the trust managers.

Source Code

```
KeystoreService keyStoreService = ITApiFactory.getService(KeystoreService.class, null);
TrustManager[] trustmanager = keyStoreService.getTrustManagers();
/**
 *Provide an alias name in getTrustManagers()
 */
```

Add the following import declarations:

Source Code

```
import javax.net.ssl.TrustManager;
import com.sap.it.api.ITApiFactory;
import com.sap.it.api.exception.InvalidContextException;
import com.sap.it.api.keystore.KeystoreService;
import com.sap.it.api.keystore.exception.KeystoreException;
```

6. To import all the key managers, use the code below:

Source Code

```
KeystoreService keyStoreService = ITApiFactory.getService(KeystoreService.class, null);
KeyManager[] keymanagers = keyStoreService.getKeyManagers();
/**
 *Provide an alias name in getKeyManagers()
 */
```

Add the following declaration:

Source Code

```
import javax.net.ssl.KeyManager;
```

Accessing On-Premise Application using Cloud Connector

Follow the steps here to extend the connectivity of Cloud Integration with remote on-premise application.

Prerequisites

You must have the access to Public APIs. For more information, see [SDK API](#).

Context

You can prepare an Adapter Development Kit (ADK) project to access any on-premise application using Cloud Connector.

Procedure

1. Depending on your requirements, add the code below to either the `*Producer.java` (receiver adapter) or `*Consumer.java` (sender adapter) file. This is for connecting to on-premise application.

a. For HTTP

Source Code

Use the below code to fetch the proxy details and additional headers.

```
CloudConnectorContext context = new CloudConnectorContext();
context.setConnectionType(ConnectionType.HTTP);
CloudConnectorProperties props = ITApiFactory.getService(CloudConnectorProperties.class, context);
String proxyHost = props.getProxyHost();
```

```
String proxyPort = props.getProxyPort();
Map<String, String> additionalHeaders = props.getAdditionalHeaders();
```

Add the following import declarations:

Source Code

```
import com.sap.it.api.ITApiFactory;
import com.sap.it.api.ccs.adapter.CloudConnectorContext;
import com.sap.it.api.ccs.adapter.CloudConnectorProperties;
import com.sap.it.api.ccs.adapter.ConnectionType;
```

Sample Code

The example here shows how you can use the above proxy details in HTTP client. Ensure to provide the location ID of the Cloud Connector using the header `SAP-Connectivity-SCC-Location_ID`.

```
DefaultHttpClient httpClient = new DefaultHttpClient();
HttpGet httpGet = new HttpGet(address);
HttpHost proxy = new HttpHost(proxyHost, proxyPort);
httpClient.getParams().setParameter(ConnRoutePNames.DEFAULT_PROXY, proxy);
httpGet.setHeader("SAP-Connectivity-SCC-Location_ID", locationId);
if (additionalHeaders != null && !additionalHeaders.isEmpty()) {
    for (Entry<String, String> header : additionalHeaders.entrySet()) {
        httpGet.setHeader(header.getKey(), header.getValue());
    }
}
HttpResponse response = httpClient.execute(httpGet);
```

b. For TCP

Source Code

Depending on the environment type there are two types of proxy authentication; via JWT access token and via Username and password. The simplest way to determine the type to be used is to look at cloud connector properties.

```
CloudConnectorContext context = new CloudConnectorContext();
context.setConnectionType(ConnectionType.TCP);
```

```
CloudConnectorProperties cloudConnectorProperties = ITApiFactory.getService(CloudConnectorProperties.class, context);
AuthType authType = cloudConnectorProperties.getAuthType();
```

Source Code

Proxy host and proxy port of the cloud connector can be accessed using the below API:

```
cloudConnectorProperties.getProxyPort();
cloudConnectorProperties.getProxyHost();
```

The following are intended to use with appropriate authentication methods only.

Source Code

For use with `AuthType.PasswordAuthentication`

```
cloudConnectorProperties.getUserName(locationId)
```

For more information, see [Username Password Authentication](#)

Source Code

For use with `AuthType.JWTTokenAuthentication`

```
cloudConnectorProperties.getJWTToken()
```

For more information, see [JWT Access Token Authentication](#)

2. Save the changes.

Versioning Rules for Custom Adapters

Follow the guidelines mentioned below to version a custom adapter.

i Note

This versioning feature is available from ADK version 1.38.0.

Versioning Rules

Version Type	Applicable For	Rules
Micro Version	Small changes such as label and tooltip updates	- Existing metadata variant should be updated with new micro version - For example, if the current version is 1.0.0, then the version has to be updated to 1.0.1 both at the component and variant level. - Relevant metadata changes has to be done in the same file.
Minor Version	Adding a new feature i Note It is not recommended to add mandatory fields. If needed, ensure that the runtime is backward compatible.	- Each variant should be in a single metadata file. If both sender and receiver version has to be updated, two separate metadata files has to be created for each. - Both the component and variant version has to be updated. - Necessary metadata changes has to be done in the newly created metadata file. - The existing version remains unchanged. So, the existing scenarios will have old user interface and it should continue to work as before. Only new scenarios will reflect the new version changes.
Major Version	Incompatible Changes i Note This is not supported.	

The step by step procedure to perform a micro and minor version update is explained below.

Micro Version Update

1. Ensure that the necessary changes has been made to the existing metadata variant.
2. Increment the micro version of the existing metadata both at component level and variant level.
3. Increment the version in `config.adk`.

Minor Version Update

1. Copy the variant of latest version to a new metadata file. Only one variant should exist within a metadata file for any new version.

i Note

The same metadata directory will contain multiple metadata files. Any naming convention can be followed for versioned files.

2. Make the minor version increment to the new metadata both at component and variant level.
3. Make the necessary changes to the metadata.
4. Make the necessary runtime changes.

i Note

Make sure the runtime is compatible with earlier version and the existing scenarios should continue to work.

5. Increment the version in `config.adk`.

Deploying a Secure Parameter Artifact

Use the secure parameter artifact to deploy confidential data for custom adapters.

Prerequisites

In the Node Explorer you have selected a tenant, in the context menu you have selected [Deploy Artifacts](#), and you have specified [Secure Parameter](#) as the artifact type.

Context

You can store secure data like passwords in a secure store and use an alias name to access this data in an integration flow. The secure parameter artifact contains this alias name and confidential data.

You can deploy the artifact on the tenant by following the procedure below.

Procedure

1. Specify the attributes on the wizard page.

Option	Description
Name	Provide a name for the artifact. The artifact name is used as an alias for the confidential data assigned by this parameter.
Description	Provide a more detailed description of the artifact.
Secure Parameter	Enter the confidential value of the attribute. i Note The possible length of the secure parameter varies: - for Cloud Foundry: 4096 characters max. (including spaces) - for Neo: 255 characters max. (including spaces)
Repeat Secure Parameter	Repeat the confidential value of the attribute.

2. Choose [Finish](#).

i Note

If correct alias is configured in the integration flow, then the runtime framework passes the secured artifact value to the component at runtime from the secure store.

Parameterizing Integration Flows Using the Partner Directory

The Partner Directory is a tenant-specific storage option that allows you to store information on business partners that are connected to the tenant in the context of a larger business network.

The Partner Directory facilitates the management of integration scenarios encompassing communication networks with numerous partners. In these scenarios, integration flows are parameterized with partner-specific parameters that are centrally stored in the Partner Directory. At runtime, integration flows can read information from the Partner Directory, based on the partner-specific values contained in the incoming request or the payload of the inbound message. This *dynamic* usage of integration flow parameters provides the advantage of adding new partners to the network without the need to edit and redeploy any integration flow, allowing for the extension of the business partner network without any downtime of the scenario.

i Note

Partner-specific information comprises information such as endpoints, XSLT mappings, XSD definitions, or certificates (to mention some examples).

You can access (read/write) the content of the partner directory through the OData API as well as during message processing through certain (parameterized) integration flow steps.

To display and maintain Partner Directory entries, there are the following options:

Option	Description
User interface access	In the Monitor section, you can access the Partner Directory, display existing entries and create new ones. See: Managing Partner Directory Entries
API access	You can access (read/write) its content through the OData API as well as during message processing through certain (parameterized) integration flow steps. See: Partner Directory

Want to Learn More

For an introduction into the concepts, see: [Partner Directory Concepts](#).

For information on the OData API access to the Partner Directory, see: [Partner Directory](#).

For information on the integration flow components that can be parameterized with Partner Directory content, see [Parameterizable Integration Flow Components](#).

Dedicated integration flow design guidelines show you how to use the Partner Directory. For more information, see [Use the Partner Directory Appropriately](#).

The following SAP Community blogs introduce you step-by-step to the topic:

- [Cloud Integration – Partner Directory – Step-by-Step Example](#) 
- [Cloud Integration – Partner Directory – Partner Dependent XML Structures and IDs](#) 
- [Cloud Integration – Partner Directory – Sender Partner Connecting with Client Certificate Authentication](#) 
- [Cloud Integration – Partner Directory – Partner Dependent User Credential Selection](#) 
- [Cloud Integration – Partner Directory – Mass Configuration](#) 

Caution

Limitations

Be aware of the following limitations when working with the Partner Directory:

Size restrictions for the different entity types:

- Maximum number of StringParameters overall: 3,000,000 (corresponds to 10,000 partners each using 300 StringParameters)
- Maximum number of BinaryParameters overall: 400,000 (corresponds to 10,000 partners each using 40 BinaryParameters)
- Maximum number of AlternativePartners overall: 1,000,000 (corresponds to 10,000 partners each using 100 AlternativePartners)
- Maximum number of AuthorizedUsers overall: 500,000 (corresponds to 10,000 partners each using 50 AuthorizedUsers)

Limit for certificate-to-user mapping (when using the Neo environment): 2 MB (corresponds to about 2000 X.509 certificates)

This component stores data on your tenant. Note that the tenant space is limited and shared with other tenant data like message processing logs.

i Note

The maximum size of a keystore is 2 MB (when using the Neo environment).

- The 2 MB limit corresponds to around 2000 X.509 certificates.
- A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 600 key pairs in the keystore.

The maximum size of a keystore is 6 MB (when using the Cloud Foundry environment).

- The 6 MB limit corresponds to around 6000 X.509 certificates.
- A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 1800 key pairs in the keystore.

If you upload a whole keystore (.jks file) to the tenant, the maximum keystore size is limited to 2 MB.

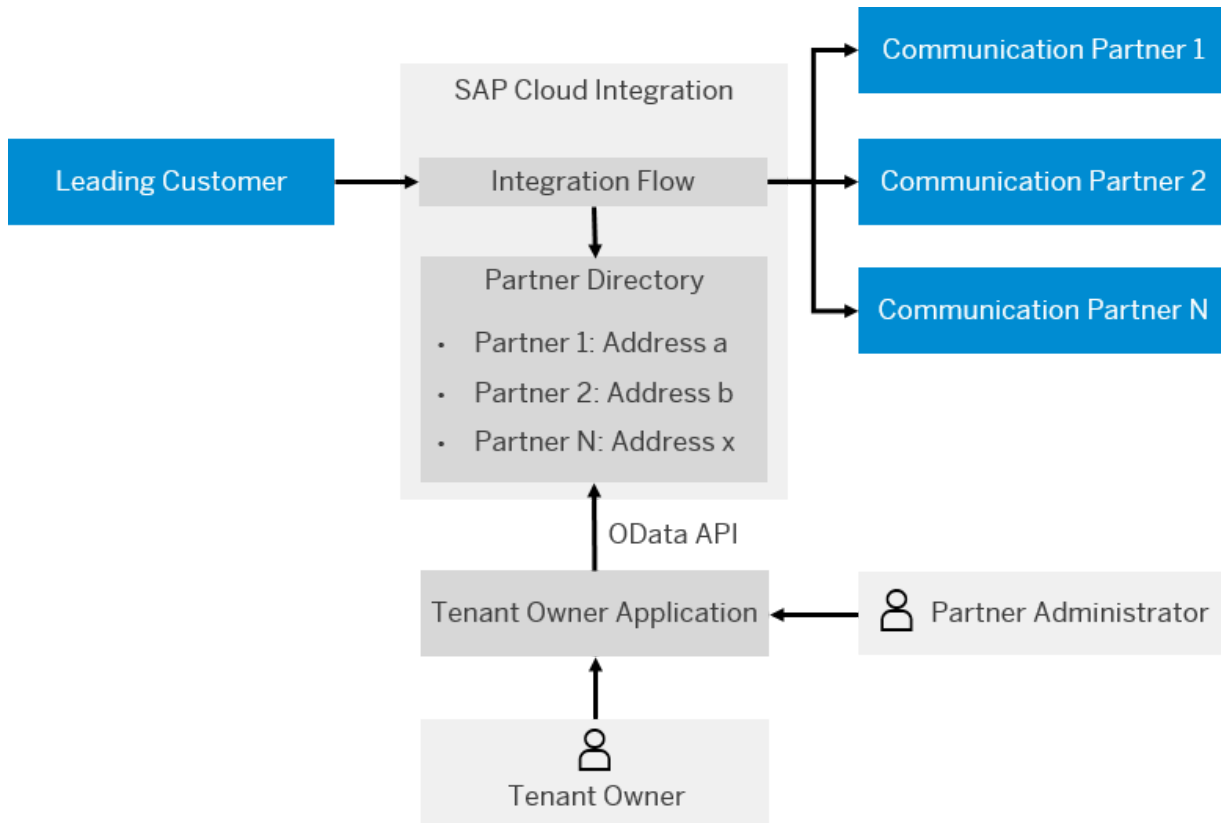
Partner Directory Concepts

In the context of a business-to-business (B2B) scenario involving a partner network, the person or organization that is responsible for the B2B scenario as a whole is also the owner of the SAP Cloud Integration tenant.

Administrators at the side of each business partner use a dedicated application (referred to as tenant owner application) to maintain entities in the Partner Directory (through an OData API). The tenant owner application is provided to the partners by the tenant owner.

Note that such a tenant owner application is not part of the feature set of SAP Cloud Integration. However, SAP Cloud Integration comes with a set of OData APIs that allow access to the Partner Directory (and can be used to implement such a tenant owner application).

As illustrated in the figure, the Partner Directory is embedded in the system landscape in the following way.



Partner Directory Entities

The Partner Directory contains the following entities (which can be accessed using an OData API):

- Partner ID (PID)

A partner has an ID (PID) that is unique within the Partner Directory. The uniqueness of the PID is ensured by the tenant owner application.

- Alternative Partner

A partner can have several alternative identifiers (Alternative Partner). The same concepts are applied to the Alternative Partner as to party identifiers in SAP Process Integration: Each Alternative Partner has three string fields: Agency, Scheme, and ID.

- Agency

Name of the organization that defines the identification scheme (or schema) and issues names for the objects to be identified.

- Scheme

The reference framework within which objects are uniquely identified by names.

- ID

An ID or name that identifies the object within the given scheme.

For more information on the alternative partner ID and how it is related to the partner ID, check out the following SAP Community blog: [Cloud Integration – Partner Directory – Partner Dependent XML Structures and IDs](#).

For more information on the usage of the fields Agency, Scheme, and ID, see the documentation of SAP Process Integration at <http://help.sap.com>.

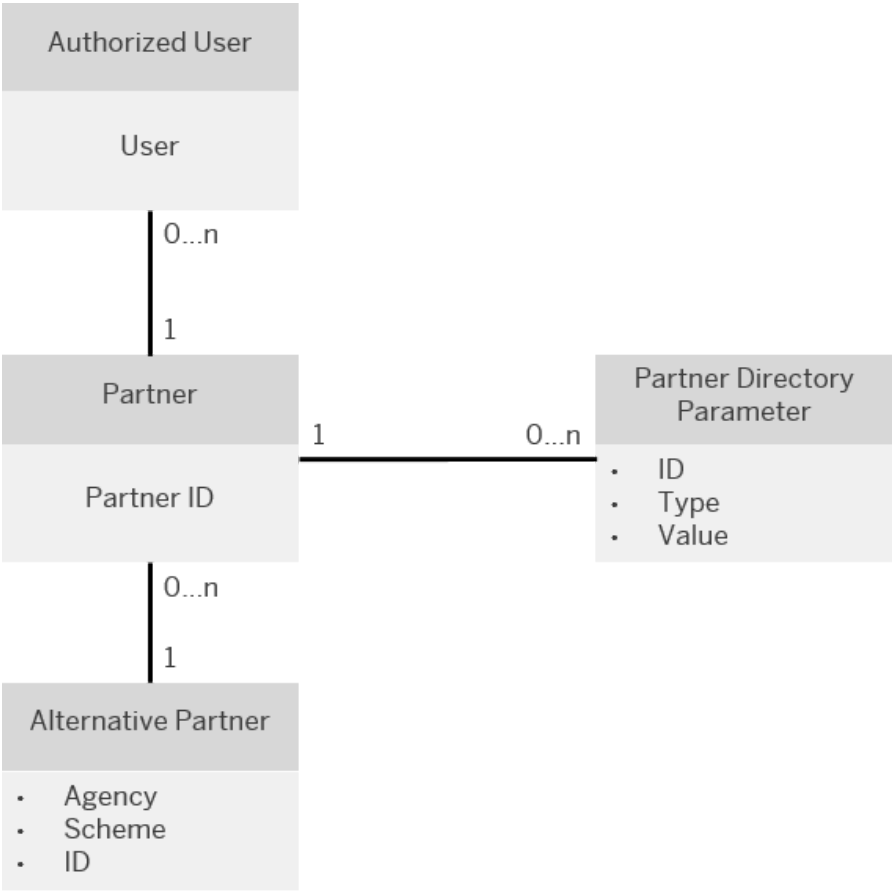
- Authorized User

This user authorizes a sending partner system to log in to SAP Cloud Integration (inbound communication).

If the partner uses HTTPS with client certificate authentication to connect to SAP Cloud Integration, certificate-to-user mappings are applied. One or more Authorized Users can be defined for each partner.

- Partner Directory Parameters for the parameterization of integration flows

The following figure provides an overview of the Partner Directory entities.



The Partner Directory entities related to a partner are maintained by an administrator at the partner organization using the tenant owner application. The Partner Directory entities are accessed using an OData application interface.

Related Information

- [Partner Authorization \(Inbound\)](#)
- [Dynamically Reading XSLT Mappings from the Partner Directory](#)
- [Dynamically Reading XSD Files from the Partner Directory](#)
- [Partner Directory Cache](#)

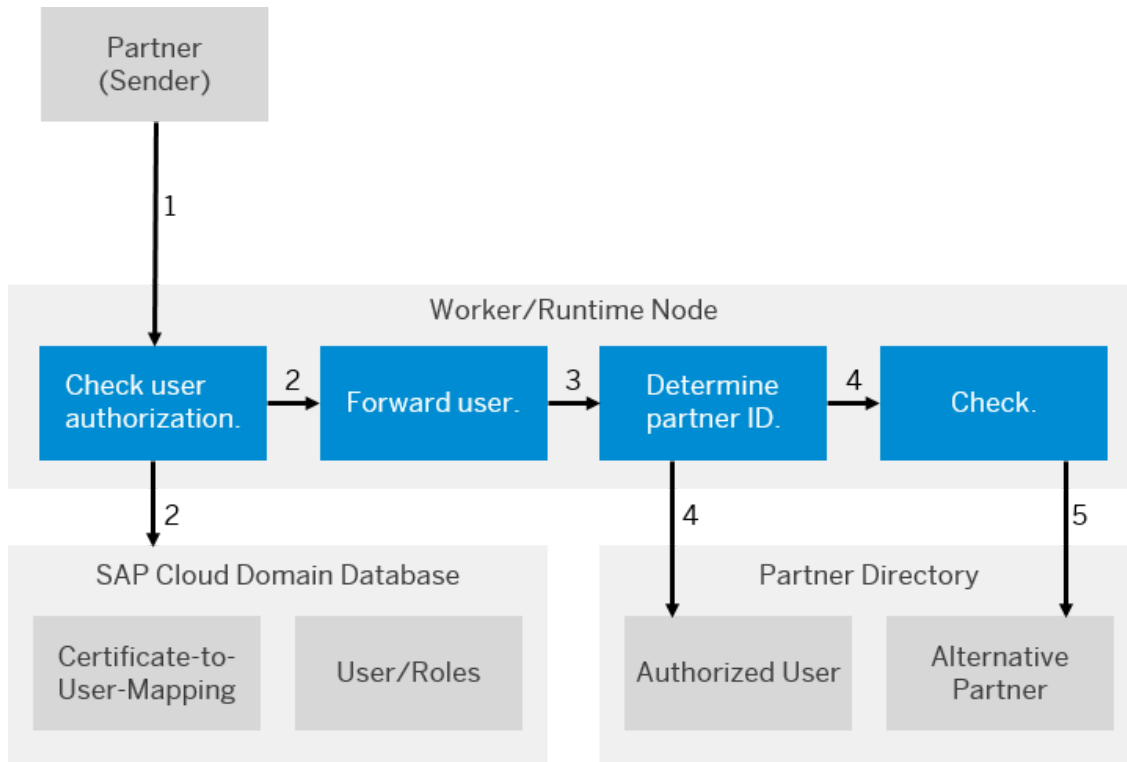
Partner Authorization (Inbound)

For inbound calls (when a partner sends a message to the integration platform), a partner authorization check evaluates whether the logged-in user has permission to execute messages with a particular sender partner ID.

Basic authentication and client certificate authentication using certificate-to-user mappings are both supported.

Note that the authorization check is implemented based on a script contained in the deployed integration flow (referred to as the *authorization check script* in the following).

The following figure shows how the partner authorization check works (the numbers in the figure correspond to those in the numbered list below).



1. The partner sends a message to the integration platform (Cloud Integration) and the user associated with the sender system is logged in.

The sender calls the tenant either by basic authentication, client certificate authentication, or OAuth authentication with client certificate grant.

- If basic authentication is configured, the user is a user from the SAP ID service.
- If client certificate authentication is configured, it is a user defined in a certificate-to-user mapping.
- If OAuth authentication is configured, the user name has the form `oauth_client_<client ID>` where `<client ID>` is the client ID of the OAuth client.

2. Cloud Integration checks whether the role `ESBMessaging.send` is assigned to the logged-in user.

3. The sender adapter forwards the logged-in user as the subject in the Camel header `SapAuthenticatedUserName`.

4. From the user (contained in the Camel header `SapAuthenticatedUserName`, which is read using a [Content Modifier](#) step), the authorization check script determines the Partner IDs by checking the list of Authorized Users in the Partner Directory.

5. The authorization check script reads the Partner ID or the Alternative Partner from the message content. If an Alternative Partner is read from the message content, it is mapped to the Partner ID stored in the Partner Directory.

The authorization check script compares the Partner ID determined from the Authorized User table with the Partner ID derived from the message. If they are identical, the message has been processed.

Note that for each Authorized User (provided by a partner), a user-to-role assignment (to role `ESBMessaging.send`) is automatically generated by the system (using a REST API from SAP BTP) prior to the execution of the scenario.

Dynamically Reading XSLT Mappings from the Partner Directory

You can configure the XSLT Mapping step in an integration flow to dynamically read XSLT mappings from the Partner Directory. The Mapping step will then point to an XSLT mapping defined in the Partner Directory.

To do this, in the XSLT Mapping step, specify the name of a header that points to the XSLT mapping in the Partner Directory.

Select [Header](#) as the [Source](#) in the XSLT Mapping step.

In order for the scenario to work, there must be an additional integration flow step (either a Script step or a Content Modifier step) that sets the header so that the XSLT Mapping step can refer to it.

In this additional step, the header must be specified as a URI in the following format:

`pd:<Partner ID>:<Parameter ID>:Binary`

This URI specifies a `Binary` `Parameter` from the Partner Directory. At runtime, the XSLT Engine resolves this URI.

i Note

XSL documents that are stored in the Partner Directory can be referenced in other XSL and XML mappings by specifying it in the Partner Directory URI (using the `xsl:import` or "document" feature).

For detailed information, check out the [blog post](#) about partner dependent XML structures and IDs.

Example

The integration flow contains a Content Modifier step followed by an XSLT Mapping step. You can configure both steps so that an XSLT mapping is dynamically read from the Partner Directory.

In the **XSLT Mapping** step, specify the following settings:

In the **Source** field, select **Header**, and in the **Header Name** field, specify `XSLT_FROM_PD`. This is the name of the header, which must be filled in a previous step (in our example, a Content Modifier).

In the **Content Modifier** step, specify the following settings (under **Message Header**):

Attribute	Specify With the Following Entry ...
Name	Enter <code>XSLT_FROM_PD</code> .
Type	Select Expression .
Data Type	Enter <code>java.lang.string</code> .
Value	Enter <code>pd:\${property.SENDER}:xslt:Binary</code>. The expression <code>\${property.SENDER}</code> makes sure that the Partner ID is dynamically read from the Partner Directory, the entry <code>xslt</code> is the name of the parameter for the XSLT mapping in the Partner Directory, and the expression <code>Binary</code> classifies it as a binary parameter.

Dynamically Reading XSD Files from the Partner Directory

You can configure the XML Validator step in an integration flow to dynamically read XML schema (XSD) files from the Partner Directory. The XML Validator step will then point to an XSD file in the Partner Directory.

To do this, in the XML Validator step, specify the name of a header that points to the XSD file in the Partner Directory.

In the **XML Schema** field, enter the Camel simple expression for the header that should point to the corresponding Partner Directory entity.

In order for the scenario to work, there must be an additional integration flow step (either a Script step or a Content Modifier step) that sets the header so that the XML Validator step can refer to it.

In the additional step, the header must be specified as a URI in the following format:

`pd:<Partner ID>:<Parameter ID>:Binary`

This URI specifies a `Binary Parameter` from the Partner Directory. At runtime, the XSLT Engine resolves this URI.

i Note

XSL documents that are stored in the Partner Directory can be referenced in other XSL and XML mappings by specifying it in the Partner Directory URI (using the `xsl:import` or "document" feature).

For detailed information, check out the [blog post](#) about partner dependent XML structures and IDs.

Example

The integration flow contains a Content Modifier step followed by an XML Validator step. You can configure both steps so that an XSD file is dynamically read from the Partner Directory.

In the **XML Validator** step, enter the following simple expression as the **XML Schema**: `${header.XSD_FROM_PD}`.

`XSD_FROM_PD` is the name of the header, which must be filled in a previous step (in our example, a Content Modifier).

In the **Content Modifier** step, specify the following settings (under **Message Header**):

Attribute	Specify With the Following Entry ...
Name	Enter XSD_FROM_PD.
Type	Select Expression.
Data Type	Enter java.lang.string.
Value	Enter pd:\${property.SENDER}:xsd:Binary. The expression \${property.SENDER} makes sure that the Partner ID is dynamically read from the Partner Directory, the entry xsd is the name of the parameter for the XSD file in the Partner Directory, and the expression Binary classifies it as a binary parameter.

Partner Directory Cache

To improve performance, Partner Directory is cached.

Entities from the Partner Directory are cached on runtime nodes. From there, they can be accessed with good performance from Script steps in integration flows deployed on the same runtime node.

The following Partner Directory entities are cached:

- StringParameter
- BinaryParameter
- AuthorizedUser

If an object (for a requested key) is not found in the cache, it is loaded from the Partner Directory.

Maximum cache size is restricted to 5% of the total heap size. Elements are evicted from the cache according to the Least Recently Used order.

An entry in the cache is invalidated when the entry is updated or deleted. This invalidation can take a few minutes. The invalidation time depends on the number of changes that are done during design time. The more changes are done at the same time, the longer the invalidation takes.

Parameterizable Integration Flow Components

The following integration flow components are parameterizable so that partner-specific information (such as partner endpoint address, specific mapping, client certificates for inbound calls) can be used at runtime.

- XSLT: XSLT files are specified by each partner
- XML Schema Validator: XSD files are specified by each partner
- HTTP receiver adapter: receiver address (however, one common user or client certificate is used to call the partner system)
- HTTP receiver adapter: user credential
- HTTP sender adapter: sender partner-specific user or client certificate
- SOAP receiver adapter: user credential
- Script: for accessing partner-specific information and performing partner-specific operations

Partner Directory Parameters for AS2 Adapter

The AS2 sender reads the following parameters from the Partner Directory:

Field	Parameter	Value
Decrypt Message	SAP_AS2_Inbound_Decrypt_Message	- true - false
Private Key Alias	pd:<pd parameter name>	Parameter name of private key alias in Partner Directory.
Verify Signature	SAP_AS2_Inbound_Verify_Signature	- notRequired - trustedCertificate

Field	Parameter	Value
		trustedRootCertificate
Public Key Alias	pd:<pd parameter name>	Parameter name of public key in Partner Directory.

AS2 MDN sender reads the following parameters from Partner Directory:

Field	Parameter	Value
Verify Signature	SAP_AS2_Inbound_Mdn_Verify_Signature	- true - false
Verify MIC	SAP_AS2_Inbound_Mdn_Verify_Mic	- true - false
Public Key Alias	pd:<pd parameter name>	Parameter name of public key in Partner Directory.

i Note

PD parameters are shown in the MPL log as MPL properties.

API Documentation

Cloud Integration provides various application programming interfaces (APIs) with which you can access data.

Overview

Cloud Integration provides SDK APIs (or Java APIs) and OData APIs for accessing various aspects of the platform. Furthermore, OData APIs can be easily found and tested in the SAP Business Accelerator Hub. For further information, simply click on the tiles below:

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.



Please note that image maps are not interactive in PDF outputs.

SDK API

APIs can be accessed using either Groovy Script or Java Script as the programming language. Besides, they can also be used when developing SAP Cloud Integration adapter using the Software Development Kit (short: SDK). For more information, see [SDK API](#).

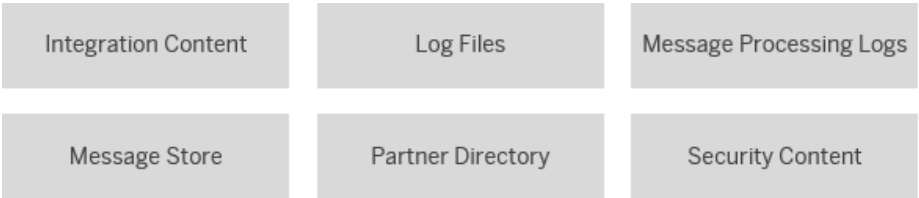
OData API

OData APIs are implemeted as REST API and use the Open Data Protocol as a technical protocol. For more information, see [OData API](#).

OData APIs on SAP Business Accelerator Hub

Cloud Integration OData APIs can be tested and explored on SAP Business Accelerator Hub under <https://api.sap.com/package/CloudIntegrationAPI/odata>. They are structured around entity types or resources. Click on the tiles below to find more information on the entity types and aspects.

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.



Please note that image maps are not interactive in PDF outputs.

The OData APIs are protected by basic authentication (username and password). For more information, click on the tile below:

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.

Authentication

Please note that image maps are not interactive in PDF outputs.

Additionally, the OData APIs make use of common query options:

This image is interactive. Hover over each area for a description. Click highlighted areas for more information.

Query Options

Please note that image maps are not interactive in PDF outputs.

SDK API

SDK APIs refer to the overall set of APIs like the Generic API, Adapter API, Script API.

Application programming interfaces (APIs) are available for the following use cases:

- Using a programming language (Java Script or Groovy), you can control how a message is being processed on a tenant.
- Using an API, you can build your own custom adapter in the context of the Adapter Development Kit.

i Note

The Java standard libraries of Java 8 can be used.

Cloud Integration supports the XML Document Object Model (DOM) to process XML documents.

You can implement logging by using Simple Logging Façade for Java (SLF4J) API package. We recommend using SLF4J framework over using your own logging frameworks.

Public APIs in Maven Central Repository

Some of the public APIs are available in the Maven Central Repository. Look out for the specific APIs with their associated coordinates. Also, for detailed information about these APIs, refer to their corresponding JavaDocs mentioned in the next paragraphs.

Adapter API

Use Adapter APIs if you want to develop custom adapters.

To access the Adapter API from [Maven Central Repository](#) , do the following steps:

1. Look out for the latest public API in Maven Central Repository. The group ID is `com.sap.cloud.adk` and artifact ID is `adapter.api`.
2. Add the maven dependency for the API in your adapter pom file. For example, the sample that follows is a way to consume the adapter API.

Sample Code

```
<dependency>
  <groupId>com.sap.cloud.adk</groupId>
  <artifactId>adapter.api</artifactId>
  <version>XX.YY.ZZ</version>
  <!-- replace XX.YY.ZZ with the latest version -->
</dependency>
```

3. Add `com.sap.cloud.adk` to the `excludeGroupIds` section in the **Configuration Settings** of the **Maven Dependency Plugin**. For example:

Sample Code

```
<excludeGroupIds>com.sap.cloud.adk,org.apache.camel,org.slf4j,log4j</excludeGroupIds>
```

From the [versions](#) ↗ tab on Maven central repository, choose [Browse](#) for the respective version to access the [javadoc](#) ↗ . The javadoc for Adapter API contains the complete set of packages and classes.

i Note

On how to consume cluster lock feature from the package, see [Component Metadata](#).

Script API

Use the Script APIs (version 1 or 2) if you want to use the Script step in your integration flow.

Look out for latest public script API [version 1](#) ↗ and [version 2](#) ↗ from Maven Central Repository. Use the maven coordinates that follow in your project to consume the script API.

For Script version 1 (v1):

≡ Sample Code

```
<dependency>
  <groupId>com.sap.cloud.script</groupId>
  <artifactId>script.api</artifactId>
  <version>XX.YY.ZZ</version>
  <!-- replace XX.YY.ZZ with the latest version -->
</dependency>
```

The [javadoc for Script v1 API](#) contains the complete set of packages and classes.

For Script version 2 (v2):

≡ Sample Code

```
<dependency>
  <groupId>com.sap.cloud.script</groupId>
  <artifactId>script.v2.api</artifactId>
  <version>XX.YY.ZZ</version>
  <!-- replace XX.YY.ZZ with the latest version -->
</dependency>
```

The [javadoc for Script v2 API](#) ↗ contains the complete set of packages and classes.

Generic API

Generic APIs are common for Script step and developing custom adapters.

To access the Generic API from [Maven Central Repository](#) ↗ , do the following steps:

1. Look out for the latest public API in Maven Central Repository. The group ID is `com.sap.cloud.adk` and artifact ID is `generic.api`.
2. Add the maven dependency for the API in your adapter pom file. For example, the sample that follows is a way to consume the generic API.

≡ Sample Code

```
<dependency>
  <groupId>com.sap.cloud.adk</groupId>
  <artifactId>generic.api</artifactId>
  <version>XX.YY.ZZ</version>
  <!-- replace XX.YY.ZZ with the latest version -->
</dependency>
```

3. Add `com.sap.cloud.adk` to the `excludeGroupIds` section in the [Configuration Settings](#) of the [Maven Dependency Plugin](#). For example:

≡ Sample Code

```
<excludeGroupIds>com.sap.cloud.adk,org.apache.camel,org.slf4j,log4j</excludeGroupIds>
```

From the [versions](#)  tab on Maven central repository, choose [Browse](#) for the respective version to access the javadoc. The [javadoc](#)  for Generic API contains the complete set of packages and classes.

Mapping API

The [javadoc for Mapping API](#) contains the complete set of packages and classes.

Related Information

[Developing Custom Adapters](#)

Setting and Getting SOAP Headers

Understand how SOAP headers can be accessed, set, and read through an API.

A SOAP message consists of SOAP headers and a SOAP body. SOAP Headers need to be accessed by an API for further processing.

SOAP Header Attributes

The attributes that can be used in the SOAP header are conform to the SOAP specifications. You can set two attributes for the SOAP header:

- **MustUnderstand** defines whether the header needs to be processed by the recipient or not. If the value is set to [true], the processing of the header becomes mandatory and you need to specify the header in the WSDL file. Otherwise, the header will be rejected. If the value is set to [false], the header does not have to be processed.
- **Actor** defines who of the processors has to react to the message. It can be used for your own purposes as it is not used at runtime. If you specify more than one actor, it is possible to route the message to multiple recipients with only the specific information that is relevant to each of the recipient.

How to Set and Get SOAP Headers

There are three methods which can be used to process SOAP headers:

- **GetSoapHeaders**: returns a list (ArrayList) of SOAP headers represented by objects of type SOAPHeader.
- **SetSoapHeaders**: sets SOAP headers in a request or response message before the message is sent out. This method can be used in an integration flow as often as required.
- **ClearSoapHeaders**: removes headers that were received, so that they won't be propagated.

You can create a SOAP header instance using a DOM tree element or a Java string class instance.

Check out the related blog if you want to know how to process SOAP headers in an integration flow. The blog describes a detailed integration flow example using a script step: [Cloud Integration - Accessing and Setting SOAP Headers in an Integration Flow](#) 

Using Script API Methods in Groovy Scripts

Prerequisites

- You have installed Groovy plugins for Eclipse.

Context

If you are creating a script based on the [Message](#) class from SAP, you can use a jar file that contains the definition of the [Message](#) class in your script project to make your life easier. By using this jar file, Eclipse provides content assist while writing scripts for the [Message](#) class. This enables you to complete the script easily.

Here's how you can import this jar into your script project:

Procedure

1. Download the JAR from the [Using Script API](#) section of the [Cloud Integration Tools Page](#).
2. Create a new [Groovy Project](#).
3. In the **Build Settings**, choose **Add External JARs** to add the downloaded JAR file to your groovy project.

4. When you create a new groovy class, reference the groovy script project where you have added the JAR.

Results

When you are creating the script, Eclipse automatically suggests options when you are inserting methods from [Message](#) class.

Creating an Exception Response for SOAP-Based Adapters

Understand how to populate the fields in the exception response for SOAP-based adapters via API by populating/overwriting the generated exception.

i Note

This topic is related to SOAP-based adapters. This includes the following adapter types:

- SOAP SOAP 1.x
- SOAP SAP RM
- IDoc
- XI

If an error is thrown during processing, the user receives a detailed fault description. Overwrite that generated exception with exception responses adapted to the workflow's needs.

The `com.sap.gateway.ip.core.customdev.processor.SoapFaultFactory` class provides the following methods:

Method	Description
<code>throwSoapFault(SoapFaultParam param)</code>	Use this method for SOAP sender channels. It allows the setting of all attributes that the SOAP specification describes, especially the SOAP version. For details, see the SOAP specification. Be aware that there are some differences between SOAP 1.1 and SOAP 1.2. In particular, SOAP 1.2 only allows certain fault codes. • <code>@param param</code>: Object to set all parameters according to the SOAP specification. The parameter detail is expected to be an Element object of a DOM tree. The parameter namespaces can be used to add additional namespace declarations for parameter detail if required. • <code>@throws: UnknownSoapVersionException</code> • <code>@throws UnknownFaultCodeException</code>
<code>throwSoapFault(String faultstring, Element detail, Map<String, String> namespaces)</code>	Use this method for SOAP sender channels. It provides a minimum set of parameters to only set text parameters. All other parameters are set to their defaults. The SOAP version is set to SOAP 1.1. • <code>@param faultstring</code>: See also the SOAP specification. This is a simple error text. • <code>@param detail</code>: See also the SOAP specification. Here you can set additional information. The SOAP fault exception object of the SOAP adapter required here is an Element object of a DOM tree. • <code>@param namespaces</code>: If the detail parameter requires additional namespace declarations, then they can be set here.
<code>throwXiSoapFault(XiSoapFaultParam param)</code>	Use this method for XI sender channels. It throws a SOAP fault exception that fulfills the XI protocol specification. This specification basically restricts the detail attribute of the SOAP fault to a certain format. For the SOAP adapter, this can be freely chosen. In the XI adapter, the detail has to follow the XI error header specification. In addition, the SAP category is always set to 'XI Server' and the SAP code is set to 'INTERNAL.MESSAGE'. All other codes and categories are reserved. P1 to P4, additional text, and stack can be freely chosen. • <code>@param param</code>: Object to set all possible attributes of an XI SOAP fault. • <code>@throws XMLStreamException</code>

Check out the related blog if you want to know how to process SOAP headers in an integration flow. The blog describes a detailed integration flow example using a script step: [Cloud Integration - Accessing and Setting SOAP Headers in an Integration Flow](#) 

Related Information

[Creating Scripts in a Script Collection](#)

[XI Adapter](#)

OData API

The Cloud Integration application programming interface (API) allows you to access Cloud Integration resources, for example, monitoring data.

The OData API is implemented as a REST API and the technical protocol is Open Data Protocol (OData). This means that you can use standard HTTP methods (for example, the GET method) to call the API.

You can find the OData API on SAP Business Accelerator Hub at:

[SAP Cloud Integration](#) 

A prerequisite for using the OData API is that your user is connected to Cloud Integration and that you have access to an OData client.

Individual actions that you can perform using the OData API are protected by an authorization concept. To find out the required role for the task that you want to perform with the OData API, see [Tasks and Permissions](#).

The OData API covers the following aspects:

OData API Overview	
Section	Description
Integration Content	Get deployed integration content or deploy/undeploy integration artifacts.
Log Files, Neo Environment	Get an overview of HTTP and trace log files.
Message Processing Logs	Get an overview of the messages processed on a tenant and get the details for individual messages.
Message Stores	Get data from the Message Store, such as message payloads, headers, properties, or attachments for processed messages.
Partner Directory	Get, write, or delete data of the Partner Directory. This content can be used to parameterize integration flows.
Security Content	Get, write, or delete security content, for example, keystore entries, user credentials, or certificates to user mappings.
	Access your Edge Integration Cell data even during offline mode when the Operations Cockpit user interface is unavailable. You can retrieve information such as component status, pods, event logs, metrics, and runtime parameters.

The OData API is structured by a number of entities, grouped on SAP Business Accelerator Hub according to the sections listed in the previous table. Each entity (resource) is specified by a number of properties, to which filtering options can also be applied.

You can address the collection of entries or a single entry within a collection.

How the entities are related to each other is described in the entity data model. This model also provides information on how to navigate between entities using the API.

This documentation provides an overview of the available resources and summarizes the supported HTTP methods and query options for each resource.

→ Tip

For more information about the programming model and how the HTTP methods and query options work, see [OData - the Best Way to REST](#) .

OData specification version 2.0 is supported by the Cloud Integration API.

HTTP Calls and URI Components

More information: [HTTP Calls and URI Components](#)

Recommendations to Improve OData Request Performance

Using the OData API allows you to run certain requests on data processed by and stored on SAP BTP. The load that these requests put on the platform depends on the URL pattern and the respective parameters specified in the request, as well as the amount of data available. The latter depends directly on the details of message processing.

⚠ Caution

SAP has set a maximum time for the execution of an OData request. It is the customer's responsibility to design OData requests so that the set of returned data sets remains manageable (as far as this suits the requirements of the client making the request).

To keep the load on the network manageable, we therefore recommend that you use query options as follows:

- Make requests as selective as possible by using the `$filter` option. For example, when you access message processing logs (MPLs) using the API, you can filter in such a way that the request retrieves MPLs in a restricted time interval only, with the log end time between two nearby datetime literals.
- Use the `$select` option to make the API return only those properties that are required by the client. This reduces network load.
- If it is necessary to navigate a large number of data sets (>1000), use the `$top` and `$skip` options to iterate over the result set. Consider designing the client application in such a way that it offers paging options.
- Note that when you use the OData API, you navigate to MessageStore entities starting from the MessageProcessingLog entity. Apply filter options to the MessageProcessingLog rather than the MessageStore entities.

i Note

The DataStore and Variables APIs do not support the following query options:

- - `$filter`
 - `$inlinecount`
 - `$orderby`
 - `$skip`
 - `$top`
 - `$expand`
 - `$select`

Related Information

[Integration Content](#)
[Log Files, Neo Environment](#)
[Message Processing Logs](#)
[Message Stores](#)
[Partner Directory](#)
[Security Content](#)
[Query Options](#)
[Authentication](#)

API Details

The API details correspond to the details shown in the SAP Business Accelerator Hub for : [SAP Cloud Integration API Package](#) 

Related Information

[Integration Content](#)
[Log Files, Neo Environment](#)
[Message Processing Logs](#)
[Message Stores](#)
[Partner Directory](#)
[Security Content](#)

Integration Content

Manage integration artifacts for your tenant.

You can access the OData API at:

[Integration Content](#) 

This documentation provides additional information.

Overview

Operating an integration scenario requires certain integration artifacts. Using the OData API you can access integration artifacts.

Integration artifacts contain the information required for the runtime components to process messages in the context of a certain integration scenario. First and foremost, an integration flow artifact contains all information about the connectivity settings and processing steps modeled in an integration flow model with the Web UI (in the [Design](#) section). When an integration developer has finished the modeling tasks, they deploy the integration - as an *integration artifact* - on the tenant and, that way, makes it executable.

There are more kinds of artifacts, as summarized in the following table.

Integration Artifact Types

Artifact Type	Description
Integration flow	An executable integration flow model An integration flow specifies the way how a message is to be processed between different systems. In particular, you define in an integration flow: the senders and receivers of a message, how the senders and receivers are connected to the tenant (adapters), and the steps that define the message processing. You can design an integration flow in the Web UI Design section.
Value mapping	An executable value mapping definition A value mapping allows you to transform one representation of an object to another representation. In most situations when data is exchanged between two systems, two ways of representing the same data need to be translated. For example, one specific person is referred to as a particular customer number in one system and as an employee number in another system. Both numbers refer to the same person. You can design a value mapping in the Web UI Design section. i Note You can create a new Value Mapping artifact from any existing/downloaded Value Mapping project. You can upload either a .zip or a .jar file format.
OData API	An executable OData API model You can use an OData API to expose data sources as OData endpoints. You can design an OData in the Web UI Design section. The system generates from an OData API an integration flow that contains by default an OData sender adapter.

Alternatively, you can access integration content through the OData API.

Permissions

To authorize an API client to access the OData API, perform the steps as described at:

- In the **Cloud Foundry** environment, perform the steps described at: [Setting Up Inbound HTTP Connections \(for API Clients\)](#)
Assign the role template as listed for the desired task (for example, download deployed integration flow) under [Tasks and Permissions](#).
- In the **Neo** environment, perform the steps described at: [Setting Up Inbound HTTP Connections \(for API Clients\)](#), [Neo Environment](#)
Assign one or more roles as listed for the desired task (for example, download deployed integration flow) under [Tasks and Permissions](#).

To be able to maintain custom tags:

- In the **Cloud Foundry** environment, assign the following role template:
WebToolingSettingsProductProfiles.savetenantconfiguration (part of role collection PI_Administrator)
- In the **Neo** environment assign the following role:
WebToolingSettingsProductProfiles.savetenantconfiguration (part of authorization group AuthGroup.Administrator)

Resources

The following table contains a list of resources and their respective descriptions.

Resource	Description
Integration Package - Discover	Represents an integration package in the Cloud Integration Discover section. You can use the <code>CopyIntegrationPackage</code> resource to copy an integration package from the Discover section to the Design section.
Integration Packages - Design	Represents an integration package in the Cloud Integration Design section. You can use resource <code>IntegrationPackages</code> to access (create, update, delete, read) integration packages. You can read an entire integration package, or only an integration flow or you can read all the packages in design time.
Custom Tag Configurations	Represents a set of attributes (custom tags) defined on tenant level. Once defined, the integration package owners need to maintain these attributes when creating new integration packages. The tenant administrator can maintain integration package-related attributes that are also referred to as <i>custom tags</i>. Integration developers can use custom tags to classify their integration packages. If the tenant administrator has classified the custom tag as mandatory, the integration developer has to maintain the attribute when creating an integration package. Otherwise, the integration developer can't save the integration package. Custom tags make it easier for integration developers to organize integration content across a large organization. → Tip Example The tenant administrator can define the mandatory custom tag <code>Author</code>. As a result, for each newly created integration package the integration developer has to specify the author name. This makes it easy for all involved integration developers to keep track of the ownership of the individual integration packages. The tenant administrator can maintain custom tags using one of the following options: - Using the Cloud Integration Settings section. - Using the OData API (Custom Tags interface) More information: Creating Custom Tags SAP Cloud Integration – Custom Tags  (SAP Community blog) i Note For more information on how to access custom tags specified for an individual integration package, see: SAP Cloud Integration – Custom Tags  (SAP Community blog)
Integration Flow	Represents an integration flow model. You can use resource <code>IntegrationDesignTimeArtifacts</code> to read, download, create, upload, deploy, and delete integration flows. i Note - You can't delete content from configure-only package. - You can't delete from a locked artifact.
Configurations of Integration Flow	Represents externalized configuration parameters for an integration flow. You can read or update integration flow configurations. You can access integration flow configurations through the <code>IntegrationDesignTimeArtifacts</code> resource.
Resources of Integration Flow	Represents integration flow resources. You can read, download, add, or update integration flow resources. You can access integration flow resources through the <code>IntegrationDesignTimeArtifacts</code> resource. i Note You can use the <code>ReferencedResourceType</code> parameter if you want to refer to a file type with another file type. For example, you can refer to an xsd file with a wsdl file. In this case, the value of the parameter is wsdl. You can't download resources from configure-only content.

Resource	Description
	i Note For script type resources, the Resource ODATA APIs only support version v1.
Message Mapping	Represents a message mapping. You can read, create, and delete message mappings. You can access message mappings through the <code>MessageMappingDesignTimeArtifacts</code> resource.
Script Collection	Represents a script collection. You can create and upload a script collection, add resources to a script collection, and deploy a script collection. You can access script collections through the <code>ScriptCollectionDesignTimeArtifacts</code> resource.
Build and Deploy Status	Represents the build and deploy status. This status indicates the deployment status of the artifact that is triggered for deployment. You can access integration flow configurations through the <code>BuildAndDeployStatus</code> resource. For more information on the possible status values, see Build and Deploy Status . For integration artifacts that have reached the worker node, another status becomes relevant, the runtime status. The runtime status indicates if a deployed artifact is ready to operate (see Runtime Status).
MDI Delta Token	Represents the delta token for the SAP Master Data Integration receiver adapter (MDI receiver adapter). See:
Runtime Artifacts	Represents the runtime artifact associated with a design time artifact (for example, an integration flow model or a value mapping). You can use resource <code>IntegrationRuntimeArtifacts</code> to read, deploy, and undeploy integration content. i Note To deploy a runtime artifact, you've to include the artifact that you want to deploy as a binary stream into the body of your POST request. Refer to the OData Protocol specification for details on how to use Media Entities. You can only deploy <code>BUNDLE</code> type integration artifacts (integration flows, value mappings, or OData services).
Error Information of Runtime Artifact	Represents error information related to a runtime artifact. You can read error information related to deployed integration artifacts (such as integration flows, for example). You can access error information through the <code>IntegrationRuntimeArtifacts</code> resource.
Endpoints of Runtime Artifacts	Represents endpoints of deployed integration content (for example, integration flow endpoints). You can use resource <code>ServiceEndpoints</code> to read all endpoints provided for integration flows and to get the number of endpoints.
Value Mappings	Represents value mapping. You can use resource <code>ValueMappingDesignTimeArtifacts</code> to read, download, create, upload, deploy value mappings, and to delete entries from value mappings.
Integration Adapter	Represents an integration adapter (only available in the Cloud Foundry environment). You can use resource <code>IntegrationAdapterDesignTimeArtifacts</code> to import, deploy, or delete an integration adapter.

For general information about query options, see [Query Options](#).

i Note

To access integration content through a user interface, you can use the Cloud Integration [Design](#) application. To access predefined integration packages, you can go to the Cloud Integration [Discover](#) application.

Example Requests

You find various example requests on SAP Business Accelerator Hub at [Integration Content](#).

For more example requests, see [Integration Content Example Requests](#).

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first. You can do that in the following way: First, send a GET call to the API with a header `X-CSRF-Token` with the value `Fetch`. As a response, you get the value of the token in the header `X-CSRF-Token`. When sending the modifying request, add the header `X-CSRF-Token` with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under [API Environment](#)). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

This is custom documentation. For more information, please visit [SAP Help Portal](#).

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

Build and Deploy Status

Indicates the status of an artifact that is triggered for deployment.

You can access integration flow configurations through the BuildAndDeployStatus resource.

For integration artifacts that have reached the worker node, another status becomes relevant, the runtime status. The runtime status indicates if a deployed artifact is ready to operate (see [Runtime Status](#)).

There are the following status values for the build and deploy status.

Status Values

Status	Description
SUCCESS	The artifact is deployed on worker node(s).
FAIL	The artifact isn't deployed on worker node(s), for example, because of a validation error/CMD checks error in the currently deployed integration flow.
DEPLOYING	The artifact is in the process of being deployed.
FAIL_ON_LICENSE_ERROR	The artifact isn't deployed on worker node(s) because the integration flow contains components that require an upgrade to the user's existing license.

Related Information

[Runtime Status](#)
[Deployment Status View](#)

Integration Content Example Requests

You find various example requests on SAP Business Accelerator Hub at <https://api.sap.com/api/IntegrationContent/>🔗.

For more example requests, see:

Runtime artifacts and error information	Runtime Artifacts and Error Information Example Requests
Integration flows and resources	Integration Flow Example Requests Get Runtime Status of Deployed Integration Flow
Service endpoints	ServiceEndpoints Example Requests
Integration packages	Integration Package Example Requests
Custom tags	Get Custom Tags Defined on the Tenant Create New Custom Tags Configuration
Value mappings	Value Mapping Example Requests
Integration adapter	Integration Adapter Example Requests, Cloud Foundry Environment

Get Custom Tags Defined on the Tenant

To get all custom tags defined by the tenant administrator, you need to perform the following call:

Method	Resource Path	Purpose
GET	/CustomTagConfigurations('CustomTags')/\$value	Get all custom tags.

Assume that you like to get all custom tags defined by the tenant administrator.

Send the following GET request:

```
https://<host address>/api/v1/CustomTagConfigurations('CustomTags')/$value
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

If a mandatory custom tag with name **Author** is defined on the tenant, you get the following response:

```
{"customTagsConfiguration":[{"tagName":"Author","isMandatory":true}]}
```

If for the mandatory custom tag with name **Author** also permitted values (for example, Mr. Bean and Ms. Bean) are defined, you get the following response:

```
{"customTagsConfiguration":[{"tagName":"Author","permittedValues":["Mr. Bean, Ms. Bean"],"isMandatory":true}]}
```

Create New Custom Tags Configuration

To create custom tags, you need to perform the following call:

Method	Resource Path	Purpose
POST	/CustomTagConfigurations	Create custom tags.

As request body, provide the following JSON content:

```
{
  "CustomTagsConfigurationContent": "<base64-encoded custom tag configurations content>"
}
```

Assume that you like to create a mandatory custom tag with the name **Owner**.

The related JSON representation is:

```
{"customTagsConfiguration":[{"tagName":"Owner","isMandatory":true}]}
```

After base64 encoding, you get the following string:

```
eyJjdXN0b2lUYWdzQ29uZmVndXJhdGlvbiI6W3sidGFhbnRmZSI6I6W3bmVvIiwiaXNNYW5kYXRvcnkiOnRydWV9XX0=
```

Send the following POST request:

```
https://<host address>/api/v1/CustomTagConfigurations
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As request body, send:

```
{
  "CustomTagsConfigurationContent": "eyJjdXN0b2lUYWdzQ29uZmVndXJhdGlvbiI6W3sidGFhbnRmZSI6I6W3bmVvIiwiaXNNYW5kYXRvcnkiOnRydWV9XX0="
}
```

If a custom tags configuration is already available on the tenant, add the following query parameter to the request:

```
Overwrite=true
```

Update and Validate Configuration Parameters

Update and validate configuration parameters of an integration flow.

1. Update configuration parameters of the integration flow via an OData API, perform the below call:

Method	Resource Path	Purpose	Response Status
GET	api/v1 /IntegrationDesigntimeArtifacts(Id='{Id}',Version='{Version}')/Configurations	Get configurations of an integration flow by Id and version.	o 200: Retrieval of configurations of an integration flow by Id and version. Exa Collection Integration San { d } o 400: Bad Request: structure of parameters o 401: Unauthorized: missing authentication information o 403: Forbidden: user does not have access to the resource o 404: Not Found: integration flow could not be found o 429: Too many requests: request rate exceeded. o default: Error Exa odata.error San { error* }

Method	Resource Path	Purpose	Response Status
GET	api/v1/IntegrationDesignTimeArtifacts(Id='{Id}',Version='{Version}')/Configurations/\$count	Get number of configuration parameters.	200: Number of configuration parameters. 400: Bad Request. Invalid structure of parameters. 401: Unauthorized. Missing authentication information. 403: Forbidden. No access to the resource. 404: Not Found. Resource could not be found. 429: Too many requests. Rate limit exceeded. - default: Error. ❖ Example <pre> { "odata.error": { "message": "The address field must begin with '/' and c", "sourceObject": "ComponentMetadataEditor/etype:MessageFl", "resourcePath": "src/main/resources/scenarioflows/integ", "resourceName": "iflw1.iflw" } }</pre>
PUT	api/v1/IntegrationDesignTimeArtifacts(Id='{Id}',Version='{Version}')/\$links/Configurations('parameterKey')	Updates configuration parameter value. Note For Neo environment, this call performs the validation of integration flow as well.	202

2. Once, all the configuration parameters are updated, validate the integration flow along with the configuration parameters via an OData API, perform the below call::

Method	Resource Path	Purpose	Response Status
POST	api/v1/ValidateIntegrationDesignTimeArtifact?Id='{Id}'&Version='{Version}'	Perform validation checks and return results	202 - Passed: Checkexecutionresult: Passed - Failed: Checkexecutionresult: Failed + detailed error list ❖ Example Sample Code <pre> Checkexecutionresult: Failed[{ "severity": "Error", "message": "The address field must begin with '/' and c", "sourceObject": "ComponentMetadataEditor/etype:MessageFl", "resourcePath": "src/main/resources/scenarioflows/integ", "resourceName": "iflw1.iflw" }]</pre> 404

Method	Resource Path	Purpose	Response Status
			❖ Example ⌵ Sample Code <pre> { "error": { "code": "Not Found", "message": { "lang": "en", "value": "Integration design time artifact not found." } } } </pre> ○ 400

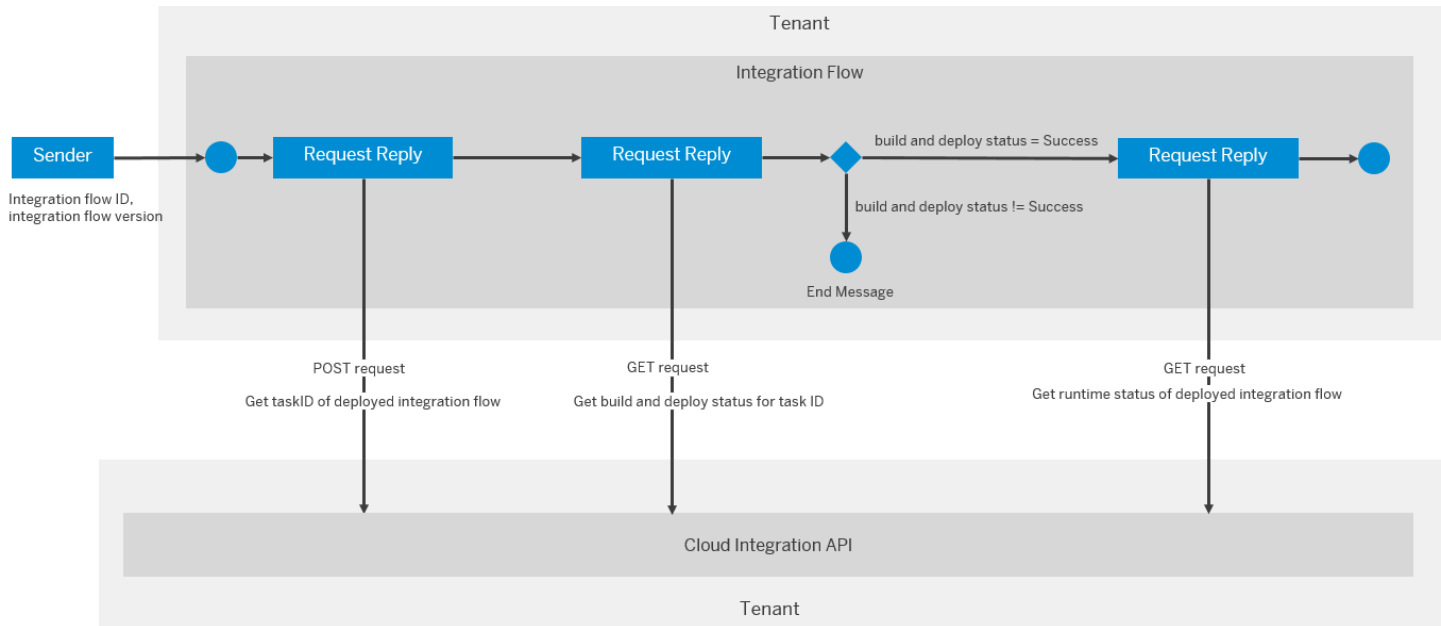
Get Runtime Status of Deployed Integration Flow

Get the runtime status of a currently deployed integration flow.

Perform three subsequent calls:

Method	Resource Path	Purpose
POST	/DeployIntegrationDesignTimeArtifact? Id='<bundleId>'&Version='<version>'	Get taskID of currently deployed integration flow.
GET	/BuildAndDeployStatus(TaskId=' <taskId>')	Get build and deploy status of currently deployed integration flow.
GET	/IntegrationRuntimeArtifacts(' <bundleId>')	If build and deploy status is Success, get runtime status of currently deployed integration flow.

To set up such a sequence of requests, you can design an integration flow with the following sequence of OData API calls:



The integration flow performs the following steps:

1. An HTTP client (for example, Postman) calls the integration flow. The request body contains the Id and version of the integration flow for which you like to request the runtime status.

A content modifier (not shown in the diagram) stores Id and version as exchange properties (let's give them the names: `bundleId` and `version`).

2. The first Request Reply step calls the Cloud Integration OData API through an HTTP receiver adapter (for the tenant on which the integration flow in question is deployed).

Cloud Integration calls the OData API with the following URL and query:

```
https://<host address>/api/v1/DeployIntegrationDesignTimeArtifact?
Id='${property.bundleId}'&Version='${property.version}' (POST method)
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

The OData API sends back a response that contains the `taskId`.

A content modifier (not shown in the diagram) stores the `taskId` as exchange property.

3. The second Request Reply step calls the OData API (through an HTTP receiver adapter) with the following URL and query:

```
https://<host address>/api/v1/BuildAndDeployStatus(TaskId='${property.taskId}')
```

The OData API provides a response with the following content:

```
<m:properties>
  <d:TaskId>abcd-1234-5678-xyz</d:TaskId>
  <d:Status>Success</d:Status>
</m:properties>
```

A content modifier (not shown in the diagram) stores the status as exchange property.

4. A routing step leads to two different routes, depending in the value of the status:
 - The default route leads to a message end event.
 - The route taken when the status provided by the OData API is `Success` contains another Request Reply step that calls the OData API through the HTTP receiver adapter. It specifies a call with the following URL and query:

```
https://<host address>/api/v1/IntegrationRuntimeArtifacts('${property.bundleId}')
```

The response contains attributes for the retrieved integration flow.

The `Status` attribute contains the actual runtime status.

The integration flow sends back the runtime status to the HTTP client.

Related Information

[Integration Content](#)
[Runtime Status](#)

Runtime Artifacts and Error Information Example Requests

Get Runtime Artifacts in Status Error

Perform the following call:

Method	Resource Path
GET	/IntegrationRuntimeArtifacts?\$filter=Status eq 'ERROR'

To get all integration artifacts in status `ERROR`, send the following GET request:

```
https://<host address>/api/v1/IntegrationRuntimeArtifacts?$filter=Status eq 'ERROR'
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

To get error information related to the integration artifact with Id `Mail_Sender_Integration_Flow`, send the following GET request:

`https://<host address>/api/v1/IntegrationRuntimeArtifacts('Mail_Sender_Integration_Flow')/ErrorInformation`

You get the following response:

```
<entry ...>
  <id>https://<host address>/api/v1/RuntimeArtifactErrorInformations('Mail_Sender_Integration_Flow')</id>
  <title type="text">RuntimeArtifactErrorInformations</title>
  <updated>2021-03-18T11:19:04.598Z</updated>
  <category term="com.sap.hci.api.RuntimeArtifactErrorInformation" scheme="http://schemas.microsoft.com/ado/2007/08/dataserv
  <link href="RuntimeArtifactErrorInformations('Mail_Sender_Integration_Flow')" rel="edit" title="RuntimeArtifactErrorInform
  <link href="RuntimeArtifactErrorInformations('Mail_Sender_Integration_Flow')/$value" rel="edit-media" type="application/oc
  <content type="application/octet-stream" src="RuntimeArtifactErrorInformations('Mail_Sender_Integration_Flow')/$value"/>
  <m:properties>
    <d:Id>Mail_Sender_Integration_Flow</d:Id>
  </m:properties>
</entry>
```

To get the error information in JSON format (for this example), send the following GET request:

`https://<host address>/api/v1/IntegrationRuntimeArtifacts('<artifact name>')/ErrorInformation/$value`

Integration Flow Example Requests

You can use resource `IntegrationDesignTimeArtifacts` to read, download, create, upload, deploy, and delete integration flows.

This section provides various example requests and additional information. Also check out the following SAP Community blog: [Cloud Integration – Remote OData API's for Integration Flows](#) 🐦.

i Note

Update Integration Flow Name

To update an Integration flow name, and save as a draft, send the following call:

Method	Resource Path
PUT	<code>/IntegrationDesignTimeArtifacts(Id='<integration flow ID>',Version='active')</code>

To update an Integration flow with Id `My_Integration_Flow` with the new name `My Integration Flow New`, and save as a draft, send the following PUT request:

`https://<host address>/api/v1/IntegrationDesignTimeArtifacts(Id='My_Integration_Flow',Version='active')`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Provide the following request body:

```
{
  "Name": "My Integration Flow New",
  "ArtifactContent": "UESDBBQACAgIAER6cIIAAAAAAAAA ..... vLnByb3BQSwUGAAAAAYABgDWAQAA8xIAAAAA"
}
```

i Note

For the `ArtifactContent` property, you need to provide the base64-encoded content of the integration flow bundle (for example, of the zip file that contains the integration flow model).

To get the integration flow Id, open the integration flow model and select the **General** tab (ID field).

Update Integration Flow with Specific Version

To copy an Integration flow with a new version, send the following call:

Method	Resource Path
POST	/IntegrationDesigntimeArtifactSaveAsVersion?Id='<integration flow ID>'&SaveAsVersion='<new version>'

To save Integration flow with Id My_Integration_Flow to version 1.0.0, send the following POST request:

```
https://<host address>/api/v1/IntegrationDesigntimeArtifactSaveAsVersion?
Id='My_Integration_Flow'&SaveAsVersion='1.0.0'
```

i Note

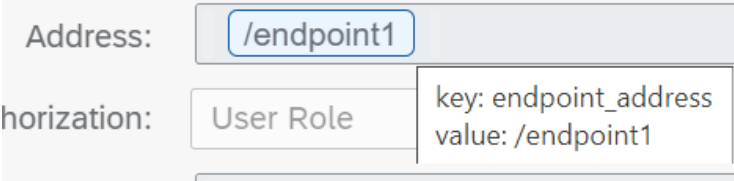
If you would like to update an integration flow artifact with a specific version, the *PUT* method must be executed first.

Update Integration Flow Configuration Parameters

Method	Resource Path
PUT	/IntegrationDesigntimeArtifacts(Id='{Id}',Version='{Version}')/\$links/Configurations('{ParameterKey}')

Assume that you've an integration flow with an externalized parameter for the HTTPS sender adapter address. When defining the HTTPS adapter address as externalized parameter, you've given it the parameter name `endpoint_address` and the initial value `endpoint1`.

The figure shows the externalized parameter in the **Connection** tab of the HTTPS sender adapter.



To change the value of the externalized parameter to `endpoint1`, send the following PUT request:

```
https://<host address>/api/v1/IntegrationDesigntimeArtifacts(Id='My_Integration_Flow',Version='1.0.1')/$links
/Configurations('endpoint_address')
```

Provide the following request body:

```
{
  "ParameterValue": "/endpoint2",
  "DataType": "xsd:string"
}
```

You can also do a batch request, for example:

```
⌵ Sample Code

--batch_34fcd829-64e4-4139-b880-f24aa4ee9235
Content-Type: multipart/mixed; boundary=changeset_5e8e492f-2e3f-490f-9629-a4cd82a7aafd

--changeset_5e8e492f-2e3f-490f-9629-a4cd82a7aafd
Content-Type: application/http
Content-Transfer-Encoding: binary

PUT IntegrationDesigntimeArtifacts(Id='TestingCustomScheduler',Version='1.0.3')/$links/Configurations('Receiver1_host_33')
Accept: application/json
Content-Type: application/json

{
  "ParameterKey" : "Receiver1_host_33",
  "ParameterValue" : "blr002",
  "DataType" : "xsd:string"
}

--changeset_5e8e492f-2e3f-490f-9629-a4cd82a7aafd
Content-Type: application/http
```

Content-Transfer-Encoding: binary

PUT IntegrationDesigntimeArtifacts(Id='TestingCustomScheduler',Version='1.0.3')/\$links/Configurations('Sender_host_14') HTTP/1.1

Accept: application/json

Content-Type: application/json

```
{
  "ParameterKey" : "Sender_host_14",
  "ParameterValue" : "wdfsp001",
  "DataType" : "xsd:string"
}
```

--changeset_5e8e492f-2e3f-490f-9629-a4cd82a7aafd--

--batch_34fcd829-64e4-4139-b880-f24aa4ee9235--

Perform Multiple Operations on Resources

You can perform multiple operations on resources in a single call using a batch request:

Sample Code

--batch_abc

Content-Type: multipart/mixed; boundary=changeset_123

--changeset_123

Content-Type: application/http

Content-Transfer-Encoding: binary

POST IntegrationDesigntimeArtifacts(Id='ResourceViewTest',Version='1.0.1')/Resources HTTP/1.1

Content-Type: application/json

```
{
  "Name": "Convert1.xsl",
  "ResourceType": "xslt",
  "ResourceContent": "<Base64 encoded content>"
}
```

--changeset_123

Content-Type: application/http

Content-Transfer-Encoding: binary

POST IntegrationDesigntimeArtifacts(Id='ResourceViewTest',Version='1.0.1')/Resources HTTP/1.1

Content-Type: application/json

```
{
  "Name": "Convert2.xsl",
  "ResourceType": "xslt",
  "ResourceContent": "<Base64 encoded content>"
}
```

--changeset_123--

--batch_abc--

Note

The *Name* and *ResourceType* must not be null.

ResourceContent should be in base64-encoded format.

For more information, check out the following SAP Community blog: [Cloud Integration – Remote OData API's for Integration Flows](#) .

Query Parameters Supported for IntegrationDesigntimeArtifact Requests

IntegrationDesigntimeArtifacts request supports system query parameters for controlling the amount and order of data.

System Query Options are query string parameters that a client specifies to control the amount and order of the data that an OData API returns for the resource identified by the URI. The names of all System Query Options are prefixed with a "\$" character.

Reading DesigntimeArtifact

Query	Description	Example
\$select	You can select response properties based on the requirement.	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')?\$select=Id,Version</code>

Reading Resources

Query	Description	Example
\$select	You can select response properties based on the requirement.	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$select=Name,ResourceType</code>
\$count	Returns the total count of resources found in the artifact.	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources/\$count</code>
\$orderby	Orders the collection of entries sorted either in ascending or descending order. By default the entries are sorted in ascending order. Use desc to sort all entries returned in descending order.	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$orderby=Name</code> or <code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$orderby=Name,ResourceType desc</code>
\$filter	Returns a subset of the entries, which matches the filter condition. The following filter conditions are supported: - Eq - ne The following string functions are supported: - substringof - startswith - endswith	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$filter=Name eq 'script1.groovy'</code> <code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$filter=Name ne 'script1.groovy'</code> <code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$filter=substringof('script1', Name) eq true</code>
\$format	Response is received either in JSON or Atom XML format. The default format is Atom XML .	<code>https://<tmnurl>/api/v1/IntegrationDesigntimeArtifacts(Id='<bundleid>',Version='<version>')/Resources?\$format=json</code>

Reading Configurations

Query	Description	Example
\$select	You can select response properties based on the requirement.	<code>https://<tmnurl>/api/v1/IntegrationDesignTimeArtifacts(Id='<bundleid>',Version='<version>')/Configuration?\$select=ParameterKey,ParameterValue</code>
\$count	Returns the total count of resources found in the artifact.	<code>https://<tmnurl>/api/v1/IntegrationDesignTimeArtifacts(Id='<bundleid>',Version='<version>')/Configuration?\$count=1</code>
\$orderby	Orders the collection of entries sorted either in ascending or descending order. By default the entries are sorted in ascending order. Use <code>desc</code> to sort all entries returned in descending order.	<code>https://<tmnurl>/api/v1/IntegrationDesignTimeArtifacts(Id='<bundleid>',Version='<version>')/Configuration?\$orderby=ParameterKeyorhttps://<tmnurl>/api/v1/IntegrationDesignTimeArtifacts(Id='<bundleid>',Version='<version>')/Configuration?\$orderby=ParameterKey,ParameterValue desc</code>
\$filter	Returns a subset of the entries, which matches the filter condition. The following filter conditions are supported: - Eq - ne The following string functions are supported: - substringof - startswith - endswith	<code>https://<tmnurl>/api/v1/IntegrationDesignTimeArtifacts(Id='<bundleid>',Version='<version>')/Configuration?\$filter=ParameterKey eq '<key>'</code> <code>https://<tmnurl>/api/v1/IntegrationDesignTimeArtifacts(Id='<bundleid>',Version='<version>')/Configuration?\$filter=ParameterKey ne '<key>'</code> <code>https://<tmnurl>/api/v1/IntegrationDesignTimeArtifacts(Id='<bundleid>',Version='<version>')/Configuration?\$filter=substringof('<substring of key>', ParameterKey) eq true</code>
\$format	Response is received either in JSON or Atom XML format. The default format is Atom XML .	<code>https://<tmnurl>/api/v1/IntegrationDesignTimeArtifacts(Id='<bundleid>',Version='<version>')/Configuration?\$format=json</code> <code>https://<tmnurl>/api/v1/IntegrationDesignTimeArtifacts(Id='<bundleid>',Version='<version>')/Configuration?\$format=atomxml</code>

ServiceEndpoints Example Requests

The ServiceEndpoints resource enables you to obtain information about the service endpoints exposed by Cloud Integration on a tenant.

The Service Endpoints also provides you with the adapter-specific protocol information. The protocol is the type of the APIs such as SOAP, REST, and OData. The API protocol includes the protocol version, if applicable.

Adapter	Protocol Type
SOAP	SOAP
IDoc	SOAP
OData V2	ODATAV2
AS2	AS2
AS4	AS4

Adapter	Protocol Type
HTTPS	REST

Specific Example Requests for Service Endpoints

- To return information about all service endpoints with only the endpoints information expanded, send the following GET request:

`https://<host address>/api/v1/ServiceEndpoints?$expand=EntryPoints`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

An endpoint is an array of type `EntryPoint`. Following are the properties of the `EntryPoint` entity:

Property	Type	Description
Name	String	(Required) Name of the entry point.
URL	URL	(Required) Fully qualified endpoint or the base path of the API.
Type	Enumerated String	(Optional) Possible values are DEV, TEST, PROD, SANDBOX.

- To return information about all service endpoints with only the API definitions expanded, send the following GET request:

`https://<host address>/api/v1/ServiceEndpoints?$expand=ApiDefinitions`

An API definition is an array of type `APIDefinition` containing links to the machine-readable API definitions like Open API specification files, OData metadata, and so on. Following are the properties of the `APIDefinition` entity:

Property	Type	Description
URL	URL	(Required) Fully qualified endpoint to the API definitions file.
Type	Enumerated String	(Required) Name of the API definition endpoint. Possible values are oas-yaml, oas-json, raml, edmx, wsd.

- To return the service endpoints only for the integration flow with name `Getting Started Flow`, send the following GET request:

`https://<host address>/api/v1/ServiceEndpoints?$expand=EntryPoints&$filter=Name eq 'Getting Started Flow'`

- To return only the service endpoints information of protocol type SOAP, send the following GET request:

`https://<host address>/api/v1/ServiceEndpoints?$expand=EntryPoints&$filter=Protocol eq 'SOAP'`

Integration Package Example Requests

You can copy integration packages to your [Design](#) space. You can also create, update, delete, and search for integration packages in design time.

Import an Integration Package into the Design Section

To import an integration package into the [Design](#) section, send the following call:

Method	Resource Path
POST	/IntegrationPackages

Provide the following request body:

```
{
  "PackageContent":<base64-encoded package content>
}
```

If the package to be imported is in zip format, it has to be converted into **base 64 encoded** format.

i Note

You can't import a package if there's already a package in the [Design](#) section with the same name. You can use the `Overwrite` parameter to overwrite the existing package. Use the following request:

```
https://<host address>/api/v1/IntegrationPackages?Overwrite=true
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Download an Integration Package

To download an integration package as .zip file from the [Design](#) section, send the following call:

Method	Resource Path
GET	/IntegrationPackages('{Id}')/\$value

i Note

Download fails if the package contains one or more artifacts in draft state.

Value Mapping Example Requests

You can read, query, deploy, download, and upload a Value Mapping artifact using the OData API. You can also read and update the configurations of value mapping design time artifacts

Filter All Value Mappings Based on Source and Target Value for Specific Bidirectional Agency Identifier

To filter all the value mappings based on the source and target value for a specific bidirectional agency identifier, send the following call:

Method	Resource Path
GET	/ValueMappingDesignTimeArtifacts(Id=<Id>,Version=<version>)/ValMapSchema(SrcAgency=<Agency_name>,SrcId=<source_Id,TgtAgency=<Agency_Name>,TgtId=<target_Id>)/ValMaps?\$filter=Value/SrcValue eq <Source_Value> and Value/TgtValue eq <target_Value>

Create Value Mapping Using Function Import

To create a value mapping using function import [UpsertValMaps](#), send the following call::

Method	Resource Path
POST	/UpsertValMaps?Id=<Id>&Version=<version>&SrcAgency=<Agency_name>&SrcId=<source_Id>&TgtAgency=<Agency_Name>&TgtId=<target_Id>&SrcValue=<value>&TgtValue=<value>&IsConfigured=<true or false>

i Note

- *IsConfigured* is used to change the state of the Bidirectional mapping. If the value is true, then the *ValMapSchema* is configured and if false it is meant for nonconfigured *ValMapSchema*.
- Once a *ValMapSchema* is configured, it can't be changed. The value of *IsConfigured* should always be true for further operations as you cannot edit a configured content.

Value Mapping Batch Requests

Batch operation with multiple queries and changesets:

☰, **Sample Code**

```

--batch
Content-Type: multipart/mixed; boundary=changeset_77162fcd-b8da-41ac-a9f8-9357efbbd621

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd621
Content-Type: application/http
Content-Transfer-Encoding: binary

POST <operation> HTTP/1.1

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd621
Content-Type: application/http
Content-Transfer-Encoding: binary

POST <operation> HTTP/1.1

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd621--

--batch
Content-Type: multipart/mixed; boundary=changeset_77162fcd-b8da-41ac-a9f8-9357efbbd622

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd622
Content-Type: application/http
Content-Transfer-Encoding: binary

POST <operation> HTTP/1.1

--changeset_77162fcd-b8da-41ac-a9f8-9357efbbd622--

--batch
Content-Type: application/http
Content-Transfer-Encoding: binary

GET <operation> HTTP/1.1

--batch
Content-Type: application/http
Content-Transfer-Encoding: binary

GET <operation> HTTP/1.1

--batch
Content-Type: application/http
Content-Transfer-Encoding: binary

GET <operation> HTTP/1.1

--batch--

```

Integration Adapter Example Requests, Cloud Foundry Environment

i Note

This information is relevant only when you use SAP Cloud Integration in the Cloud Foundry environment.

Deploy an Integration Adapter

To deploy an integration adapter, send the following call:

Method	Resource Path
POST	/api/v1/DeployIntegrationAdapterDesignTimeArtifact? Id='SubsystemSymbolicName1'

- Import an integration adapter with the symbolic name *SubsystemSymbolicName1*.
- Send the request as given in the following example.

❖ Example

For deploying an integration adapter, the URL is given in the following format: `https://<host address>/api/v1/DeployIntegrationAdapterDesignTimeArtifact?Id='SubsystemSymbolicName1'`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

To delete an integration adapter, send the following call:

Method	Resource Path
DELETE	/api/v1/IntegrationAdapterDesignTimeArtifacts(Id='SubsystemSymbolicName1')

- Import the integration adapter with the symbolic name *SubsystemSymbolicName1*.
- Send the request as given in the following example.

 Example

For deleting an integration adapter, the URL is given in the following format: `https://<tenant url>/api/v1/IntegrationAdapterDesignTimeArtifacts(Id='SubsystemSymbolicName1')`

Log Files, Neo Environment

Access technical system logs written during message processing on the worker/runtime node.

i Note

This information is relevant only when you use SAP Cloud Integration in the Neo environment.

You can access the OData API at:

There, you find the basic operations.

<https://api.sap.com/api/LogFiles/>

This documentation provides additional information.

Permissions

To authorize an API client to access the OData API, perform the steps as described at:

In the **Neo** environment, perform the steps described at: [Setting Up Inbound HTTP Connections \(for API Clients\), Neo Environment](#)

Assign the following roles:

`IntegrationOperationServer.read,AuditLog.Read` (see [Tasks and Permissions](#))

Resources

Resource	Description
Log File Archives	Represents a log file. Two kinds of log files are created during message processing: • HTTP access logs Contain information about inbound HTTP requests. You can use this information to analyze authentication and authorization errors for inbound HTTP communication. • Default trace logs Contain technical processing information.
Log Files	Represent a set of log files in a compressed file (zip format).

i Note

The property `NodeScope` identifies the node type associated with the log file.

For example, the following request returns a list of all HTTP access and default trace log files available for all runtime nodes (*worker nodes*) of the tenant cluster:

`https://<host address>/api/v1/LogFiles$filter=NodeScope eq 'worker'`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

i Note

In future, this list probably contains log files from other node types. Therefore, we recommend the use of the variant with the `NodeScope` filter parameter, as it guarantees that only runtime nodes (and no other node types) are considered.

There's a user interface that allows you to access log files. Go to the **Monitor** application under **Access Logs** (only in the Neo environment).

More information: [Access Logs, Neo Environment](#)

For general information about query options, see [Query Options](#).

Example Requests

You find various example requests on SAP Business Accelerator Hub at <https://api.sap.com/api/LogFiles/> .

For more example requests, see [Log Files Example Requests](#).

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under **API Environment**). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

`https://<host address>/api/v1/<relative resource path>`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

Related Information

[Log Files Example Requests](#)

Log Files Example Requests

You find various example requests on SAP Business Accelerator Hub at:

<https://api.sap.com/api/LogFiles/> .

For more specific example requests, see the following list:

- HTTP access files
 - `https<tmn>/api/v1/LogFileArchives(Scope='all',LogFileType='http',NodeScope='worker')/$value`
Downloads a log file (zip) archive that contains all existing HTTP access log files (specified by `Scope='all'`)
 - `https://<tmn>/api/v1/LogFileArchives(Scope='latest',LogFileType='http',NodeScope='worker')/$value`
Downloads a log file (zip) archive that contains the latest HTTP access log file per runtime node (specified by `Scope='latest'`). The zip archive for the scope `latest` contains only a single log file for each runtime node. Nevertheless, if some runtime nodes have been restarted and appropriate log files still exist for the previous instances of runtime notes, these files are also downloaded.
 - `https://<tmn>/api/v1/LogFileArchives(Scope='all',LogFileType='http',NodeScope='worker')/$value?modifiedAfter=2017-03-25T13:54:51Z`
Downloads a log file (zip) archive that contains all HTTP access log files created after the indicated time (specified by the `modifiedAfter` attribute).

i Note

When filtering using `modifiedAfter`, the API checks if access log files have been modified after the specified time. This means that the files contain entries that were created after the specified time and can contain entries that were already written before the specified time. It doesn't reflect the time when the file has been created.

- Default trace log files
 - `https://<tmn>/api/v1/LogFiles(Name='ljs_trace_a40a97f_2017-12-15.log',Application='krt01iflmap')`
Returns the metadata of a single default trace log file.

- `https://<tmn>/api/v1/LogFiles(Name='ljs_trace_a40a97f_2017-12-15.log ',Application='krt0liflmap')/$value`
Downloads a single default trace log file (as gzip file).
- `https://<tmn>/api/v1/LogFileArchives(Scope='all',LogFileType='trace',NodeScope='worker')/$value`
Download a log file (zip) archive that contains all existing default trace log files (specified by Scope='all').
- `https://<tmn>/api/v1/LogFileArchives(Scope='latest',LogFileType='trace',NodeScope='worker')/$value`
Downloads a log file (zip) archive that contains the latest default trace log file per runtime node (specified by Scope='latest').

Message Processing Logs

Get an overview of the messages processed on a tenant and get the details for individual messages. The message processing log (MPL) stores data about the messages processed on a tenant. Furthermore, it stores information about the individual processing steps for each processed message.

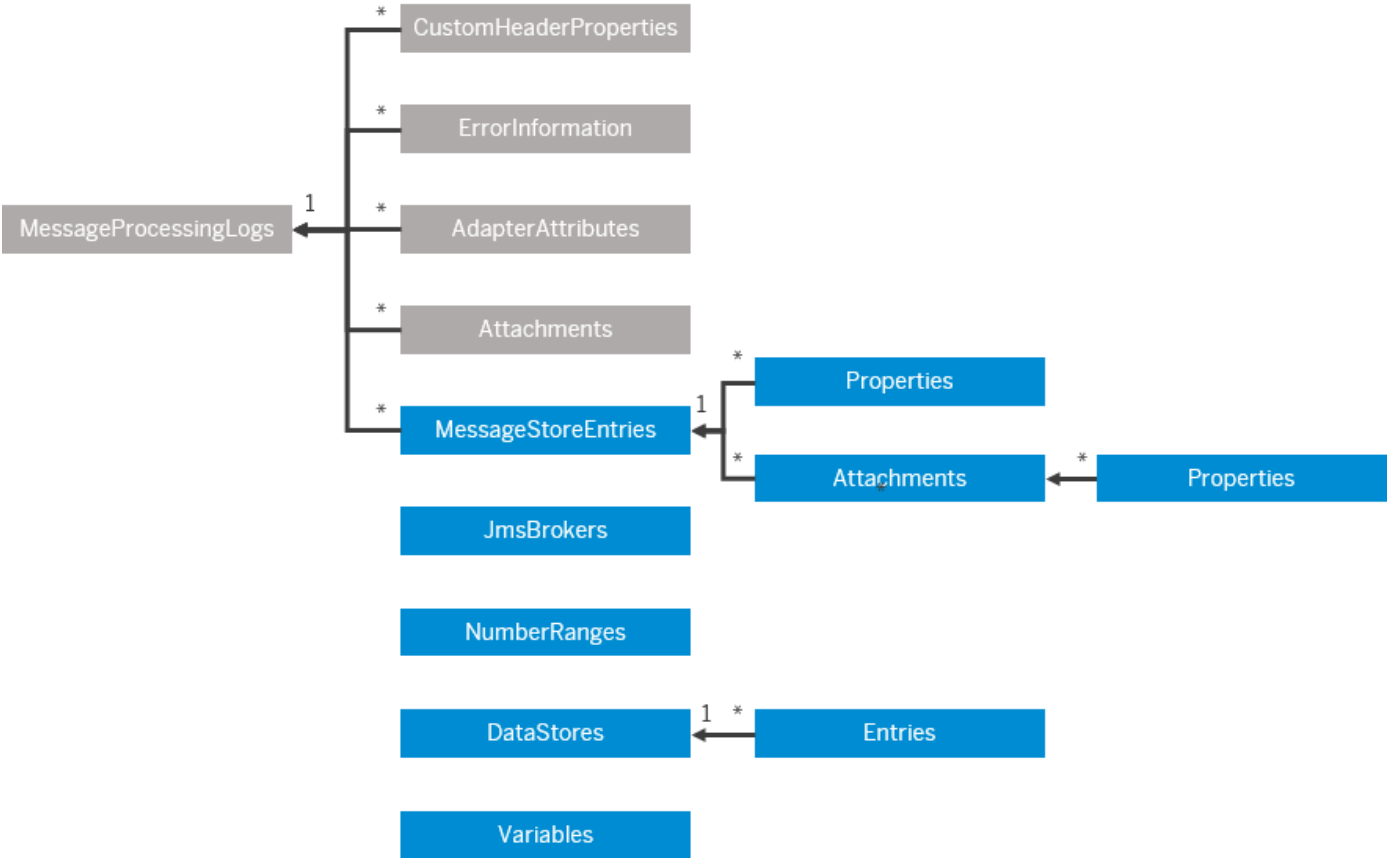
You can access the OData API and learn about its basic operations at:

- [Message Processing Logs](#) for Cloud Integration.
- [Message Processing Logs](#) for Edge Integration Cell

This documentation provides additional information.

Entity Data Model

The following diagram shows how the entities related to `MessageProcessingLog` and `MessageStoreEntry` are related to each other:



Permissions

To authorize an API client to access the OData API:

- In the **Cloud Foundry** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\)](#)
Assign the following role template:
MonitoringDataRead (see [Tasks and Permissions](#))
To access the ID Mapper, assign role template **MessageProcessingLocksRead**.

- In the **Neo** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\), Neo Environment](#)

Assign the following roles:

`IntegrationOperationServer.read`, `NodeManager.read` (see [Tasks and Permissions](#))

To access the ID Mapper, assign role template `MessageProcessingLocks.Read`.

Resources

Caution

Note the following, if you plan to get message processing logs with status “Discarded”, using the OData API:

- Discarded messages don’t contain error information.
- Calling frequent queries, for error information, in discarded messages, can impose an unnecessary high load on the system.

Therefore, decide the usage for such specific queries, diligently.

Resource	Description
Logs	Represents a message processing logs object. More information: Monitor Message Processing Message Status Message Processing Log
Adapter Attributes	Represents adapter-specific attributes. Each adapter has a defined set of attributes that are described for every adapter type as part of the adapter configuration. The <code>AdapterId</code> property identifies the adapter that logs something (the component ID of the adapter in the component metadata). The <code>AdapterMessageId</code> property uniquely identifies a message in context of a particular adapter.
Attachments	Represents attachments of message processing logs. i Note To be able to view attachments, you need the following role in the NEO environment :<code>esbmessagesstorage.read</code>. In the Cloud Foundry environment, you need the role template <code>MessagePayloadsRead</code>).
Custom Header Properties	Represents custom header properties of the message processing log.  Caution Restriction for the usage of \$orderby Regarding the \$orderby option with the <code>MessageProcessingLogCustomHeaderProperty</code> entity, \$orderby supports only the combinations <code>\$orderby=Name,Value</code> and <code>\$orderby=Name desc,Value desc</code>. If you need additional options, we recommend that you implement these options on the client side. More information: Use Custom Header Properties to Search for Message Processing Logs
Error Information	Represents error information for message processing logs.
ID Mapper	Allows you to get all target IDs for the given source ID (stored in the ID mapper) or to get all source IDs for the given target ID (stored in the ID mapper). i Note Not supported in.
Idempotent Repository	Represents the idempotent repository. i Note Not supported in . i Note The <code>Idempotent Repository</code> resource of the <code>Message Processing Logs</code> API has been deprecated and replaced by a new one. The idempotent repository contains information about files that have already been consumed from the connected server in scenarios where one or more of the following features are used: Aggregation step, FTP, adapter, JMS adapter, SFTP adapter, or XI adapter. Files that are stored in the idempotent repository can be identified by the file name. When Cloud Integration tries to process the file, the

Resource	Description
	system can detect if the file has already been consumed (based on its idempotent repository entry) and that way can prevent it from being consumed a second time from the server. More information: • Define Aggregator • FTP Adapter • JMS Adapter • SFTP Adapter • XI Adapter Certain values (when indicated as <hexaLias>) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string my aLias is expressed by the following decimal numbers: 109 121 32 97 108 105 97 115 because, according to the American Standard Code for Information Interchange (ASCII), the letter m is translated to the decimal number 109, and so forth. If you convert each number to a hexadecimal number, you get: 6D 79 20 61 6C 69 61 73. Note that 6D is the hexadecimal representation of 109, and so forth. If you want to specify my aLias in an example request, enter 6D7920616C696173.
External Logging (only available in the Cloud Foundry environment.)	Allows you to activate or deactivate external logging. More information: External Logging Cloud Foundry Environment
Data Archiving (only available in the Cloud Foundry environment.)	Allows you to activate data archiving. More information: Archiving Data, Cloud Foundry Environment

For general information about query options, see [Query Options](#).

i Note

To access message processing logs through a user interface, you can use the **Monitor** application (**Monitor Message Processing** section).

Example Requests

You can find various example requests on SAP Business Accelerator Hub at [Message Stores](#).

For more example requests, see [Message Processing Logs Example Requests](#).

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first. You can do that in the following way: First, send a GET call to the API with a header X-CSRF-Token with the value Fetch. As a response, you get the value of the token in the header X-CSRF-Token. When sending the modifying request, add the header X-CSRF-Token with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under **API Environment**). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

https://<host address>/api/v1/<relative resource path>

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

i Note

A sandbox environment isn't available for .

Related Information

[Monitor Message Processing](#)

Message Processing Logs Example Requests

You find various example requests on SAP Business Accelerator Hub at [Message Processing Logs](#).

For more example requests, see:

Logs	Get Message Processing Logs for a Dedicated Time Period
Custom Header Properties	Get Custom Header Properties

Get Message Processing Logs for a Dedicated Time Period

Get the message processing logs for all completed messages processed within a dedicated time period.

Perform the following call:

Method	Resource Path
GET	/MessageProcessingLogs?\$inlinecount=allpages&\$filter=Status eq 'COMPLETED' and LogStart gt datetime'{beginning of time period}' and LogEnd lt datetime'{end of time period}'

Assume that you like to get all message processing logs for completed messages processed in the time period of February 1 to February 14 2021.

Send the following GET request:

```
https://<host address>/api/v1/MessageProcessingLogs?$inlinecount=allpages&$filter=Status eq 'COMPLETED' and LogStart gt datetime'2021-02-01T00:00:00' and LogEnd lt datetime'2021-02-14T00:00:00'
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Get Custom Header Properties

Get custom header properties for a dedicated message ID.

Perform the following call:

Method	Resource Path
GET	/MessageProcessingLogs(' {MessageGuid} ') /CustomHeaderProperties

You can set custom header properties in a Script step. After processing the integration flow, custom header properties are displayed in the **Monitor** application (under **Monitor Message Processing**). However, you can also use the OData API to retrieve custom header properties.

There's an integration flow design guideline that shows you how to set custom header properties using the Script step (see [Use Custom Header Properties to Search for Message Processing Logs](#)).

When you've deployed and processed this integration flow, the corresponding message processing log shows the custom header property `po_number` with a certain value.

Assume that the message ID is ABCD - 1234 - XYZ. To get the custom header property, send the following GET request:

```
https://<host address>/api/v1/MessageProcessingLogs(' ABCD-1234-XYZ ')/CustomHeaderProperties'
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

The response contains the following part:

```
<m:properties>
<d:Id>999</d:Id>
<d:Name>po_number</d:Name>
<d:Value>12345</d:Value>
</m:properties>
```

Message Stores

Get data from Message Stores for processed messages and resources of the used JMS queues.

You can access the OData API and learn about its basic operations at:

- [Message Stores](#) for Cloud Integration.
- [Message Stores](#) for

This documentation provides additional information.

Overview on Message Store Resources

Message content is stored by the [Persist](#) step of an integration flow. You can use this API to access the stored message and analyze it at a later point in time.

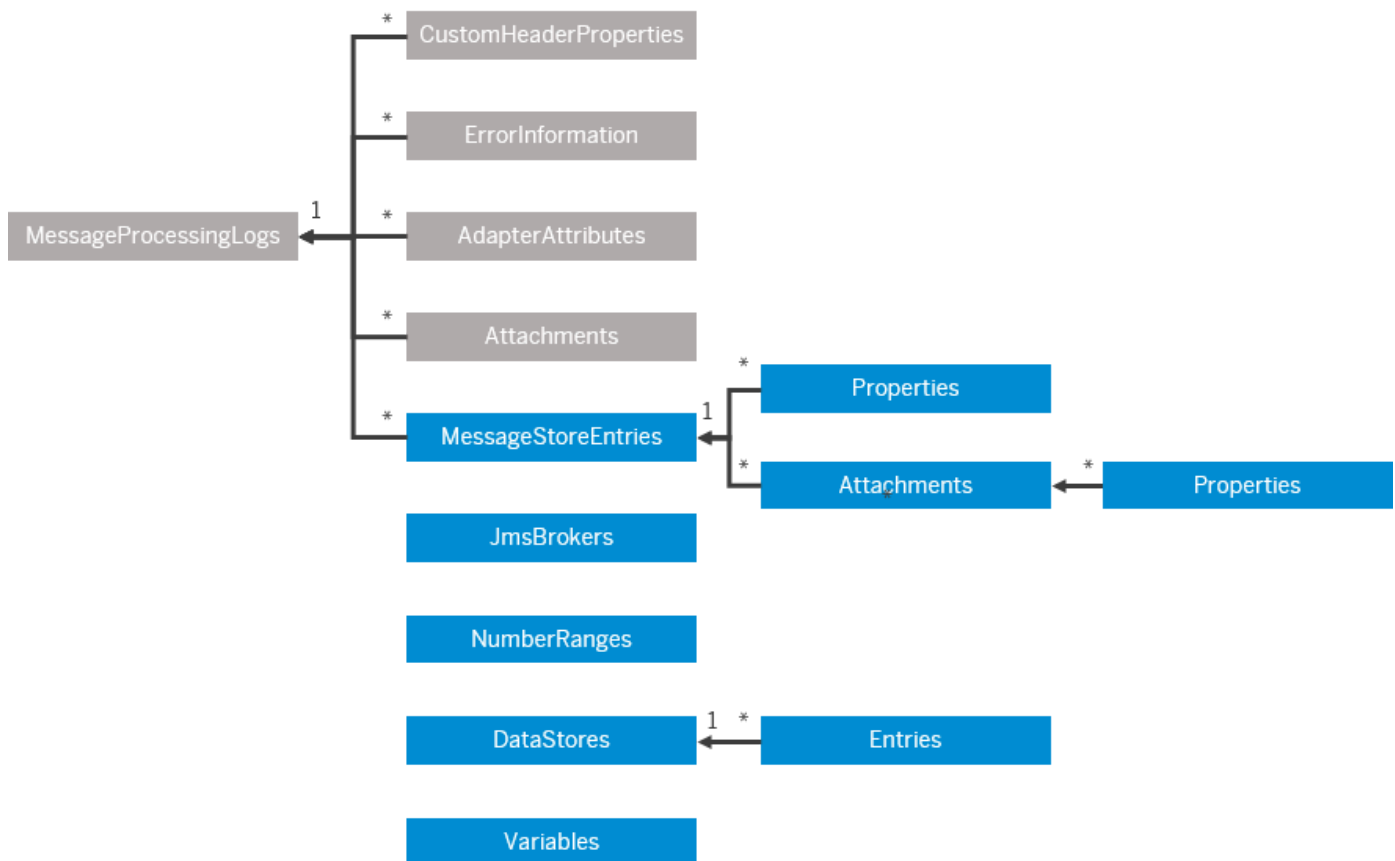
You can also use this API to get information about the JMS resources used in asynchronous messaging scenarios.

You can access the OData API at:

<https://api.sap.com/api/MessageStore/>

Entity Data Model for Message Store Resources

The following diagram shows how the `MessageProcessingLogs` resources (gray) are related to the `MessageStoreEntries` resources (and related resources) of the `Message Stores` API.



Overview on Data Store/Variables Resources

During the processing of an integration flow, you can temporarily store data in the tenant database. This data can be accessed by steps within the same integration flow or another integration flow, deployed on the same tenant.

The following data can be stored:

- Message payloads and headers as data store entries

Data store entries can be accessed by Data Store Select or Data Store Get steps.

- Any data as variable

Variables can be accessed by dedicated integration steps, for example, by a content modifier to store the variable as property or header.

To do that, you use the [Data Store Write](#) step (for data stores and data store entries) or the [Write Variables](#) step (for variables) of an integration flow. With the Data Store Get or Select step, you can read data store content during the processing of an integration flow.

For an overview of the content of the tenant database, see [Using Data Storage Features When Designing Integration Flows](#).

Depending on the data store or variable type, the data is shared by all integration flows (deployed on the tenant) or only by one specific integration flow.

- **Global:** Data is shared across all integration flows deployed on the tenant.
- **Integration Flow:** Data can only be used within 1 integration flow.

Permissions

To authorize an API client to access the OData API:

- In the **Cloud Foundry** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\)](#).

Depending on the use case, assign the following role templates (see [Tasks and Permissions](#)):

- To view message store content: `MonitoringDataRead`
 - To view data store entries: `DataStoresAndQueuesRead`
 - To download the message payload (stored as data store entry) and to view variables: `DataStorePayloadsRead`
 - To delete data store entries/variables: `DataStoresAndQueuesDelete`
 - To view JMS resources: `DataStoresAndQueuesRead`
 - In the **Neo** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\), Neo Environment](#)
- Depending on the use case, assign the following roles (see [Tasks and Permissions](#)):
- To view message store content: `IntegrationOperationServer.read, NodeManager.read`
 - To view data store entries: `IntegrationOperationServer.read, ESBDDataStore.read`
 - To download the message payload (stored as data store entry) and to view variables: `IntegrationOperationServer.read, ESBDDataStore.readPayload`
 - To delete data store entries/variables: `IntegrationOperationServer.read, ESBDDataStore.read, ESBDDataStore.delete`
 - To view JMS resources: `IntegrationOperationServer.read, ESBDDataStore.read`

Resources

Resource	Description
Entries	Represents a message store entry. A message store entry is created by a Persist step of an integration flow (see Persist Messages).
Entry Attachments	Represents an attachment of a message from the message store A message store entry is created by a Persist step of an integration flow (see Persist Messages).
Entry Attachment Properties	Represents properties of message attachments from the message store. A message store entry is created by a Persist step of an integration flow (see Persist Messages).
Entry Properties	Represents a header of a message from the message store. A message store entry is created by a Persist step of an integration flow (see Persist Messages).
JMS Resources	Represents available resources of the used Java Message Service (JMS) queues. JMS queues are accessed to by integration flows that contain an AS2, a JMS, or an XI adapter (see AS2 Adapter , JMS Adapter , and XI Adapter).
Data Stores	Represents a data store. A data store entry is created by a Data Store Write step of an integration flow (see Define Data Store Write Operations). i Note The DataStore API does not support the following query options: • ◦ <code>\$filter</code> ◦ <code>\$inlinecount</code>

Resource	Description
	◦ \$orderby ◦ \$skip ◦ \$top ◦ \$expand ◦ \$select
Data Store Entries	Represents a data store entry. A data store entry is created by a Data Store Write step of an integration flow (see Define Data Store Write Operations).
Variables	Represents a variable. A variable is created by a Write Variables step of an integration flow (see Define Write Variables). i Note The Variables API does not support the following query options: • ◦ \$filter ◦ \$inlinecount ◦ \$orderby ◦ \$skip ◦ \$top ◦ \$expand ◦ \$select
Number Ranges i Note Not supported in.	Represents a Number Ranges object (see Managing Number Ranges). While sending out a document when using EDI processing, a unique interchange number must be added to each document. To add such an interchange number, you can use the Number Range object. If EDIFACT is used, the outgoing EDIFACT messages have an Interchange Control Reference with a length of 1 to 9 digits.

For general information about query options, see [Query Options](#).

i Note

Cloud Integration doesn't provide a user interface to access the Message Store (data stored by the Persist step). You can only access this data using the OData API.

To access JMS resources through a user interface, you can use the **Monitor** application (**Message Queues** tile under **Manage Stores**).

To access Number Ranges objects through a user interface, you can use the **Monitor** application (**Number Ranges** tile under **Manage Stores**).

Cloud Integration provides a user interface for accessing data stores and variables (go to the **Monitor** application and select the tile **Data Stores** or **Variables** under **Manage Stores**).

Example Requests

You find various example requests on the SAP Business Accelerator Hub at [Message Stores](#) .

For more example requests, see .

- [Message Stores Example Requests](#)
- [Data Store Example Requests](#)
- [JMS Resources Example Requests](#)

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first. You can do that in the following way: First, send a GET call to the API with a header X-CSRF-Token with the value Fetch. As a response, you get the value of the token in the header X-CSRF-Token. When sending the modifying request, add the header X-CSRF-Token with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under **API Environment**). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

i Note
A sandbox environment isn't available for .

Message Stores Example Requests

You find various example requests on SAP Business Accelerator Hub at [Message Stores](#).

For more example requests, see:

Entry Properties	Get Message Store Entry Properties
Number Ranges	Add a Number Ranges Object Update a Number Ranges Object

Get Message Store Entry Properties

Get message store entry properties.

To get message store entry properties, you need to perform two subsequent calls:

Method	Resource Path	Purpose
GET	/MessageProcessingLogs('{MessageGuid}') /MessageStoreEntries	Get all message store entries created during the processing of an integration flow (for a dedicated MessageGuid).
GET	/MessageStoreEntries('{MessageStoreEntryId}')	Get entry properties for an individual message store entry (identified by MessageStoreEntryId).

In detail, perform the following subsequent calls (example).

1. Get all message store entries created during the processing of an integration flow.

Note that each Persist step in an integration flow creates a message store entry.

Each message processing run is identified by a **Message ID** that you can find when selecting the associated integration flow using the **Monitor** application (under **Monitor Message Processing**).

To get message store entries for a dedicated message processing run (for example, with a message ID ABCD1234XYZ), send the following GET request:

https://<host address>/api/v1/MessageProcessingLogs(' ABCD1234XYZ')/MessageStoreEntries

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a response, you get a message that contains for each message store entry a passage with the following structure:

```
<m:properties>  
  <d:Id>sap-it-abcd-1234-uvw</d:Id>  
  <d:MessageGuid>ABCD1234XYZ</d:MessageGuid>  
  <d:MessageStoreId>Persist1</d:MessageStoreId>  
  <d:TimeStamp>2021-02-25T12:49:48.83</d:TimeStamp>  
  <d:HasAttachments>>false</d:HasAttachments>  
</m:properties>
```

The property Id uniquely identifies the message store entry.

2. To get entry properties for an individual message store entry, perform a second call. To get properties for the message store entry with Id sap-it-abcd-1234-uvw, you perform the following GET request:

https://<host address>/api/v1/MessageStoreEntries(' sap-it-abcd-1234-uvw')/Properties

As response, you get a message with an entry for each message header with the following part (example for message header myheader):

```
<content type="application/xml">
  <m:properties>
    <d:MessageId>sap-it-abcd-1234-uvw</d:MessageId>
    <d:Name>myheader</d:Name>
    <d:Value>myheader_value</d:Value>
  </m:properties>
</content>
```

Related Information

[Use the Persist Step](#)

Add a Number Ranges Object

Add a **Number Ranges** object to the tenant.

Perform the following call:

Method	Resource Path
POST	/NumberRanges

Example: To add a **Number Ranges** object to your tenant, send the following POST request:

`https://<host address>/api/v1/NumberRanges`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As request body, provide (example):

≡ Sample Code

```
{ "CurrentValue": "0",
  "Name": "My NRO Object",
  "MinValue": "0",
  "MaxValue": "9999",
  "Description": " Number Range Object ",
  "Rotate": "true",
  "FieldLength": "4" }
```

You find the newly created **Number Ranges** object using the **Monitor** application (**Number Ranges** tile under **Manage Stores**).

Update a Number Ranges Object

Perform the following call:

Method	Resource Path
PUT	/NumberRanges(' {objectName}')

For example, to update a Number Ranges object with name **My NRO Object**), send the following PUT request:

`https://<host address>/api/v1/NumberRanges( 'My NRO Object' )`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

In the request body, provide the new properties of the Number Ranges object (example):

≡ Sample Code

```
{
  "CurrentValue": "0",
  "Name": "My NRO Object",
  "MinValue": "0",
  "MaxValue": "9999",
  "Description": " Number Range Object ",
  "Rotate": "true",
  "FieldLength": "4"
}
```

i Note

The **Current Value** returned by the API corresponds to the **Next Value** shown in the Monitoring tab of the UI. The difference is only in terminology.

You find the updated **Number Ranges** object using the **Monitor** application (**Number Ranges** tile under **Manage Stores**).

Data Store Example Requests

Data Stores	Get All Data Stores with Overdue Messages
Data Store Entries	Get Single Data Store Entry Get All Data Store Entries for a Data Store Get All Data Store Entries for a Specific Message ID
Variables	Download a Variable

Get All Data Stores with Overdue Messages

To get data stores that contain overdue messages, perform the following call:

Method	Resource Path	Purpose
GET	/DataStores?overdueonly=true	Gets all data stores that contain overdue messages.

If `overdueonly` is set to `false` or not available, all data stores are shown.

For each retrieved data store entry,the response contains a part with the following structure:

```
<m:properties>
<d:DataStoreName>My_Datastore</d:DataStoreName>
<d:IntegrationFlow>My_Integration_Flow</d:IntegrationFlow>
<d:Type/>
<d:NumberOfMessages>19</d:NumberOfMessages>
<d:NumberOfOverdueMessages>19</d:NumberOfOverdueMessages>
</m:properties>
```

Get Single Data Store Entry

To get a single data store entry, perform the following call:

Method	Resource Path
GET	/DataStoreEntries(Id='{EntryId}',DataStoreName='{DataStoreName}',IntegrationFlow='{IntegrationFlowName}',Type='{Type}')

To get a single entry with entry ID 1234-abcd-4567-xyz from the local data store `MyStore`, perform the following GET request:

```
https://<host address>/api/v1/DataStoreEntries(Id='{1234-abcd-4567-xyz}',DataStoreName='{DataStoreName}',IntegrationFlow='{MyIntegrationFlow}',Type='{Type}')
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

The response contains an entry with the following elements:

```
<m:properties>
  <d:Id>1234-abcd-4567-xyz</d:Id>
  <d:DataStoreName>MyStore</d:DataStoreName>
  <d:IntegrationFlow>MyIntegrationFlow</d:IntegrationFlow>
  <d:Type></d:Type>
  <d:Status>Waiting</d:Status>
  <d:MessageId>ABCD_XYZ</d:MessageId>
  <d:DueAt>2021-04-16T13:53:35.28</d:DueAt>
  <d:CreatedAt>2021-04-14T13:53:35.28</d:CreatedAt>
  <d:RetainUntil>2021-07-13T13:53:35.28</d:RetainUntil>
</m:properties>
```

Get All Data Store Entries for a Data Store

To get data store entries of specific data store, perform the following call:

Method	Resource Path	Purpose
GET	<code>/DataStores(DataStoreName='<data store name>',IntegrationFlow='<integration flow ID>',Type='')/Entries</code>	Gets all entries of a specific data store.

To retrieve all data store entries of data store `MyDataStore` created by integration flow `MyIntegrationFlow`, perform the following request:

```
https://<host address>/api/v1
/DataStores(DataStoreName='MyDataStore',IntegrationFlow='MyIntegrationFlow',Type='')/Entries
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

For each data store entry, the response contains a part with the following structure:

```
<m:properties>
  <d:Id>1234-abcd-7890-xyz</d:Id>
  <d:DataStoreName>MyDataStore</d:DataStoreName>
  <d:IntegrationFlow>MyIntegrationFlow</d:IntegrationFlow>
  <d:Type></d:Type>
  <d:Status>Waiting</d:Status>
  <d:MessageId>ABCD1234XYZ</d:MessageId>
  <d:DueAt>2021-07-08T09:15:05.837</d:DueAt>
  <d:CreatedAt>2021-07-06T09:15:05.837</d:CreatedAt>
  <d:RetainUntil>2021-08-05T09:15:05.837</d:RetainUntil>
</m:properties>
```

Get All Data Store Entries for a Specific Message ID

To get data store entries that are associated with a specific message ID, perform the following call:

Method	Resource Path	Purpose
GET	<code>/DataStoreEntries?messageid=<messageId></code>	Gets all data store entries for a given message ID.

To retrieve all data store entries created during the message processing run identified by the message ID `ABCD1234XYZ`, perform the following request:

```
https://<host address>/api/v1/DataStoreEntries?messageid=ABCD1234XYZ
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

For each data store entry, the response contains a part with the following structure:

```
<m:properties>
<d:Id>1234-abcd-7890-xyz</d:Id>
<d:DataStoreName>My_Datastore</d:DataStoreName>
<d:IntegrationFlow>My_Integration_Flow</d:IntegrationFlow>
<d:Type/>
<d:Status>Overdue</d:Status>
<d:MessageId>ABCD1234XYZ</d:MessageId>
<d:DueAt>2021-02-06T14:24:24.016</d:DueAt>
<d:CreatedAt>2021-02-04T14:24:24.016</d:CreatedAt>
<d:RetainUntil>2021-05-05T14:24:24.016</d:RetainUntil>
</m:properties>
```

Download a Variable

To download a single variable, perform the following call:

Method	Resource Path	Purpose
GET	/Variables(VariableName='{VariableName}',IntegrationFlow='{IntegrationFlowName}')/\$value	Downloads a single variable.

To download variable timestamp for integration flow MyIntegrationFlow, perform the following request:

```
https://<host address>/api/v1/Variables(VariableName='timestamp',IntegrationFlow='MyIntegrationFlow')/$value
```

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

JMS Resources Example Requests

Get information about JMS message queues.

Method	Resource Path	Purpose
GET	/JmsBrokers('Broker1')?\$expand=QueueStates	Get the queue status of each message queue (0 OK, 1 warning, 2 critical).
GET	/JmsBrokers('Broker1')?\$expand=InactiveQueues	Get stopped queues.

You can use this API in an integration flow to set up notifications for critical resource situations. For more information, see the following SAP Community blog: [Cloud Integration – Automated Notification for Critical or Exhausted JMS Resources](#)

Partner Directory

The Partner Directory contains information on partners that are connected to a tenant in the context of a larger business partner network.

You can access the OData API and learn about its basic operations at:

- <https://api.sap.com/api/PartnerDirectory/> for Cloud Integration.
- [Overview | Partner Directory | SAP Business Accelerator Hub](#) for Edge Integration Cell.

i Note

Read-only access.

This documentation provides additional information.

i Note

There's also a Java API to access the Partner Directory using a programming language like Java Script or Groovy. For more information, check out the JavaDoc linked to at [SDK API](#) (for example, check out the package com.sap.it.api.pd).

 Caution

The data is stored unencrypted in the Partner Directory. Therefore, make sure that the data does not contain any sensitive information (for example, passwords or personal information).

Permissions

To authorize an API client to access the OData API, perform the steps as described at:

- In the **Cloud Foundry** environment, perform the steps described at: [Setting Up Inbound HTTP Connections \(for API Clients\)](#)

Assign the following role collection:

`AuthGroup_TenantPartnerDirectoryConfigurator` (see [Permissions](#))

- In the **Neo** environment, perform the steps described at: [Setting Up Inbound HTTP Connections \(for API Clients\)](#), [Neo Environment](#)

Assign the following authorization group:

`AuthGroup.TenantPartnerDirectoryConfigurator` (see [Permissions](#))

Resources

More information on the concepts: [Partner Directory Concepts](#)

Entity Type	Description
Alternative Partners	Represents an alternative partner. You can access (read, write, delete) an alternative partner. See: Partner Directory Entity Types
Authorized Users	Represents the user with which a partner sender system can log in into SAP Cloud Integration. You can access (read, write, delete) an authorized user. See: Partner Directory Entity Types
Binary Parameters	You can access (read, write, delete) a binary parameter of the Partner Directory (for example, for an XSD document). Here, you can also specify the encoding for xml, xsl, xsd, json, and text (separated by semicolon), for example: <code>xml;encoding=UTF-8</code> See: Partner Directory Entity Types
Partners	You can read all partners or delete a partner from the Partner Directory. A partner is identified by an internal identifier (referred to as <code>Pid</code>) (which is unique within the tenant partner directory and has a maximum length of 60 characters). The <code>PID</code> may consist of alphanumeric characters (A-Z, a-z, 0-9), as well as the following special characters: <code>'-', '!', '!', '!', '~', '<', '>',</code> and <code>'@'</code>. A partner can have parameters (<code>StringParameter</code>, <code>BinaryParameter</code>, or <code>UserCredentialParameter</code>). The parameter is uniquely identified by the <code>Pid</code> of the partner to which the parameter belongs and its <code>Id</code>. The ID of this entity may consist of alphanumeric characters (A-Z, a-z, 0-9), as well as the following special characters: <code>'-', '!', '!', '!', '~', '<', '>',</code> and <code>'@'</code>.
String Parameters	You can access (read, write, delete) a string parameter of the Partner Directory. See: Partner Directory Entity Types
User Credential Parameters	Represents a User Credentials artifact (that is to be used for outbound calls to a partner system). You can access (read, write, delete) a User Credentials artifact. You use this parameter to configure and deploy User Credentials artifacts. The User Credentials parameter contains a user and a password (both maximum length of 200 characters). The ID of this entity may consist of alphanumeric characters (A-Z, a-z, 0-9), as well as the following special characters: <code>'-', '!', '!', '!', '~', '<', '>',</code> and <code>'@'</code>. Note that the user credentials (user and password) relate to the receiver (partner) system that is targeted by an outbound call (where these credentials are used for basic authentication). i Note For user credentials artifacts created by the OData API, an alias with the following naming convention is generated: <code>pd:<Pid>:<Id>:UserCredential</code>

Entity Type	Description
	Pid is the internal identifier of the Partner and Id is the Id of the User Credential Parameter. i Note User credentials artifacts created with the API are displayed in the Web UI under Manage Security (Security Material tile).

For general information about query options, see [Query Options](#).

Example Requests

You find various example requests on SAP Business Accelerator Hub at <https://api.sap.com/api/PartnerDirectory/>.

For more example requests, see [Partner Directory Example Requests](#).

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first. You can do that in the following way: First, send a GET call to the API with a header X-CSRF-Token with the value Fetch. As a response, you get the value of the token in the header X-CSRF-Token. When sending the modifying request, add the header X-CSRF-Token with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under [API Environment](#)). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

`https://<host address>/api/v1/<relative resource path>`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

Caution

Limitations

Be aware of the following limitations when working with the Partner Directory:

Size restrictions for the different entity types:

- Maximum number of StringParameters overall: 3,000,000 (corresponds to 10,000 partners each using 300 StringParameters)
- Maximum number of BinaryParameters overall: 400,000 (corresponds to 10,000 partners each using 40 BinaryParameters)
- Maximum number of AlternativePartners overall: 1,000,000 (corresponds to 10,000 partners each using 100 AlternativePartners)
- Maximum number of AuthorizedUsers overall: 500,000 (corresponds to 10,000 partners each using 50 AuthorizedUsers)

The maximum size of a keystore is 2 MB (when using the Neo environment).

- The 2 MB limit corresponds to around 2000 X.509 certificates.
- A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 600 key pairs in the keystore.

Limit for certificate-to-user mapping (when using the Neo environment): 2 MB (corresponds to about 2000 X.509 certificates)

The maximum size of a keystore is 6 MB (when using the Cloud Foundry environment).

- The 6 MB limit corresponds to around 6000 X.509 certificates.
- A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 1800 key pairs in the keystore.

If you upload a whole keystore (.jks file) to the tenant, the maximum keystore size is limited to 2 MB.

For detailed step-by-step descriptions how to use the Partner Directory, see the following blogs:

- [Cloud Integration – Partner Directory – Step-by-Step Example](#)
- [Cloud Integration – Partner Directory – Partner Dependent XML Structures and IDs](#)
- [Cloud Integration – Partner Directory – Sender Partner Connecting with Client Certificate Authentication](#)

- [Cloud Integration – Partner Directory – Partner Dependent User Credential Selection](#)
- [Cloud Integration – Partner Directory – Mass Configuration](#)

Related Information

- [Parameterizing Integration Flows Using the Partner Directory](#)
- [Partner Directory Concepts](#)
- [Partner Directory Example Requests](#)
- [Managing Partner Directory Entries](#)

Permissions

Permissions, Cloud Foundry Environment

The role template `AuthGroup_TenantPartnerDirectoryConfigurator` grants permissions to access (read and write) the Partner Directory.

More information:

- [Personas](#)
- [Tasks and Permissions](#)

Permissions, Neo Environment

The authorization group `AuthGroup.TenantPartnerDirectoryConfigurator` provides permissions to read and write Partner Directory content (`StringParameter`, `BinaryParameter`, `AuthorizedUser`, `AlternativePartnerId`). You can assign this authorization group to the technical user that uses the Partner Directory remote API.

Read access is protected by the role `TenantPartnerDirectory.read`. This role is assigned to the following authorization groups:

- `AuthGroup.Administrator` (Tenant Administrator)
- `AuthGroup.IntegrationDeveloper`
- `AuthGroup.ReadOnly` (Supporter)
- `AuthGroup.SystemDeveloper`
- `AuthGroup.TenantPartnerDirectoryConfigurator`

Write access is protected by the role `TenantPartnerDirectory.write`. This role is assigned to the following authorization groups:

- `AuthGroup.Administrator` (Tenant Administrator)
- `AuthGroup.TenantPartnerDirectoryConfigurator`

Additionally, the authorization group `AuthGroup.TenantPartnerDirectoryConfigurator` contains the roles `NodeManager.deploycredentials` and `NodeManager.readcredentials` for deploying and reading `UserCredentialParameters` via the Partner Directory OData API.

More information:

- [Personas](#)
- [Tasks and Permissions](#)

Partner Directory Example Requests

You find various example requests on SAP Business Accelerator Hub at:

<https://api.sap.com/api/PartnerDirectory/>

For more example requests, see:

Alternative Partner	Get Alternative Partner by Key Fields Get Alternative Partners (Filter with Select) Create Alternative Partner
---------------------	--

	Update an Alternative Partner
Authorized User	Requests for String_Parameter, Binary_Parameter, and Authorized User
String Parameter	
Binary Parameter	
User Credential Parameters	Requests for UserCredentialParameter

See also: [Mass Create/Change/Delete Operations in One Transaction for StringParameter, BinaryParameter, AlternativePartner, and AuthorizedUser](#)

i Note

The charset encoding of the JSON documents in the requests must be UTF-8. The charset encoding of the JSON documents of the responses is also UTF-8.

Get Alternative Partner by Key Fields

Get an alternative Partner through key fields Agency, Scheme, and Id.

Perform the following call:

Method	Resource Path
GET	/AlternativePartners(Hexagency=' {Hexagency}' ,Hexscheme=' {Hexscheme}' ,Hexid=' {Hexid}')

Assume that you like to get the alternative partner associated with agency AgencyA, schema SchemeA, and Id Partner123.

Therefore, you need to send the following GET request providing the hexadecimal values for the key parameters:

```
https://<host address>/api/v1/AlternativePartners(Hexagency='4167656E637941',Hexscheme='536368656D6541',Hexid='506172746E6572313233')
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a response, you get a message that contains the following part:

```
<content type="application/xml">
  <m:properties>
    <d:Hexagency>4167656E637941</d:Hexagency>
    <d:Hexscheme>536368656D6541</d:Hexscheme>
    <d:Hexid>506172746E6572313233</d:Hexid>
    <d:Agency>AgencyA</d:Agency>
    <d:Scheme>SchemeA</d:Scheme>
    <d:Id>Partner123</d:Id>
    <d:Pid>PartnerZ</d:Pid>
    <d:LastModifiedBy><user who modified alternative partner></d:LastModifiedBy>
    <d:LastModifiedTime>2021-03-16T08:25:53.556</d:LastModifiedTime>
    <d:CreatedBy><user who created alternative partner></d:CreatedBy>
    <d:CreatedTime>2020-07-01T11:57:26.096</d:CreatedTime>
  </m:properties>
</content>
```

Certain values (for example, for the agency when indicated in the example request as Hexagency) have to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my agency` is expressed by the following decimal numbers: 109 121 32 97 103 101 110 99 121 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 67 65 6E 63 79. Note that 6D is the hexadecimal representation of 109, and so forth. That means that, for example, the value `my agency` is given in hexadecimal notation by 6D79206167656E6379. Note that the hex values are built from the string values by first transforming the string value to a UTF-8-encoded byte array and then by calculating a hex string (with uppercase letters 'A', 'B', 'C', 'D', 'E').

Java example for calculating hex values:

```
byte[] bytes = "testValue".getBytes(StandardCharsets.UTF_8);
String hexString = DatatypeConverter.printHexBinary(bytes);
```

However, the properties Hexagency, Hexscheme, and Hexid are not supported in the filter. If you use these properties, you get an empty result.

Get Alternative Partners (Filter with Select)

Get all alternative partners for a dedicated agency, schema, and Id. Furthermore, in the result set show only the Pid parameter.

Perform the following call:

Method	Resource Path
GET	/AlternativePartners?\$filter=Agency eq '{Agency}' and Scheme eq '{Scheme}' and Id eq '{Id}'&\$select=Pid

Assume that you like to get the Pid value for the Partner Directory entry associated with agency `agency1`, schema `schema1`, and Id `Id1`.

Therefore, you need to send the following GET request:

```
https://<host address>/api/v1/AlternativePartners?$filter=Agency eq 'agency1' and Scheme eq 'schema1' and Id eq 'Id1'&$select=Pid
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a response, you get a message that contains the following part:

```
<entry>
<id>https://<host address>/api/v1/AlternativePartners(Hexagency='6167656E637931',Hexscheme='736368656D6131',Hexid='496431')</id>
<title type="text">AlternativePartners</title>
<updated>2021-03-16T08:08:41.059Z</updated>
<category term="com.sap.hci.api.AlternativePartner" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices/scheme"/>
<link href="AlternativePartners(Hexagency='6167656E637931',Hexscheme='736368656D6131',Hexid='496431')" rel="edit" title="Alter
<content type="application/xml">
  <m:properties>
    <d:Pid>partnerX</d:Pid>
  </m:properties>
</content>
</entry>
```

Certain values (for example, for the agency when indicated in the example request as Hexagency) are also provided by the API in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my agency` is expressed by the following decimal numbers: 109 121 32 97 103 101 110 99 121 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 67 65 6E 63 79. Note that 6D is the hexadecimal representation of 109, and so forth. That means that, for example, the value `my agency` is given in hexadecimal notation by 6D79206167656E6379.

However, the properties Hexagency, Hexscheme, and Hexid aren't supported in the filter. If you use these properties, you get an empty result.

Create Alternative Partner

Create an alternative partner.

Perform the following call:

Method	Resource Path
POST	/AlternativePartners

Provide the following request body:

```
{
  "Agency": "string",
  "Scheme": "string",
  "Id": "string",
  "Pid": "string"
}
```

If you set the optional query parameter `user`, this user is used for audit logging instead of the user that is initiating the request.

Assume that you like to create a new alternative partner with the following attributes:

- Agency: myagency
- Scheme: myschema
- Id: myId
- Pid: partner1234

Therefore, you need to send the following POST request:

`https://<host address>/api/v1/AlternativePartners`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Provide the following request body:

```
{
  "Agency": "myschema",
  "Scheme": "myschema",
  "Id": "myId",
  "Pid": "partner1234"
}
```

As a response, you get a message that contains the following part:

```
<m:properties>
  <d:Hexagency>6D79736368656D61</d:Hexagency>
  <d:Hexscheme>6D79736368656D61</d:Hexscheme>
  <d:Hexid>6D794964</d:Hexid>
  <d:Agency>myschema</d:Agency>
  <d:Scheme>myschema</d:Scheme>
  <d:Id>myId</d:Id>
  <d:Pid>partner1234</d:Pid>
  <d:LastModifiedBy><user who modified alternative partner></d:LastModifiedBy>
  <d:LastModifiedTime>2021-03-16T07:31:05.761</d:LastModifiedTime>
  <d:CreatedBy><user who created alternative partner></d:CreatedBy>
  <d:CreatedTime>2021-03-16T07:31:05.761</d:CreatedTime>
</m:properties>
```

Certain values (for example, for the agency when indicated in the example request as Hexagency) are also provided by the API in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my agency` is expressed by the following decimal numbers: 109 121 32 97 103 101 110 99 121 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 67 65 6E 63 79. Note that 6D is the hexadecimal representation of 109, and so forth. That means that, for example, the value `my agency` is given in hexadecimal notation by 6D79206167656E6379.

Update an Alternative Partner

Update an alternative Partner.

Perform the following call:

Method	Resource Path
PUT	<code>/AlternativePartners(Hexagency='{Hexagency}',Hexscheme='{Hexscheme}',Hexid='{Hexid}')</code>

Provide the following request body (with the new Pid value).

```
{
  "Pid": "string"
}
```

If you set the optional query parameter `user`, this user is used for audit logging instead of the user that is initiating the request.

Assume that you like to update a new alternative partner with the following attributes:

- Agency: AgencyA
- Scheme: SchemaA
- Id: Partner123
- New Pid: PartnerZ

Therefore, you need to send the following PUT request, providing the hexadecimal values for the key parameters:

```
https://<host
address>/api/v1/AlternativePartners(Hexagency='4167656e637941',Hexscheme='536368656d6541',Hexid='506172746e6572313233')
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Provide the following request body:

```
{
  "Pid": "PartnerZ"
}
```

Certain values (for example, for the agency when indicated in the example request as `Hexagency`) have to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my agency` is expressed by the following decimal numbers: 109 121 32 97 103 101 110 99 121 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 67 65 6E 63 79. Note that 6D is the hexadecimal representation of 109, and so forth. That means that, for example, the value `my agency` is given in hexadecimal notation by 6D79206167656E6379. Note that the hex values are built from the string values by first transforming the string value to a UTF-8-encoded byte array and then by calculating a hex string (with uppercase letters 'A', 'B', 'C', 'D', 'E').

Java example for calculating hex values:

```
byte[] bytes = "testValue".getBytes(StandardCharsets.UTF_8);
String hexString = DatatypeConverter.printHexBinary(bytes);
```

Requests for String Parameter, Binary Parameter, and Authorized User

Create Authorized User

Purpose	Method	Resource Path
Create authorized user.	POST	/AuthorizedUsers

Assume that you like to create an authorized user `MyUser` that is to be associated to partner with Pid `PartnerZ`.

Perform a POST request to the following address:

```
https://<host address>/api/v1/AuthorizedUsers
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Provide the following request body:

```
{
  "User": "MyUser",
  "Pid": "PartnerZ"
}
```

As a response, you get a message that contains the following part:

```

<content type="application/xml">
  <m:properties>
    <d:User>myuser</d:User>
    <d:Pid>PartnerZ</d:Pid>
    <d:LastModifiedBy><modifying user></d:LastModifiedBy>
    <d:LastModifiedTime>2021-03-16T11:46:13.969</d:LastModifiedTime>
    <d:CreatedBy><creating user></d:CreatedBy>
    <d:CreatedTime>2021-03-16T11:46:13.969</d:CreatedTime>
  </m:properties>
</content>

```

i Note

If you want to read `AuthorizedUsers` with a filter expression containing user values, such as `/AuthorizedUser?$filter=User eq 'myuser'`, you must always use lowercase characters (Locale:English) for the user value.

Create and Get a String Parameter

Purpose	Method	Resource Path
Create String parameter.	POST	/StringParameters
Get String parameter.	GET	/StringParameters

Assume that you like to create a String parameter `ReceiverAddress` that contains the endpoint address of a receiver system called by Cloud Integration .

To create the String parameter, perform a POST request to the following address:

`https://<host address>/api/v1/StringParameters?user=myuser`

i Note

The custom query option `user` is optional, but recommended if a technical user is used for the communication. The query option `user` is the logged-in user of the system that initiates the call.

Assume that you like to create a String parameter with the following attributes:

- Pid: PartnerZ
- Id: ReceiverAddress
- Value: `http://receiverZ.receiver_org.com`

Provide the following request body:

```

{
  "Pid": "PartnerZ",
  "Id": "ReceiverAddress",
  "Value": "http://receiverZ.receiver_org.com"
}

```

As a response, you get a message that contains the following part:

```

<content type="application/xml">
  <m:properties>
    <d:Pid>PartnerZ</d:Pid>
    <d:Id>ReceiverAddress</d:Id>
    <d:LastModifiedBy>myuser</d:LastModifiedBy>
    <d:LastModifiedTime>2021-03-16T11:25:07.916</d:LastModifiedTime>
    <d:CreatedBy>myuser</d:CreatedBy>
    <d:CreatedTime>2021-03-16T11:25:07.916</d:CreatedTime>
    <d:Value>http://receiverZ.receiver_org.com</d:Value>
  </m:properties>
</content>

```

You've the several options to read a String parameter.

To read String parameters associated with a partner with Pid **PartnerZ**, perform the following GET request:

```
https://<host address>/api/v1/StringParameters?$filter=Pid eq 'PartnerZ'
```

i Note

For better readability, the URI `https://<host address>/api/v1/StringParameters?$filter=Pid eq 'PartnerZ'` isn't URI encoded. The correct URI is `https://appl-tmn.avt.hana.ondemand.com/api/v1/StringParameters$filter=Pid%20eq%20%27PartnerZ%27`.

As a response, you get a message that contains the following part:

```
<content type="application/xml">
  <m:properties>
    <d:Pid>PartnerZ</d:Pid>
    <d:Id>ReceiverAddress</d:Id>
    <d:LastModifiedBy>myuser</d:LastModifiedBy>
    <d:LastModifiedTime>2021-03-16T12:06:08.856</d:LastModifiedTime>
    <d:CreatedBy>myuser</d:CreatedBy>
    <d:CreatedTime>2021-03-16T12:06:08.856</d:CreatedTime>
    <d:Value>http://receiverZ.receiver_org.com</d:Value>
  </m:properties>
</content>
```

To read String parameters with paging and select, you can perform the following GET request:

```
https://<host address>/api/v1/StringParameters?$select=Value
```

As response, you get a message with one item per String parameter. Each item contains a part such like:

```
<entry>
  <id>https://<host address>/api/v1/StringParameters(Pid='PartnerZ',Id='ReceiverAddress')</id>
  <title type="text">StringParameters</title>
  <updated>2021-03-16T12:27:14.51Z</updated>
  <category term="com.sap.hci.api.StringParameter" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices/scheme"
  <link href="StringParameters(Pid='PartnerZ',Id='ReceiverAddress') " rel="edit" title="StringParameter"/>
  <content type="application/xml">
    <m:properties>
      <d:Value>http://receiverZ.receiver_org.com</d:Value>
    </m:properties>
  </content>
</entry>
```

This GET request selects only the value field content of the first 1000 string parameter entities. To get the next 1000 entries, you've to call:

```
https://<host address>/api/v1/StringParameters?$select=Value&$format=json&$skiptoken=1000
```

The paging size for binary parameters is 30.

The following GET request reads the single entity by key values:

```
https://<tmn>/api/v1/StringParameters(Pid='PartnerZ',Id='ReceiverAddress')
```

As response, you get with one item with the following part:

```
<m:properties>
  <d:Pid>PartnerZ</d:Pid>
  <d:Id>ReceiverAddress</d:Id>
  <d:LastModifiedBy>myuser</d:LastModifiedBy>
  <d:LastModifiedTime>2021-03-16T12:06:08.856</d:LastModifiedTime>
  <d:CreatedBy>myuser</d:CreatedBy>
  <d:CreatedTime>2021-03-16T12:06:08.856</d:CreatedTime>
  <d:Value>http://receiverZ.receiver_org.com</d:Value>
</m:properties>
```

For more information, check out the following SAP Community blog: [Cloud Integration – Partner Directory – Step-by-Step Example](#)👉.

Create Binary Parameter

Purpose	Method	Resource Path
Create Binary parameter.	POST	/BinaryParameters

Assume that you like to create a Binary parameter `MyUser` that is to be associated to partner with `Pid PartnerZ`.

Perform a POST request to the following address:

`https://<host address>/api/v1/BinaryParameters`

Provide the following request body:

```
{
  "Pid": "PartnerZ",
  "Id": "Z_XSD",
  "ContentType": "xsd",
  "Value": "PHhz0nNjaGVtYSBhdHRy2R1Y3RzIj4KICAgIDx4czpj . . . . ."
}
```

As `Value`, in our example provide a base64-encoded XSD file.

As response, you get:

```
<m:properties>
  <d:Pid>PartnerZ</d:Pid>
  <d:Id>Z_XSD</d:Id>
  <d:LastModifiedBy>myuser</d:LastModifiedBy>
  <d:LastModifiedTime>2021-03-16T15:29:16.151</d:LastModifiedTime>
  <d:CreatedBy>myuser</d:CreatedBy>
  <d:CreatedTime>2021-03-16T15:29:16.151</d:CreatedTime>
  <d:ContentType>xsd</d:ContentType>
  <d:Value>PHhz0nNjaGVtYSBhdHRyYWJldGVGb3JtRG. . . M6c2NoZWlhPg==</d:Value>
</m:properties>
```

i Note

Note that possible values for `ContentType` are: xml, xsl, xsd, json, text, zip, gz (for GZIP files), zlib, crt (for DER-encoded X.509 certificate).

The maximum length of the `Value` field is restricted to 260 KB.

The PEM-DER string of a certificate must be base64 encoded for the value field of the binary parameter in the POST or PUT request.

For more information on Binary parameters, check out the following SAP Community blog: [Cloud Integration – Partner Directory – Partner Dependent XML Structures and IDs](#).

Update Authorized User

Purpose	Method	Resource Path
Update binary parameter.	PUT	/BinaryParameters(Pid='partner1',Id='bp1')?user=userXYZ

To update a binary parameter associated with user `userXYZ`, perform a PUT request to the following address:

`https://<host address>/api/v1/BinaryParameters(Pid='partner1',Id='bp1')?user=userXYZ`

Provide the following request body:

```
{
  "ContentType": "string",
  "Value": "string"
}
```

Mass Create/Change/Delete Operations in One Transaction for StringParameter, BinaryParameter, AlternativePartner, and AuthorizedUser

You can perform mass processing through an OData batch. Batch processing allows you to group multiple operations. Batch requests with one change set are allowed. One change set is treated as an atomic operation (transaction). Create/change/delete operations can be executed in one change set for the entity types `StringParameter`, `BinaryParameter`, `AlternativePartner`, and `AuthorizedUser`. The following example shows how.

```
POST https://<host address>/api/v1/$batch
Accept: application/json
Content-type: multipart/mixed; boundary=batch_34fcd829-64e4-4139-b880-f24aa4ee9235
--batch_34fcd829-64e4-4139-b880-f24aa4ee9235
Content-Type: multipart/mixed; boundary=changeset_845b320f-7f06-423d-b10a-a24048a27b1c
--changeset_845b320f-7f06-423d-b10a-a24048a27b1c
Content-Type: application/http
Content-Transfer-Encoding: binary

POST StringParameters?user=userAbc HTTP/1.1
Content-Length: 46
Accept: application/json
content-type: application/json

{"Pid":"partner7","Id":"sp11","Value":"sp11v"}
--changeset_845b320f-7f06-423d-b10a-a24048a27b1c
Content-Type: application/http
Content-Transfer-Encoding: binary

POST StringParameters?user=userAbc HTTP/1.1
Content-Length: 46
Accept: application/json
content-type: application/json

{"Pid":"partner7","Id":"sp12","Value":"sp12v"}
--changeset_845b320f-7f06-423d-b10a-a24048a27b1c
Content-Type: application/http
Content-Transfer-Encoding: binary

PUT StringParameters(Pid='partner2',Id='sp3')?user=AriabaUserId HTTP/1.1
Content-Length: 53
Accept: application/json
content-type: application/json

{"Value":"sp3v_modified"}
--changeset_845b320f-7f06-423d-b10a-a24048a27b1c--
--batch_34fcd829-64e4-4139-b880-f24aa4ee9235--
Batch response:
--batch_46440294-4efe-4c78-8d38-6414e44382d0
Content-Type: multipart/mixed; boundary=changeset_8a1989b1-9382-446c-8479-f9330108b1f4
--changeset_8a1989b1-9382-446c-8479-f9330108b1f4
Content-Type: application/http
Content-Transfer-Encoding: binary
HTTP/1.1 201 Created
DataServiceVersion: 2.0
Location: https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp11')
Content-Type: application/json
Content-Length: 328

{"d":{"__metadata":{"id":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp11')","uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp11')"},"uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp11')"},"uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp11')"}
--changeset_8a1989b1-9382-446c-8479-f9330108b1f4
Content-Type: application/http
Content-Transfer-Encoding: binary
HTTP/1.1 201 Created
DataServiceVersion: 2.0
Location: https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp12')
Content-Type: application/json
Content-Length: 328

{"d":{"__metadata":{"id":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp12')","uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp12')"},"uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp12')"},"uri":"https://<host address>/api/v1/StringParameters(Pid='partner7',Id='sp12')"}
--changeset_8a1989b1-9382-446c-8479-f9330108b1f4
Content-Type: application/http
Content-Transfer-Encoding: binary
HTTP/1.1 204 No Content
DataServiceVersion: 2.0
--changeset_8a1989b1-9382-446c-8479-f9330108b1f4--
--batch_46440294-4efe-4c78-8d38-6414e44382d0--
```

For more information, check out the following SAP Community blog: [Cloud Integration – Partner Directory – Mass Configuration](#) .

Requests for UserCredentialParameter

⚠ Caution

The charset encoding of the JSON documents in the requests must be UTF-8. The charset encoding of the JSON documents of the responses is also UTF-8.

Create or Update a User Credentials Parameter

Method	Resource Path
POST	/UserCredentialParameters
PUT	

Assume that you like to create a User Credentials parameter with Pid PartnerZ and Id Id1.

Perform a POST request to the following address:

<https://<host address>/api/v1/UserCredentialParameters>

The part <https://<host address>/api/v1> is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

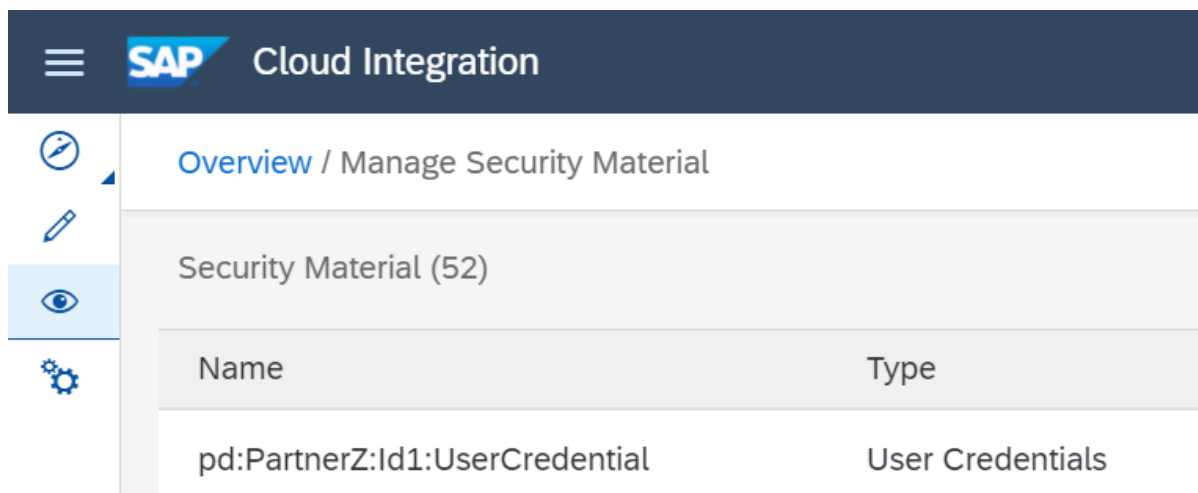
Provide the following request body (example):

```
{
  "Pid": "PartnerZ",
  "Id": "Id1",
  "User": "myUser",
  "Password": "Abcd1234"
}
```

As a response, you get a message that contains the following part:

```
<content type="application/xml">
  <m:properties>
    <d:Pid>PartnerZ</d:Pid>
    <d:Id>Id1</d:Id>
    <d:LastModifiedBy>P12345678</d:LastModifiedBy>
    <d:LastModifiedTime>2021-03-17T10:28:31.026</d:LastModifiedTime>
    <d:CreatedBy>P12345678</d:CreatedBy>
    <d:CreatedTime>2021-03-17T10:28:31.026</d:CreatedTime>
    <d:User>myUser</d:User>
    <d>Password m:null="true"/>
  </m:properties>
</content>
```

You can find a new User Credentials artifact in the [Monitor](#) section ([Security Material](#) tile under [Manage Security](#)). The newly created artifact has the name `pd:PartnerZ:Id1:UserCredential`.



SAP Cloud Integration

Overview / Manage Security Material

Security Material (52)

Name	Type
pd:PartnerZ:Id1:UserCredential	User Credentials

Note

The specification of a user query option User (such as `?user=aribaUser`) has no effect. It is always the logged in technical user (in the above example P12345678 that appears in the properties LastModifiedBy and CreatedBy).

You can also use the POST request to update a User Credentials parameter with the same values for PID and Id.

There is no difference between the properties LastModifiedTime and CreatedTime. They always contain the time of the last POST request or the tuple PID and Id.

There is no difference between the properties LastModifiedB and CreatedBy. Both fields contain the logged in user of the last POST request for the tuple PID and Id.

Caution

PUT requests are not supported.

Get User Credentials Parameters

Method	Resource Path
GET	/UserCredentialParameters

Assume that you like to get the User Credentials parameter with Pid PartnerZ.

Perform a GET request to the following address:

`https://<host address>/api/v1/UserCredentialParameters?user=AnyUser&$filter=Pid eq 'PartnerZ'`

If there are two User Credentials parameters with Pid PartnerZ, you get the following response:

```
<feed ...>
...
<entry>
  <id>https://d0262-tmn.hci.eu1.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id1')</id>
  <title type="text">UserCredentialParameters</title>
  <updated>2021-03-17T10:42:46.843Z</updated>
  <category term="com.sap.hci.api.UserCredentialParameter" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices" />
  <link href="UserCredentialParameters(Pid='PartnerZ',Id='Id1')" rel="edit" title="UserCredentialParameter"/>
  <content type="application/xml">
    <m:properties>
      <d:Pid>PartnerZ</d:Pid>
      <d:Id>Id1</d:Id>
      <d:LastModifiedBy>P12345678</d:LastModifiedBy>
      <d:LastModifiedTime>2021-03-17T10:28:31.101</d:LastModifiedTime>
      <d:CreatedBy>P12345678</d:CreatedBy>
      <d:CreatedTime>2021-03-17T10:28:31.101</d:CreatedTime>
      <d:User>myUser</d:User>
      <d>Password m:null="true"/>
    </m:properties>
  </content>
</entry><entry>
  <id>https://d0262-tmn.hci.eu1.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id2')</id>
  <title type="text">UserCredentialParameters</title>
  <updated>2021-03-17T10:42:46.843Z</updated>
  <category term="com.sap.hci.api.UserCredentialParameter" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices" />
  <link href="UserCredentialParameters(Pid='PartnerZ',Id='Id2')" rel="edit" title="UserCredentialParameter"/>
  <content type="application/xml">
    <m:properties>
      <d:Pid>PartnerZ</d:Pid>
      <d:Id>Id2</d:Id>
      <d:LastModifiedBy>P12345678</d:LastModifiedBy>
      <d:LastModifiedTime>2021-03-17T10:42:20.822</d:LastModifiedTime>
      <d:CreatedBy>P12345678</d:CreatedBy>
      <d:CreatedTime>2021-03-17T10:42:20.822</d:CreatedTime>
      <d:User>myUser</d:User>
      <d>Password m:null="true"/>
    </m:properties>
  </content>
</entry>
</feed>
```

Note

The returned value for the Password property is always null.

You need to specify a filter for property Pid. If you don't specify a filter for this property, you get an error.

If you specify a `User` in the query options (such as `?user=AnyUser` in the example request), this user appears in the audit log.

Assume that you like to get the `User Credentials` parameter with `Pid PartnerZ` and select only the properties `Id` and `CreatedBy`.

Perform a GET request to the following address:

`https://<host address>/api/v1/UserCredentialParameters?user=AnyUser&$filter=Pid eq 'PartnerZ'&$select=Id,CreatedBy`

If there are two `User Credentials` parameters with `Pid PartnerZ`, you get the following response:

```
<feed ...>
...
<entry>
  <id>https://d0262-tmn.hci.eu1.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id1')</id>
  <title type="text">UserCredentialParameters</title>
  <updated>2021-03-17T10:50:15.729Z</updated>
  <category term="com.sap.hci.api.UserCredentialParameter" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices" />
  <link href="UserCredentialParameters(Pid='PartnerZ',Id='Id1')" rel="edit" title="UserCredentialParameter"/>
  <content type="application/xml">
    <m:properties>
      <d:Id>Id1</d:Id>
      <d:CreatedBy>P12345678</d:CreatedBy>
    </m:properties>
  </content>
</entry>
<entry>
  <id>https://d0262-tmn.hci.eu1.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id2')</id>
  <title type="text">UserCredentialParameters</title>
  <updated>2021-03-17T10:50:15.731Z</updated>
  <category term="com.sap.hci.api.UserCredentialParameter" scheme="http://schemas.microsoft.com/ado/2007/08/dataservices" />
  <link href="UserCredentialParameters(Pid='PartnerZ',Id='Id2')" rel="edit" title="UserCredentialParameter"/>
  <content type="application/xml">
    <m:properties>
      <d:Id>Id2</d:Id>
      <d:CreatedBy>P12345678</d:CreatedBy>
    </m:properties>
  </content>
</entry>
</feed>
```

Assume that you like to get the `User Credentials` parameter with `Pid PartnerZ` and `Id Id1` and select only the properties `User` and `CreatedBy`.

Perform a GET request to the following address:

`https://<host address>/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id1')?user=AnyUser&$select=User,CreatedBy`

You get the following response:

```
<entry ...>
...
  <content type="application/xml">
    <m:properties>
      <d:User>myUser</d:User>
      <d:CreatedBy>P12345678</d:CreatedBy>
    </m:properties>
  </content>
</entry>
```

Use Custom Query Option to Get Password in Hash Format

You can use the query option `returnHashedPassword=SHA256` to request that the password is returned in hash format. You get a value in the `password` property with the following format:

`SAHA256.<SHA256 hash value of the password in HEX>`

If you do not use this option, you get a null value for the password.

For example, perform the following GET request:

```
https://<host address>/api/v1/UserCredentialParameters(Pid='PartnerZ',Id='Id1')?
user=AnyUser&returnHashedPassword=SHA256
```

You get the following response:

```
<entry ...>
...
  <content type="application/xml">
    <m:properties>
      <d:Pid>PartnerZ</d:Pid>
      <d:Id>Id1</d:Id>
      <d:LastModifiedBy>P12345678</d:LastModifiedBy>
      <d:LastModifiedTime>2021-03-17T10:28:31.101</d:LastModifiedTime>
      <d:CreatedBy>P12345678</d:CreatedBy>
      <d:CreatedTime>2021-03-17T10:28:31.101</d:CreatedTime>
      <d:User>myUser</d:User>
      <d>Password>SHA256.3f21a8490cef2bfb60a9702e9d2ddb7a805c9bd1a263557dfd51a7d0e9dfa93e</d>Password>
    </m:properties>
  </content>
</entry>
```

Performing Batch Operations

There are restrictions for change sets in batch:

- A change set can only contain exactly one request with the `UserCredentialParameter` entity type.
- If a change set contains a `UserCredentialParameter` request, it cannot contain other requests with different entity types.

POST `https://<host address>/api/v1/$batch`:

```
--batch_06a89345-8056-41c4-91c2-55636fc844fe
Content-Type: multipart/mixed; boundary=changeset_d8938fea-cd05-464a-a7c8-4522e5e7b1e4
--changeset_d8938fea-cd05-464a-a7c8-4522e5e7b1e4
Content-Type: application/http
Content-Transfer-Encoding: binary
POST UserCredentialParameters?user=AribaUser HTTP/1.1
Content-Length: 83
Accept: application/json
Content-Type: application/json
{"Pid":"pid_batch","Id":"id_batch","User":"user_batch","Password":"password_batch"}
--changeset_d8938fea-cd05-464a-a7c8-4522e5e7b1e4--
--batch_06a89345-8056-41c4-91c2-55636fc844fe--
Batch response
--batch_6264c4f6-6117-4159-87e7-82d2e370850e
Content-Type: multipart/mixed; boundary=changeset_0033d380-2e75-4284-8fb1-d7333a40dc9a
--changeset_0033d380-2e75-4284-8fb1-d7333a40dc9a
```

```
Content-Type: application/http
Content-Transfer-Encoding: binary
```

```
Response:
--batch_6264c4f6-6117-4159-87e7-82d2e370850e
Content-Type: multipart/mixed; boundary=changeset_0033d380-2e75-4284-8fb1-d7333a40dc9a
--changeset_0033d380-2e75-4284-8fb1-d7333a40dc9a
Content-Type: application/http
Content-Transfer-Encoding: binary
HTTP/1.1 201 Created
DataServiceVersion: 2.0
Location: https://appl-tmn.avt.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='pid_batch',Id='id_batch')
Content-Type: application/json
Content-Length: 527
{"d":{"__metadata":{"id":"https://appl-tmn.avt.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='pid_batch',Id='id_batch')",
"uri":"https://appl-tmn.avt.hana.ondemand.com:443/api/v1/UserCredentialParameters(Pid='pid_batch',Id='id_batch')",
"type":"com.sap.hci.api.model.UserCredentialParameter"},
"PId":"pid_batch",
"Id":"id_batch",
"LastModifiedBy":"P12345678",
"LastModifiedTime":"\\/Date(1453721123280)\\/",
"CreatedBy":"P12345678",
"CreatedTime":"\\/Date(1453721123280)\\/",
"User":"user_batch","Password":null}}
--changeset_0033d380-2e75-4284-8fb1-d7333a40dc9a--
--batch_6264c4f6-6117-4159-87e7-82d2e370850e--
```

Security Content

Manage security content on the tenant that is required to configure secure connections with remote systems.

Depending on the kind of connection, the applied authorization and authorization options, and the direction of the request (either inbound or outbound), different kind of security content needs to be created and deployed on the tenant.

You can access the OData API at <https://api.sap.com/api/SecurityContent/> where you can find the basic operations.

This documentation provides additional information.

Permissions

To authorize an API client to access the OData API, perform the steps as described in:

- In the **Cloud Foundry** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\)](#).

Assign the following role template:

`MonitoringDataRead` (see [Tasks and Permissions](#))

- In the **Neo** environment, perform the steps described in: [Setting Up Inbound HTTP Connections \(for API Clients\), Neo Environment](#)

Assign the following roles:

`IntegrationOperationServer.read`, `NodeManager.read` (see [Tasks and Permissions](#))

Resources

Resource	Description
Certificate	Represents a trusted certificate (keystore entry) (see Managing Keystore Entries and X.509 Certificates).
Key Pair	Represents a key pair of the tenant keystore (see Creating a Key Pair/SSH Key Pair). You can use the API to create, import, or export a private key pair. You can also create a certificate signing request, or import and export the related certificate chain.
Keystore Entry	Represents an entry of the tenant keystore (see Managing Keystore Entries). The keystore contains private/public key pairs and certificates that are used in various scenarios. Typically, a keystore contains keys and certificates for connections in both directions (inbound and outbound). Keys can be used for protecting the communication both on the transport-level and on the message-level. Keystore entries have to be maintained in accordance with the security content owned by the connected sender and receiver systems to ensure that the communication is protected based on public key technology. For example, for scenarios where SAP Cloud Integration receives encrypted messages from a sender system, the tenant keystore needs to contain a private/public key pair (where the private key is used by the Decryptor step of the integration flow). The public key (associated with the private key) needs to be available in the corresponding key storage of the sender system. A keystore typically contains entries that belong to the tenant administrator and entries that are owned by SAP. Each entry is uniquely identified by an alias. Keystore entries have a dedicated lifecycle, which means that they have an expiration date and need to be renewed in a coordinated way. Various properties are available for a keystore entry, for example, alias, key type, key size, the validation period, Subject DN and Issuer DN, and administrative information such as the time when the entry was last modified. You can find a user interface to manage keystore entries in the Keystore tile of the Monitor section under Manage Security .
Keystore	Represents a tenant keystore (see Managing Keystore Entries). You can use this resource to manage the keystore content as a whole. For managing individual keystore entries, use the Keystore Entry and Keystore resources. You can import a keystore (containing multiple keystore entries) or export the public content of the keystore. You can back up keystore entries owned by the tenant administrator and, vice versa, retrieve backed-up entries. You can also trigger a mass deletion of keystore entries. ⚠ Caution You need to use UTF-8 encoding for the charset of the JSON documents in the requests. The charset of the JSON documents of the responses is also UTF-8 encoded. Hex representation is used for the <code>Alias</code> field; the server doesn't allow slashes or backslashes in a URI, even if they're percent encoded. Hex representation of the alias does mean that a UTF-8-encoded byte array is built from the alias string. A hex string is then calculated from this byte array. i Note The maximum size of a keystore is 2 MB (when using the Neo environment).

Resource	Description
	- The 2 MB limit corresponds to around 2000 X.509 certificates. - A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 600 key pairs in the keystore. The maximum size of a keystore is 6 MB (when using the Cloud Foundry environment). - The 6 MB limit corresponds to around 6000 X.509 certificates. - A key pair with a chain of three X.509 certificates consumes about 3 KB, so if the keystore only contains key pairs of this type, then you can store around 1800 key pairs in the keystore. If you upload a whole keystore (. j ks file) to the tenant, the maximum keystore size is limited to 2 MB.
Keystore History	Represents the SAP Key History keystore (see Managing the Lifecycle of Keys). This keystore contains old (expired) keys.
User Credentials	Represents a User Credentials artifact (see Deploying a User Credentials Artifact). To set up outbound connections using basic authentication, you need to deploy a User Credentials artifact. The artifact contains a user name and password and, if applicable, other parameters for the user associated with the outbound call.
Secure Parameter	Represents a Secure Parameter artifact (see Deploying a Secure Parameter Artifact). You use this artifact type to deploy confidential data, for example, for custom adapters.
OAuth2 Client Credentials	Represents an OAuth2 Client Credentials artifact (see Deploying an OAuth2 Client Credentials Artifact). When an OAuth 2.0 client credentials grant is implemented, the client gets access to the protected resources in two steps: After presenting a set of client credentials, the client fetches an access token from the token service. In a subsequent step, the client uses the access token to get access to the protected resources.
Certificate-to-User-Mapping (Neo environment)	Represents a Certificate-to-User-Mapping artifact required for inbound calls based on client certificate authentication and User Role authorization (see Managing Certificate-to-User Mappings, Neo Environment). i Note Certificate-to-user mappings are only available when using SAP Cloud Integration in the Neo environment. The sender authenticates itself against Cloud Integration based on a digital client certificate. This certificate is also mapped to a user (based on the information contained in a Certificate-to-User Mapping artifact deployed on the tenant). ! Restriction Be aware of the following restrictions: - Only 1:n relationships between user and certificate are supported. The same user can be used for different certificates. - When using an entity in the context of the Partner Directory, only 290 entries for certificate-to-user mappings are allowed in the Partner Directory. - The user name (user attribute) can have a maximum length of 150 characters. - The key field Id contains the MD5 of the certificate in hex format. Note that certificates with the same MD5 value overwrite each other. The MD5 is calculated from the DER-encoded bytes (not on the PEM format) and the hex format uses lower case characters (maximum length of 32 characters). - The Certificate attribute contains an X.509 certificate in PEM-DER format (maximum length of 8192 characters). - When using an entity in the context of the Partner Directory, note that the Partner Directory creates the necessary user-to-role assignments for these users so that the users can execute integration flows on the runtime node. When deleting a certificate-to-user mapping, there are restrictions for change sets in a batch: - A change set can only contain one request with the CertificateUserMapping entity type. - If a change set contains a CertificateUserMapping request, it can't contain other requests with other entity types.
Access Policies	Access policies allow you to restrict the access to integration artifacts and their associated data. See: Managing Access Policies
Artifact References	Artifact references define for which artifacts (for example, integration flows) an access policy applies. See: Managing Access Policies

For general information about query options, see [Query Options](#).

i Note

This is custom documentation. For more information, please visit [SAP Help Portal](#).

TheCloud Integration **Monitor** application provides a user interface to access these resources:

- For User Credentials and OAuth2 Client Credentials, see the **Security Material** tile under **Manage Security**.
- For Certificate, Key Pair, Keystore Entry, Keystore History, and Keystore, see the **Keystore** tile under **Manage Security**.
- For Certificate-to-User-Mapping (only available in the Neo environment), see the **Certificate-to-User Mappings** tile under **Manage Security**.

Example Requests

You find various example requests on SAP Business Accelerator Hub at [Security Content](#).

For additional example requests, see [Security Content Example Requests](#).

Certain values (for example, for the alias when indicated in the example request as <hexalias>) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string my alias is expressed by the following decimal numbers: 109 121 32 97 108 105 97 115 (because, according to the American Standard Code for Information Interchange (ASCII), the letter m is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 6C 69 61 73. Note that 6D is the hexadecimal representation of 109, and so forth. If you like to specify my alias in an example request, enter 6D7920616C696173.

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first. You can do that in the following way: First, send a GET call to the API with a header X-CSRF-Token with the value Fetch. As a response, you get the value of the token in the header X-CSRF-Token. When sending the modifying request, add the header X-CSRF-Token with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant (to be configured under **API Environment**). If you want to perform an API call against your custom tenant using an HTTP client such like Postman, make sure that the request URL is composed in the following way:

https://<host address>/api/v1/<relative resource path>

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub.

Related Information

- [Manage Security](#)
- [Concepts of Secure Communication](#)
- [Keystore](#)

Security Content Example Requests

You find various example requests on SAP Business Accelerator Hub at [Security Content](#).

For more example requests, see:

Certificate	Export Certificate Import and Update Certificate
Key Pair	Generate a Key Pair
Keystore Entry	Get All Keystore Entries Get Keystore Entry by Alias Rename the Alias of a Keystore Entry Export Public Key in OpenSSH Format
Keystore	Get Keystore Properties and Entries Import/Back up a Keystore Trigger Mass Deletion of Keystore Entries
Keystore History	Restore an Entry from the SAP Key History Keystore to the New SAP Keys Keystore
Certificate-to-User Mapping	Create a Certificate-to-User Mapping, Neo Environment Get Certificate-to-User Mappings with Specific User, Neo Environment

Export Certificate

Export a certificate from the keystore.

Perform the following call:

Method	Resource Path
GET	/KeystoreEntries('{Hexalias}')/Certificate/\$value

Assume that your tenant keystore contains a certificate with alias `smtp.mail.yahoo.com` (server certificate for the Yahoo mail server) and you like to export it. The hexadecimal value of the alias is: `736d74702e6d61696c2e7961686f6f2e636f6d`.

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: `109 121 32 97 108 105 97 115` (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number `109`, and so forth). If you convert each number to a hexadecimal number, you get: `6D 79 20 61 6C 69 61 73`. Note that `6D` is the hexadecimal representation of `109`, and so forth. If you like to specify `my alias` in an example request, enter `6D7920616C696173`.

Therefore, you need to send the following GET request:

`https://<host address>/api/v1/KeystoreEntries('736d74702e6d61696c2e7961686f6f2e636f6d')/Certificate/$value`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a result, you get the certificate in the following form (content type: `application/pkix-cert`):

```
-----BEGIN CERTIFICATE-----
MIITDCCBzSgAwIBAgIQDBxHETUvPf18xKE7JdDoNjANBgkqhkiG9w0BAQsFADBw
MQswCQYDVQQGEwJVUzEVMBMGA1UEChMMRGlnaUNlcnQgSW5jMRkwFwYDVQQLExB3
d3cuZGlnaWNlcnQuY29tMS8wLQYDVQQDEyZEaWdpQ2VydCBTSEEyIEhpZ2ggQXNz

...

F8jCgaR+bTz+hYoLTFN9mow1kr5jNgc68HALdKF8S1uCyZS5jFbD0M4HQay16pU
J++y5pv1BMTzQ57107itrSQ32praT6whMLCBy9pAnJIKzVM4IDM5H1drHc6shhuo
+XiCLYEJRHstbLVQYFwB2w==
-----END CERTIFICATE-----
```

Import and Update Certificate

Import and update a certificate.

Perform the following call:

Method	Resource Path
PUT	/CertificateResources('{Hexalias}')/\$value

Caution

Although an HTTP PUT method is used, a new OData entity is created with the sample request.

You like to import a certificate to the tenant keystore (with an alias `mycertificate`). The hexadecimal value of the alias is `6d796365727469666963617465`.

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: `109 121 32 97 108 105 97 115` (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number `109`, and so forth). If you convert each number to a

hexadecimal number, you get: 6D 79 20 61 6C 69 61 73. Note that 6D is the hexadecimal representation of 109, and so forth. If you like to specify my alias in an example request, enter 6D7920616C696173.

Send the following PUT request:

https://<host address>/api/v1/CertificateResources('6d796365727469666963617465')/\$value

The part https://<host address>/api/v1 is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Add the certificate content with the request body (example content):

```
[-----BEGIN CERTIFICATE-----
MIIDdzCCA1+gAwIBA...
....
R9I4LtD+gdwyah617jzV/0eBHRnDJELqYzmp
-----END CERTIFICATE-----]
```

Generate a Key Pair

Generate a key pair.

Perform the following call:

Method	Resource Path
POST	/KeyPairGenerationRequests

Input Properties

The following table indicates whether the properties are mandatory or not and also contains the default values:

Property	Mandatory	Type	Default Value	Description
Hexalias	X	String		Any dummy hex value System calculates the correct hex value from the specified Alias property.
Alias	X	String		Keystore entry alias
KeyType		String	"RSA"	Key type Possible values are "RSA", "DSA", "EC". If not specified, the default value is used.
SignatureAlgorithm		String	"SHA-512/RSA"	Signature Algorithm Possible values depend on the KeyType value. If KeyType="RSA", possible values are: "SHA-512/RSA", "SHA-256/RSA", "SHA-384/RSA", "SHA-224/RSA", "SHA-1/RSA". If KeyType="DSA", possible values are: "SHA-256/DSA", "SHA-224/DSA", "SHA-1/DSA". If KeyType="EC", possible values are: "SHA-512/ECDSA", "SHA-256/ECDSA", "SHA-1/ECDSA".
KeySize		Integer	4096	KeySize must be specified for KeyType="RSA" and "DSA"; in the case of KeyType="EC", either this property or the property "KeyAlgorithmParameter" must be specified. For KeyType="EC", the value must be either null or from 112 through 571.
KeyAlgorithmParameter		String	null	Currently only used for KeyType="EC": Either this property or the property "KeySize" must then be specified. This

Property	Mandatory	Type	Default Value	Description
				property can contain the standard name of the Elliptic Curve domain parameters. Possible values: secp160k1,secp160r1,secp160r2,secp192k1,secp192r1,NIST P-192,X9.62 prime192v1,secp224k1,secp224r1,NIST P-224,secp256k1,secp256r1,NIST P-256,X9.62 prime256v1,secp384r1,NIST P-384,secp521r1,NIST P-521,X9.62 prime192v2,X9.62 prime192v3,X9.62 prime239v1,X9.62 prime239v2,X9.62 prime239v3,sect163k1, NIST K-163,sect163r1,sect163r2,NIST B-163,sect193r1,sect193r2,sect233k1,NIST K-233,sect233r1,NIST B-233,sect239k1,sect283k1,NIST K-283,sect283r1,NIST B-283,sect409k1,NIST K-409,sect409r1,NIST B-409,sect571k1,NIST K-571,sect571r1,NIST B-571,X9.62 c2tnb191v1,X9.62 c2tnb191v2,X9.62 c2tnb191v3,X9.62 c2tnb239v1,X9.62 c2tnb239v2,X9.62 c2tnb239v3,X9.62 c2tnb359v1,X9.62 c2tnb431r1 We recommend using the following curves. • Curves over F_p – 192 bits: secp192k1 and secp192r1. – 224 bits: secp224k1 and secp224r1. – 256 bits: secp256k1 and secp256r1. – 384 bits: secp384r1. – 521 bits: secp521r1. • Curves over F_{2^m} – 163 bits: sect163k1, sect163r1, and sect163r2. – 233 bits: sect233k1 and sect233r1. – 239 bits: sect239k1. – 283 bits: sect283k1 and sect283r1. – 409 bits: sect409k1 and sect409r1. – 571 bits: sect571k1 and sect571r1.
CommonName	X	String		Common name of the subject distinguished name of the certificate
OrganizationUnit		String		Organization unit of the subject distinguished name of the certificate
Organization		String		Organization of the subject distinguished name of the certificate
Locality		String		Locality of the subject distinguished name of the certificate
State		String		State/province of the subject distinguished name of the certificate
Country	X	String		Country or region of the subject distinguished name of the certificate, two characters required.
Email		String		E-mail of the subject distinguished name of the certificate
ValidNotBefore		Date	Current time	Lower boundary of the validity period of the certificate
ValidNotAfter		Date	Current time + 3 years	Upper boundary of the validity period of the certificate

When the KeyType "EC" is used together with a KeySize value, the NIST-recommended elliptic curve domain parameters will be used according to the following table:

KeySize	Parameter Used	Domain Parameter	keysize
≤ 192	P-192	192	≤ 224
$192 < \text{keysize} \leq 224$	P-224	224	$224 < \text{keysize} \leq 256$
$256 < \text{keysize} \leq 384$	P-256	256	$384 < \text{keysize} \leq 521$

Certain values (for example, for the alias when indicated in the example request as <hexalias>) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number

instead. For example: The string `my alias` is expressed by the following decimal numbers: 109 121 32 97 108 105 97 115 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 6C 69 61 73. Note that 6D is the hexadecimal representation of 109, and so forth. If you like to specify `my alias` in an example request, enter 6D7920616C696173.

Request Example

For example, you can use the following request body:

```
{
  "Hexalias": "<any dummy hex value",
  "Alias": "cust_alias",
  "KeyType": "RSA",
  "SignatureAlgorithm": "SHA256/RSA",
  "KeySize": 2048,
  "CommonName": "cpi.test.wdf.sap.com",
  "OrganizationUnit": "My Organization Unit",
  "Organization": "My Organization",
  "Locality": "My Location",
  "State": "My State",
  "Country": "DE",
  "Email": "mymail@myorganization.com",
  "ValidNotBefore": "/Date(1469685029780)/",
  "ValidNotAfter": "/Date(1469775029780)"}
}
```

Get All Keystore Entries

Get all entries of the tenant keystore.

Perform the following call:

Method	Resource Path
GET	/KeystoreEntries

Assume that your tenant keystore contains a keystore entry with alias `smtp.mail.yahoo.com` (server certificate for the Yahoo mail server).

Send the following GET request:

`https://<host address>/api/v1/KeystoreEntries`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a response, you get a message that contains for each keystore entry a passage with the following structure (assuming that your tenant keystore contains a keystore entry with alias `smtp.mail.yahoo.com`):

```
<m:properties>
  <d:Hexalias>736D74702E6D61696C2E7961686F6F2E636F6D</d:Hexalias>
  <d:Alias>smtp.mail.yahoo.com</d:Alias>
  <d:KeyType>RSA</d:KeyType>
  <d:KeySize>2048</d:KeySize>
  <d:ValidNotBefore>2019-10-04T00:00:00Z</d:ValidNotBefore>
  <d:ValidNotAfter>2020-04-01T12:00:00Z</d:ValidNotAfter>
  ....
</m:properties>
```

You can design an integration flow that evaluates the validity of each keystore entry and sends out a notification if the validity of certain entries soon expires.

For an example scenario (checking for the validity of certificate-to-user mappings), check out the following SAP Community blog: [Cloud Integration – Automated Notification for Client Certificates Reaching Expiry](#) 📄.

Get Keystore Entry by Alias

Get a single keystore entry by alias.

Perform the following call:

Method	Resource Path
GET	/KeystoreEntries('{Hexalias}')

Assume that your tenant keystore contains a keystore entry with alias `smtp.mail.yahoo.com` (server certificate for the Yahoo mail server) and you like to export it. The hexadecimal value of the alias is: `736d74702e6d61696c2e7961686f6f2e636f6d`

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: `109 121 32 97 108 105 97 115` (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number `109`, and so forth). If you convert each number to a hexadecimal number, you get: `6D 79 20 61 6C 69 61 73`. Note that `6D` is the hexadecimal representation of `109`, and so forth. If you like to specify `my alias` in an example request, enter `6D7920616C696173`.

Therefore, you need to send the following GET request:

```
https://<host address>/api/v1/KeystoreEntries('736d74702e6d61696c2e7961686f6f2e636f6d')
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

As a response, you get a message for the keystore entry that contains the following part:

```
<m:properties>
  <d:Hexalias>736D74702E6D61696C2E7961686F6F2E636F6D</d:Hexalias>
  <d:Alias>smtp.mail.yahoo.com</d:Alias>
  <d:KeyType>RSA</d:KeyType>
  <d:KeySize>2048</d:KeySize>
  <d:ValidNotBefore>2019-10-04T00:00:00Z</d:ValidNotBefore>
  <d:ValidNotAfter>2020-04-01T12:00:00Z</d:ValidNotAfter>
  ....
</m:properties>
```

You can design an integration flow that evaluates the validity of the keystore entry and sends out a notification if the entry soon expires.

For an example scenario (checking for the validity of certificate-to-user mappings), check out the following SAP Community blog: [Cloud Integration – Automated Notification for Client Certificates Reaching Expiry](#)👉.

Rename the Alias of a Keystore Entry

Rename an alias of a tenant keystore entry.

Perform the following call:

Method	Resource Path
PUT	/KeystoreEntries('{Hexalias}')

The request body must contain a property of the `KeystoreEntry` entity type with a null value. Otherwise, you get an exception.

Make sure that you specify the value for the old alias in hexadecimal format. The new alias must be URL-encoded.

Assume that your tenant keystore contains a keystore entry with alias `mykeypair` and you like to rename it to `mykeypair-new`. The hexadecimal value of `mykeypair` is: `6d796b657970616972`

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: `109 121 32 97 108 105 97 115` (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number `109`, and so forth). If you convert each number to a hexadecimal number, you get: `6D 79 20 61 6C 69 61 73`. Note that `6D` is the hexadecimal representation of `109`, and so forth. If you like to specify `my alias` in an example request, enter `6D7920616C696173`.

Therefore, you need to send the following PUT request:

```
https://<host address>/api/v1/KeystoreEntries('6d796b657970616972')?renameAlias=mykeypair-new
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Export Public Key in OpenSSH Format

Export a public key in OpenSSH format.

Perform the following call:

Method	Resource Path
GET	/KeystoreEntries(' {Hexalias}')/Sshkey/\$value

The following keys are supported: rsa-encrypted and dsa-encrypted keys. Other algorithms (for example, elliptic curve (EC)) aren't supported.

Assume you like to get the public key of keystore entry `baltimore cybertrust root` (hexadecimal value: `62616c74696d6f7265206379626572747275737420726f6f74`) in OpenSSH format, send the following GET request:

`https://<host address>/api/v1/KeystoreEntries(' 62616c74696d6f7265206379626572747275737420726f6f74 ')/Sshkey/$value`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: `109 121 32 97 108 105 97 115` (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number `109`, and so forth). If you convert each number to a hexadecimal number, you get: `6D 79 20 61 6C 69 61 73`. Note that `6D` is the hexadecimal representation of `109`, and so forth. If you like to specify `my alias` in an example request, enter `6D7920616C696173`.

As response, you get:

```
ssh-rsa AAAAB3NzaC1yc2E ..... dKoanTZDcJxo5 public key for alias baltimore cybertrust root
```

Get Keystore Properties and Entries

Get the time when the keystore was modified last (LastModifiedTime) and the keystore entries of the **Current** keystore.

Perform the following call:

Method	Resource Path
GET	/Keystores(' system')? \$expand=Entries&\$select=LastModifiedTime,Entries

Import/Back up a Keystore

Import/back up a keystore.

Perform the following call:

Method	Resource Path
POST	/KeystoreResources

In the request body, provide the following properties:

Properties		
Property	Possible Values	Description
Name	system	Key field name of the keystore Currently, only <code>system</code> is allowed (SAP supports only one keystore).

Property	Possible Values	Description
Resource	<base64-encoded JKS or JCEKS keystore>	Keystore in JKS or JCEKS format
password	password	Keystore password and password of all private keys in the keystore  Caution All private keys must have the same password.

i Note

If the imported keystore contains entries that don't represent an X.509 certificate entry or a key-pair entry with a certificate chain of X.509 certificates, these entries are ignored during the import. The returned keystore entries don't contain an entry for these ignored original entries.

i Note

Only if the custom query option `returnKeystoreEntries` was `true`, the response contains the `KeystoreEntry` instances of the merged keystore, as well as the nonimported and removed entries (HTTP response code 200). A returned `KeystoreEntry` instance contains information about whether the instance was newly created, updated, unchanged, not imported, or removed in the property status. If the entry wasn't imported, information on the reason for this behavior is also provided in the `status` property.

Trigger Mass Deletion of Keystore Entries

Delete several entries owned by the tenant administrator from the keystore.

Perform the following call:

Method	Resource Path
PUT	<code>/KeystoreResources('system')?deleteEntries=true</code>

The aliases of the entries that you want to delete must be specified in the `Aliases` property of the request body.

For example, send the following request:

```
https://<host address>/api/v1/KeystoreResources('system')?deleteEntries=true
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

In the request body, provide the aliases of the entries to be deleted.

```
{"Aliases": "alias1;alias2;alias3"}
```

i Note

- If an alias contains the character `'`, it must be escaped.
Example:
`"Company A; Company B" -> "Company A\; Company B";`
- If an alias contains the character `\`, it must be escaped.
Example:
`Alias1="Company\A", Alias2="CompanyB" -> "Aliases": "Company\\A\;CompanyB"`

Restore an Entry from the SAP Key History Keystore to the New SAP Keys Keystore

Restore an entry from the [SAP Key History](#) keystore to the [New SAP Keys](#) keystore.

Perform the following call:

Method	Resource Path
PUT	/HistoryKeystoreEntries('{Hexalias}')

You can specify an optional destination alias (URL-encoded).

If no destination alias is specified, the original alias contained in the metadata of the `KeyHistory` entry is used as destination alias.

Make sure to specify the alias as hexadecimal number.

Certain values (for example, for the alias when indicated in the example request as `<hexalias>`) need to be provided in hexadecimal notation. String characters are stored by the computer as numbers. When the hexadecimal notation is required, the text is to be provided not as decimal number but as a hexadecimal number instead. For example: The string `my alias` is expressed by the following decimal numbers: 109 121 32 97 108 105 97 115 (because, according to the American Standard Code for Information Interchange (ASCII), the letter `m` is translated to the decimal number 109, and so forth). If you convert each number to a hexadecimal number, you get: 6D 79 20 61 6C 69 61 73. Note that 6D is the hexadecimal representation of 109, and so forth. If you like to specify `my alias` in an example request, enter 6D7920616C696173.

Send the following PUT request:

`https://<host address>/api/v1/HistoryKeystoreEntries('{Hexalias}')?copy=true&destinationAlias={URL-encoded destination alias}`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Create a Certificate-to-User Mapping, Neo Environment

Create a certificate-to-user mapping.

i Note

This information is relevant only when you use SAP Cloud Integration in the Neo environment.

Perform the following call:

Method	Resource Path
POST	/CertificateUserMappings

i Note

Make sure that the PEM-DER string of the certificate is base64-encoded for the `Certificate` field.

Even if the ID specified in the POST request isn't correctly calculated, the POST response returns the correct ID.

The user supplied in the query part of the URI is only used for audit logging. It doesn't appear in the `CreatedBy` or `LastModifiedBy` property of the entity. The logged-in 'technical user' is used for the `CreatedBy` and `LastModifiedBy` properties. This behavior is different to the other entity types `StringParameter`, `BinaryParameter`, `AuthorizedUser`, and `AlternativePartnerId`.

You can execute the POST request with the same certificate several times. Unlike the other entity types, no duplicate key exception is raised. You can use the POST request to update the user.

You like to create a certificate-to-user mapping with a user `myuser` and a base64 encoded certificate that is represented by a string like the following one `LS0tLS1CRUdJTiBDRVJUV2T0Y2 . . . . AotLS0tLUV0RCBDRVJUSUZJQ0FURS0tLS0t` (shortened example string).

Send the following POST request:

`https://<host address>/api/v1/CertificateUserMappings`

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

Add the following request body (example content):

```
{
  "User": "myuser",
  "Certificate": "LS0tLS1CRUdJTiBDRVJUV2T0Y2 . . . . AotLS0tLUV0RCBDRVJUSUZJQ0FURS0tLS0t"
}
```

If the request was successful, you get a response that contains the following part:

```
<m:properties>
  <d:Id>abcd123...xyz</d:Id>
  <d:User>myuser</d:User>
  <d:Certificate>LS0tLS1CRUdJTiBDRVJUV2T0Y2...AotLS0tLUVORCBDRVJUSUZJQ0FURS0tLS0t</d:Certificate>
  <d:LastModifiedBy>your_user</d:LastModifiedBy>
  <d:LastModifiedTime>2021-02-23T08:57:25.021</d:LastModifiedTime>
  <d:CreatedBy>your_user</d:CreatedBy>
  <d:CreatedTime>2021-02-23T08:57:25.021</d:CreatedTime>
  <d:ValidUntil m:null="true"/>
</m:properties>
```

Get Certificate-to-User Mappings with Specific User, Neo Environment

Get certificate-to-user mappings with a specific user.

i Note

This information is relevant only when you use SAP Cloud Integration in the Neo environment.

Perform the following call:

Method	Resource Path
GET	/CertificateUserMappings

Request Example

You like to get all certificate-to-user mappings with user `myUser`.

Send the following GET request:

```
https://<host address>/api/v1/CertificateUserMappings?$filter=User eq 'myUser'
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

If the request was successful, you get a response that contains one or multiple parts with the following attributes (example):

```
<m:properties>
  <d:Id>abcd123...xyz</d:Id>
  <d:User>myuser</d:User>
  <d:Certificate>LS0tLS1CRUdJTiBDRVJUV2T0Y2...AotLS0tLUVORCBDRVJUSUZJQ0FURS0tLS0t</d:Certificate>
  <d:LastModifiedBy>your_user</d:LastModifiedBy>
  <d:LastModifiedTime>2021-02-23T08:57:25.021</d:LastModifiedTime>
  <d:CreatedBy>your_user</d:CreatedBy>
  <d:CreatedTime>2021-02-23T08:57:25.021</d:CreatedTime>
  <d:ValidUntil>1747094340000</d:ValidUntil>
</m:properties>
```

Read Expiration Date of a Client Certificate, Neo Environment

Read the expiration date of a client certificate associated with a certificate-to-user mapping.

i Note

This information is relevant only when you use SAP Cloud Integration in the Neo environment.

Perform the following call:

Method	Resource Path
GET	/CertificateUserMappings?\$select=Id,User,ValidUntil

The following example request reads the expiration date (`ValidUntil`) of the client certificate associated with all deployed certificate-to-user mappings (together with `Id` and target user of the certificate-to-user mapping).

```
https://<host address>/api/v1/CertificateUserMappings?$select=Id,User,ValidUntil
```

The part `https://<host address>/api/v1` is also referred to as service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

If the request was successful, you get a response that contains one or multiple parts with the following attributes (example):

```
<m:properties>
<d:Id>abcd123...xyz</d:Id>
<d:User>myuser</d:User>
<d:ValidUntil>1747094340000</d:ValidUntil>
</m:properties>
```

Using such a request, you can design an integration flow that evaluates the expiration dates of all client certificates (associated with certificate-to-user mappings) and sends out a notification if the entry soon expires.

For an example scenario, check out the following SAP Community blog: [Cloud Integration – Automated Notification for Client Certificates Reaching Expiry](#) .

B2B Scenarios

Use the **B2B Scenarios OData API** to access and manage data related to inter-enterprise business integration scenarios.

With this API, you can securely query business documents and this way support efficient data exchange and business process collaboration between enterprises.

Find the OData API and the basic operations at [B2B Scenarios](#)  on SAP Business Accelerator Hub. Note that the **Try out** feature is currently unavailable.

Permissions

To authorize an API client to access the OData API, perform the steps described in [Setting Up Inbound HTTP Connections \(for API Clients\)](#) in the Cloud Foundry environment.

Then, assign the following role templates:

- MonitoringDataRead
- B2bInterchangeOperate
- B2bInterchangePayloadRead
- B2bInterchangeRead

See also [Tasks and Permissions](#).

Resources

The following resources are included in the API:

Resource	Description
Orphaned Interchanges	Retrieve a list of all orphaned interchanges available in the system.
Business Documents	Retrieve a list of all inter-enterprise business documents available in the system.
Business Document Additional Field	Retrieve business document notes.
Sender Technical Acknowledgment	Retrieve sender technical acknowledgment.
Receiver Technical Acknowledgment	Retrieve receiver technical acknowledgment.
Sender Functional Acknowledgment	Retrieve sender functional acknowledgment.
Receiver Functional Acknowledgment	Retrieve receiver functional acknowledgment.
Custom Objects	Retrieve custom objects.
Error Details	Retrieve error details.
Business Document Processing Event	Retrieve processing events.
Business Document Payloads	Retrieve documents payloads.
Business Document Protocol Headers	Retrieve protocol headers.
Business Document Notes	Retrieve business document notes.

Resource	Description
Technical Acknowledgment	Retrieve technical acknowledgments.
Functional Acknowledgment	Retrieve functional acknowledgments.
Data Archiving	Allows you to activate data archiving. See Archiving Data .

For general information about query options, see [Query Options](#).

Example Requests

To trigger the modifying actions (POST, PUT, and DELETE), you need to fetch a CSRF token first, as described in the following:

1. Send a GET call to the API with a header X-CSRF-Token with the value Fetch.
2. As a response, you get the value of the token in the header X-CSRF-Token.
3. When sending the modifying request, add the header X-CSRF-Token with the token retrieved from the first call.

On SAP Business Accelerator Hub, you can test API calls against a sandbox tenant or against a custom tenant, which you configure under [API Environment](#). If you want to perform an API call against your custom tenant using an HTTP client such as Postman, make sure that the request URL is composed according to the following pattern:

https://<host address>/api/v1/<relative resource path>

i Note

https://<host address>/api/v1 is the service root URI of the API call. For more information on the address of an API call, see [HTTP Calls and URI Components](#).

You can find the relative resource path for each operation on SAP Business Accelerator Hub. For example requests, see [B2B Scenarios](#) .

Get Orphaned Interchanges for a Dedicated Time Period

In the following example, you want to get the orphaned interchanges within a dedicated time period. In this case, assume that the time period is 2025-10-07T11:23:17.152 to 2025-10-08T11:23:17.152.

1. Perform a GET call with the following resource path:

OrphanedInterchanges/\$count?\$filter=(Date ge datetime'2025-10-07T11:23:17.152' and Date le datetime'2025-10-08T11:23:17.152')
2. Then, send the following GET request:

https://<host address>/api/v1/OrphanedInterchanges/\$count?\$filter=(Date ge datetime'2025-10-07T11:23:17.152' and Date le datetime'2025-10-08T11:23:17.152')

Get Document Payloads

In the following example, you want to retrieve business document payloads for a specific business document.

1. Perform a GET call with the following resource path:

/BusinessDocuments('9b12fbad477049ff907ac7f5a8411528')/BusinessDocumentPayloads

With this API, you can retrieve all business document payloads associated with a specific business document. It's typically used to view all raw data content (such as attachments or messages) related to a business document.
2. Assume that the document ID is 9b12fbad477049ff907ac7f5a8411528. To get the custom header property, send the following GET request:

https://<host address>/api/v1/BusinessDocuments('9b12fbad477049ff907ac7f5a8411528')/BusinessDocumentPayloads

Authentication

The API is protected by basic authentication and OAuth.

Related Information

[Setting Up Inbound HTTP Connections \(for API Clients\)](#)

HTTP Calls and URI Components

An HTTP call is composed of a **service root URI**, **resource path**, and **query options**.

To find the **resource path** and the supported **query options**, go to SAP Business Accelerator Hub and select the API. For example, to find the resource paths, the supported operations and query options for message processing logs, check out the [Message Processing Logs](#) page on SAP Business Accelerator Hub.

How to get the service root URI, depends on the environment:

- SAP BTP, Cloud Foundry environment:

Open SAP BTP cockpit, go to the subaccount and choose **Instances and Subscriptions**. View the details of the service key that you have created when setting up inbound authentication for API access, and find the `url` parameter (see [Setting Up Inbound HTTP Connections \(for API Clients\)](#)). Throughout this documentation, we refer to this part of the address also as **host address**.

i Note

If you have set up inbound API access with basic authentication of an IdP user (according to [Basic Authentication of an IdP User for API Clients](#)), no service key is in place. In this case, create a service key (with plan api as described at [Creating Service Instance and Service Key for Inbound Authentication](#)), get the `url` parameter and delete the service key after you have retrieved the service root URI.

The service root URI then is derived by appending `/api/v1`.

The service root URI is then: `<url parameter from service key>/api/v1`.

For example, to get all message processing logs with status **Completed**, use the following call:

```
https://12345abc.it-xyz789.cfapps.us10-001.hana.ondemand.com/api/v1/MessageProcessingLogs?$filter=Status eq 'COMPLETED'
```

In the example, these are the URI components:

- Service root URI:

```
https://12345abc.it-xyz789.cfapps.us10-001.hana.ondemand.com/api/v1 (where https://12345abc.it-xyz789.cfapps.us10-001.hana.ondemand.com is the url parameter of the service key)
```

- Resource path:

```
MessageProcessingLogs
```

- Query option:

```
$filter=Status eq 'COMPLETED'
```

- Neo environment:

Check out the address of your Cloud Integration user interface (Web UI). It ends with `/itspaces/shell/integration`.

Remove the `/itspaces/shell/integration` and replace it by `/api/v1`.

The service root URI is then: `<Cloud Integration user interface address without /itspaces/shell/integration>/api/v1`.

For more information about the service root URI, resource path, and query options, see [URI Conventions \(OData Version 2.0\)](#).

⚠ Caution

The browser translates space characters to `%20`. If you paste a URL from this documentation to the browser, the OData call therefore works. The string `%20` is left out of the example URLs to make them easier to read.

Query Options

Query options allow you to control the amount and order of the data that a data service returns for the resource identified by the URI.

OData System Query Options Supported by the Cloud Integration API

OData system query options are provided by the OData framework and documented in detail in the OData specification at <http://www.odata.org/documentation/odata-version-2-0/uri-conventions>.

This section provides a summary of the supported query options.

⚠ Caution

Not all query options listed below are always supported by each entity type of the Cloud Integration OData API.

For example, The DataStore and Variables APIs do not support the following query options:

- - `$filter`
 - `$inlinecount`
 - `$orderby`
 - `$skip`
 - `$top`
 - `$expand`
 - `$select`

For more information on the supported query options, check out the description of the individual resources on SAP Business Accelerator Hub.

Supported System Query Options

System Query Option	... Does the Following
<code>\$count</code>	Returns the number of records according to the resource path section of the URI (or, if a filter is applied, the number of records matching the filter).
<code>\$expand</code>	Returns related entities for a given navigation property in line with the entities being retrieved. For example, adding the query <code>\$expand=AdapterAttributes</code> to an OData request addressing the <code>MessagingProcessingLog</code> entity returns adapter-specific attributes in line with the message processing log data.
<code>\$filter</code>	Returns a subset of records according to the resource path section of the URI and to the filter expression.
<code>\$inlinecount</code>	Specifies that the response contains a count of the number of records in the collection of entries identified by the resource path section of the URI.
<code>\$metadata</code>	Returns the data model (which is the structure of all resources).
<code>\$orderby</code>	Specifies an expression for determining which values are used to order the collection of records identified by the resource path section of the URI. ⚠ Caution Limitations for the usage of <code>\$orderby</code> Regarding the <code>\$orderby</code> option with the <code>MessageProcessingLogCustomHeaderProperty</code> entity, note the following: <code>\$orderby</code> supports only the combinations: <code>\$orderby=Name, Value</code> as well as <code>\$orderby=Name desc, Value desc</code> . In case you need additional options, we recommend you to implement these at the client side.
<code>\$select</code>	Returns a limited set of information on the entities identified by the resource path section of the URI. i Note Using the <code>\$select</code> option can have a positive impact on the performance of a request.
<code>\$skip</code>	Identifies a subset of records (according to the resource path section of the URI), where the subset is defined by seeking <code>n</code> entries and selecting only the remaining entries (starting with entry <code>n+1</code>).
<code>\$top</code>	Identifies a subset of records (according to the resource path section of the URI), where the subset is defined by selecting only the first <code>n</code> items of the set.
<code>\$value</code>	When applied with the entity <code>RuntimeArtifactErrorInformation</code> , this query returns the error message in JSON format.

Filter System Query Option (`$filter`)

Examples for the `$filter` query option:

⚠ Caution

The browser translates space characters to %20. If you paste a URL from this documentation to the browser, the OData call therefore works. The string %20 is left out of the example URLs to make them easier to read.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=MessageGuid eq 'AFQaBF-k2zSgMgIYekzBzk07Bs-T'`

Addresses the message processing log with the MessageGuid AFQaBF-k2zSgMgIYekzBzk07Bs-T. This is equivalent to the query `https://<tmn>/api/v1/MessageProcessingLogs('AFQaBF-k2zSgMgIYekzBzk07Bs-T')`.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=ApplicationMessageId eq 'applicationID'`

Addresses all message processing logs with the ApplicationMessageId applicationID.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=ApplicationMessageType eq 'application-message-type'`

Addresses all message processing logs with the ApplicationMessageType application-message-type

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=LogEnd gt datetime'2014-09-01T00:00:00' and LogEnd lt datetime'2014-09-02T00:00:00'`

Addresses all message processing logs with log end time between the two datetime literals 2014-09-01T00:00:00Z and 2014-09-02T00:00:00.

You can also use `https://<tmn>/api/v1/MessageProcessingLogs?$filter=LogEnd gt datetime'2014-09-01T00:00:00' and LogStart lt datetime'2014-09-02T00:00:00'` to get all Message Processing Logs inside the chosen time interval.

Currently, parsing of time zone information in datetime literals is not supported. All datetime literals are interpreted as UTC (Universal Time Coordinated). Therefore, a zone offset cannot be interpreted. For example, a call with the following syntax will not be interpreted correctly:

`https://<tmn>/api/v1/MessageProcessingLogs?$filter=LogEnd gt datetime'2014-09-01T00:00:00Z' and LogEnd lt datetime'2014-09-02T00:00:00Z'`.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=Sender eq 'sender'`

Returns all message processing logs, where the associated messages were sent by sender.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=Receiver eq 'receiver'`

Returns all message processing logs, where the associated messages were addressed to receiver.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=CustomStatus eq 'ERROR'`

Returns all message processing logs, where the associated messages are in CustomStatus, here ERROR.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=Status eq 'ERROR'`

Returns all message processing logs, where the associated messages are in ERROR status.

- `https://<tmn>/api/v1/MessageProcessingLogs?$filter=CorrelationId eq 'AbCd-jshkdgsz23jds-T'`

Returns all messages that are correlated by the CorrelationId AbCd-jshkdgsz23jds-T.

→ Tip

You can combine other filters into one OData request, but it is recommended to use filters containing MessageGuid, CorrelationID and ApplicationMessageID separately from other filters.

Server-Side Paging (\$skiptoken System Query Option)

The Cloud Integration API provides 1000 entries for a call maximum. This feature also protects the Cloud Integration runtime environment if unfiltered queries enforce the return of huge amount of data.

If a query returns more than 1000 entries, the response will contain a *Next* link, which can be used to initiate the return of the additional entries.

Example:

```
<feed ...>
  <id><tmn>/api/v1/MessageProcessingLogs</id>
  <title type="text">MessageProcessingLogs</title>
  ...
  <link href="https://<tmn>/api/v1/MessageProcessingLogs?$skiptoken=1000" rel="next" />
</feed>
```

Filtering Message Processing Logs for Custom Header Properties

You can set custom header properties to filter message processing logs. The **Monitor** application shows these custom header properties in the filtering section of the message processing log screen.

When requesting message processing logs (MPLs) using the OData API, you can use additional query options to filter MPLs along custom header properties as well as to get information on the available custom header properties.

You can combine filter for custom properties with other query parameters such like `filter`, `format`, `orderby`, and `top`, for example.

Let's assume that you have MPLs with the custom header property `po_number` which can have different values (12345, for example).

If you like to return MPLs with given custom header properties and values (for the example above), perform one of the following API calls:

```
/MessageProcessingLogs?filterCustomHeaderProperties='po_number' eq '12345'
```

```
/MessageProcessingLogs?$format=json&...&$filter=<different-filters>&filterCustomHeaderProperties='po_number' eq '12345'
```

The response contains the MPL details.

Let's assume that you don't know which custom properties and values are available and you like to get a list of all custom header properties used in different MPLs, perform the following API call:

```
/MessageProcessingLogCustomHeaderProperties
```

The response contains one or many of the following sections with the custom header properties and their values (for example):

```
<content type="application/xml">
  <m:properties>
    <d:Id>1234</d:Id>
    <d:Name>po_number</d:Name>
    <d:Value>12345</d:Value>
  </m:properties>
</content>
```

You can use following request to get information about MPLs using a specific custom header property:

```
/MessageProcessingLogCustomHeaderProperties('1234')/Log
```

To return information about a specific custom header property, perform the following call:

```
/MessageProcessingLogCustomHeaderProperties?$filter=Name eq 'custom_property_name' and Value eq 'custom_Property_value'
```

For example, assume that there are MPLs with the custom header property `po_number` and you perform the following API call:

```
/MessageProcessingLogCustomHeaderProperties?$filter=Name eq 'po_number' and Value eq '12345'
```

The response contains the following section with the custom header property, its value, and an Id:

```
<content type="application/xml">
  <m:properties>
    <d:Id>1234</d:Id>
    <d:Name>po_number</d:Name>
    <d:Value>12345</d:Value>
  </m:properties>
</content>
```

Now that you know the custom header property Id, you can use following request to get information about MPLs associated with the custom header property with this Id:

```
/MessageProcessingLogCustomHeaderProperties('1234')/Log
```

The response contains the MPL details.

→ Tip

For more information on how to set custom header properties, see [Use Custom Header Properties to Search for Message Processing Logs](#).

Support Components

If you find an issue with the OData API, you can raise a ticket.

The following table lists the relevant support components for the different resources of the API.

API	Resource	Support Component
Integration Content	CopyIntegrationPackage	LOD-HCI-PI-WT-IFL
Integration Content	IntegrationPackages	LOD-HCI-PI-WT-IFL
Integration Content	IntegrationRuntimeArtifacts	LOD-HCI-PI-OP-NM
Integration Content	IntegrationDesignTimeArtifacts	LOD-HCI-PI-WT-IFL
Integration Content	IntegrationDesignTimeArtifactsSaveAsVersion	LOD-HCI-PI-WT-IFL
Integration Content	DeployIntegrationDesignTimeArtifacts	LOD-HCI-PI-WT-IFL
Integration Content	ServiceEndpoints	LOD-HCI-PI-WT-IFL
Integration Content	ValueMappingDesignTimeArtifacts	LOD-HCI-PI-MAP
Integration Content	DeployValueMappingDesignTimeArtifacts	LOD-HCI-PI-MAP
Integration Content	ValueMappingDesignTimeArtifactSaveAsVersion	LOD-HCI-PI-MAP
Integration Content	UpsertValMaps	LOD-HCI-PI-MAP
Integration Content	UpdateDefaultValMap	LOD-HCI-PI-MAP
Integration Content	DeleteValMaps	LOD-HCI-PI-MAP
Integration Content	IntegrationAdapterDesignTimeArtifacts	LOD-HCI-PI-WT-IFL
Integration Content	DeployIntegrationAdapterDesignTimeArtifacts	LOD-HCI-PI-WT-IFL
Integration Content	CustomTagConfigurations	LOD-HCI-PI-WT-IFL
Integration Content	MessageMappingDesignTimeArtifacts	LOD-HCI-PI-MAP
Integration Content	DeployMessageMappingDesignTimeArtifact	LOD-HCI-PI-MAP
Integration Content	MessageMappingDesignTimeArtifactSaveAsVersion	LOD-HCI-PI-MAP
Integration Content	BuildAndDeployStatus	LOD-HCI-PI-WT-IFL
Integration Content	ScriptCollectionDesignTimeArtifacts	LOD-HCI-PI-WT-IFL
Integration Content	DeployScriptCollectionDesignTimeArtifact	LOD-HCI-PI-WT-IFL
Log Files	LogFiles	LOD-HCP-PI-OP-SRV
Log Files	LogFileArchives	LOD-HCP-PI-OP-SRV
Message Processing Logs	MessageProcessingLogs	LOD-HCP-PI-OP-SRV
Message Processing Logs	IdMapTolds	LOD-HCP-PI-OP-SRV
Message Processing Logs	GenericIdempotentRepositoryEntries	LOD-HCP-PI-OP-SRV
Message Processing Logs	activateArchivingConfiguration	LOD-HCP-PI-OP-SRV
Message Processing Logs	ArchivingConfigurations	LOD-HCP-PI-OP-SRV
Message Processing Logs	ArchivingKeyPerformanceIndicators	LOD-HCP-PI-OP-SRV
Message Stores	MessageProcessingLogs	LOD-HCP-PI-OP-SRV
Message Stores	MessageStoreEntries	LOD-HCI-PI-CON-SOAP
Message Stores	MessageStoreEntryAttachments	LOD-HCI-PI-CON-SOAP
Message Stores	MessageStoreEntryAttachmentProperties	LOD-HCI-PI-CON-SOAP
Message Stores	JMSBrokers	LOD-HCI-PI-CON-SOAP
Message Stores	NumberRanges	LOD-HCI-PI-CON-SOAP
Message Stores	DataStores	LOD-HCI-PI-CON-SOAP
Message Stores	DataStoreEntries	LOD-HCI-PI-CON-SOAP

API	Resource	Support Component
Message Stores	Variables	LOD-HCI-PI-CON-SOAP
Partner Directory	AlternativePartners	LOD-HCI-PI-CON-SOAP
Partner Directory	AuthorizedUsers	LOD-HCI-PI-CON-SOAP
Partner Directory	BinaryParameters	LOD-HCI-PI-CON-SOAP
Partner Directory	Partners	LOD-HCI-PI-CON-SOAP
Partner Directory	StringParameters	LOD-HCI-PI-CON-SOAP
Partner Directory	UserCredentialParameters	LOD-HCI-PI-CON-SOAP
Security Content	Certificate Resources	LOD-HCI-PI-CON-SOAP
Security Content	KeystoreEntries	LOD-HCI-PI-CON-SOAP
Security Content	KeyPairGenerationRequests	LOD-HCI-PI-CON-SOAP
Security Content	SSHKeyGenerationRequests	LOD-HCI-PI-CON-SOAP
Security Content	KeyPairResources	LOD-HCI-PI-CON-SOAP
Security Content	CertificateChainResources	LOD-HCI-PI-CON-SOAP
Security Content	Keystores	LOD-HCI-PI-CON-SOAP
Security Content	KeystoreResources	LOD-HCI-PI-CON-SOAP
Security Content	HistoryKeystoreEntries	LOD-HCI-PI-CON-SOAP
Security Content	UserCredentials	LOD-HCI-PI-CON-SOAP
Security Content	OAuth2ClientCredentials	LOD-HCI-PI-MAP
Security Content	CertificateUserMappings	LOD-HCI-PI-CON-SOAP

Addressing the CISUsers Entity (for User Creation)

If you want to connect your Cloud Integration account to a separate Cloud Identity service, you can use the OData API to create users that a client of the Cloud Integration tenant can use to send HTTP messages via basic authentication to the runtime nodes.

The OData API allows you to create, update, read, and delete users within the Cloud Identity service that is connected to your account.

Create User

https://<tmn>/api/v1/CISUsers

HTTP Method: POST

HTTP Headers:

Name	Value	Description
Content-Type	application/json;charset=utf-8	Content type must be JSON and the charset UTF-8.
Accept	application/json	If not set, the body of the HTTP response will contain XML data. If set, the response will be JSON data.

HTTP request body (example):

```
{
  "Email": "xxx@sap.com"
}
```

HTTP response, when Accept "application/json" is set and creation was successful (HTTP response code 201):

```
{
  "d": {
    "__metadata": {
      "id": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000051')",

```

```
    "uri": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000051')",
    "type": "com.sap.hci.api.CISUser"
  },
  "Email": "xxx@sap.com",
  "CisId": "P000051"
}
```

You have to specify the `CisId` property as well as the `Email` property because `CisId` is a key field.

Properties in the Request Body

Name	Mandatory	Description
Email	Yes	E-mail address can only be used once to create a user.
CisId	No	CisId will contain the internal user ID after a user has been created. CisId will be ignored within an HTTP POST request body.
LoginName	No	Can be used to set a login name for the user.
Type	No	Not used yet
Password	No	You can use this property to set an initial password. When a password is set, the account is activated, but an e-mail is still sent to the specified e-mail address.
FamilyName	No	You can use this property to set the family name of the user.
GivenName	No	You can use this property to set the given name of the user.

Response Status and Error Codes

Response Code	Response Description
201 OK	User successfully created.
400 Bad Request	Wrong properties used.
401 Unauthorized	User of the call to the OData API does not exist in the Cloud Identity service or the password is wrong.
403 Forbidden	User not assigned the appropriate role. Assign either the <code>AuthGroup.TenantPartnerDirectoryConfigurator</code> or <code>AuthGroup.Administrator</code> authorization group or the <code>CloudIdentityService.Write</code> role. Compare the user name in the role assignment (case-sensitive).
500 Internal Server Error	Creation of user failed. For more details, check the Java trace files. A possible reason for the error is that the e-mail address is already used.

Read User

```
https://<tmn>/api/v1/CISUsers('<CisId>')
```

Example:

```
https://<tmn>/api/v1/CISUsers('P000095')
```

HTTP Method: GET

HTTP Headers:

Name	Value	Description
Accept	application/json	If not set, the body of the HTTP response will contain XML data. If set, the response will be JSON.

HTTP response, when Accept "application/json" is set and creation was successful (HTTP response code 200):

```
{ "d": {
  "__metadata": {
```

```
{
  "id": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000095')",
  "uri": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000095')",
  "type": "com.sap.hci.api.CISUser"
},
"Email": "test@test.com",
"CisId": "P000095",
"LoginName": "franz",
"Type": null,
"Password": null,
"FamilyName": "Smith",
"GivenName": "John"
}}
```

The returned value for the Password property will always be null even if a password is set.

You can also specify the properties that are returned in the query part of the HTTP request URI, for example:

```
https://<tmn>/api/v1/CISUsers('P000095')?$select=FamilyName,GivenName
```

This request only returns the FamilyName and GivenName property:

```
{
  "d": {
    "__metadata": {
      "id": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000095')",
      "uri": "https://<account>-tmn.hci.hana.ondemand.com:443/api/v1/CISUsers('P000095')",
      "type": "com.sap.hci.api.CISUser"
    },
    "FamilyName": "Smith",
    "GivenName": "John"
  }
}
```

Response Status and Error Codes	
Response Code	Response Description
200 OK	CISUser was found and successfully returned.
400 Bad Request	Wrong properties used.
401 Unauthorized	User of the call to the OData API does not exist in the Cloud Identity service or the password is wrong.
403 Forbidden	User not assigned the appropriate role. Assign either the AuthGroup.TenantPartnerDirectoryConfigurator, AuthGroup.Administrator, AuthGroup.IntegrationDeveloper, AuthGroup.SystemDeveloper, or AuthGroup.ReadOnly, authorization group or the CloudIdentityService.Read role.
404 Not Found	No CISUser was found for the specified CisId.
500 Internal Server Error	Read failed. For more details, check the Java trace files.

Change User

```
https://<tmn>/api/v1/CISUsers('<CisId>') )
```

Example:

```
https://<tmn>/api/v1/CISUsers('P000095')
```

HTTP Method: PATCH

We recommend using the HTTP method PATCH:

In OData 2.0 specification, the HTTP method PATCH has merge semantics, whereas the HTTP method PUT is used for an update of all properties of an OData entity (property values are replaced either by taking the values indicated in the request body, or reset to their default values if not mentioned in the request). The HTTP PUT method is not supported.

Some HTTP clients do not support the PATCH method. In this case, you can try to use theX-HTTP-Method-Override request propertyy

HTTP Headers:

Name	Value	Description
Content-Type	application/json;charset=utf-8	Content type must be JSON and the charset UTF-8.

Name	Value	Description
Accept	application/json	If not set, the body of the HTTP response will contain XML data. If set, the response will be JSON.

HTTP request body (example):

```
{"Email":"test3@test.com", "LoginName":"SmithJ", "FamilyName":"Smith", "GivenName":"John", "Password":"password1234"}
```

The properties provided in the request are merged with the existing properties. Therefore, you can only specify a subset of the properties. For example, if you only want to change the password, then you only specify the Password property.

HTTP request body (example of changing the password only):

```
{"Password":"Pass1234"}
```

Properties whose values are null in the request body are ignored during the merge.

Example:

```
{"LoginName":null}
```

If you want to initialize a property, use the empty value, for example:

```
{"LoginName":""}
```

Empty values are not possible for the properties Email and Password.

Properties in the request body:

Properties in the Request Body

Name	Mandatory	Description
Email	No	E-mail address must be unique.
LoginName	No	Login name of the user
Type	No	Not used yet
Password	No	
FamilyName	No	
GivenName	No	You can use this property to set the given name of the user.

Response Status and Error Codes

Response Code	Response Description
204 No Content	User was merged using the provided properties.
400 Bad Request	Wrong properties used.
401 Unauthorized	User of the call to the OData API does not exist in the Cloud Identity service or the password is wrong.
403 Forbidden	User not assigned the appropriate role. Assign either the <code>AuthGroup.TenantPartnerDirectoryConfigurator</code> or <code>AuthGroup.Administrator</code> authorization group or the <code>CloudIdentityService.Write</code> role.
500 Internal Server Error	Changing the user properties failed. For more details, check the Java trace files. Possible reasons for the error are: The user with the specified Cisd no longer exists. In this case, the response body contains a message such as <code>Update of the user with ID P000095 failed. HTTP response code of Identity Service is: 400 - User with [id] not found.</code> The provided password is insufficient, because it does not fulfill the requirements of a valid password.

Response Code	Response Description
	The provided Email value is already used with another CISUser.

Delete User

```
https://<tmn>/api/v1/CISUsers('<CisId>') )
```

Example:

```
https://<tmn>/api/v1/CISUsers('P000095')
```

HTTP Method: DELETE

HTTP response code for successful deletion: 200:

Response Status and Error Codes

Response Code	Response Description
200 OK	CISUser was successfully deleted.
400 Bad Request	Wrong properties used.
401 Unauthorized	User of the call to the OData API does not exist in the Cloud Identity service or the password is wrong.
403 Forbidden	User not assigned the appropriate role. Assign either the AuthGroup.TenantPartnerDirectoryConfigurator or AuthGroup.Administrator authorization group or the CloudIdentityService.Write role.
500 Internal Server Error	User deletion failed. If the returned body contains a message such as No CISUser found for CisId P000095, deletion was not possible because a user with the specified CisId does not exist. For more details, check the Java trace files.

Related Information

Local OData API Access for Edge Integration Cell

For Edge Integration Cell users, it is possible to configure local OData API access to call monitoring APIs directly from Edge Integration Cell.

You can configure local OData API access to call monitoring APIs directly from your Edge Integration Cell runtime, without requiring intervention from the Cloud Integration UI.

The available APIs cover the following:

- [Message Processing Logs](#)
- [Message Stores](#)
- [Partner Directory](#)
-

These APIs are published as a package on the SAP Business Accelerator Hub at [Overview | SAP Edge Integration Cell | SAP Business Accelerator Hub](#).

For a more detailed version of this topic, see [Local OData API Access](#).

How to Work with the Example Integration Flows

For each guideline, one or more example integration flows are provided that help you to quickly understand the topic. These integration flows are kept as simple as possible and can be set up and executed quickly.

Integration Packages

The integration flows are contained in a set of dedicated integration packages published on SAP Business Accelerator Hub. To learn how to access the integration packages and deploy the integration flows, see [Copying the Integration Package and Deploying the Integration Flows](#).

HTTP Client as Sender Component

Many integration flows are triggered by an HTTP call.

You can use any HTTP client software to execute these integration flows, for example, Postman. To facilitate running the scenario when using Postman, we provide Postman collections that allow you to run the HTTP requests with minimal effort.

The Postman collections associated with the example integration flows are contained in the integration packages. To learn how such a Postman collection can be downloaded, imported into Postman, and executed, see [Working with Postman Collections](#).

Generic Receiver

For the example integration flows, we provide a generic receiver. This measure saves you the effort of configuring a receiver system. The generic receiver is implemented with a separate integration flow (with name **GenericReceiver**) that creates a data store entry for each receiver call.

Each integration package (as listed under [Copying the Integration Package and Deploying the Integration Flows](#)) has its own **Generic Receiver** integration flow with an integration package-specific address.

As a prerequisite to run an example integration flow, you also need to deploy the **Generic Receiver** integration flow contained in the same integration package as the example integration flow.

For more information about this integration flow, see [Generic Receiver](#).

OData Calls to an External Component

Certain reference integration flows read data from an external WebShop application.

These integration flows are connected to a WebShop application that we provide for training and demonstration purposes.

For more information, see [WebShop Example Application](#)

Copying the Integration Package and Deploying the Integration Flows

To work with the example integration flows, you need to perform the following steps:

1. Copy the integration package that contains the example integration flows for the guideline into your workspace.

i Note

For more information, try out the tutorial [Getting Started with Integration Flow Design Guidelines - Timer Initiated Scenario](#).

The integration flows are published in dedicated integration packages on SAP Business Accelerator Hub. For each guideline category (for example, [Handle Errors Gracefully](#)), you can find a dedicated integration package as summarized in the following table:

Guideline Category and Documentation	Integration Package Name
Learn the Basics	Integration Flow Design Guidelines - Learn the Basics 
Run an Integration Flow Under Well-Defined Boundary Conditions	Integration Flow Design Guidelines - Run an Integration Flow Under Well-Defined Boundary Conditions 
Relax Dependencies to External Components	Integration Flow Design Guidelines - Relax Dependencies to External Components 
Keep Readability in Mind	Integration Flow Design Guidelines - Keep Readability in Mind 
Handle Errors Gracefully	Integration Flow Design Guidelines - Handle Errors Gracefully 
Use Prepackaged Integration Content Provided by SAP	(no integration package available)
Apply the Highest Security Standards	Integration Flow Design Guidelines - Apply Highest Security Standards 
Use the Partner Directory Appropriately	Integration Flow Design Guidelines - Use the Partner Directory Appropriately 
Use Scripting Appropriately	Integration Flow Design Guidelines - Scripting Guidelines 
Guidelines to Implement Specific Integration Patterns	Integration Flow Design Guidelines - Enterprise Integration Patterns 

2. Open the package it in the **Design** section of the Web UI.

The integration flows are in the **Artifacts** tab and the Postman collections are in the **Documents** tab.

3. Search for the integration flows that belong to the guideline that you want to explore.

The integration flow names follow the naming pattern **<Category> - <Guideline> - <extension>**.

4. Deploy each required integration flow.

Make sure that the **Generic Receiver** integration flow is also deployed. This integration flow serves as a common receiver mock up for all the integration flows.

For more information, see [Generic Receiver](#).

Working with Postman Collections

Each guideline or pattern has associated example integration flows and 1 Postman collection, which can be used to trigger the example.

The integration flows, as well as the associated Postman collections, are contained in the related integration package.

Downloading the Postman Collection

The name of the Postman collection starts with the name of the guideline and ends with the suffix **_PostmanCollection**. You can find it in the **Documents** tab of the integration package. Download the compressed file and unpack it to any folder. After that, import the unpacked **.json** file into your Postman by clicking **Import** in the Postman user interface.

Executing a Postman Collection

After having imported the Postman collection, you must first define an environment that contains the connection parameters of your tenant. In the upper-right corner of the Postman user interface, select **Manage Environments** and add a new environment with an arbitrary name. Add the following 3 variables with their corresponding values to this environment:

username: The name of your user

password: Your password

host: The fully qualified hostname of your Cloud Integration tenant

After you've performed this task, you can start the Postman runner. On the **Collection Runner** user interface, choose your collection, select the environment, and start the run. Now, all requests of the collection are sent and the tests, if any, are executed.

i Note

When the example integration flow is protected against Cross-Site-Request-Forgery (CSRF) and the associated setting is activated in the HTTPS sender adapter, note the following: If the Postman collection contains an HTTP Post request, typically 2 requests are necessary:

- A GET request where the X-CSRF-Token is fetched from the target system
- Another POST request using the value of the fetched X-CSRF-Token

In such cases, the Postman collection contains both requests.

More information:

[HTTPS Sender Adapter](#)

[Send the HTTP Request and Process the Integration Flow](#)

i Note

Some integration flows are triggered by a Timer event. In that case, the body of the message that represents the inbound call is created initially within the integration flow in a Content Modifier step. No HTTP request is required to start the integration flow. The integration flow is started on deployment.

Generic Receiver

The generic receiver is implemented with a separate integration flow (with name GenericReceiver) that creates a data store entry for each receiver call.

When setting up integration scenarios, a significant effort is typically required to configure the connected sender and receiver systems. One simple receiver component available to everyone for free is an email account that can be connected with SAP Cloud Integration using the Mail adapter.

However, even setting up a connection with an email server requires additional steps such as:

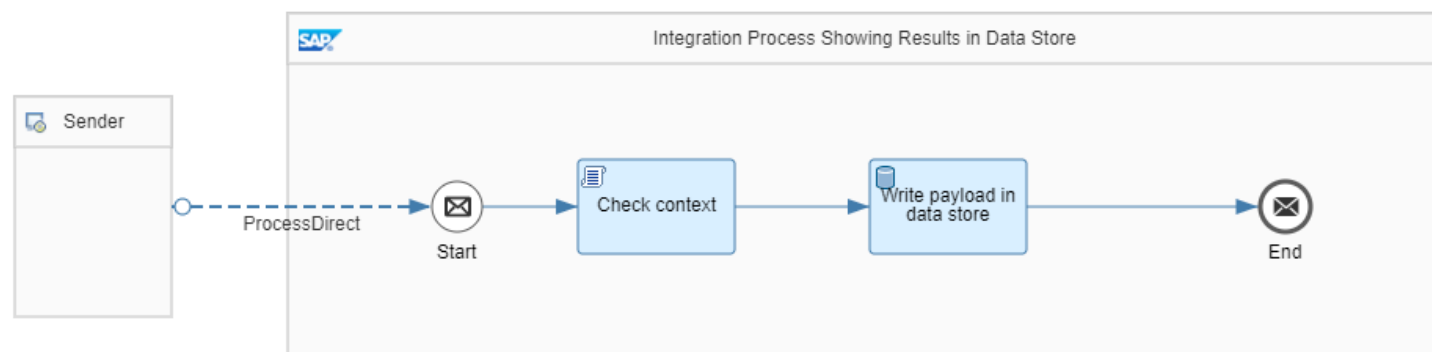
- Performing specific security settings for the email account
- Uploading the email provider's server certificate to the keystore

To save these efforts for you and to enable you to focus on the guideline, a generic receiver has been added to certain integration flows. The generic receiver component is implemented using a dedicated integration flow: [Generic Receiver](#).

i Note

As a prerequisite to read data store content, your user needs to have the [AuthGroup.BusinessExpert](#) authorization group assigned.

Each integration flow for which a receiver component is modeled, calls the integration flow [Generic Receiver](#) through the ProcessDirect adapter (always using the same address). When called, the [Generic Receiver](#) integration flow creates a data store entry on your tenant with the following components:



The [Generic Receiver](#) integration flow is called from a sender integration flow through the ProcessDirect adapter. The sender integration flow provides a header with the name [context](#). This value represents the pattern or guideline that is covered by the calling integration flow. In some cases, the sender integration flow provides an additional header with the name [receiver](#).

The [Check context](#) Script step checks the value of [context](#) header that is provided by the calling integration flow. If no value is provided, the default value [Result](#) is set.

The Data Store Write step [Write payload in data store](#) creates a data store entry with the following attributes:

Data Store Name	Generated from the value of the context header (through the dynamic expression \${header.context}).
Entry ID	Generated from the value of the receiver header (through the dynamic expression \${header.receiver}). If the calling integration flow doesn't provide any such value, a unique identifier is generated. i Note If the calling integration flow sends the message potentially to more than 1 receiver (for example, in one of the variants that showcase the Content-Based Routing pattern), the data store could have different entries. The Entry ID value for the actual message processing run is generated based on the actual value of the receiver header.

i Note

Each integration package (as listed under [Copying the Integration Package and Deploying the Integration Flows](#)) has its own [Generic Receiver](#) integration flow with an integration package-specific address.

Example

The [Pattern Content Based Routing - Ignore If No Receiver](#) integration flow (for pattern variant [Variant: Ignore](#)) sets the following header values:

Header	Value
context	ContentBasedRouting-IgnoreIfNoReceiver This value identifies the pattern variant.
receiver	The value depends on the value of the shippingCountry element in the actual message processing run.

WebShop Example Application

This is custom documentation. For more information, please visit [SAP Help Portal](#).

10/6/26, 11:57 AM

Various integration flows are connected to a WebShop application that contains data from a fictitious product catalog.

Certain reference integration flows read data from an external WebShop application.

This WebShop application serves training and demonstration purposes. It contains data from a fictitious product catalog and provides an API that allows remote components, such as SAP Cloud Integration, to read certain data sets such as product information.

The API of the WebShop application supports the Open Data Protocol (OData). For our scenarios, we use the ESPM WebShop application, which is based on the Enterprise Sales and Procurement Model (ESPM) provided by SAP. You can access the user interface of the demo application at the following address: <https://refapp-espm-ui-cf.cfapps.eu10.hana.ondemand.com/webshop/index.html>. You can see fictitious products on this website. Note that some example integration flows expect a **productId** value provided with the HTTP client request. You can check out this website to find the defined **productId** values a product.

SAP Cloud Integration connects to this application through the OData receiver adapter.